

Artificial Intelligence

Lecture slides

Tudor Mihoc

UBB

February 26, 2026

Introduction

Course organization

- 14 Lectures

- 14 Practical Laboratories

- Evaluation:

- Each Assignment graded G_i (from 1 to 10).
- Written examination graded G_e (from 1 to 10).
- Final grade: $\left(\frac{\sum_{i=1}^{no. \text{ assignments}} G_i}{no. \text{ assignments}} + G_e \right) / 2$

What You Will Learn in This Course

Course focus

- Fundamental algorithms used in Artificial Intelligence.
- Classical AI models and methods (not limited to deep learning).
- Simple, hands-on implementations of core techniques.
- Interpretation and critical analysis of model results.

Important clarification

- This course does **not** aim to build Artificial General Intelligence (AGI).
- The primary objective is to understand the **principles and mechanisms** underlying AI systems.

The goal is to understand how AI systems work, not to treat them as black boxes.

What is Artificial Intelligence?

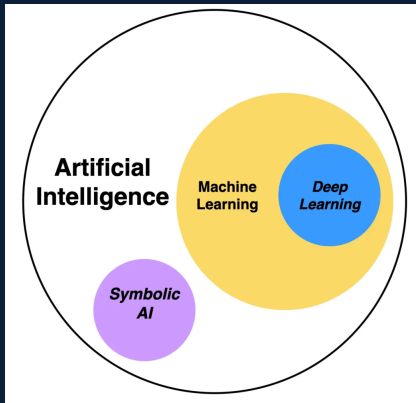
- Artificial Intelligence (AI) is a field of computer science dedicated to creating machines or systems capable of performing tasks that typically require human intelligence.
- AI refers to **algorithms** and **models** designed to enable machines to simulate cognitive functions like *learning, reasoning, problem-solving, and decision-making*.

First Classification

STRONG AI - a hypothetical form of AI that possesses general intelligence, capable of understanding, learning, and reasoning across domains, comparable to a human mind.

WEAK AI - AI systems designed to perform specific tasks or solve narrow problems, without genuine understanding or consciousness.

General facts on AI



AI IS NOT ML!!

AI = Mathematics + Algorithms

Artificial Intelligence is fundamentally based on mathematical models and algorithmic procedures.

Foundational components:

- **Data Structures** → graphs, trees, state spaces
- **Algorithms** → search, dynamic programming, numerical optimization
- **Mathematics** → linear algebra, probability theory, statistics

How learning actually happens:

- Models are defined as *parameterized functions*.
- Training corresponds to *minimizing a loss function*.
- Learning is performed via *optimization algorithms* (e.g., gradient-based methods).

Critical clarification

AI systems **do not** “think” or
“understand”!

Intelligence emerges from formal models
+ computation, **not** cognition!

Classical AI vs Modern AI

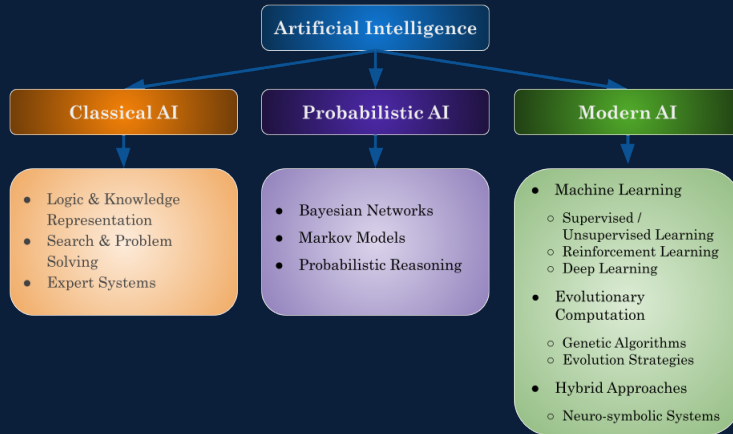
Classical AI (Symbolic / Rule-Based)

- Knowledge represented explicitly using rules, logic, and symbols
- Intelligence achieved through search, reasoning, and inference
- System behavior is interpretable and deterministic
- Performance depends on hand-crafted knowledge
- Typical methods: state-space search, expert systems, logic-based reasoning

Modern AI (Data-Driven / Statistical)

- Knowledge represented implicitly through parameters learned from data
- Intelligence achieved through optimization and learning
- System behavior is probabilistic and data-dependent
- Performance depends on data availability and model capacity
- Typical methods: machine learning, deep learning, probabilistic models

AI fields from a historical perspective



Historical Roots of Artificial Intelligence

- The idea of intelligent machines dates back to ancient myths and philosophy.
- In Greek mythology, Hephaestus was said to create automata—self-operating mechanical servants—anticipating the concept of artificial beings.
- These early narratives inspired later philosophical inquiry into intelligence and artificial life.
- Philosophers such as René Descartes questioned the nature of reason, consciousness, and mechanistic thinking, asking whether machines could ever truly think.

Turing and the Birth of Computational AI

- The emergence of Artificial Intelligence as a **computational discipline** is closely tied to the work of **Alan Turing**.
- Turing introduced the **Turing Machine**, a theoretical model capable of performing any computation expressible as an algorithm, establishing the foundations of modern computer science.
- This framework demonstrated that machines could, in principle, carry out tasks previously associated with human intelligence.
- Turing later proposed the **Turing Test**, shifting the focus of intelligence from internal mechanisms to **observable behavior**.

The AI Winter

- Despite early optimism, progress in Artificial Intelligence was **not linear**.
- Periods known as **AI Winters** marked significant slowdowns in AI research and development.
- The first major AI Winter occurred in the **1970s**, driven by:
 - overly ambitious expectations of human-level intelligence;
 - limited computational power and data availability;
 - AI systems failing to scale to complex, real-world problems.
- These limitations led to growing scepticism about AI's feasibility.
- As a consequence, **research funding declined** and many AI projects were abandoned or postponed.

AI Resurgence and Modern Advances

- AI experienced a major resurgence in the **late 20th and early 21st centuries**.
- This revival was enabled by:
 - advances in **machine learning** and **deep learning**;
 - the availability of **large-scale datasets**;
 - significant improvements in **computational power**.
- Deep learning, based on artificial neural networks, proved especially effective at modeling complex patterns.
- Modern AI systems achieved notable successes in:
 - image recognition,
 - natural language processing,
 - autonomous systems.
- As a result, AI evolved from a struggling research field into a **core technology across multiple industries**.

Key Milestones in AI Research

- ★ **The Dartmouth Conference (1956)**
 - term *Artificial Intelligence*; AI as a distinct research field; the idea that aspects of intelligence could be formally described; promoted symbolic reasoning as the dominant early paradigm.
- ★ **The Development of Early AI Programs**
 - creation of rule-based and logic-driven systems; early successes; machine-based reasoning; limited by computational power and handcrafted knowledge.
- ★ **Rise of Machine Learning and Deep Learning**
 - shift from rule-based systems to data-driven models; increased focus on statistical learning and optimization; breakthroughs enabled by large datasets and GPUs; deep neural networks achieved state-of-the-art performance.
- ★ **Recent Developments in AI**
 - widespread AI in industry and society; advances in natural language processing and computer vision; attention to ethics, fairness, and interpretability; emergence of hybrid and foundation models.

Strategic Positioning of This Course

Course Strategy

We begin with the **modern paradigm (Machine Learning)** to establish mathematical and statistical foundations.

Afterwards, we return to the **symbolic paradigm of Classical AI** to compare reasoning mechanisms and optimization principles.

AI is a unified field built on complementary paradigms.

Some mathematical prerequisites

Linear Algebra in Artificial Intelligence

The language of data and transformations

- Almost all AI models operate on **vectors and matrices**.

Concepts

- Vectors and matrices
- Matrix multiplication
- Eigenvalues and eigenvectors
- Dot products and vector norms

Why it matters

- Data is represented as vectors.
- Neural networks are composed of stacked matrix multiplications.
- Embeddings for text and images reside in high-dimensional vector spaces.

Linear Algebra: Essential Formulas

AI represents data as vectors and transforms it using matrices.

■ **Vector representation:**

$$x = (x_1, x_2, \dots, x_d) \in \mathbb{R}^d$$

■ **Linear transformation:**

$$y = Wx + b$$

■ **Similarity** (dot product) and **magnitude** (norm):

$$\langle x, y \rangle = \sum_{i=1}^d x_i y_i, \quad \|x\|_2 = \sqrt{\sum_{i=1}^d x_i^2}$$

Linear Algebra: Meaning Behind the Formulas

- **Vector representation:** $x = (x_1, x_2, \dots, x_d)$.
 - Data points and embeddings are vectors in $\mathbb{R}^d$.
- **Linear transformation:** $y = Wx + b$
 - Example: a neural network layer that applies a linear mapping.
- **Similarity (dot product):** $\langle x, y \rangle$
 - Measures alignment or similarity between data points.

Understanding the meaning of the formula matters more than computing it by hand.

Linear Algebra: A Simple Numerical Example

- **Vector representation** (a data sample described by two numerical features)

$$x = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \in \mathbb{R}^2$$

- **Linear transformation** (transforms the original data into a new representation)

$$W = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \quad y = Wx + b = \begin{pmatrix} 5 \\ 1 \end{pmatrix}$$

- **Similarity and magnitude**

$$\langle x, y \rangle = 1 \cdot 5 + 2 \cdot 1 = 7, \quad \|x\|_2 = \sqrt{1^2 + 2^2} = \sqrt{5}$$

Calculus

Learning in AI is formulated as an optimization problem.

Concepts:

- Derivatives and partial derivatives
- Gradients
- Chain rule
- Optimization methods (e.g., gradient descent)

Importance:

- Training corresponds to minimizing a loss function.
- Backpropagation relies on multivariable calculus.
- Gradients are employed for updating model parameters.

Optimization Theory

Optimization is the engine behind training AI models.

Concepts:

- Convex vs. non-convex optimization
- Constraints
- Lagrange multipliers
- Stochastic optimization methods

Importance:

- Training modern models is a difficult optimization problem.
- Many practical AI failures are optimization-related.
- Optimization links theory with real-world system performance.

Optimization Problem: Formal Definition

An optimization problem consists of finding a value of a decision variable that minimizes or maximizes a given objective function, possibly subject to constraints.

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

- x – decision variables
- $f(x)$ – objective function
- $g_i(x)$, $h_j(x)$ – constraints

Find the best feasible solution.

Information Theory

Information theory quantifies uncertainty and information content.

■ Main concepts

- Entropy
- Cross-entropy
- Kullback–Leibler divergence
- Mutual information

■ Why it matters

- Widely used loss functions in classification.
- Fundamental to language modeling.
- Important for compression and representation learning.

Graph Theory

Some AI problems are about relationships, not vectors.

- **Main Concepts**

- Nodes and edges
- Paths and connectivity
- Adjacency matrices

- **Why it matters**

- Modeling social and interaction networks.
- Knowledge graph representations.
- Graph neural networks and recommender systems.

Logic and Discrete Mathematics

Symbolic reasoning complements numerical methods in AI.

■ Main Concepts

- Propositional and predicate logic
- Boolean algebra
- Combinatorics

■ Why it matters

- Rule-based and knowledge-based systems.
- Decision trees and logical inference.
- Planning and symbolic reasoning.

Probability Theory

AI systems reason under uncertainty using probabilities.

■ Main Concepts

- Random variables
- Probability distributions
- Expectation and variance
- Conditional probability

■ Why it matters

- Classification models output probability estimates.
- Language models predict likely next tokens.
- Explicit uncertainty handling is essential in real systems.

Statistics

Statistics turns data into reliable conclusions.

■ **Main Concepts**

- Estimation and hypothesis testing
- Regression analysis
- Bias–variance tradeoff
- Sampling and confidence intervals

■ **Why it matters**

- Model evaluation and comparison.
- Understanding overfitting and generalization.
- Designing meaningful experiments and benchmarks.

Calculus & Optimization: How Models Learn

- **Model as a parameterized function:**

$$f(x; w)$$

- **Training objective (empirical risk minimization):**

$$\min_w \frac{1}{N} \sum_{i=1}^N \ell(f(x_i; w), y_i)$$

- **Gradient-based update (Gradient Descent / SGD):**

$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

Learning = minimizing a loss via gradients and optimization.

Calculus & Optimization: Meaning Behind the Formulas

■ Parameterized model $f(x; w)$

- w represents the adjustable parameters of the model.
- Learning consists of finding values of w that produce good predictions.

■ Loss minimization

- The loss function $\ell(\cdot)$ measures how wrong a prediction is.
- Training means reducing the average error over the dataset.

■ Gradient-based updates

- The gradient indicates how the loss changes with respect to parameters.
- Gradient descent moves parameters in the direction of decreasing error.

Learning is an optimization process, not a cognitive one.

Probability: Uncertainty and Prediction

- **Model outputs probabilities:**

$$p(y \mid x)$$

- **Bayes' rule (updating beliefs):**

$$p(y \mid x) = \frac{p(x \mid y) p(y)}{p(x)}$$

- **Expectation (average behavior under uncertainty):**

$$\mathbb{E}[X] = \sum_x x p(x) \quad \text{or} \quad \mathbb{E}[X] = \int x p(x) dx$$

AI systems reason and decide under uncertainty using probabilities.

Probability: Meaning Behind the Formulas

- **Probabilistic predictions** $p(y \mid x)$
 - Models output degrees of belief, not absolute answers.
 - Decisions are made based on likelihood, not certainty.
- **Bayesian updating**
 - Prior knowledge $p(y)$ is updated using observed data x .
 - New evidence refines, rather than replaces, existing beliefs.
- **Expectation**
 - Represents average behavior under uncertainty.
 - Many AI objectives are defined as expected losses or rewards.

Uncertainty is handled explicitly, not ignored.

Artificial Intelligence

Lecture slides - 2

Tudor Mihoc

UBB

March 4, 2026

What You Will Learn in This Particular Lecture:

- Introduction on Machine Learning:
 - Definition of Models
 - Process of Training in general
 - Validation in general

We will examine an example of a parametric model used as baseline model for comparison while validating: Linear regression.

Notes on Machine Learning

What Is a Model in Machine Learning?

- A **model** is a function (usually parameterized) that maps **features** to a **prediction**:

$$\hat{y} = h^{(w)}(x)$$

- The parameters w are learned from data.

What the model is NOT:

- Not the dataset
- Not the training algorithm
- Not the loss function

The model is the object we *deploy and use* to make predictions — training is only the process used to obtain it.

Model vs Algorithm vs Objective

Model

- A parameterized function

$$h^{(w)}(x)$$

- Maps features to predictions
- What we *use and deploy*

Algorithm

- Procedure to learn parameters
- Solves an optimization problem
- Runs during training only

Objective

- Quantifies what “good” means
- Function to be minimized
- Encodes the learning goal

We train an *algorithm* to optimize an *objective* in order to obtain a *model*.

Examples: Model vs Algorithm vs Objective

Model

- Linear model
- Neural network
- Decision tree

Algorithm

- Gradient descent
- SGD, FedAvg
- Newton's method

Objective

- Squared loss
- Cross-entropy
- Regularized ERM

M.L. Framework

Machine Learning (ML) aims to learn a *hypothesis* (prediction rule)

$$h \in \mathcal{H}$$

from a hypothesis space $\mathcal{H}$, such that h accurately predicts the *label* of a data point based solely on its *features*.

A crucial step in applying ML is defining:

- what exactly constitutes a *data point*,
- which properties are used as *features*,
- and what is considered the *label*.

These choices strongly influences the performance of ML methods.

Data Points, Features, and Labels

A data point is denoted by $\mathbf{z}$ and is represented by:

$$\mathbf{z} = (\mathbf{x}, y),$$

where:

- $\mathbf{x} \in \mathbb{R}^d$ is the *feature vector* (d is the number of features).
- $y \in \mathbb{R}$ is the *label* (continuous in regression problems).

Strictly speaking, a data point may contain additional information. A more precise notation is:

$$\mathbf{x} = \mathbf{x}(\mathbf{z}), \quad y = y(\mathbf{z}),$$

highlighting that features and labels are *functions* of the data point.

Example: Weather Prediction

We use weather prediction as an example.

Each data point corresponds to a weather measurement at a certain FMI station.

- Station name (e.g., TurkuRajakari)
- Latitude and longitude (lat, lon)
- Timestamp (YYYY-MM-DD HH:MM:SS)

These features are stacked into a feature vector $\mathbf{x}$.

The label $y \in \mathbb{R}$ is the *maximum daytime temperature* (°C).

Prediction and Loss

Given features $\mathbf{x}$, a hypothesis h produces a prediction:

$$\hat{y} = h(\mathbf{x}).$$

Predictions are generally imperfect:

$$h(\mathbf{x}) \neq y.$$

The prediction error is quantified using a *loss function*:

$$L((\mathbf{x}, y), h).$$

The choice of loss function strongly affects both statistical and computational properties of ML methods.

Empirical Risk Minimization (ERM) or Minimizing the Average Loss

Given a training dataset

$$\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})\},$$

ML aims to minimize the average loss:

$$\hat{h} \in \arg \min_{h \in \mathcal{H}} \frac{1}{m} \sum_{r=1}^m L((\mathbf{x}^{(r)}, y^{(r)}), h).$$

In **practice** is known as *Empirical Risk Minimization* or *Minimizing the Average Loss*.

What Is the Real Goal of a Model?

- The goal is **not** to perform well on the training data.
- The real goal is to make **accurate predictions** on **unseen, future data**.

Formally, we want to minimize the **expected loss** (risk):

$$\mathbb{E}_{(\mathbf{x}, y) \sim p(\mathbf{x}, y)} \{L((\mathbf{x}, y), h)\}.$$

Important: Low training loss does **not** guarantee good test performance.

What Is $p(\mathbf{x}, y)$? (Data-Generating Distribution)

Consider a data point $(\mathbf{x}, y)$

Definition

$p(\mathbf{x}, y)$ denotes the **(unknown) joint probability distribution** of features and labels. We write:

$$(\mathbf{x}, y) \sim p(\mathbf{x}, y)$$

meaning that observed data points are generated according to $p(\mathbf{x}, y)$.

- The **true goal** of learning is to perform well on *new* samples from $p(\mathbf{x}, y)$
- Since $p(\mathbf{x}, y)$ is unknown, ERM replaces expectations by empirical averages on a dataset

Empirical Risk Minimization versus Minimizing the Average Loss

They often coincide in practice, but they are not the same conceptually.

- Minimizing the Average Loss $\longleftrightarrow$ optimization viewpoint
- Empirical Risk Minimization $\longleftrightarrow$ statistical learning viewpoint

Minimizing the Average Loss

Given a fixed training set

$$\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})\},$$

we can define the *average loss* objective:

$$\min_{h \in \mathcal{H}} \frac{1}{m} \sum_{r=1}^m L((\mathbf{x}^{(r)}, y^{(r)}), h).$$

Interpretation (deterministic):

- The dataset is fixed; the objective is a number we can compute.
- Focus: **how to solve** the minimization efficiently.

Typical questions: convexity, gradients, iteration complexity, runtime.

Empirical Risk Minimization

Assume data points are i. i. d. samples from an unknown distribution $p(\mathbf{x}, y)$: $(\mathbf{x}, y) \sim p(\mathbf{x}, y)$.
The ideal goal is to minimize the *expected risk*:

$$\min_{h \in \mathcal{H}} \mathbb{E}\{L((\mathbf{x}, y), h)\}.$$

Since $p(\mathbf{x}, y)$ is unknown, we replace the expectation by the empirical average over $\mathcal{D}$:

$$\hat{h} \in \arg \min_{h \in \mathcal{H}} \frac{1}{m} \sum_{r=1}^m L((\mathbf{x}^{(r)}, y^{(r)}), h).$$

Interpretation (probabilistic): training loss is a proxy for test performance.

Same Formula, Different Meaning

Both views often lead to the same optimization problem:

$$\min_{h \in \mathcal{H}} \frac{1}{m} \sum_{r=1}^m L((\mathbf{x}^{(r)}, y^{(r)}), h),$$

but the **interpretation differs**.

Minimizing average loss

- Optimization task on a fixed dataset
- Focus: algorithms and compute
- Output: a minimizer of a concrete objective

ERM

- Statistical principle (i. i. d. assumption)
- Focus: generalization guarantees
- Output: an estimate of the risk-minimizer

Training

Training

Training an ML model means solving ERM to obtain $\hat{h}$.

Two fundamental questions arise:

- **Computational:** How much computation is required to solve ERM?
- **Statistical:** How accurate is $\hat{h}(x)$ on unseen data?

These questions are central to the analysis of a ML systems.

Computational Aspects of ERM

A principled approach to designing ML methods is to apply *optimization algorithms* to solve the ERM problem:

$$\hat{h} \in \arg \min_{h \in \mathcal{H}} \frac{1}{m} \sum_{r=1}^m L((x^{(r)}, y^{(r)}), h).$$

Most optimization methods operate *iteratively*.

Iterative Optimization

Starting from an initial hypothesis $h^{(0)}$, an optimization method constructs a sequence:

$$h^{(0)}, h^{(1)}, h^{(2)}, \dots$$

Each iterate is intended to be a more accurate approximation of a solution $\hat{h}$ to ERM.

Computational complexity is often measured by:

- the number of iterations required,
- to reach a prescribed level of accuracy.

Gradient-Based Optimization

For parametric models with a smooth loss function, ERM can be solved using *gradient-based methods*.

Starting from initial parameters $w^{(0)}$, we iterate:

$$w^{(k)} = w^{(k-1)} - \eta \nabla f(w^{(k-1)}),$$

where $\eta > 0$ is the step size (learning rate).

Next lecture we will go deeper on gradient-based methods!

Gradient Step for Linear Regression

For linear regression, the gradient step becomes:

$$w^{(k)} = w^{(k-1)} + \frac{2\eta}{m} \sum_{r=1}^m x^{(r)} (y^{(r)} - (w^{(k-1)})^\top x^{(r)}).$$

This update requires computing a sum over all m data points.

Cost of One Iteration

Questions:

- How much computation is needed for *one* iteration?
- How many iterations are required for convergence?

For a typical setup (e.g., Python on a laptop):

- One gradient step requires $\mathcal{O}(m)$ arithmetic operations
- (additions and multiplications).

The second question (number of iterations) will be studied later.

Extreme Case: No Features

Consider the special linear model with no features:

$$h(x) = w.$$

The ERM problem for squared loss has a closed-form solution:

$$\hat{w} = \frac{1}{m} \sum_{r=1}^m y^{(r)}.$$

This corresponds to predicting the *average label*.

Computational Interpretation

Solving

$$\hat{w} = \frac{1}{m} \sum_{r=1}^m y^{(r)}$$

requires:

- summing m numbers,
- followed by one division.

Thus, the computational cost is:

$$\mathcal{O}(m)$$

elementary arithmetic operations.

Validation and Diagnosis of ML

Why Do We Need Validation?

So far, our analysis of generalization relied on a **probabilistic model** for data generation.

- Generalization bounds rely on assumptions about how data are generated
- If these assumptions are violated, the theoretical bounds may not apply
- In practice, we often need a **data-driven way** to assess a learned model

Validation evaluates whether a learned hypothesis performs similarly well *inside* and *outside* the training set.

Training Error vs Validation Error

Validation is based on data that are **not used for training**.

Basic Validation Principle

A learned hypothesis $\hat{h}$ is evaluated on:

- the training set $\mathcal{D}_{\text{train}}$
 - a separate validation set $\mathcal{D}_{\text{val}}$
-
- Training error estimates how well $\hat{h}$ fits the training data
 - Validation error estimates how well $\hat{h}$ generalizes

One Iteration of ML Training and Validation

Input:

- dataset $\mathcal{D}$
- hypothesis space $\mathcal{H}$
- loss function $L(\cdot, \cdot)$

Procedure:

- 1 Split $\mathcal{D}$ into training set $\mathcal{D}_{\text{train}}$ and validation set $\mathcal{D}_{\text{val}}$
- 2 Learn a hypothesis via ERM on $\mathcal{D}_{\text{train}}$
- 3 Compute training error E_t
- 4 Compute validation error E_v

Output: learned hypothesis $\hat{h}$, training error E_t , validation error E_v .

Diagnosing ML Models Using Validation

We diagnose a learned model by comparing:

E_t (training error), E_v (validation error), E_{ref} (baseline)

- Validation is more informative when a reference baseline is available
- Baselines help interpret whether errors are “good” or “bad”

Baselines for Model Evaluation

Possible sources for a baseline error E_{ref} :

- **Probabilistic models:**

- The Bayes estimator achieves the minimum possible risk
- Provides a theoretical lower bound

- **Existing ML methods:**

- May be too expensive or impractical
- Still useful as a performance reference

- **Human experts:**

- Example: medical diagnosis by experienced clinicians

Interpreting Training and Validation Errors

Case 1: $E_t \approx E_v \approx E_{\text{ref}}$

- Good generalization
- No clear overfitting
- Limited room for further improvement

Case 2: $E_v \gg E_t$

- Overfitting
- Remedy: smaller model, regularization, or more data

Case 3: $E_t \approx E_v \gg E_{\text{ref}}$

- Model too simple or optimization not working well

Case 4: $E_t \gg E_v$

- Possible violation of the i. i. d. assumption
- Unlucky train-validation split or dataset bias

When Validation Becomes Difficult

Validation relies on:

- sufficiently large datasets
- approximate i. i. d. behavior of data points

If these conditions fail:

- Training and validation errors become harder to interpret
- Increasing dataset size or using data augmentation may help
- Statistical tests can be used to assess i. i. d. assumptions

Regularization: Why Do We Need It?

Consider an ERM-based ML method with:

- hypothesis space $\mathcal{H}$
- training set $\mathcal{D}$

An indicator of performance is the ratio:

$$\frac{d_{\text{eff}}(\mathcal{H})}{|\mathcal{D}|},$$

where $d_{\text{eff}}(\mathcal{H})$ measures the effective model complexity.

Observation:

- Larger ratios increase the risk of overfitting
- Regularization aims to reduce this ratio

Three Approaches to Regularization

Regularization techniques reduce

$$\frac{d_{\text{eff}}(\mathcal{H})}{|\mathcal{D}|}$$

via three main approaches:

- 1 **Increase $|\mathcal{D}|$:** collect more data or use data augmentation
- 2 **Penalize complexity:** add a regularization term to the ERM objective
- 3 **Shrink the hypothesis space:** impose constraints on model parameters

These approaches are closely related and often equivalent.

Regularized ERM

A common regularization strategy is to add a penalty term to ERM:

$$\hat{h} \in \arg \min_{h \in \mathcal{H}} \frac{1}{m} \sum_{r=1}^m L((x^{(r)}, y^{(r)}), h) + \alpha R(h),$$

where:

- $R(h)$ penalizes model complexity
- $\alpha > 0$ controls the strength of regularization

Larger α typically leads to a smaller effective hypothesis space.

Regularization as Hypothesis Space Shrinking

Regularized ERM can be interpreted as:

- ERM on a *restricted* hypothesis space $\mathcal{H}(\alpha) \subseteq \mathcal{H}$
- Increasing α shrinks $\mathcal{H}(\alpha)$

Example of explicit constraints:

$$\|w\|_2 \leq 10$$

Remark: Penalization and constraint-based regularization are two views of the same mechanism.

Example: Linear Regression (LR)

An example for a parametric model:

$$\mathcal{H}^{(d)} = \{h^{(\mathbf{w})} : \mathbb{R}^{(d)} \rightarrow \mathbb{R} | h^{(\mathbf{w})}(\mathbf{x}) = \mathbf{w}^T \mathbf{x}\} \quad (1)$$

This LR minimizes the average squared error:

$$\hat{\mathbf{w}}_{\text{LR}} \in \arg \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{m} \sum_{r=1}^m (y^{(r)} - \mathbf{w}^T \mathbf{x}^{(r)})^2. \quad (2)$$

This corresponds to ERM with a quadratic loss function.

Matrix Formulation

Define the feature matrix and label vector:

$$X = \begin{pmatrix} \mathbf{x}^{(1)} \\ \vdots \\ \mathbf{x}^{(m)} \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{pmatrix}.$$

The objective function becomes:

$$f(\mathbf{w}) = \frac{1}{m} (\mathbf{w}^\top X^\top X \mathbf{w} - 2\mathbf{y}^\top X \mathbf{w} + \mathbf{y}^\top \mathbf{y}).$$

This is a *smooth and convex* function.

Example: Ridge Regression

Consider a linear model:

$$h(x) = w^T x$$

Ridge regression uses the regularizer:

$$R(h) := \|w\|_2^2$$

The learning problem becomes:

$$\hat{w}(\alpha) \in \arg \min_{w \in \mathbb{R}^d} \left[\frac{1}{m} \sum_{r=1}^m (y^{(r)} - w^T x^{(r)})^2 + \alpha \|w\|_2^2 \right]$$

Regularization as Data Augmentation

Ridge regression admits an alternative interpretation:

Each data point (x, y) is replaced by many noisy copies:

$$(x + n, y), \quad n \sim \mathcal{N}(0, \alpha I)$$

- Features are perturbed with Gaussian noise
- Labels remain unchanged
- Equivalent to minimizing a regularized loss

Conclusion: Data augmentation and loss penalization are mathematically equivalent.

Computational and Statistical Aspects

Ridge regression minimizes a convex quadratic objective:

$$\hat{w}(\alpha) \in \arg \min_{w \in \mathbb{R}^d} w^\top Q w + w^\top q,$$

with:

$$Q := \frac{1}{m} X^\top X + \alpha I, \quad q := -\frac{2}{m} X^\top y$$

- For $\alpha > 0$, matrix Q is always invertible
- Improves numerical stability compared to linear regression

The choice of α controls the bias–variance trade-off and can be guided by:

- probabilistic error analysis
- comparison of training and validation errors

Machine Learning

Artificial Intelligence

What is ML?

Definition

- Arthur Samuel (1959)
 - “field of study that gives computers the ability to learn without being explicitly programmed”
- Herbert Simon (1970)
 - “Learning is any process by which a system improves performance from experience.”
- Tom Mitchell (1998)
 - “a well-posed learning problem is defined as follows: He says that a computer program is set to learn from an experience E with respect to some task T and some performance measure P if its performance on T as measured by P improves with experience E ”
- Ethem Alpaydin (2010)
 - Programming computers to optimize a performance criterion using example data or past experience.

Necessity

- Better computational systems
 - Too difficult or too expensive to be constructed manually
 - Systems that automatically adapt
 - Spam filters
 - Systems that discover information in large database → data mining
 - Financial analysis
 - Text/image analyses
- Understanding the biological systems

How to design a ML?

- Improve of task **T**
 - Establish the goal (what has to be learned) – *objective function* – and its *representation*
 - Select a learning algorithm to perform the inference of the goal based on experience
- Respect a performance metric **P**
 - Evaluation of the algorithm's performances
- Based on experience **E**
 - Select an experience database

Examples:

- T: playing checkers
- P: percent of winning games
- E: playing the game
- T: handwritten recognition
- P: percent of correct recognized words
- E: database of images with different words
- T: separate the spams
- P: percent of correct classified emails
- E: databases with annotated emails

Objective Function

What is the function that must be learned?

Ex.: checkers game → a function that:

Selects the next move

Evaluates a move

In order to identify the best next move

Ex. for checkers game

A linear combination of *#white_pieces*, *#black_pieces*,
#white_compromised_pieces,
#black_compromised_pieces

Representation of objective function

Different representations:

- Table
- Symbolic rules
- Numeric functions
- Probabilistic functions

There is a trade-off between

- How expressive is meaning of the representation
- Easy of learning

Objective function computation

- Polynomial time
- Non-polynomial time

The Learning Algorithm

Selection

- According to the training data
- Induce the hypothesis definition that
 - Match the data
 - Generalize the unseen data
- Main principle
 - Error minimisation (cost function – loss function)

Evaluation

Experimental

- By comparing different methods on different data (cross-validation)
- Collect data based on performances
 - Accuracy, training time, testing time
- Statistical analyse of the differences

Theoretic

- Mathematical analyse of algorithms and theorem proving
 - Computational complexity
 - Ability to match the training data
 - Complexity of the most relevant sample for learning

Comparison between 2 algorithms

Performance measures

- Parameters of a statistic series
- Proportion (percent) computed for a statistical series (ex. Accuracy)

Comparing based on confidence intervals

- For a problem and 2 solving algorithms with performances p_1 and p_2
- Confidence intervals

$$I_1=[p_1-\Delta_1, p_1+\Delta_1] \text{ and } I_2=[p_2-\Delta_2, p_2+\Delta_2]$$

- If $I_1 \cap I_2 = \emptyset \rightarrow$ algorithm 1 works better than algorithm 2 (for the given problem)
- if $I_1 \cap I_2 \neq \emptyset \rightarrow$ impossible to decide

Confidence interval for the mean (average)

- For a statistical series of n data, with computed mean m and dispersion σ , determine the confidence interval of the mean μ
- $P(-z \leq (m-\mu)/(\sigma/\sqrt{n}) \leq z) = 1 - \alpha \rightarrow \mu \in [m - z\sigma/\sqrt{n}, m + z\sigma/\sqrt{n}]$
- $P = 95\% \rightarrow z = 1.96$

Confidence interval for accuracy

- For an accuracy p computed for n data, determine the confidence interval of accuracy
- $p \in [p - z(p(1-p)/n)^{1/2}, p + z(p(1-p)/n)^{1/2}]$
- $P = 95\% \rightarrow z = 1.96$

$P=1-\alpha$	z
99.9%	3.3
99.0%	2.577
98.5%	2.43
97.5%	2.243
95.0%	1.96
90.0%	1.645
85.0%	1.439
75.0%	1.151

Training database

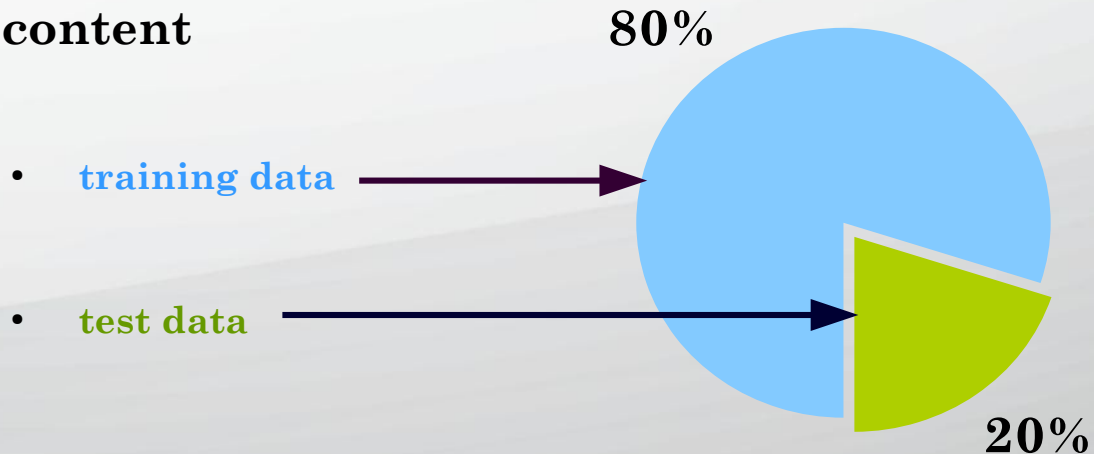
choose the training database based on:

- direct experience
 - pairs (in, out) that are useful for the objective function
 - eg. checkers game → board game annotated by correct or incorrect move
- indirect experience
 - useful feedback (unlike i/o pairs) for the objective function
 - eg. checkers game → sequences of moves and the final score of the game

data sources

- random generated examples
 - positive and negative examples
- positive examples collected by a learner
- real examples

content



characteristics

- independent data
 - otherwise → collective learning
- training and testing data must respect the same distribution law
 - otherwise → *transfer learning/inductive transfer*
 - vehicle recognition → truck recognition
 - text analyses
 - spam filters

Training database

characteristics extracted (attributes) from raw data

- quantitative characteristics → nominal or rational scale
 - continuous values → weight
 - discrete values → # of computers
 - range values → event times
- qualitative characteristics
 - nominal → colour
 - ordinal → sound intensity (low, medium, high)
- structured
 - trees – root is a generalisation of children (vehicle → car, bus, tractor, truck)

data transformation

- standardisation → numerical attributes
 - remove the scale effect (different scale and units)
 - raw values are transformed in z scores
 - $Z_{ij} = (x_{ij} - \mu_j) / \sigma_j$, where x_{ij} – value of j^{th} attribute of i^{th} instance, μ_j (σ_j) is the mean (standard deviation) of j^{th} attribute for all instances
- selection of some attributes

ML classification – goal oriented

Intelligent systems for prediction

Aim: predict the output for a new input based on a previously learned model

Eg. predicting sales of a product for a time in the future based on price, calendar month, region, average income

Intelligent systems for regression

Aim: estimation of the (uni or multi variable) function's shape based on a previously learned model

Eg.: estimate the function that models the edge of a surface

Intelligent systems for classification

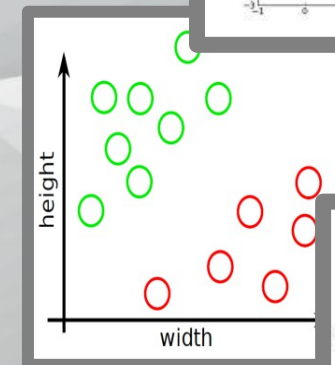
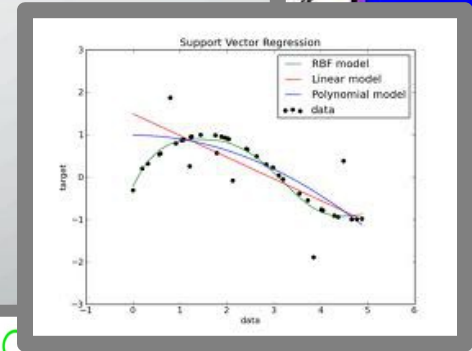
Aim: classify an object into one or more – known or unknown - categories based on their characteristics

Eg.: diagnostic systems for cancer: malign or benign or normal

Intelligent systems for planning

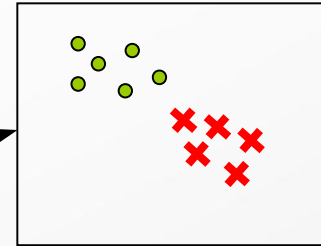
Aim: generate a sequence of optimal actions for performing a task

Eg.: planning the moves of a robot from a position to a source of energy



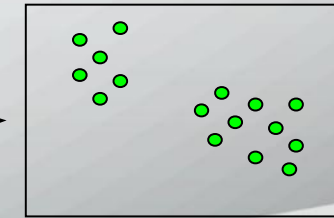
ML classification – based on learning experience

- Intelligent systems with supervised learning



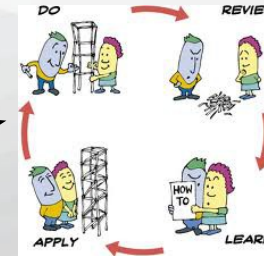
data is labeled

- Intelligent systems with unsupervised learning

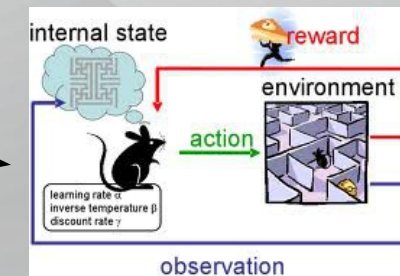


data is not labeled

- Intelligent systems with active learning



- Intelligent systems with reinforcement learning



Supervised learning

Aim

to provide a correct output for a new input

Definition

for a given data set (examples, instances, cases)

- training data – pairs $(attribute_data_i, output_i)$, where
 - $i = 1, N$ (N = number of training data pairs)
 - $attribute_data_i = (atr_{i1}, atr_{i2}, ..., atr_{im})$, m – number of attributes (characteristics, properties) for one data
 - $output_i$
 - a category from a predefined given set with k elements (number of classes) $\rightarrow$ classification problem
 - a real number $\rightarrow$ regression problem
- teste data – a set $(attribute_data_i), i = 1, n$ (n = number of test pairs).

determine

- a function (unknown) that maps the input attributes to the output on the training data set
- the output (class/value) associated with the test data (new) using the detected function

Other names

classification (regression), inductive learning

Process $\rightarrow$ 2 steps

- **Training**

learning the classification model – using an algorithm

- **Testing**

test the model with new data (not seen in the training step)

Feature

DB is labeled (for learning and sometimes also for testing)

Suitable problem types

- regression, classification

Supervised learning

learning quality

We consider a performance measure for the algorithm that is evaluated during both steps, training and testing, separately.

ex. accuracy ($\text{Acc} = \text{number of samples correct classified} / \text{total number of samples}$)

Evaluation Methods

for a large dataset	for a small dataset	for a very small dataset
disjunctive sets for training and testing	cross validation with h equal sub sets of the dataset	leave one out cross validation

Difficulties: **over-fitting** — good performance on training data but very poor on testing data

Performance measures:

- statistical, efficiency, robustness, scalability, interpret-ability, compactness, ...

Statistical metrics

Classification Accuracy

- the ratio of number of correct predictions to the total number of input samples

$$Acc = \frac{TP + TN}{N}$$

– works well only if there are equal number of samples belonging to each class.

Precision

- the number of correct positive results divided by the number of positive results predicted by the classifier.

$$Prec = \frac{TP}{TP + FP}$$

Recall

- the number of correct positive results divided by the number of all relevant samples

$$Recall = \frac{TP}{TP + FN}$$

Confusion Matrix

describes the complete performance of the model.

		Reality	
		pozitiv class	negativ class
computed results	pozitiv class	True positiv (TP)	False positiv (FP)
	negativ class	False negative (FN)	True negative (TN)

F1 Score

- is the Harmonic Mean between precision and recall.
- the range for F1 Score is [0, 1]

$$F1 = 2 * \frac{1}{\frac{1}{Prec} + \frac{1}{Recall}}$$

- how precise your classifier is (how many instances it classifies correctly),
- how robust it is (it does not miss a significant number of instances).

Mean Squared Error

- is the average of the squares of the differences between the original values and the predicted values.

$$MeanSquaredError = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- it is easier to compute the gradient

Unsupervised learning

Aim

to detect a model or an internal util structure of the data

Definition

for a given data set (examples, instances, cases)

- training data – a set $\{attribute_data_i \mid i=1, N\}$ where
 - N = number of training data pairs
 - $attribute_data_i = (atr_{i1}, atr_{i2}, \dots, atr_{im})$, m – number of attributes (characteristics, properties) for one data
- teste data – a set $(attribute_data_i), i=1, n$ (n = number of test pairs).

determine

- an unknown function that groups the training data in several classes
 - the number of classes k can be predefined or unknown
 - the data from one class are similar
- the output is the class the new input data (new) using the detected function

Examples of distances

consider two points p and q from R^m

- Euclidean $d(p, q) = \sqrt{\sum_{j=1}^m (p_j - q_j)^2}$
- Manhattan $d(p, q) = \sum_{j=1}^m |p_j - q_j|$
- Mahalanobis – measures the distance between a point p and a distribution Q
- Internal product $d(p, q) = \sum_{j=1}^m p_j q_j$
- Cosine $d(p, q) = \sum_{j=1}^m p_j q_j / (\sqrt{\sum_{j=1}^m p_j^2} \sqrt{\sum_{j=1}^m q_j^2})$
- Hamming – number of differences between p and q
- Levenshtein – the minimum number of transformations necessary to change p in q

Distance vs. Similarity

Distance $\rightarrow$ min

Similarity $\rightarrow$ max

Unsupervised learning

Other names

- clustering

Process → 2 steps

- Training
 - learning (determine), using an algorithm, the existing clusters
- Testing
 - test the model with new data (not seen in the training step)

Feature

- DB is not labeled (for learning and sometimes also for testing)

Suitable problem types

- Identifying some classes (clusters)
 - Genome analysis
 - Image processing
 - Social network analysis
 - Market segmentation
 - Astronomic data analysis
 - Computer clustering
- Dimensional reduction
- Identifying data properties (causes, explanations, etc.)
- Modeling data density

Learning quality

Internal criteria – high similarity within a cluster and reduced one between clusters

- distance inside the cluster
- distance between the clusters
- Davies-Bouldin index
- Dunn Index
- External criteria – using benchmarks made from already grouped data sets
 - Comparison with known data – almost impossible in practice
 - Precision
 - Recall
 - F1-measure

Active learning

The algorithm can receive supplementary information during the learning step in order to improve its performance.

Example: what is the subset of the training data that will facilitate the learning process.

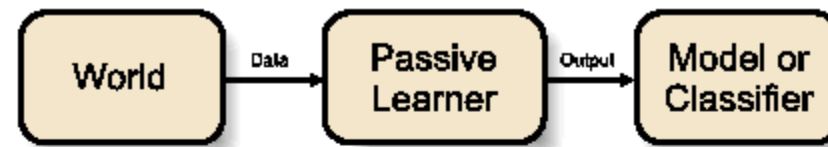


Figure 1.1: General schema for a passive learner.



Figure 1.2: General schema for an active learner.

Reinforcement learning

Aim

Learning, over a period of time, a course of action (behavior) that maximizes long-term rewards (earnings)

Characteristic

Interaction with the environment (actions → rewards)

Decision sequence

Problems' type

Ex. Training a Dog (Good and Bad Dog)

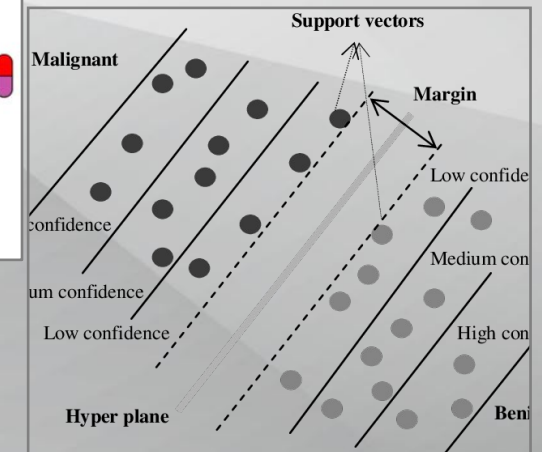
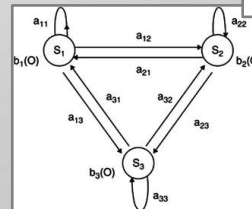
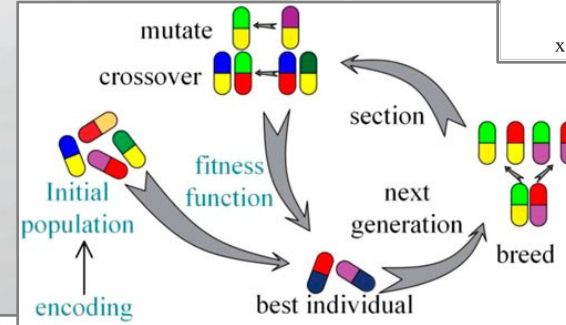
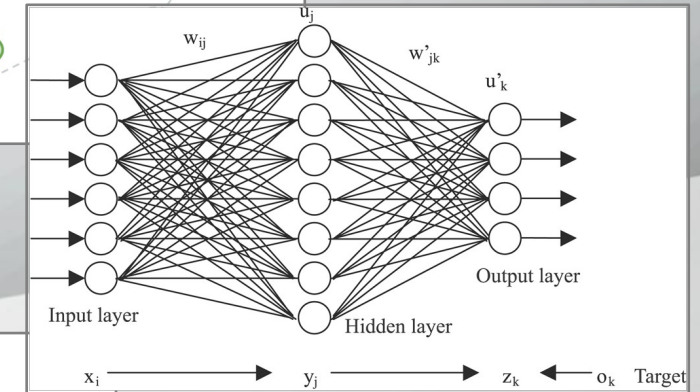
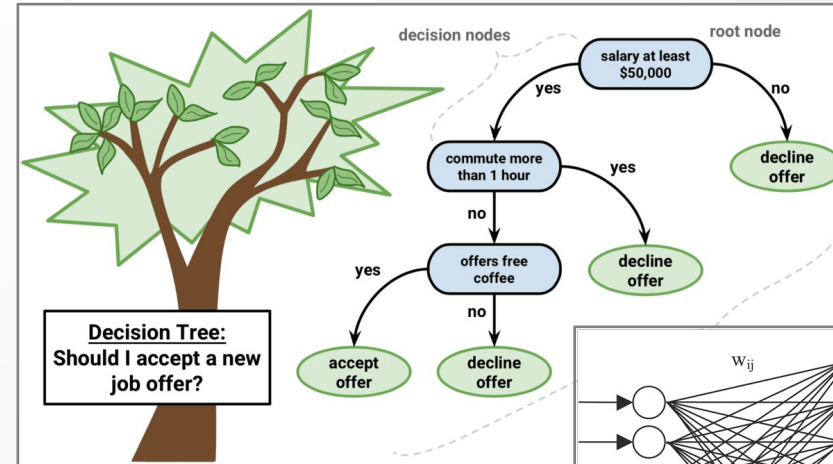
Supervised learning

Decision → consequence (malignant or benign cancer)

Intelligent Systems

Based on algorithm

- **Decision trees**
- **Artificial Neural Networks**
- **Evolutionary algorithms**
- **Support Vector Machines**
- **Hidden Markov Models**



Decision Trees (DT)

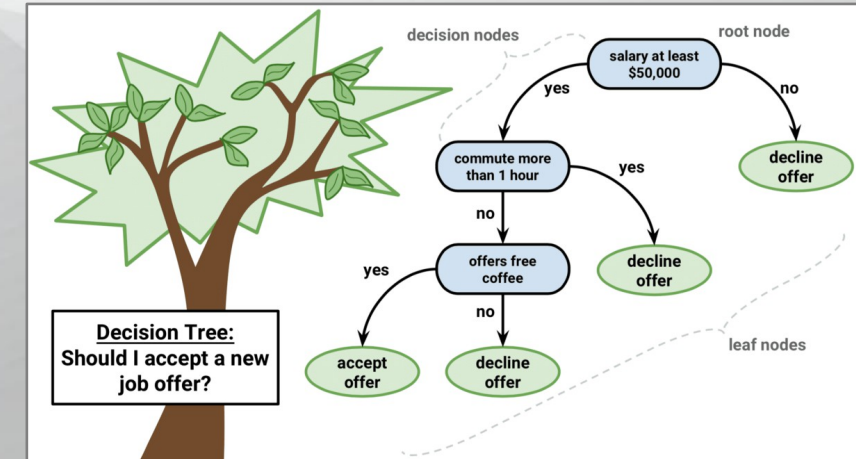
Aim

- divide a collection of articles in smaller sets by successively applying some decision rules → asking more questions
each question is addressed based on the answer of the previous question
- elements are characterized by non-metric information

Definition

- Decision tree – a special graph (bi-color and oriented tree)
 - Contains three node types:
 - Decision nodes → possibilities of decider (a test on an attribute of item that must be classified)
 - Hazard nodes → random events outside the control of decider (exam results, therapy consequences)
 - Result nodes → final states that have a utility or a label
 - Decision and hazard nodes alternate on the tree levels
 - Result nodes → leaf (terminal nodes)
 - (oriented) edges of the tree consequences of decisions (can be probabilistic)

- Each internal node corresponds to an attribute
- Each branch under a node (attribute) corresponds to the value of that attribute
- Each leaf corresponds to a class



Problems solved with DT

Properties

- problem's instances are represented by a fixed number of attributes, each attribute having a finite number of values;
- objective function takes discrete values;
- DT represents a dis-junction of more conjunctions, each conjunction being “attribute a_i has value v_j ”;
- training data could contain errors;
- training data could be incomplete;
 - Some data have not all attributes.

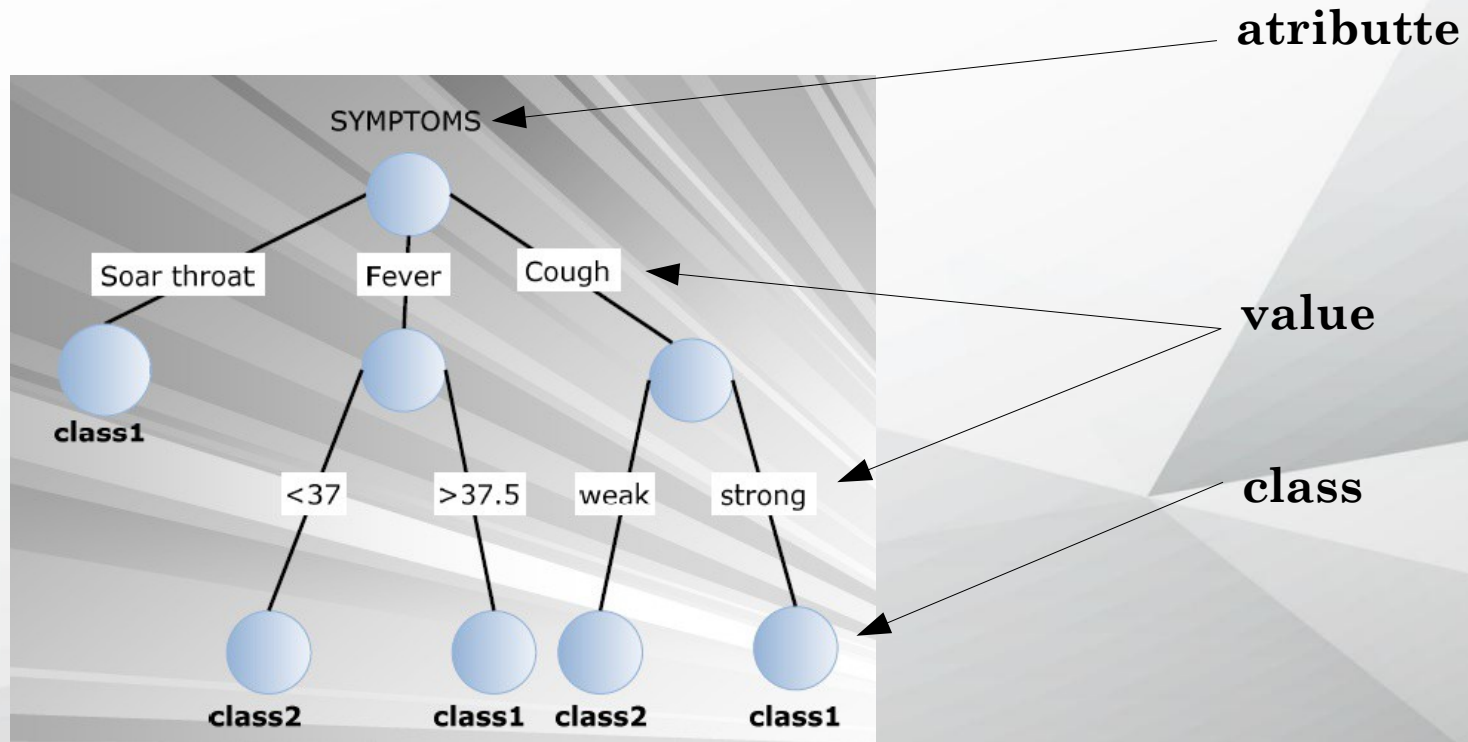
Classification problem

- Binary classifications
 - Instances are $[(attribute_{ij}, value_{ij}), class_i]$, $i=1,2,\dots,n$, $j=1,2,\dots,m$, and $class_i$ taking 2 values;
- Multi-class (k-class)
 - Instances are $[(attribute_{ij}, value_{ij}), class_i]$, $i=1,2,\dots,n$, $j=1,2,\dots,m$, and $class_i$ taking k values;

Regression problems

- instead to label each node by the label of a class, each node has associated a real value or a function that depends on the inputs of that node;
- input space is split in decision regions by parallel cuttings to Ox and Oy ;
- discrete outputs are transformed in continuous functions;
- quality of problem solving:
 - Prediction error (square or absolute)

Examples of DT



Example of train data for DT

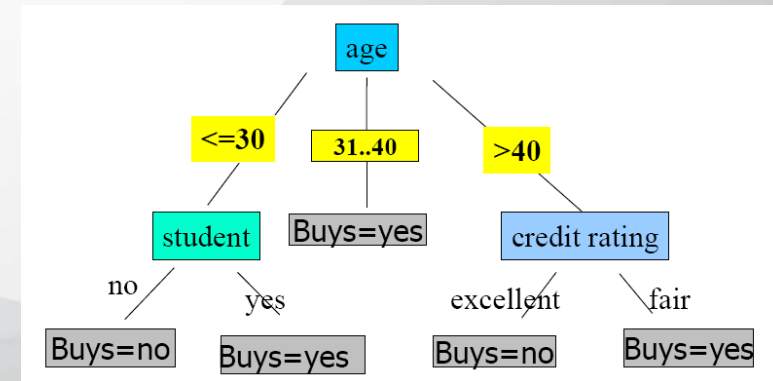
Credits

Approved or not

ID	Age	Has_Job	Own_House	Credit_Rating	Class
1	young	false	false	fair	No
2	young	false	false	good	No
3	young	true	false	good	Yes
4	young	true	true	fair	Yes
5	young	false	false	fair	No
6	middle	false	false	fair	No
7	middle	false	false	good	No
8	middle	true	true	good	Yes
9	middle	false	true	excellent	Yes
10	middle	false	true	excellent	Yes
11	old	false	true	excellent	Yes
12	old	false	true	good	Yes
13	old	true	false	good	Yes
14	old	true	false	excellent	Yes
15	old	false	false	fair	No

Examples of DT

rec	Age	Income	Student	Credit_rating	Buys_computer(CLASS)
r1	<=30	High	No	Fair	No
r2	<=30	High	No	Excellent	No
r3	31...40	High	No	Fair	Yes
r4	>40	Medium	No	Fair	Yes
r5	>40	Low	Yes	Fair	Yes
r6	>40	Low	Yes	Excellent	No
r7	31...40	Low	Yes	Excellent	Yes
r8	<=30	Medium	No	Fair	No
r9	<=30	Low	Yes	Fair	Yes
r10	>40	Medium	Yes	Fair	Yes
r11	<=30	Medium	Yes	Excellent	Yes
r12	31...40	Medium	No	Excellent	Yes
r13	31...40	High	Yes	Fair	Yes
r14	>40	Medium	No	Excellent	No



Process – DT

Tree construction (induction)

Based on training data

Works bottom-up or top-down (splitting)

Using the tree as a problem solver

All decisions performed along a path from the root to a leaf form a rule

Rules from DT are used for labeling new data

Pruning

Identify and move/eliminate branches that reflect noise or exceptions

Tree construction

Split the training data into subsets based on the characteristics of data

a node – question related to a property

branches of a node – possible answers to the question of the node

initially, all examples are located in the root

- an attribute gives the root label and its values give the branches

on next levels, examples are partitioned based on their attributes → order of attributes

- for each node, an attribute is (recursively) chosen – its values → branches

splitting – greedy decision making

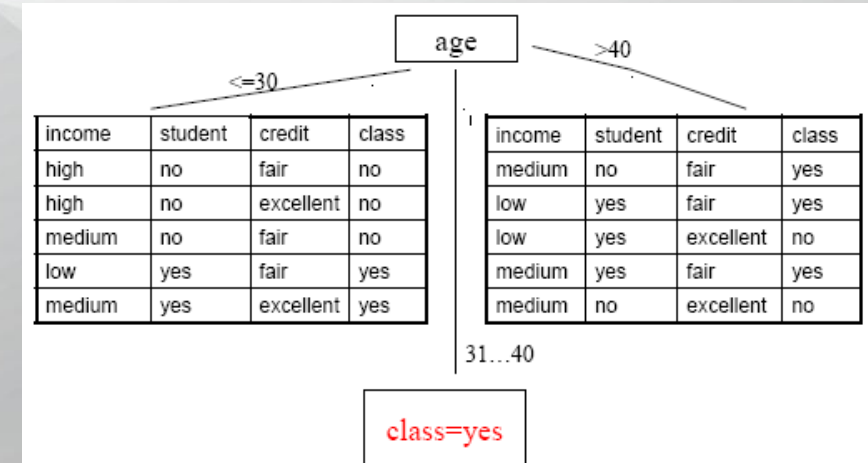
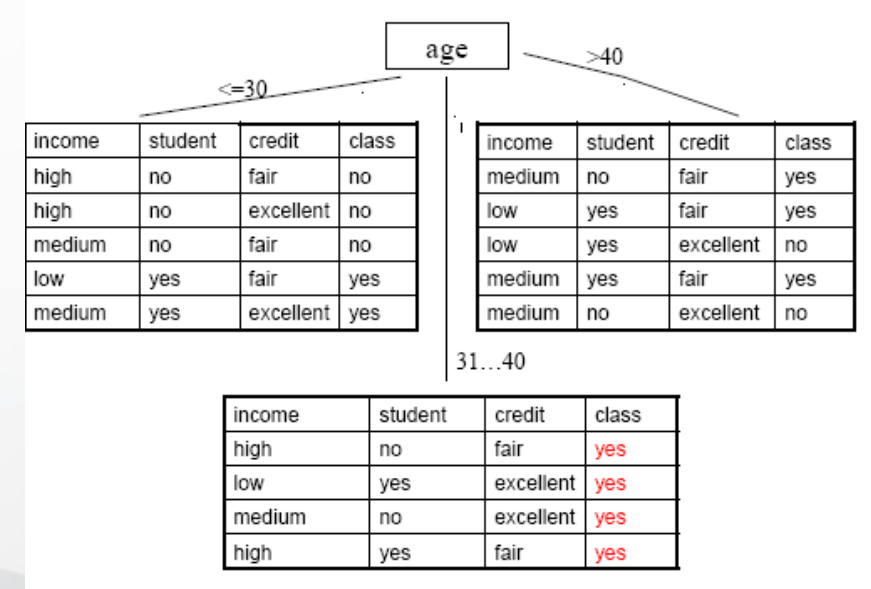
Iterative process

stop conditions:

- all examples from a node belong to the same class → node is a leaf and is labeled by $class_i$
- there are no left examples → node becomes a leaf and is labeled by the majority class of training data
- there are no attributes

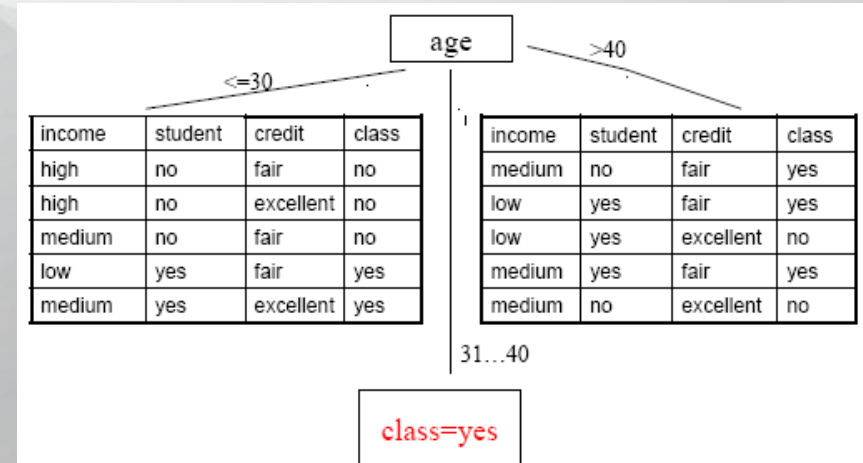
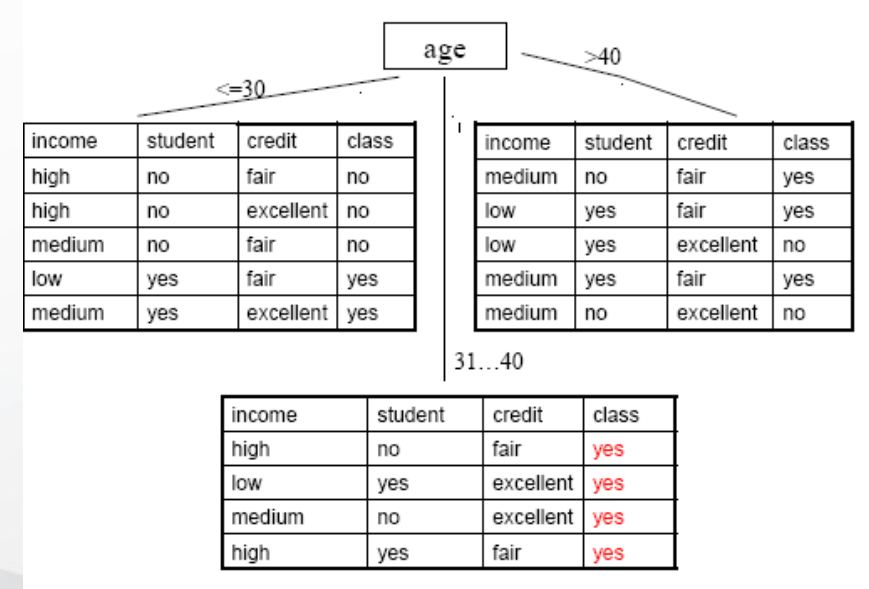
Example – tree construction

rec	Age	Income	Student	Credit_rating	Buys_computer(CLASS)
r1	<=30	High	No	Fair	No
r2	<=30	High	No	Excellent	No
r3	31...40	High	No	Fair	Yes
r4	>40	Medium	No	Fair	Yes
r5	>40	Low	Yes	Fair	Yes
r6	>40	Low	Yes	Excellent	No
r7	31...40	Low	Yes	Excellent	Yes
r8	<=30	Medium	No	Fair	No
r9	<=30	Low	Yes	Fair	Yes
r10	>40	Medium	Yes	Fair	Yes
r11	<=30	Medium	Yes	Excellent	Yes
r12	31...40	Medium	No	Excellent	Yes
r13	31...40	High	Yes	Fair	Yes
r14	>40	Medium	No	Excellent	No



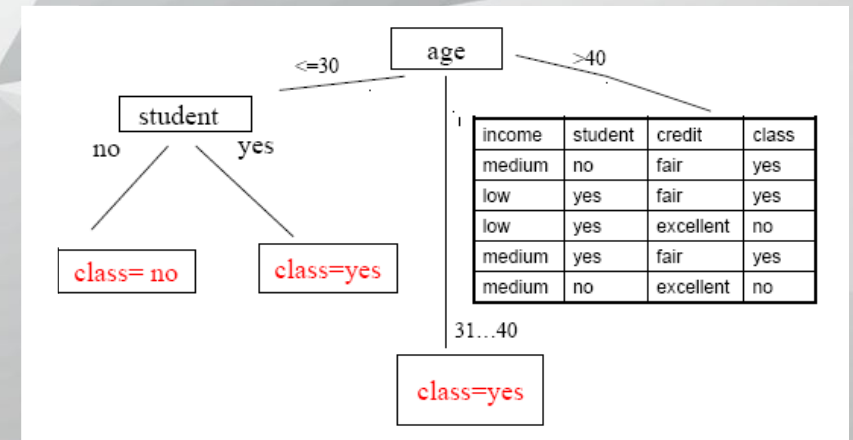
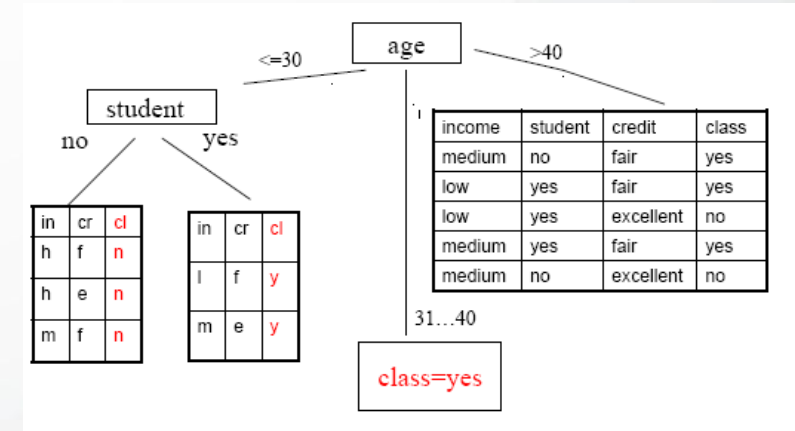
Example – tree construction

rec	Age	Income	Student	Credit_rating	Buys_computer(CLASS)
r1	<=30	High	No	Fair	No
r2	<=30	High	No	Excellent	No
r3	31...40	High	No	Fair	Yes
r4	>40	Medium	No	Fair	Yes
r5	>40	Low	Yes	Fair	Yes
r6	>40	Low	Yes	Excellent	No
r7	31...40	Low	Yes	Excellent	Yes
r8	<=30	Medium	No	Fair	No
r9	<=30	Low	Yes	Fair	Yes
r10	>40	Medium	Yes	Fair	Yes
r11	<=30	Medium	Yes	Excellent	Yes
r12	31...40	Medium	No	Excellent	Yes
r13	31...40	High	Yes	Fair	Yes
r14	>40	Medium	No	Excellent	No



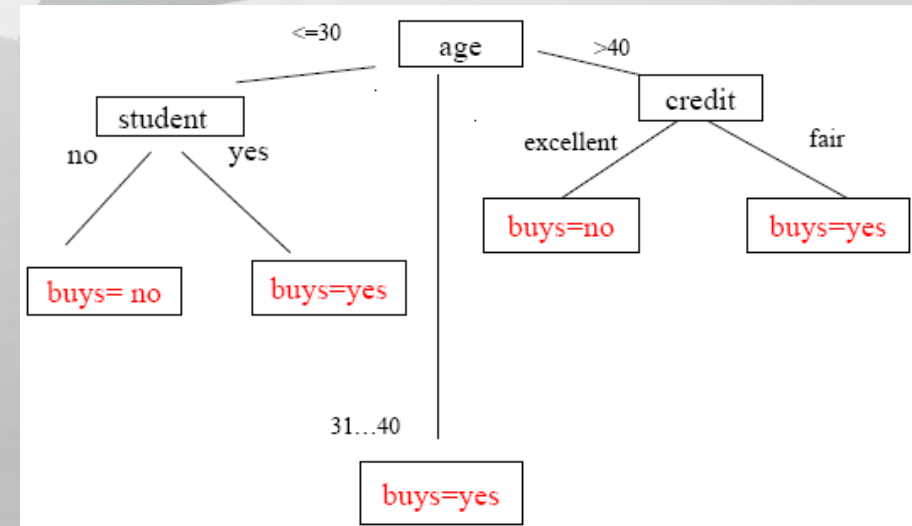
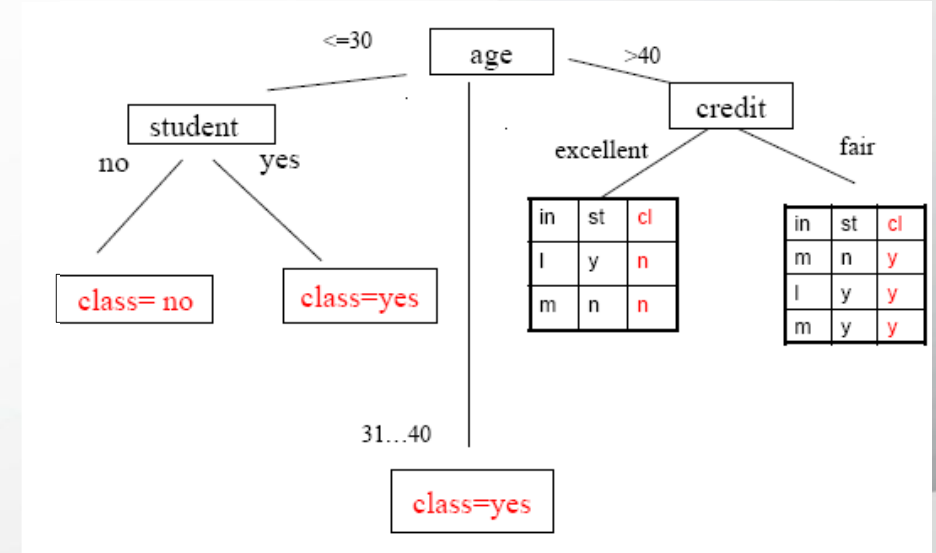
Example – tree construction

rec	Age	Income	Student	Credit_rating	Buys_computer(CLASS)
r1	<=30	High	No	Fair	No
r2	<=30	High	No	Excellent	No
r3	31...40	High	No	Fair	Yes
r4	>40	Medium	No	Fair	Yes
r5	>40	Low	Yes	Fair	Yes
r6	>40	Low	Yes	Excellent	No
r7	31...40	Low	Yes	Excellent	Yes
r8	<=30	Medium	No	Fair	No
r9	<=30	Low	Yes	Fair	Yes
r10	>40	Medium	Yes	Fair	Yes
r11	<=30	Medium	Yes	Excellent	Yes
r12	31...40	Medium	No	Excellent	Yes
r13	31...40	High	Yes	Fair	Yes
r14	>40	Medium	No	Excellent	No



Example – tree construction

rec	Age	Income	Student	Credit_rating	Buys_computer(CLASS)
r1	<=30	High	No	Fair	No
r2	<=30	High	No	Excellent	No
r3	31...40	High	No	Fair	Yes
r4	>40	Medium	No	Fair	Yes
r5	>40	Low	Yes	Fair	Yes
r6	>40	Low	Yes	Excellent	No
r7	31...40	Low	Yes	Excellent	Yes
r8	<=30	Medium	No	Fair	No
r9	<=30	Low	Yes	Fair	Yes
r10	>40	Medium	Yes	Fair	Yes
r11	<=30	Medium	Yes	Excellent	Yes
r12	31...40	Medium	No	Excellent	Yes
r13	31...40	High	Yes	Fair	Yes
r14	>40	Medium	No	Excellent	No



ID3/C4.5 algorithm

```
generate(D, A){ //D – a partitioning of training data, A – list of attributes
  create a new node N
  if examples from D belong to a single class C then
    node N becomes a leaf and is labeled by C
    return node N
  else
    if A=∅ then
      node N becomes a leaf and is labeled by majority class of D
      return node N
    else
      separation_attribute = AttributeSelection(D, A)
      label node N by separation_attribute
      for all possible values vj of separation_attribute
        let Dj – set of examples from D that have separation_attribute=vj
        if Dj = ∅ then
          add a leaf (to node N) labeled by majority class of D
        else
          add a node (to node N) return by generate(Dj, A–separation_attribute)
      return node N
}
```

Greedy, recursive, top-down, divide-and-conquer

AttributeSelection(D,A)

selects the attribute that corresponds to a node (root or internal node)

- Random
- Attribute with the fewest/most values
- Based on a pr-established order:
- Information gain
 - Gain rate
 - Distance between partitions created by the attribute

Information gain

An impurity measure

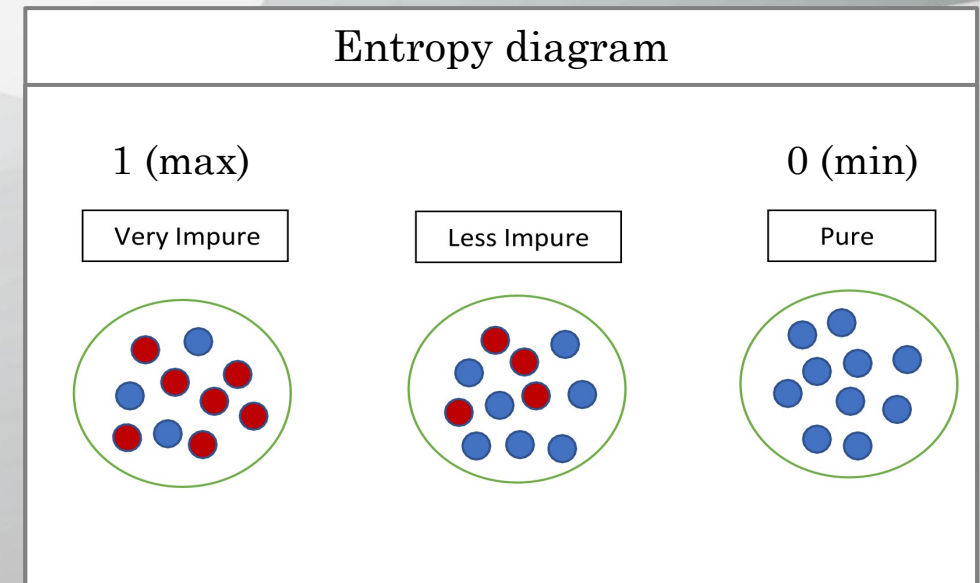
- 0 (minim) – if all examples belong to the same class
- 1 (maxim) – if examples are uniform distributed over classes

Based on data entropy

- Expected number of bits required by coding the class of an element from data
- Binary classification (2 classes): $E(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$, where
 - p_+ - proportion of positive examples in dataset S
 - p_- - proportion of negative examples in dataset S
- Multi-class classification: $E(S) = \sum_{i=1, 2, \dots, k} -p_i \log_2 p_i$
 - data entropy related to target attribute (output attribute), where
 - p_i – proportion of examples from class i in dataset S

Information gain of an attribute

- How the elimination of attribute a reduces the dataset's entropy
 - $Gain(S, a) = E(S) - \sum_{v \in \text{values}(a)} |S_v| / |S| E(S_v)$
 - $\sum_{v \in \text{values}(a)} |S_v| / |S| E(S_v)$ – expected information



Example – Information gain

	a1	a2	a3	Class
d1	big	red	circle	class 1
d2	small	red	square	class 2
d3	small	red	circle	class 1
d4	big	blue	circle	class 2

Formula for information gain

$$Gain(S, a) = E(S) - \sum_{v \in \text{values}(a)} \frac{|S_v|}{|S|} E(S_v)$$

$$S = \{d_1, d_2, d_3, d_4\} \Rightarrow p_+ = \frac{2}{4}, p_- = \frac{2}{4} \Rightarrow E(S) = -p_+ \log_2 p_+ - p_- \log_2 p_- = 1$$

$$S_{v=\text{big}} = \{d_1, d_4\} \Rightarrow p_+ = \frac{1}{2}, p_- = \frac{1}{2} \Rightarrow E(S_{v=\text{big}}) = 1$$

$$S_{v=\text{small}} = \{d_2, d_3\} \Rightarrow p_+ = \frac{1}{2}, p_- = \frac{1}{2} \Rightarrow E(S_{v=\text{small}}) = 1$$

$$S_{v=\text{red}} = \{d_1, d_2, d_3\} \Rightarrow p_+ = \frac{2}{3}, p_{\text{minus}} = \frac{1}{3} \Rightarrow E(S_{v=\text{red}}) = 0.923$$

$$S_{v=\text{blue}} = \{d_4\} \Rightarrow p_+ = 0, p_- = 1 \Rightarrow E(S_{v=\text{blue}}) = 0$$

$$S_{v=\text{circle}} = \{d_1, d_3, d_4\} \Rightarrow p_+ = \frac{2}{3}, p_{\text{minus}} = \frac{1}{3} \Rightarrow E(S_{v=\text{circle}}) = 0.923$$

$$S_{v=\text{square}} = \{d_2\} \Rightarrow p_+ = 0, p_- = 1 \Rightarrow E(S_{v=\text{square}}) = 0$$

$$\longrightarrow Gain(S, a_1) = 1 - \left(\frac{|S_{v=\text{big}}|}{|S|} E(S_{v=\text{big}}) + \frac{|S_{v=\text{small}}|}{|S|} E(S_{v=\text{small}}) \right) = 1 - \left(\frac{2}{4} 1 + \frac{2}{4} 1 \right) = 0$$

$$\longrightarrow Gain(S, a_2) = 1 - \left(\frac{|S_{v=\text{red}}|}{|S|} E(S_{v=\text{red}}) + \frac{|S_{v=\text{blue}}|}{|S|} E(S_{v=\text{blue}}) \right) = 1 - \left(\frac{3}{4} 0.923 + \frac{1}{4} 0 \right) = 0.307 \quad \star$$

$$\longrightarrow Gain(S, a_3) = 1 - \left(\frac{|S_{v=\text{circle}}|}{|S|} E(S_{v=\text{circle}}) + \frac{|S_{v=\text{square}}|}{|S|} E(S_{v=\text{square}}) \right) = 1 - \left(\frac{3}{4} 0.923 + \frac{1}{4} 0 \right) = 0.307 \quad \star$$

Gain rate

Penalises an attribute by integrating a new term that depends on spreading degree and on uniformity degree of separation – *Split information*

Split information

- entropy related to possible values of attribute a
- describes the potential worth of splitting a branch from a node

$$SplitInformation(S, a) = - \sum_{v=value(a)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}$$

where S_v is the proportion of examples from dataset S that have attribute a with value v

Information gain ratio is the ratio between the *information gain* and the *split information* value:

$$IGT(S, a) = \frac{gain(S, a)}{SplitInformation(S, a)}$$

aims to reduce a bias towards multi-valued attributes!

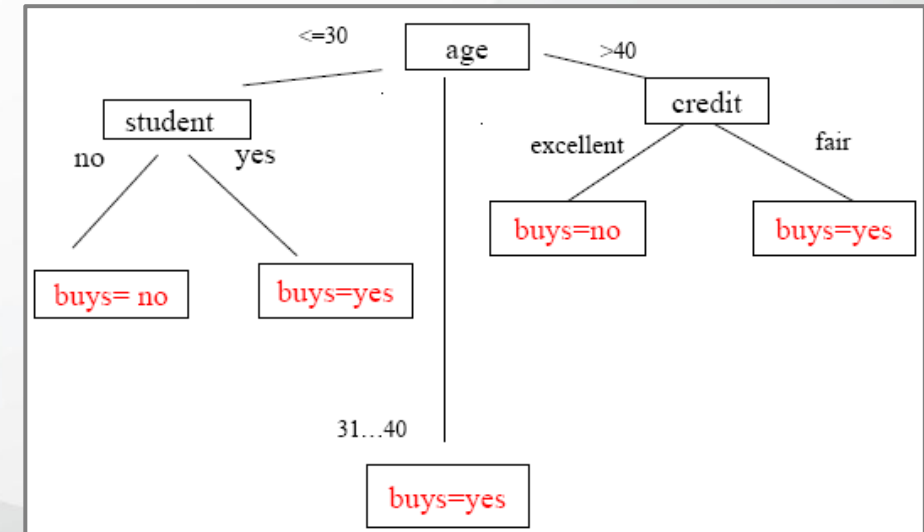
Using a tree as a solver

Extract the rules from the constructed tree

- IF $age = "<=30"$ AND $student = "no"$ THEN $buys_computer = "no"$
- IF $age = "<=30"$ AND $student = "yes"$ THEN $buys_computer = "yes"$
- IF $age = "31...40"$ THEN $buys_computer = "yes"$
- IF $age = ">40"$ AND $credit_rating = "excellent"$ THEN $buys_computer = "no"$
- IF $age = ">40"$ AND $credit_rating = "fair"$ THEN $buys_computer = "yes"$

Use the rules for classifying the test data (new data)

- for x (a data without class) – rules can be written as predicates
- IF $age(x, <=30)$ AND $student(x, no)$ THEN $buys_computer(x, no)$
- IF $age(x, <=30)$ AND $student(x, yes)$ THEN $buys_computer(x, yes)$



Difficulties

- *Underfitting* – DT constructed on training data is too simple → large classification error during training and testing
- *Overfitting* – DT constructed on training data match the training data, but it cannot generalize new data

Solutions

- Pruning – remove some branches (not useful, redundant) → smaller tree
- Cross-validation

Pruning a tree

pre-pruning

Increasing the tree is stopped during construction by stopping the division of nodes that become leaf labeled by majority class of examples from that node

post-pruning

After the DT is constructed, eliminate the branches of some nodes that become leaf $\rightarrow$ classification error reduces (on testing data)

Motivation – simplifying the tree

- After the DT is constructed, classification rules are extracted in order to represent the knowledge as *if-then* rules (easy to understand)
- A rule is create by traversing the DT from root to a leaf
- Each pair $(attribute, value) \leftrightarrow (node, edge)$ – is a conjunction in the premise of the rule (*if* part), except the last node of the path that is a leaf and represents the consequence (output, *then* part) of the rule

Conclusion

Advantages

- Easy to understand and interpret
- Can use nominal or categorized data
- Decision logic can be easy followed (rules are visible)
- Works better with large data

Disadvantages

- Instability due some change the training data
- Complexity due the representation
- Difficult to use
- The DT construction is expensive
- The DT construction requires a lot of information

Tool example:

WEKA J48



THANK YOU !

Artificial Neural Networks

Artificial Intelligence

Intelligent Systems - ANNs

Aim example

binary classification for any input data
(discrete or continuous)

- data can be separated by:
 - ♦ a line $\rightarrow ax + by + c = 0$ (if $m = 2$)
 - ♦ a plan $\rightarrow ax + by + cz + d = 0$ (if $m = 3$)
 - ♦ a hyperplan $\rightarrow \sum a_i x_i + b = 0$ (if $m > 3$)
- How do we identify the optimal values of a, b, c, d, a_i ?
 - ♦ Artificial Neural Networks (ANNs)
 - ♦ Support Vector Machines (SVMs)

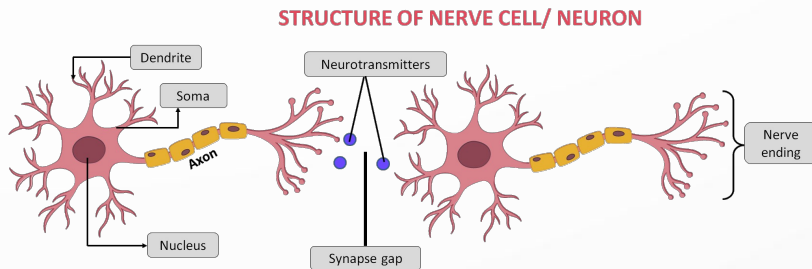
Why ANN?

some tasks can be easily done by humans, but they are difficult to be encoded as algorithms

- Shape recognition
 - Old friends
 - Handwritten
 - Voice
- Rational processes
 - Car driving
 - Piano playing
 - Basketball playing
 - Swimming

such tasks are too difficult to be formalized and done by a rational process

The learning process (brain)



Human brain

- ~10.000.000.000 of neurons connected through synapses

A neuron

- has a body (soma), an axon, and more dendrites
- can be in a given state
 - Active state – if the input information is over a given stimulation threshold
 - Passive state – otherwise

Synapse

- link between the axon of a neuron and the dendrites of other neurons
- take part to information exchange between neurons
- 5.000 connections/neuron (average)
- during a life, new connections can appear

Brain → *Neural network*

- complex system, non-linear and parallel that processes information
- information is stored and processed by the entire network, not only by a part of network

Models for information processing:

- learning
- storing
- memorizing

Learning

- a basic characteristic
- useful connections become permanent (others are eliminated)

Memory

Short time memory

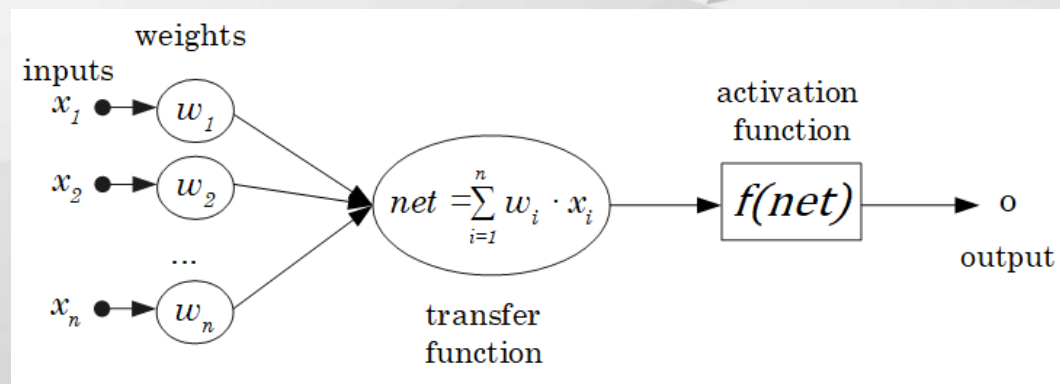
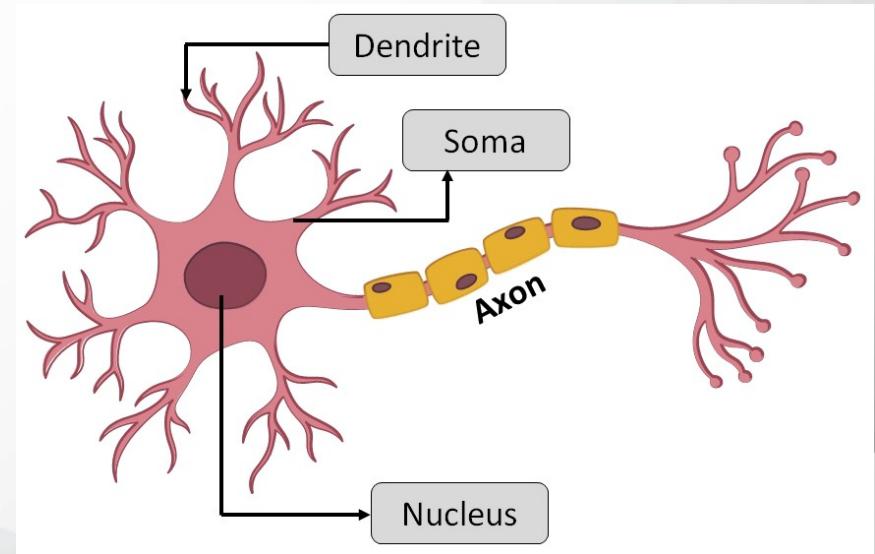
- immediately → 30 sec.
- working memory

Long term memory

- capacity
- increasing along life
- limited → learning a poetry strophe by strophe
- influenced by emotional states

Biology versus artificial

BNN	ANN
Soma	Node
Dendrite	Input
Axon	Output
Activation	Processing
Synapse	Weighted connection



The perceptron

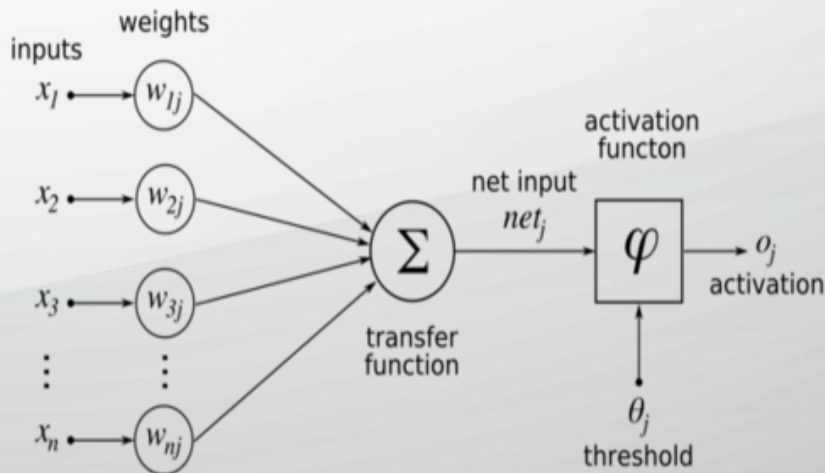
Learning Objectives

By the end of this section you should be able to answer these questions:

1. what is a perceptron?
2. what activation function has a perceptron?
3. what problems can (and can't) be solved by a perceptron?
4. how is trained a perceptron?

It is the first model of a neuron

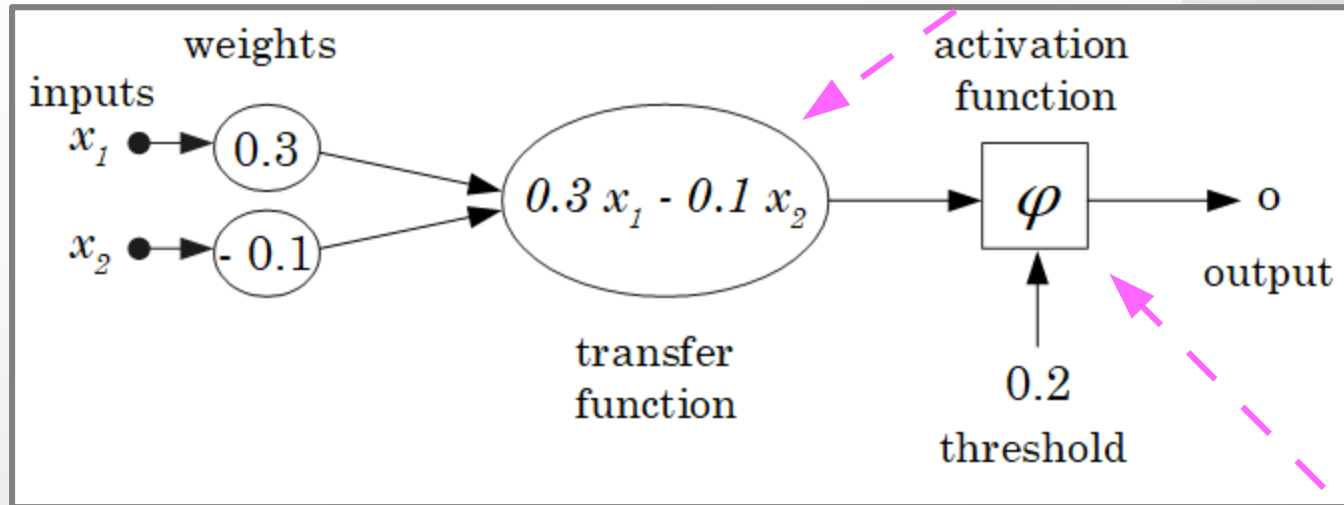
- invented by Frank Rosenblatt (1928 – 1971) an American psychologist
- the original perceptron was designed to take a number of binary inputs, and produce one binary output (0 or 1)
- uses different **weights** to represent the importance of each **input**, and that the sum of the values should be greater than a **threshold value** before making a **decision** like true or false (0 or 1)
- the original algorithm:
 1. Set a threshold value
 2. Multiply all inputs with its weights
 3. Sum all the results
 4. Activate the output



Perceptron – example

Consider the parameters: $w_1=0.3$ $w_2=-0.1$ $\theta=0.2$

$$net = \sum_{i=1}^n w_i * x_i$$



For the input: $\mathbf{x}=(x_1, x_2)=(1,0)$

We get the output: $\varphi(0.3 \times 1 - 0.1 \times 0) = \varphi(0.3) = 1$

Activation function

$$\varphi(net) = \begin{cases} 0, & \text{if } net < \theta \\ 1, & \text{if } net \geq \theta \end{cases}$$

threshold function

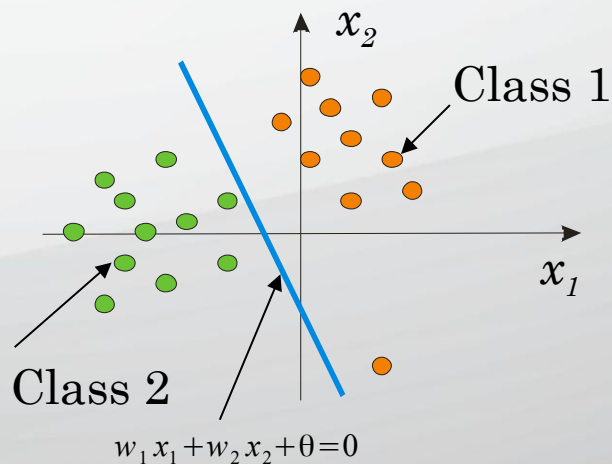
Solving capacity

Observe!

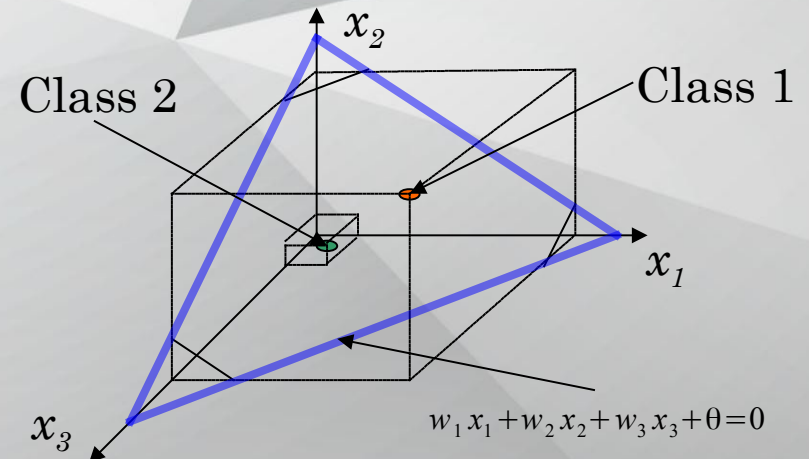
From the transfer function we get

$$y = \sum_{i=1}^n w_i x_i \quad - \text{equation of a **hyperplane**.$$

If we compose the transfer function with the threshold function (the activation) we **linearly separate** the space $\mathbf{R}^n$ with this hyperplane in two regions.



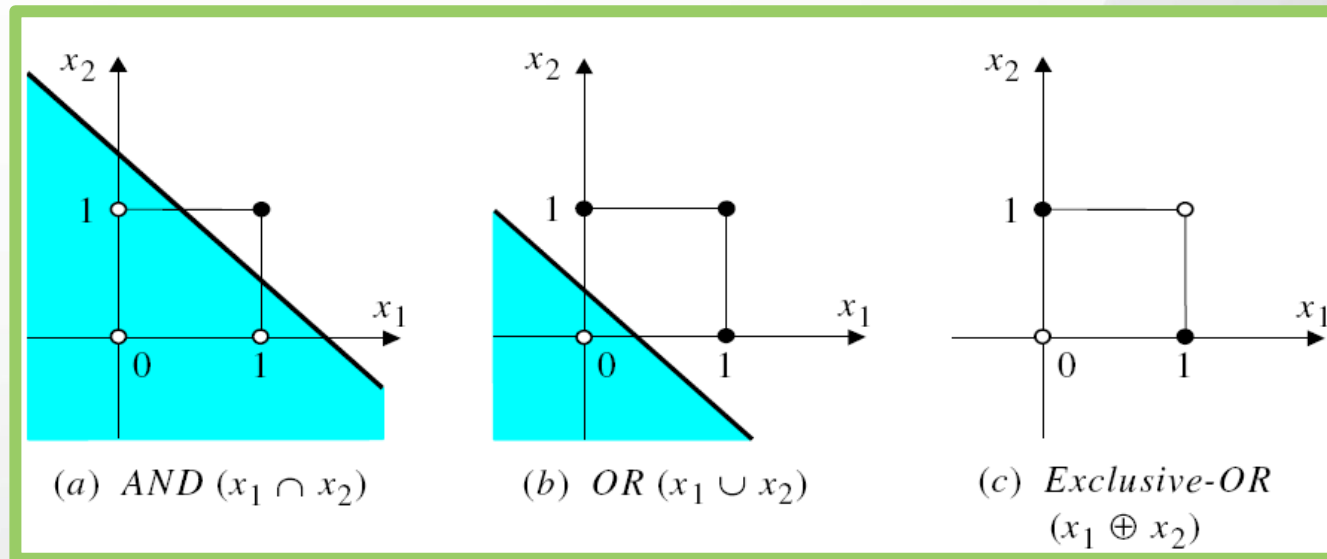
Binary classification with $m=2$ input attributes



Binary classification with $m=3$ input attributes

Perceptron – limits

Non-linear separable data can not be classified.



A perceptron can learn And and OR operations,
but it can not learn XOR operation (it is not linear separable)

Solutions

Neuron with continuous threshold
More neurons

Neuron training

Perceptron's rule → perceptron's algorithm

1. Start by some random weights
2. Determine the quality of the model create for these weights for a **single input data**
3. Re-compute the weights based on the model's quality
4. Repeat (from step 2) until a maximum quality is obtained

Delta's rule → algorithm of gradient descent

1. Start by some random weights
2. Determine the quality of the model create for these weights for **all input data**
3. Re-compute the weights based on the model's quality
4. Repeat (from step 2) until a maximum quality is obtained

Similar to perceptron's rule, but the model's quality is established based on all data (all training data).

The perceptron – learning

Aim:

To identify the optimal weights $(w_1, w_2, \dots, w_m)$ by minimising the errors.

Training data set of n data

$$\{((x_1^d, x_2^d, \dots, x_m^d), t^d) : d = 1, 2, \dots, n\}$$

with m – the number of attributes.

The error is the difference between the real output y and the output o computed by the perceptron for the input $(x_1, x_2, \dots, x_m)$.

Perceptron's algorithm:

Based on error minimisation associated to an instance of train data

Modify the weights based on error associated to an instance of train data

Perceptron's learning alg.

```
function training_perceptron( $\theta, \eta, m, n, \{((x_1^d, x_2^d, \dots, x_m^d), t^d) : d=1, 2, \dots, n\}$ )  
     $w_i = \text{random}(-1, 1)$ , where  $i=1, 2, \dots, m$  # initialise random the perceptron's weights  
  
    do repeat  
        for  $d = 1$  to  $n$  do  
            activate the neuron and determine the output  $o^d = \varphi\left(\sum_{i=1}^n w_i * x_i^d\right)$   
            compute the error  $e_d = t^d - o^d$   
            determine the weight modification  $\Delta w_i = \eta e_d x_i^d$   
            modify the weights  $w_i = w_i + \Delta w_i$   
  
    until there are no incorrect classified examples  
  
    return ( $w_1, w_2, \dots, w_m$ )  
  
end function
```


Solving *logic AND* problem

AND	0 (False)	1 (True)
0 (False)	0	0
1 (True)	0	1

threshold

$$\theta = 0.2$$

learning rate

$$\eta = 0.1$$

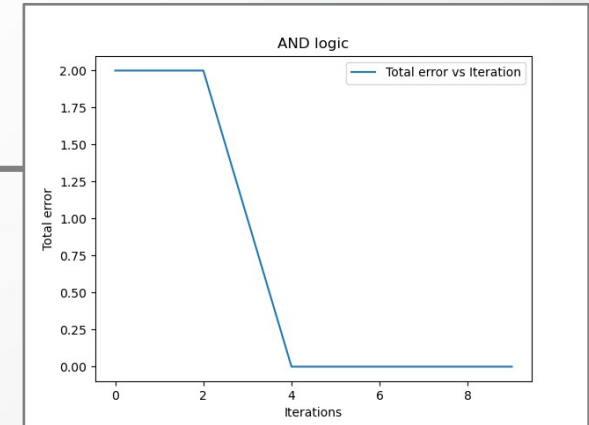
Epoch	Inputs		Desired output Y_d	Initial weights		Actual output Y	Error e	Final weights	
	x_1	x_2		w_1	w_2			w_1	w_2
1	0	0	0	0.3	-0.1	0	0	0.3	-0.1
	0	1	0	0.3	-0.1	0	0	0.3	-0.1
	1	0	0	0.3	-0.1	1	-1	0.2	-0.1
	1	1	1	0.2	-0.1	0	1	0.3	0.0
2	0	0	0	0.3	0.0	0	0	0.3	0.0
	0	1	0	0.3	0.0	0	0	0.3	0.0
	1	0	0	0.3	0.0	1	-1	0.2	0.0
	1	1	1	0.2	0.0	1	0	0.2	0.0
3	0	0	0	0.2	0.0	0	0	0.2	0.0
	0	1	0	0.2	0.0	0	0	0.2	0.0
	1	0	0	0.2	0.0	1	-1	0.1	0.0
	1	1	1	0.1	0.0	0	1	0.2	0.1
4	0	0	0	0.2	0.1	0	0	0.2	0.1
	0	1	0	0.2	0.1	0	0	0.2	0.1
	1	0	0	0.2	0.1	1	-1	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1
5	0	0	0	0.1	0.1	0	0	0.1	0.1
	0	1	0	0.1	0.1	0	0	0.1	0.1
	1	0	0	0.1	0.1	0	0	0.1	0.1
	1	1	1	0.1	0.1	1	0	0.1	0.1

Perceptron use

```
from numpy import random, dot, array
import matplotlib as mpl
```

```
def threshold(x):
    if x < 0.2 :
        return 0
    return 1
```

```
class Perceptron:
    def __init__(self, noInputs, activationFunction, learningRate):
        self.noInputs = noInputs
        self.weights = random.rand(self.noInputs)
        self.activationFunction = activationFunction
        self.output = 0
        self.learningRate = learningRate
        self.errorVariation = []
    def fire(self, inputs):
        self.output = self.activationFunction(
            inner(array(inputs), self.weights))
        return self.output
    def trainPerceptronRule(self, inputs, output):
        totalError = 0
        for i in range(len(inputs)):
            error = output[i] - self.fire(inputs[i])
            delta = self.learningRate*error*array(inputs[i])
            self.weights += delta
            totalError += error**2
        self.errorVariation.append(totalError)
```



```
def test1():
    # AND logic
    p = Perceptron(2, threshold, 0.1)
    x = [[1,1],[1,0],[0,1],[0,0]]
    t = [1,0,0,0]
    noIterations = 10
    for i in range(noIterations):
        p.trainPerceptronRule(x,t)
    print(p.weights)
    for j in range(len(x)):
        print(x[j], p.fire(x[j]))

    mpl.pyplot.plot(range(noIterations),
        p.errorVariation, label = 'Total error
        vs Iteration')

    mpl.pyplot.xlabel('Iterations')
    mpl.pyplot.ylabel('Total error')
    mpl.pyplot.legend()
    mpl.pyplot.title('AND logic')
    mpl.pyplot.show()
```

Artificial Neural Networks

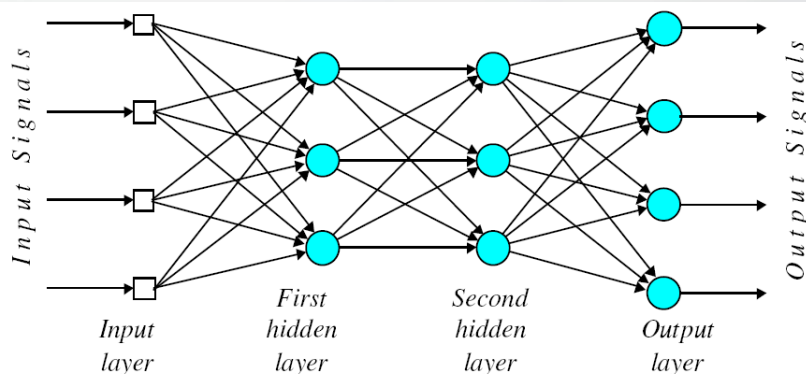
Learning Objectives

By the end of this section you should be able to answer these questions:

1. What is a *feed forward artificial neural network*?
2. How is organized an ANN?
3. What activation functions are used in ANNs?
4. How passes the signal through an ANN?
5. How to train an ANN?

A structure similar to a biological neural network

- 1943, neurophysiologist Warren McCulloch and mathematician Walter Pitts
- the original ANN was built with electrical circuits
- A set of nodes (units, neurons, processing elements) located in a graph with more layers



In a *feed forward network* the information moves in only one direction, **forward**, from the input nodes, through the hidden nodes (if any) to the output nodes.

– is the most simple type of ANN.

Artificial Neural Networks

Nodes

- Have inputs and outputs
- Perform a simple computing through an activation function
- Connected by weighted links
 - Links between nodes give the network structure
 - Links influence the performed computations

Layers

- Input layer
 - Contains m nodes (m – the number of attributes of a data)
- Output layer
 - Contains r nodes (r – the number of outputs)
- Intermediate layers
 - Different structures
 - Different sizes

Example of a feed forward network structure:

6:4:10:2 → input layer with 6 units (artificial neurons), two hidden layers with 4 and 10 units respectively and an output layer with 2 units.

this structure can be used for a problem with 6 attributes and 2 outputs

each node from the first layer has one input and one output, each unit from the first hidden layer has 6 weights and one output, each unit from the second hidden layer has 4 weights and one output, each node from the output layer has 10 weights and one output.

Artificial Neural Networks

Learning process

A training data set of n data

$$\{((x_1^d, x_2^d, \dots, x_m^d), (t_1^d, t_2^d, \dots, t_r^d)): d = 1, 2, \dots, n\}$$

where m – number of attributes and r – number of outputs

Form an ANN with m input nodes, r output nodes and an internal structure

- Some hidden layers, each layer having a given number of nodes
- With weighted connections between every 2 nodes of consecutive layers

Determine the optimal weights by minimising the error

- Difference between the real output y and the output computed by the network

Problems solved by an ANN

- Problem data can be represented by pairs (attribute-value)
- Objective function can be:
 - Single or multi-criteria
 - Discrete or continuous (real values)
- Training data can be noised
- A large training time

Neuron processing

Information is transmitted to the neuron → compute the weighted sum of inputs

$$net = \sum_{i=1}^n w_i * x_i$$

Neuron processes the information → by using an activation function

$$o = f(net)$$

- Constant function
- Step function
- Slope function
- Linear function
- Sigmoid function
- Gaussian function

Activation function

Constant function

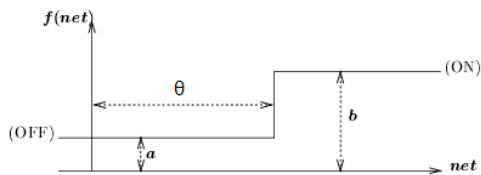
$$f(\text{net}) = \text{const.}$$



Step function

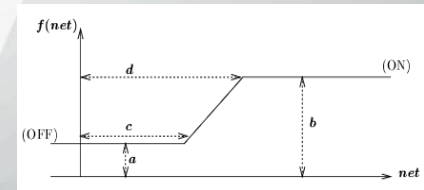
θ - threshold

$$f(\text{net}) = \begin{cases} a, & \text{if } \text{net} < \theta \\ b, & \text{if } \text{net} \geq \theta \end{cases}$$



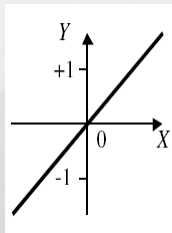
Slope function

$$f(\text{net}) = \begin{cases} a, & \text{if } \text{net} \leq c \\ b, & \text{if } \text{net} \geq d \\ a + \frac{(\text{net} - c)(b - a)}{d - c}, & \text{otherwise} \end{cases}$$



Linear function

$$f(\text{net}) = a * \text{net} + b$$

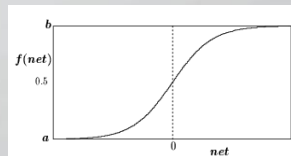


Sigmoid function

$$f(\text{net}) = z + \frac{1}{1 + \exp(-x \cdot \text{net} + y)}$$

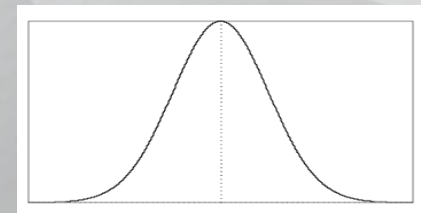
$$f(\text{net}) = \tanh(x \cdot \text{net} - y) + z$$

$$\text{where } \tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$$



Gaussian function

$$f(\text{net}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2}\left(\frac{\text{net} - \mu}{\sigma}\right)^2\right]$$



Attention! Also some other functions are in use – we will see this later!

Gradient descent

- based on the error associated to the entire set of train data
- modify the weights in the direction of the steepest slope of error reduction $E(\mathbf{w})$ for the entire set of train data

$$E(\mathbf{w}) = \frac{1}{2} \sum_{d=1}^n (t^d - o^d)^2$$

- How to determine the steepest slope? → derive E based on w (establish the gradient of error E)

$$\nabla E(\mathbf{w}) = \left(\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, \dots, \frac{\partial E}{\partial w_m} \right)$$

- Error's gradient is computed based on the neuron's activation function (that function must be differentiable → continuous)

- linear function $f(\text{net}) = \sum_{i=1}^m w_i x_i^d$

- sigmoid function $f(\text{net}) = \frac{1}{1 + e^{-\text{net}}} = \frac{1}{1 + e^{-\sum_{i=1}^m w_i x_i^d}}$

- How are modified the weights ? $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$, where $i = 1, 2, \dots, m$

Compute the error's gradient

➤ linear function

$$f(net) = \sum_{i=1}^m w_i x_i^d$$

$$\frac{\partial E}{\partial w_i} = \frac{\partial \frac{1}{2} \sum_{d=1}^n (t^d - o^d)^2}{\partial w_i} = \frac{1}{2} \sum_{d=1}^n \frac{\partial (t^d - o^d)^2}{\partial w_i} = \frac{1}{2} \sum_{d=1}^n 2(t^d - o^d) \frac{\partial (t^d - \mathbf{w}\mathbf{x}^d)}{\partial w_i}$$

$$\frac{\partial E}{\partial w_i} = \sum_{d=1}^n (t^d - o^d) \frac{\partial (t^d - w_1 x_1^d - w_2 x_2^d - \dots - w_m x_m^d)}{\partial w_i} = \sum_{d=1}^n (t^d - o^d) (-x_i^d)$$

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} = \eta \sum_{d=1}^n (t^d - o^d) x_i^d$$

➤ sigmoid function

$$f(net) = \frac{1}{1 + e^{-\mathbf{w}\mathbf{x}}} = \frac{1}{1 + e^{-\sum_{i=1}^m w_i x_i^d}}$$

$$y = s(z) = \frac{1}{1 + e^{-z}} \Rightarrow \frac{\partial s(z)}{\partial z} = s(z)(1 - s(z))$$

$$\frac{\partial E}{\partial w_i} = \frac{\partial \frac{1}{2} \sum_{d=1}^n (t^d - o^d)^2}{\partial w_i} = \frac{1}{2} \sum_{d=1}^n \frac{\partial (t^d - o^d)^2}{\partial w_i} = \frac{1}{2} \sum_{d=1}^n 2(t^d - o^d) \frac{\partial (t^d - \text{sig}(\mathbf{w}\mathbf{x}^d))}{\partial w_i} = \sum_{d=1}^n (t^d - o^d) (1 - o^d) o^d (-x_i^d)$$

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} = \eta \sum_{d=1}^n (t^d - o^d) (1 - o^d) o^d x_i^d$$

Simple and stochastic GDA

Simple GDA

Initialisation of network weights

$w_i = \text{random}(a,b)$, where $i = 1, 2, \dots, m$

While not stop condition

$\Delta w_i = 0$, where $i = 1, 2, \dots, m$

For each train example $(\mathbf{x}_d, t_d)$, where $d = 1, 2, \dots, n$

Activate the neuron and determine the output o_d

Linear activation $\rightarrow o_d = \mathbf{w} \mathbf{x}_d$

Sigmoid activation $\rightarrow o_d = \text{sig}(\mathbf{w} \mathbf{x}_d)$

For each weight w_i , where $i = 1, 2, \dots, m$

Determine the weight modification

$$\Delta w_i = \Delta w_i - \eta \frac{\partial E}{\partial w_i}$$

where η - learning rate

For each weight w_i , where $i = 1, 2, \dots, m$

Modify the weights w_i $w_i = w_i + \Delta w_i$

EndWhile

Stochastic GDA

Initialisation of network weights

$w_i = \text{random}(a,b)$, where $i = 1, 2, \dots, m$

While not stop condition

$\Delta w_i = 0$, unde $i = 1, 2, \dots, m$

For a random sample subset from the training data set
 $(\mathbf{x}_{d_i}, t_{d_i})$, where $d_i \in \{1, 2, \dots, n\}$

Activate the neuron and determine the output o_{d_i}

Linear activation $\rightarrow o_{d_i} = \mathbf{w} \mathbf{x}_{d_i}$

Sigmoid activation $\rightarrow o_{d_i} = \text{sig}(\mathbf{w} \mathbf{x}_{d_i})$

For each weight w_i , where $i = 1, 2, \dots, m$

Determine the weight modification

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

where η - learning rate

For each weight w_i , where $i = 1, 2, \dots, m$

Modify the weights w_i $w_i = w_i + \Delta w_i$

EndWhile

Neuron learning

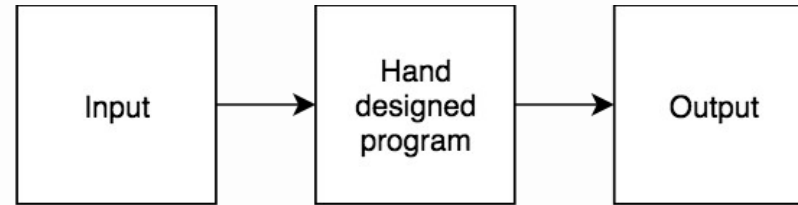
Differences	Perceptron's algorithm	Gradient descent algorithm (delta rule)
What does o^d represent?	$o^d = \text{sign}(\mathbf{w}\mathbf{x}^d)$	$o^d = \mathbf{w}\mathbf{x}^d$ or $o^d = \text{sig}(\mathbf{w}\mathbf{x}^d)$
Convergence	After a finite # of steps (until the perfect separation)	Asymptotic (to minimum error)
Solved problems	With linear separable data	Any data (linear separable or non-linear)
Neuron's output	Discrete and with threshold	Continue and without threshold

THANK YOU !

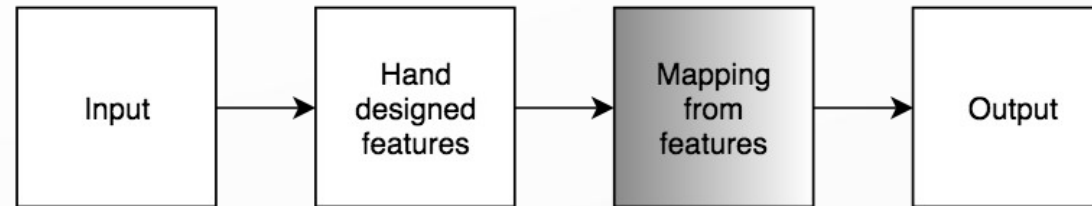
Artificial Neural Networks

Learning multiple components

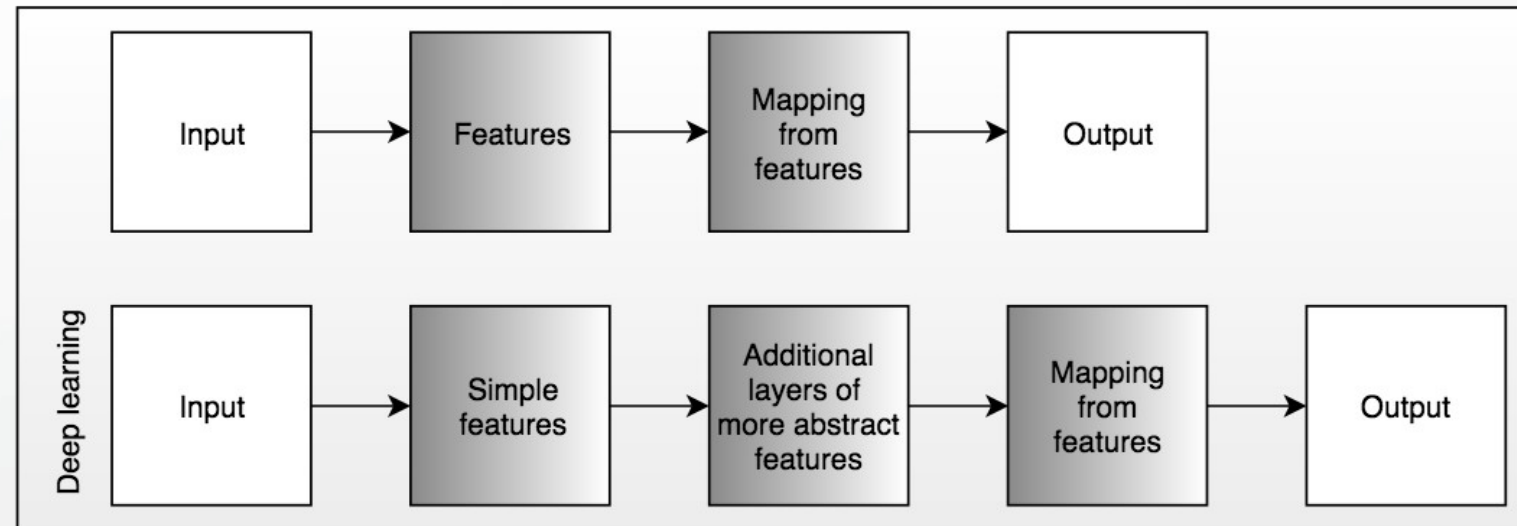
Rule-based systems



Classic machine learning

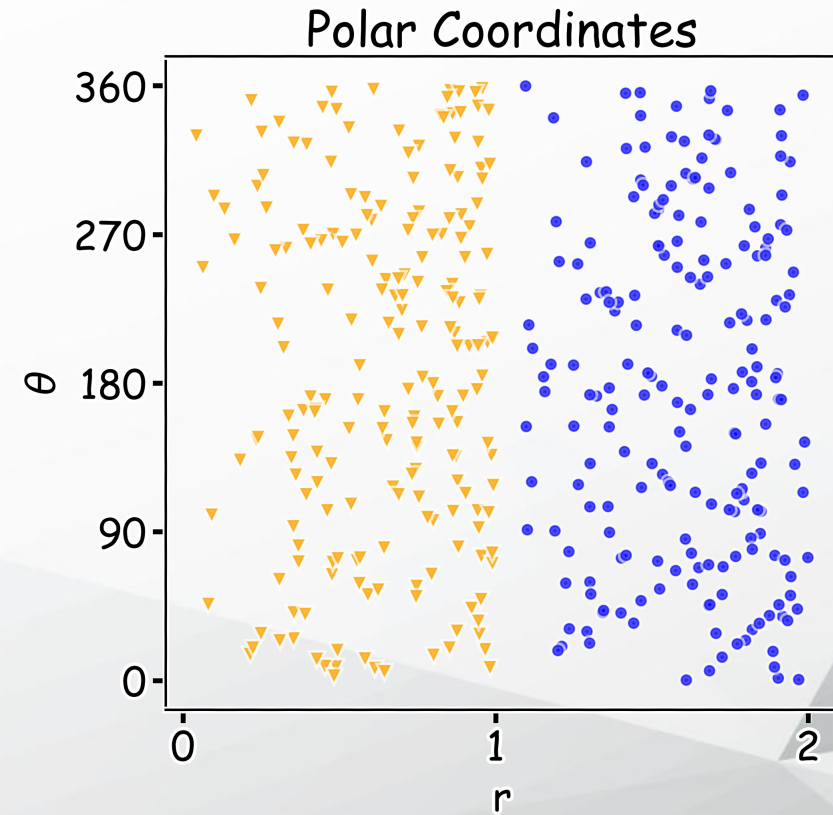
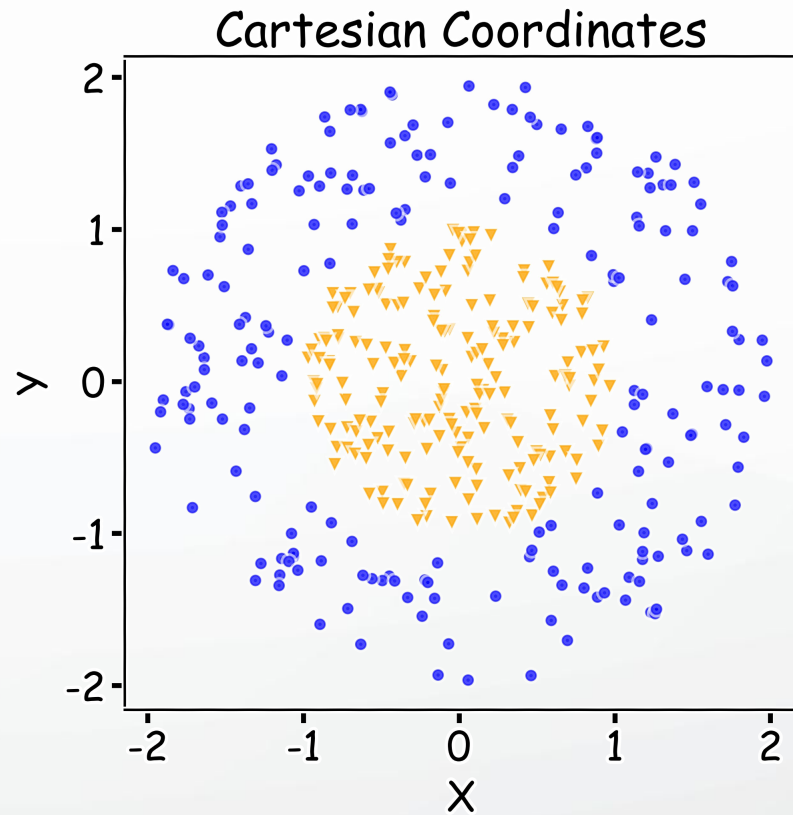


Representation learning

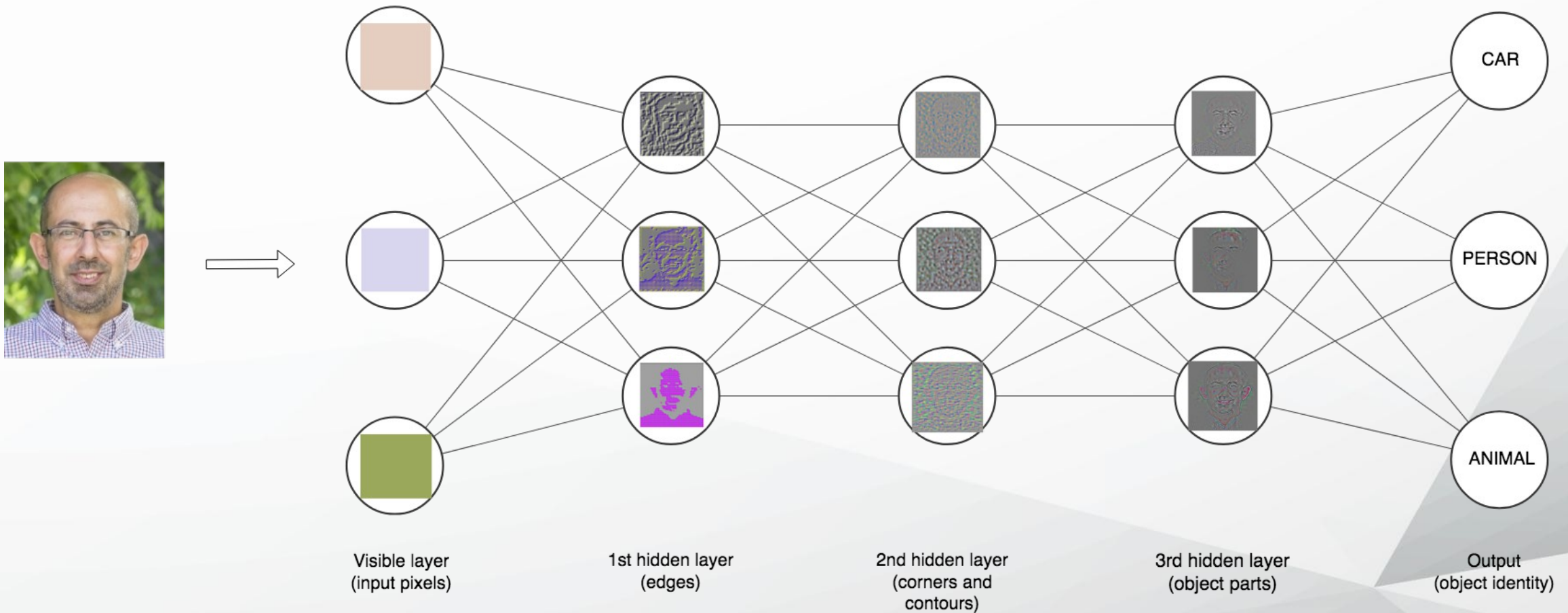


Representation

- First - Representation matters!



Depth – repeated compositions

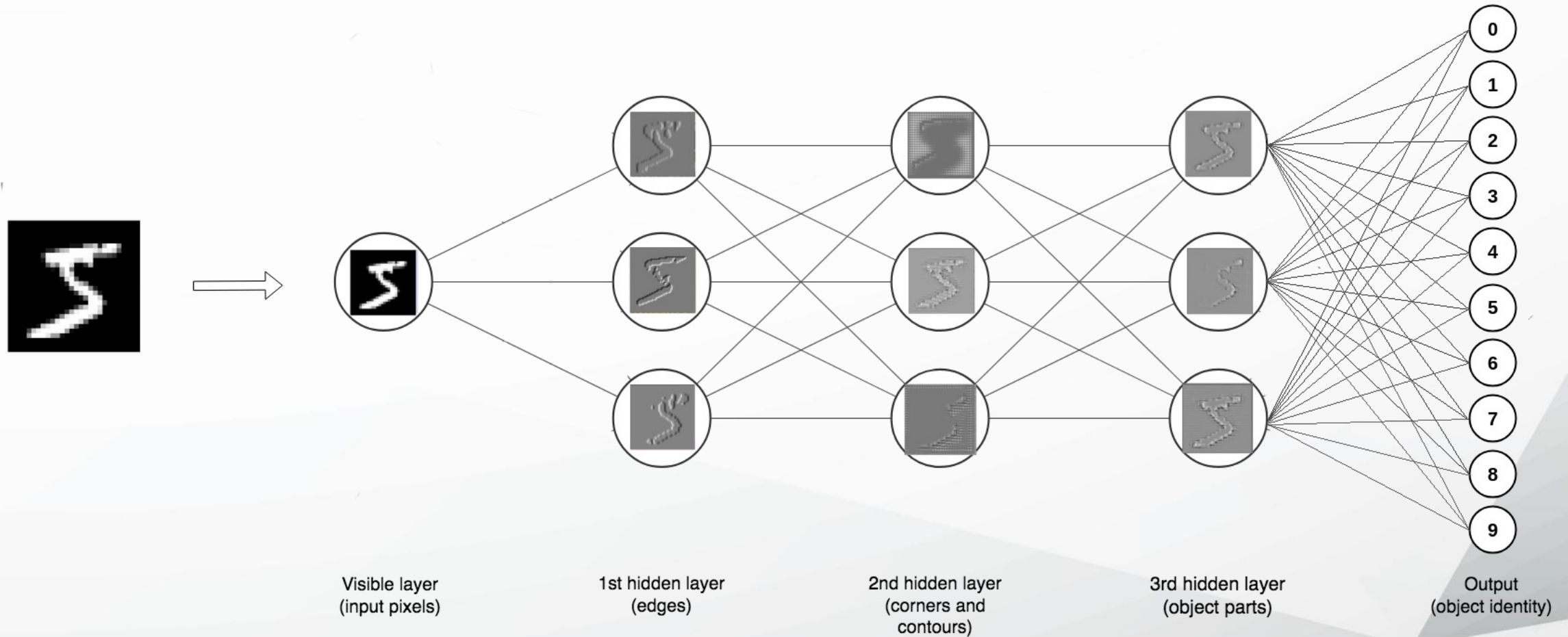


Depth – repeated compositions

MNIST – dataset of hand written digits



Depth – repeated compositions



Common types of Neural Network Architectures

- Feedforward Neural networks
- Convolutional Neural Networks
- Recurrent Neural Networks
- Long Short-term Memory networks
- Autoencoders
- Generative Adversarial Networks

Feed-forward Neural Networks

- Simplest form of ANN:
 - the perceptrons are arranged in layers
 - the first layer is taking the inputs
 - the last layer is producing the outputs
 - between them there are hidden layers
- The data flow goes in one direction:
 - each perceptron is connected with every perceptron on the next layer
 - there is no connection between the perceptrons in the same layer

Features of feed-forward ANNs

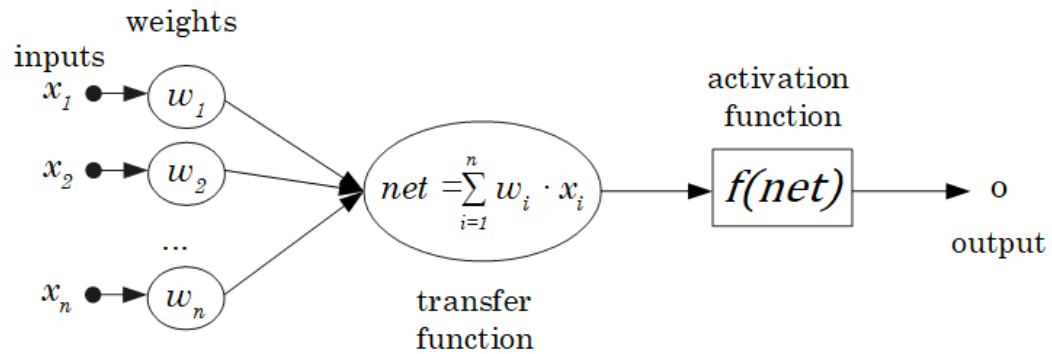
- recall that a single perceptron can classify points into two regions that are linearly separable

Consider a more complex network

- with more perceptrons that are independent of each other in the hidden layer, the points are classified in more pairs of linearly separable regions, each of it having a unique line separating the region.
- by varying the number of nodes in the hidden layer, the number of layers, and the number of input and output nodes, one can classification of points in arbitrary dimension into an arbitrary number of groups
- hence feed-forward networks are commonly used for classification.

Artificial neural network – structure

a node's structure



$$\mathbf{X} = \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{pmatrix}$$

$$\mathbf{W} = \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_n \end{pmatrix}$$

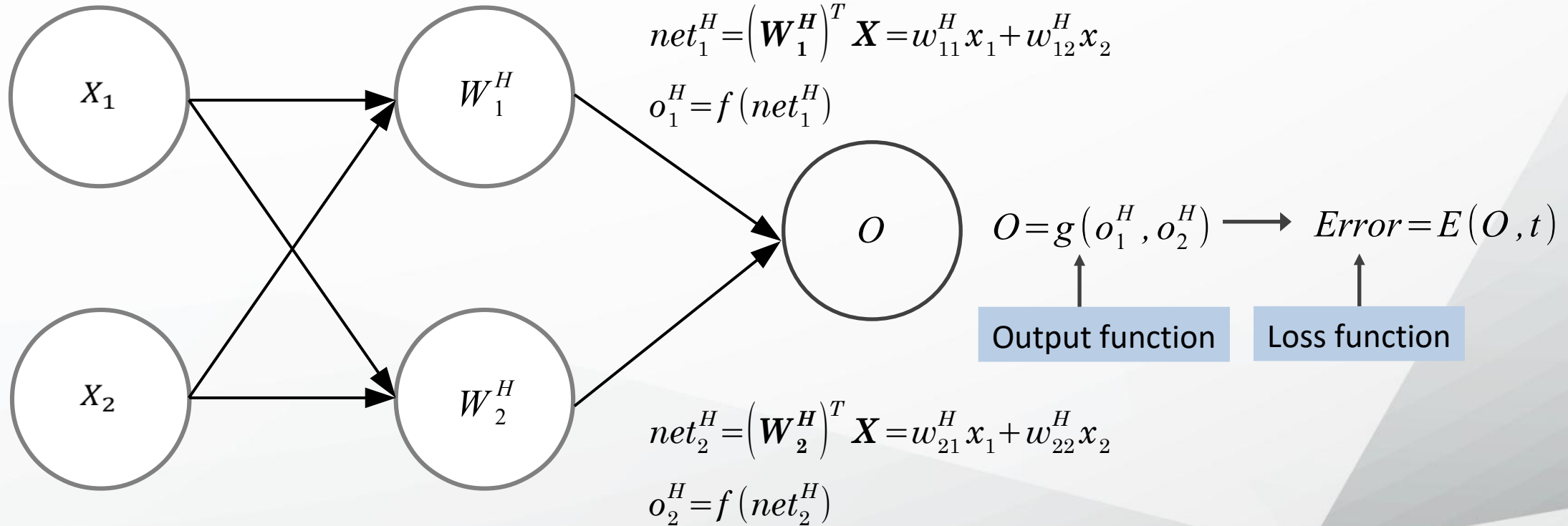
$$\mathbf{o} = f(\mathbf{X}^T \mathbf{W})$$

Artificial neural network – structure

Input layer

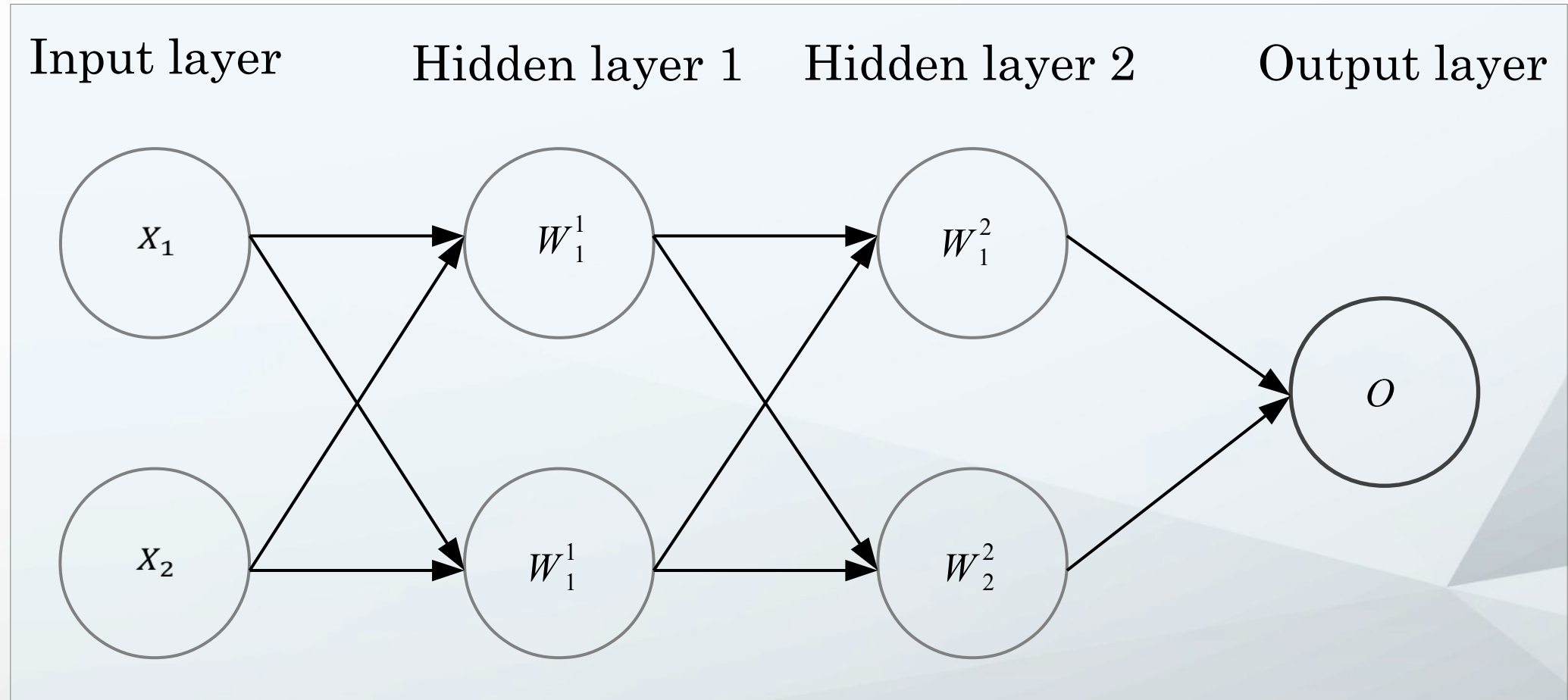
Hidden layer

Output layer



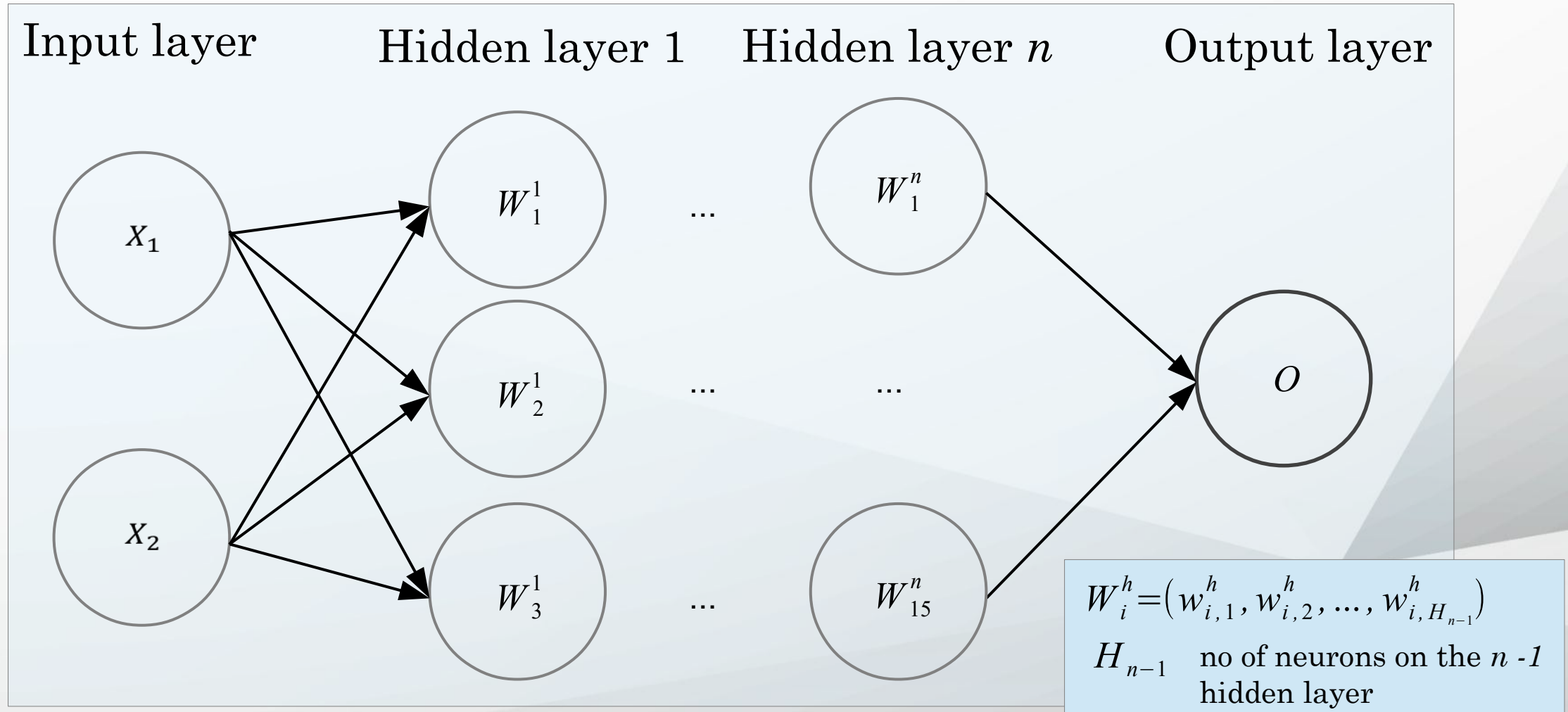
Artificial neural network – structure

Example of a **2 : 2 : 2 : 1** architecture with 2 hidden layers



Artificial neural network – structure

Example of a **2:3: ... :15:1** architecture with n hidden layers



Design choices for feed-forward ANNs

Activation function

Loss function

Output units

Architecture

Linearity versus non-linearity

- Linear models:
 - Can be fit efficiently (via convex optimization)
 - Limited model capacity
- Alternative:

$$f(\mathbf{x}) = W^T \phi(\mathbf{x})$$

- Where $\phi(\mathbf{x})$ is a *non-linear transform*

Activation functions for ANNs

The activation function should:

- Provide **non-linearity**
- Ensure **gradients remain large** through hidden unit

Common choices are

- Sigmoid
- Relu, leaky ReLU, Generalized ReLU, MaxOut
- Softplus
- Tanh
- Swish

Traditional ANNs (like feed-forward)

- Manually engineer
 - Domain specific, enormous human effort
- Generic transform
 - Maps to a higher-dimensional space
 - Kernel methods: e.g. RBF kernels
 - Over fitting: does not generalize well to test set
 - Cannot encode enough prior information

Deep Learning

- Directly learn ϕ

$$f(\mathbf{x}, \eta) = W^T \phi(\mathbf{x}, \eta)$$

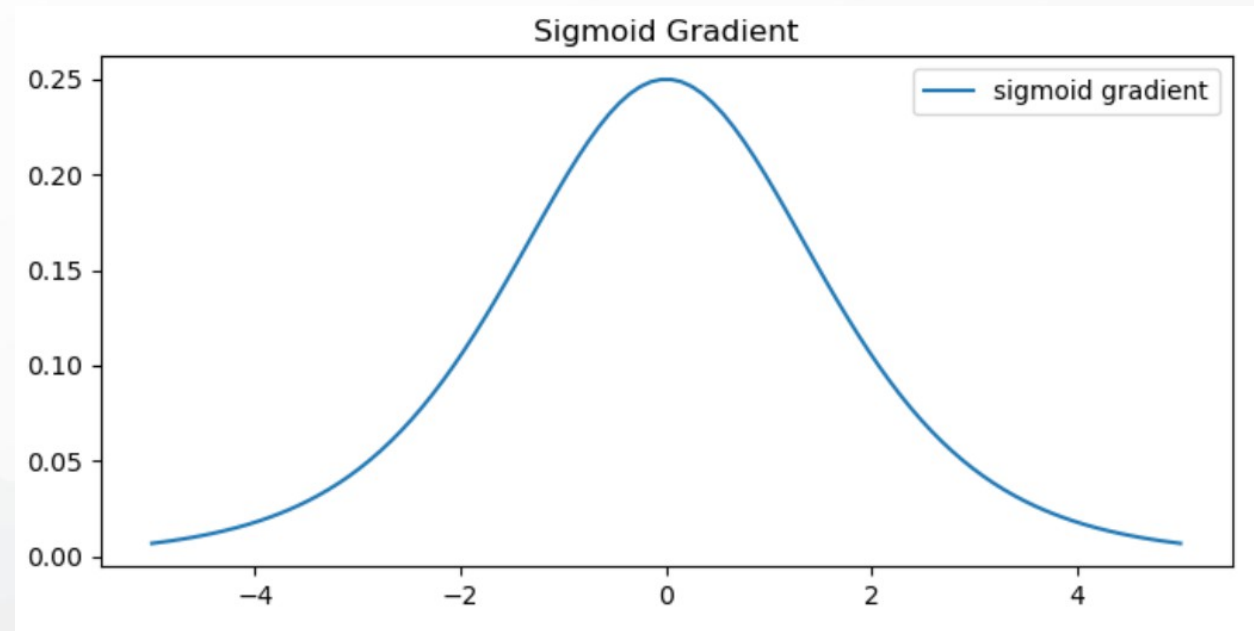
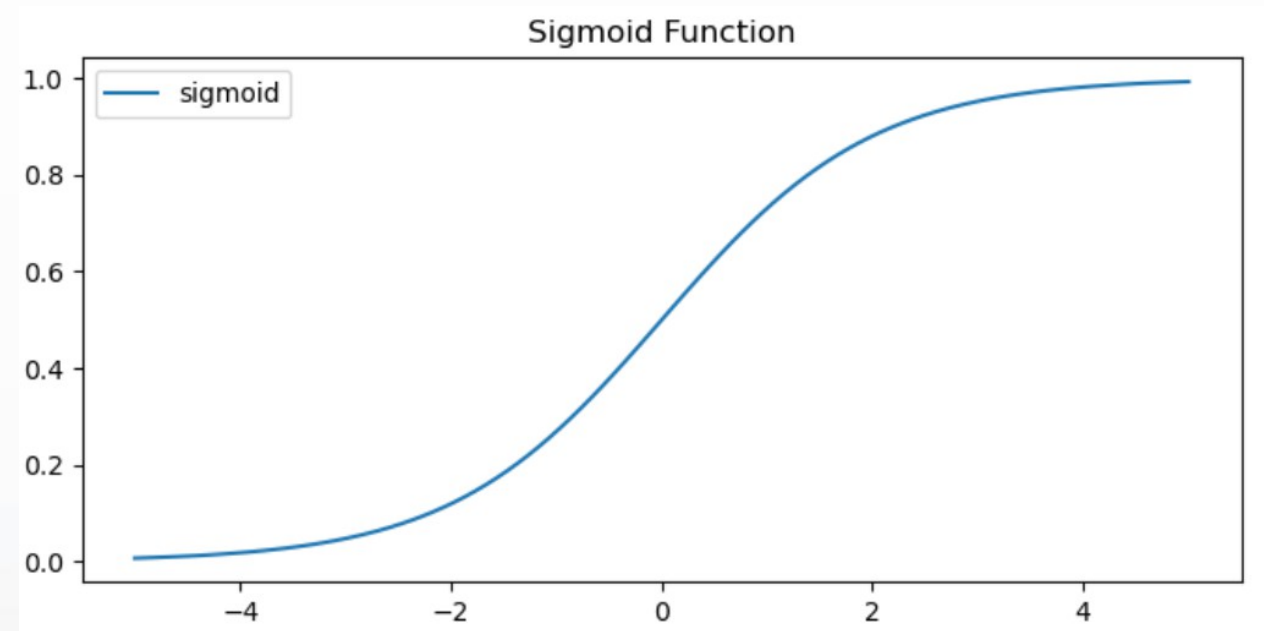
- $\phi(\mathbf{x}, \eta)$ is an automatically-learned **representation** of x
- For **deep networks**, ϕ is the function learned by the **hidden layers** of the network
- η are the learned weights

Non-convex optimization

- Can encode prior beliefs, generalizes well

Sigmoid (aka Logistic)

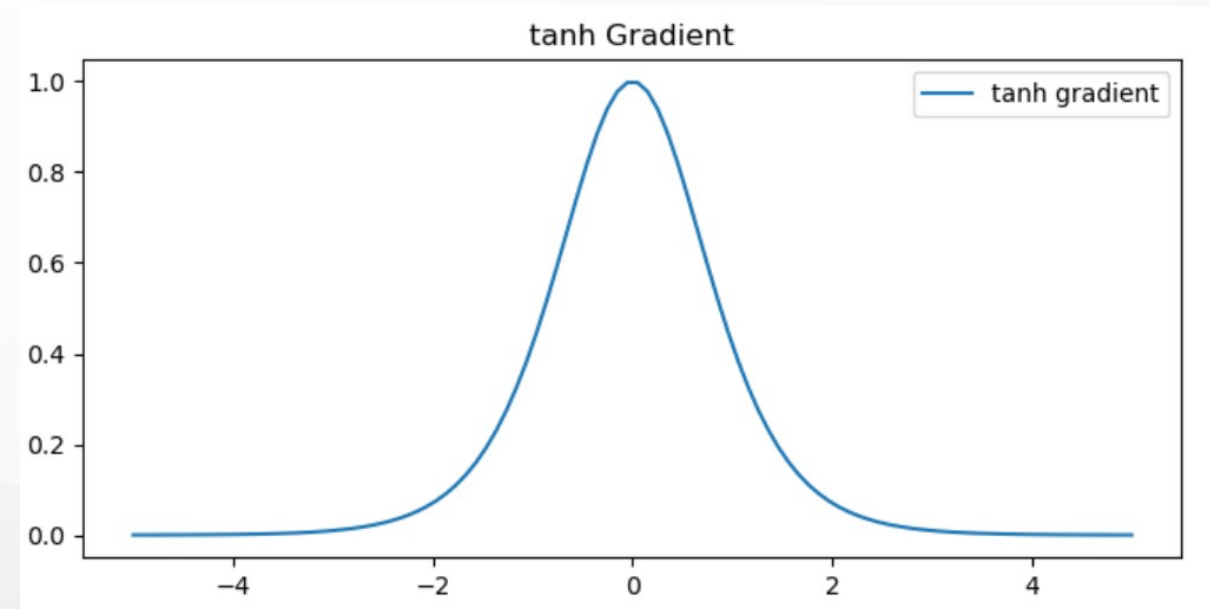
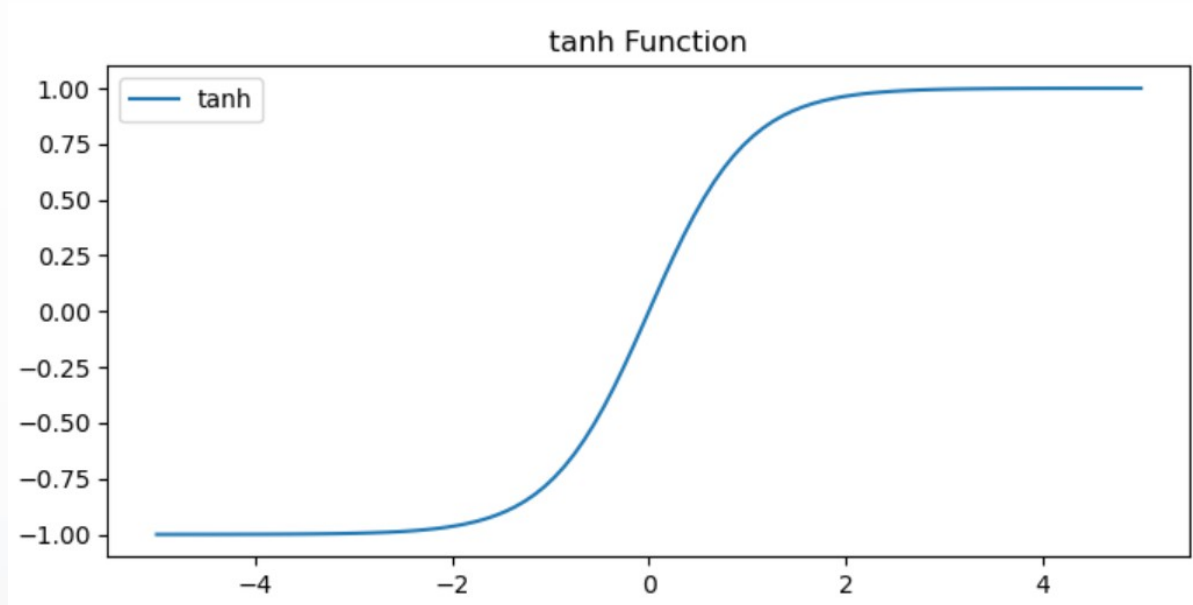
$$y = \frac{1}{1 + e^{-x}}$$



Derivative is **zero** for much of the domain. This leads to “vanishing gradients” in backpropagation.

Hyperbolic tangent (aka tanh)

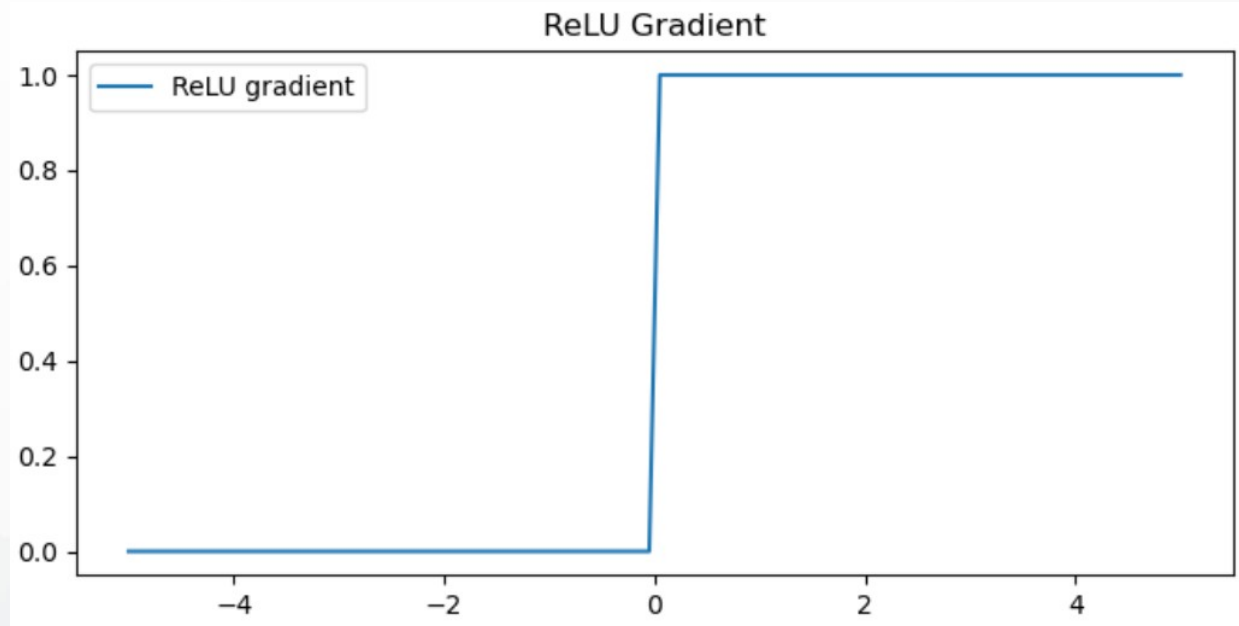
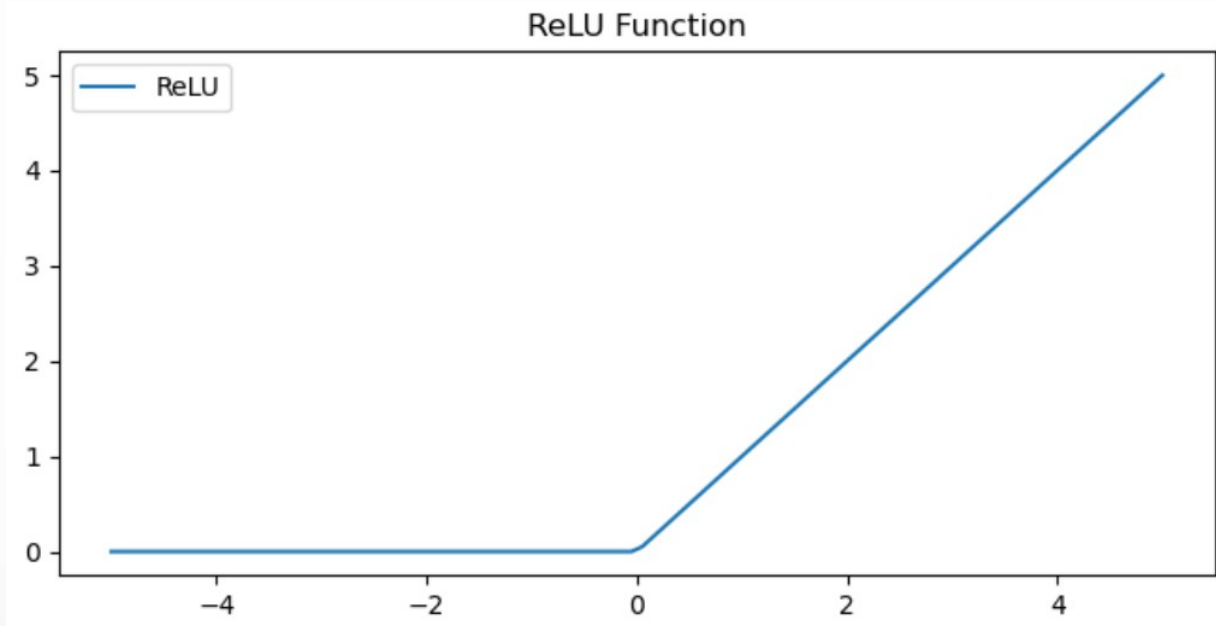
$$y = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Derivative is also **zero** for much of the domain.

Rectified Linear Unit (ReLU)

$$y = \max(0, x)$$



Two major advantages:

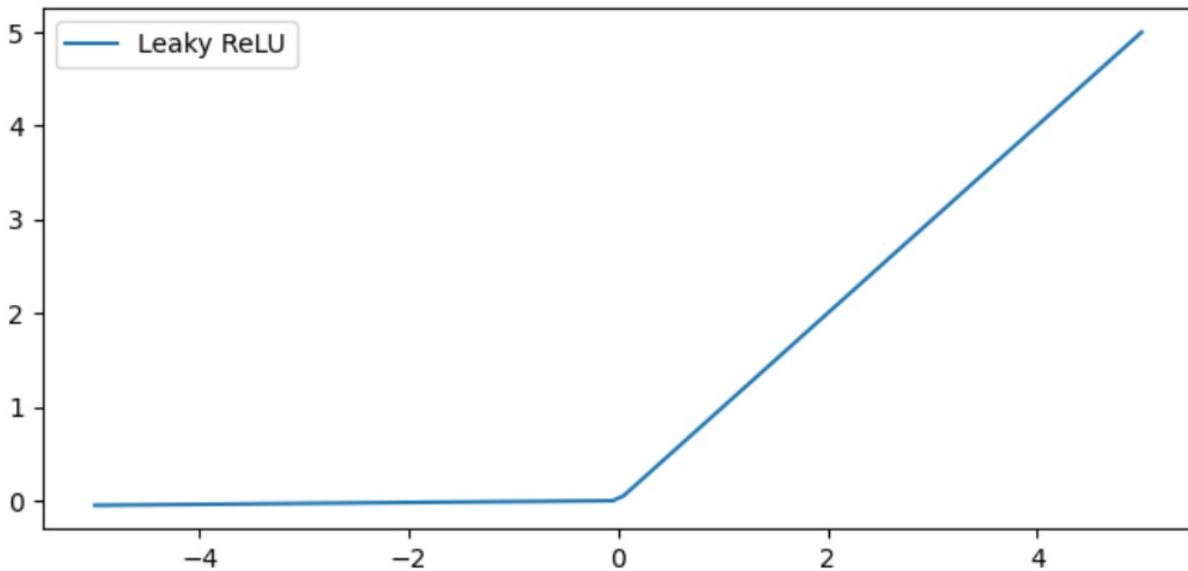
1. No vanishing gradient when $x > 0$
2. Provides sparsity (regularization) since $y = 0$ when $x < 0$

Leaky ReLU

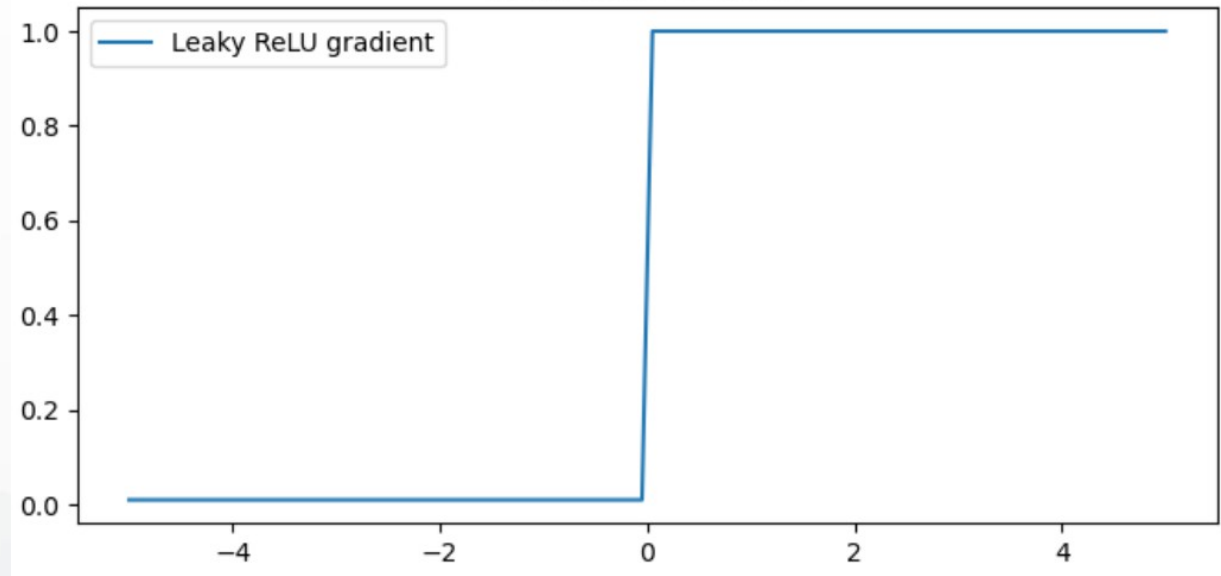
$$y = \max(0, x) + \min(\alpha x, 0)$$

α is a small positive value

Leaky ReLU Function



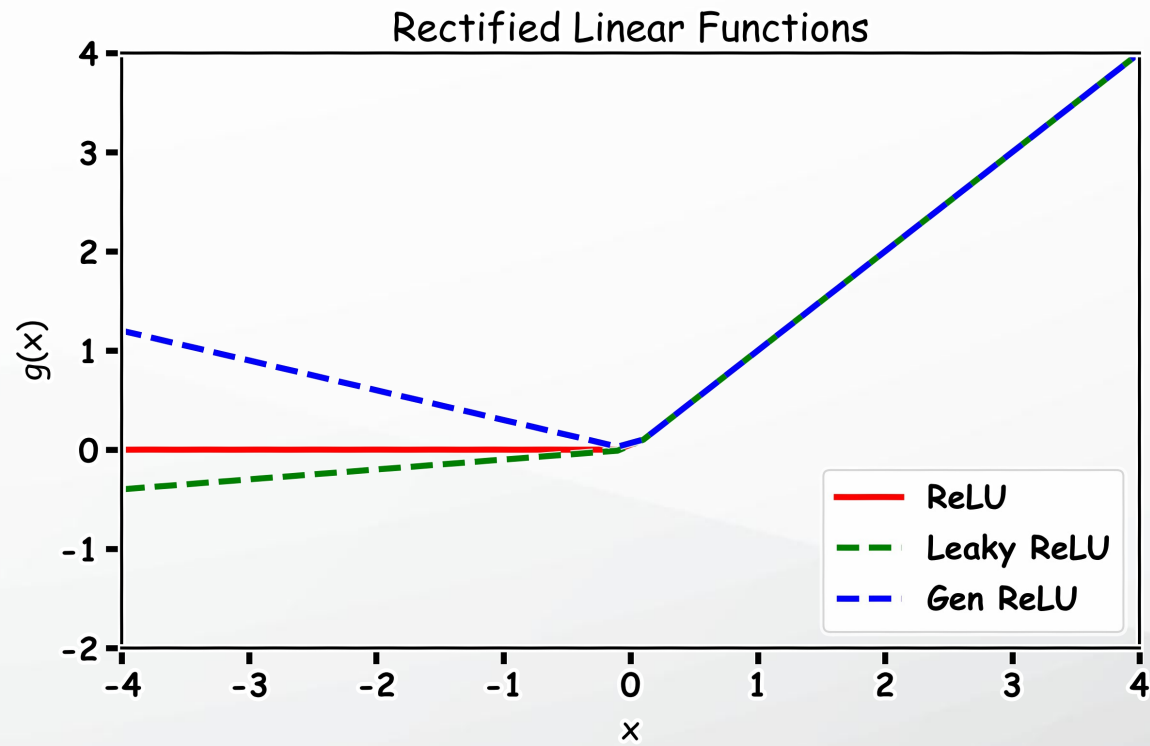
Leaky ReLU Gradient



- Tries to fix “dying ReLU” problem: derivative is non-zero everywhere.
- Some people report success with this form of activation function, but the results are not always consistent

Generalized ReLU

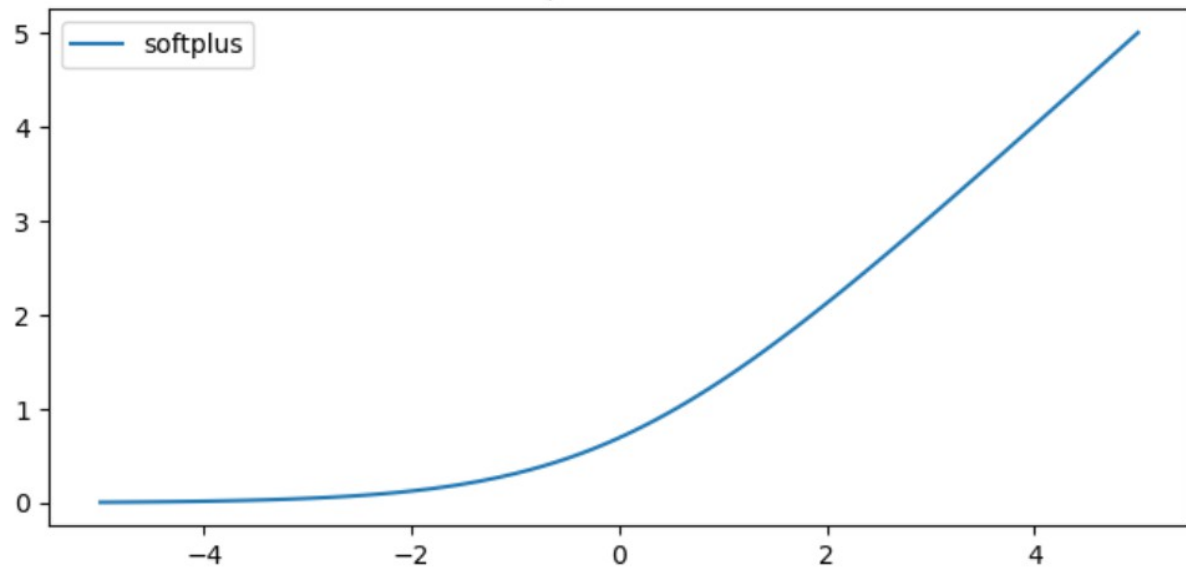
$$y = \max(0, x) + \alpha \min(x, 0)$$



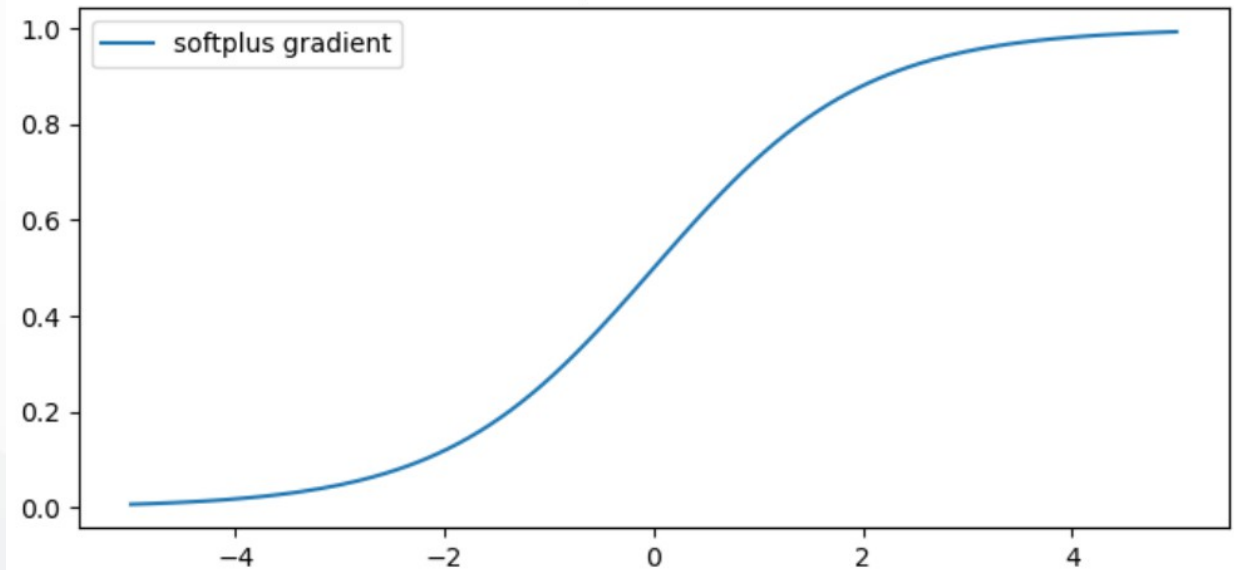
Softplus function

$$y = \log(1 + e^x)$$

softplus Function



softplus Gradient

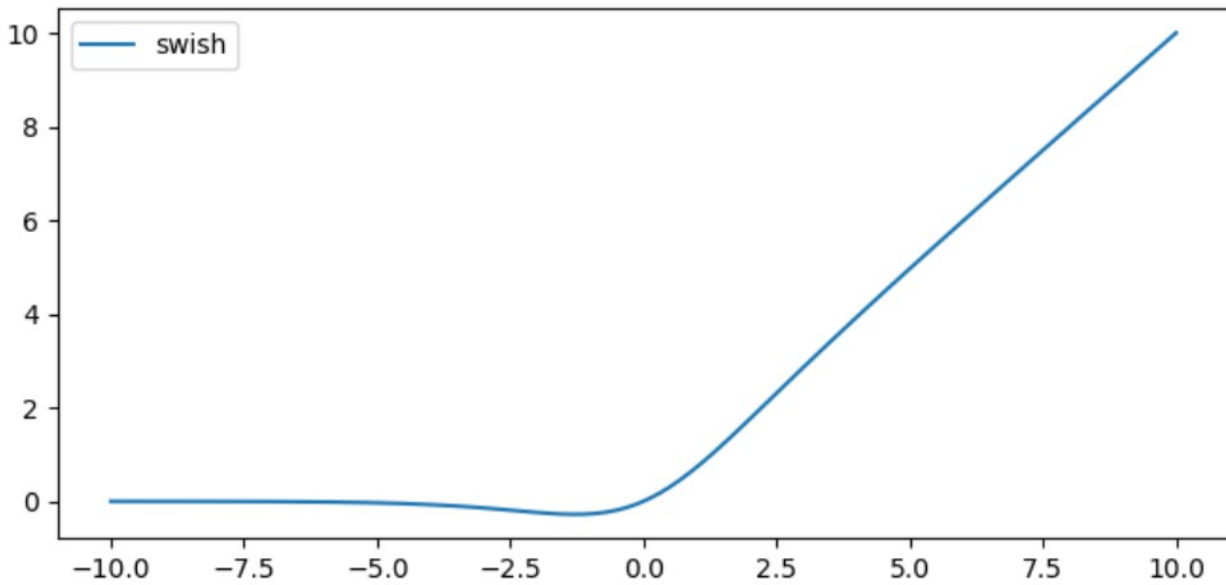


Outputs produced by sigmoid and tanh functions have upper and lower limits whereas softplus function produces outputs in scale of $(0, +\infty)$.

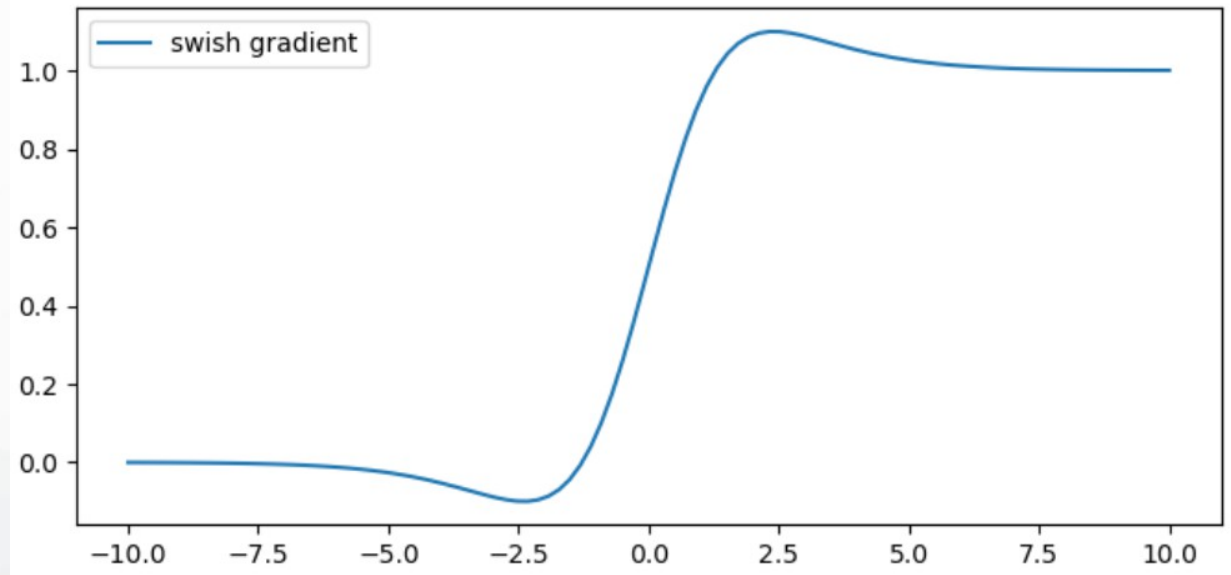
Swish function

$$y = x \operatorname{sigmoid}(x)$$

swish Function



swish Gradient



Loss functions for ANNs

- Likelihood for a given point
- Assume independence – likelihood for all measurements
- Maximize the likelihood, or equivalently maximize the log-likelihood
- Turn all this into a loss function

Common loss functions for ANNs

Mean Absolute Error Loss

Mean Squared Error Loss

Negative Log-Likelihood Loss

Cross-Entropy Loss

Hinge Embedding Loss

...

Mean Absolute Error Loss (MAE)

also called L1 Loss

- computes the average of the sum of absolute differences between actual values and predicted values.

$$\text{loss}(x, y) = |x - y|$$

- x represents the actual value and y the predicted value.

used for:

- regression problems
 - especially when the distribution of the target variable has outliers, such as small or big values that are a great distance from the mean value.

Mean Squared Error Loss (MSE)

also called L2 Loss

- computes the average of the squared differences between actual values and predicted values

$$loss(x, y) = (x - y)^2$$

- x represents the actual value and y the predicted value.

MSE is the default loss function for most regression problems.

Cross-Entropy Loss

Is the difference between two probability distributions for a provided set of occurrences or random variables.

In the discrete setting, given two probability distributions p and q , their cross-entropy is defined as

$$H(p, q) = - \sum_{x \in X} p(x) \log(q(x))$$

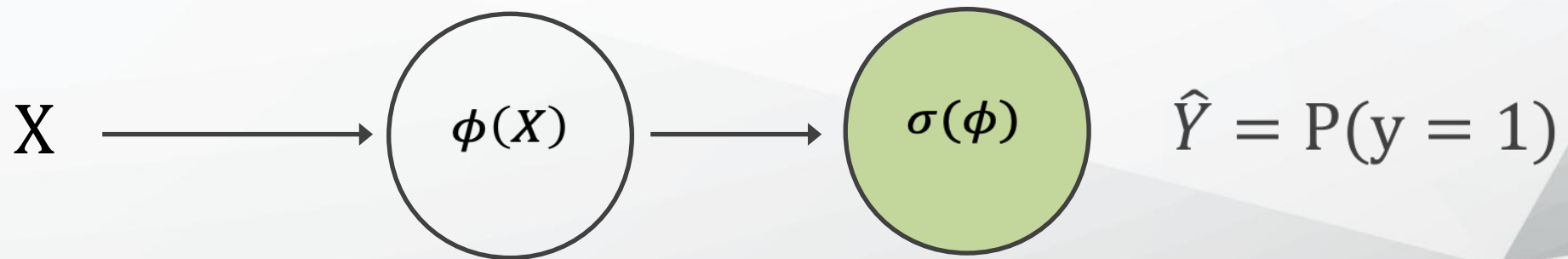
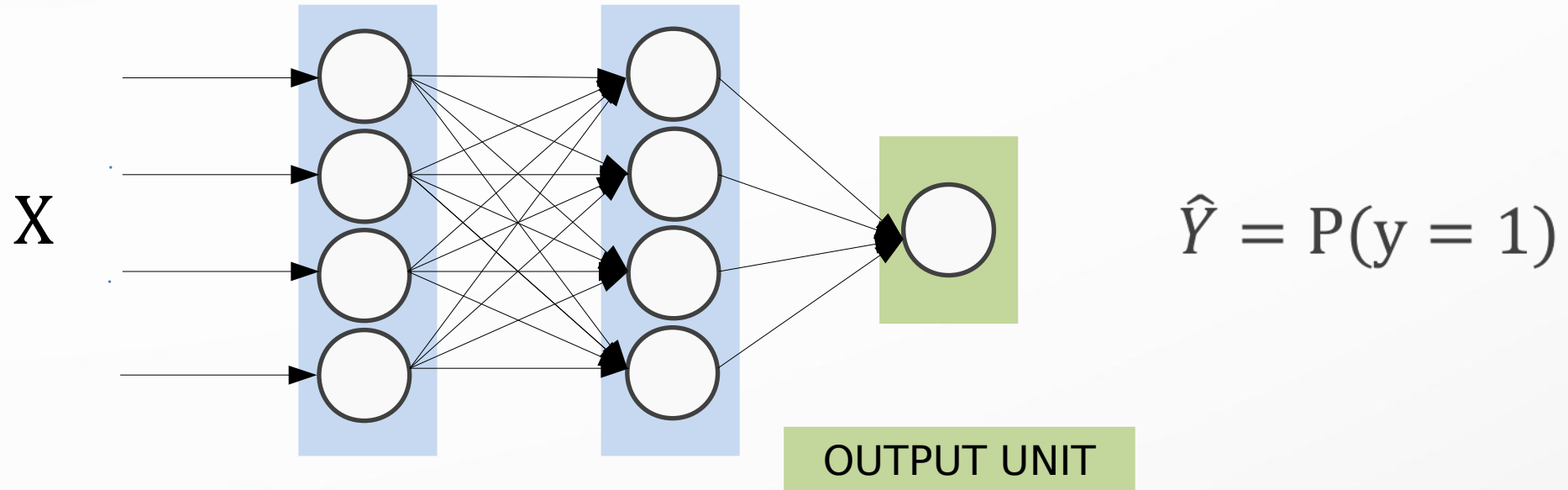
To compute the loss we apply the cross-entropy operator between the desired output and to the real output of our model after we used the softmax operator on the output.

Used in classifications

Output functions

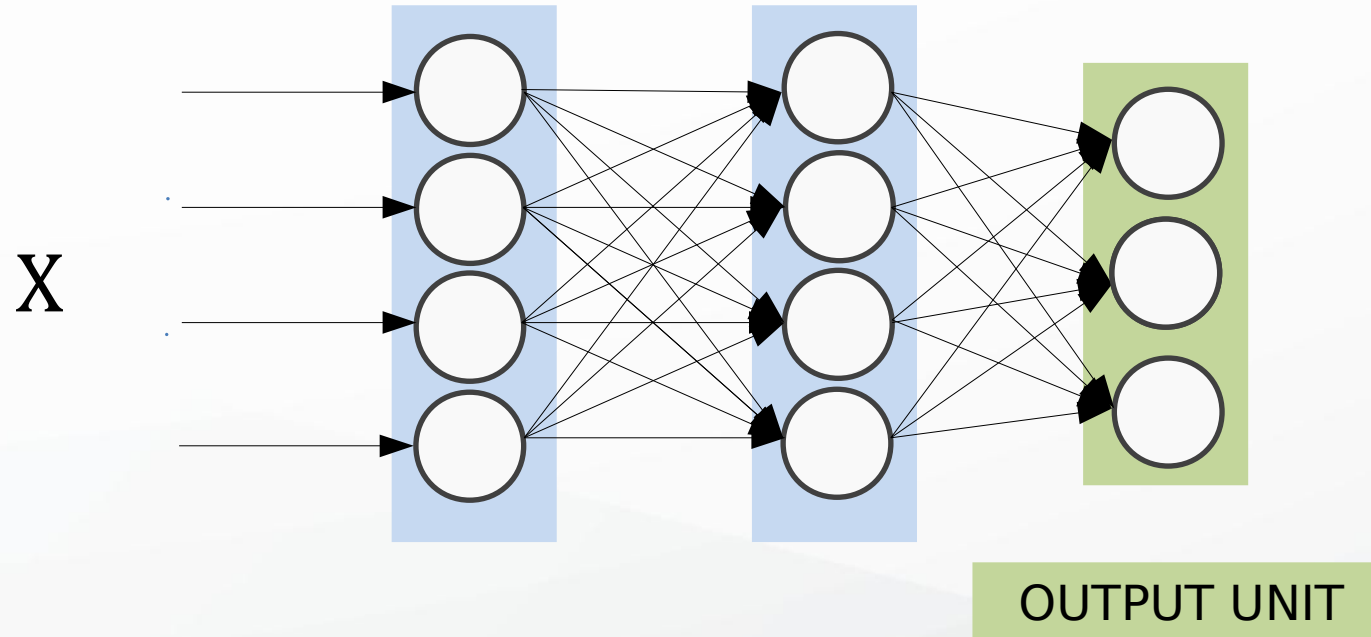
Output Type	Output Distribution	Output layer	Cost Function
Binary	Bernoulli	Sigmoid	Binary Cross Entropy
Discrete	Multinoulli	Softmax	Cross Entropy
Continuous	Gaussian	Linear	MSE
Continuous	Arbitrary	-	GANS

Output unit for binary classification

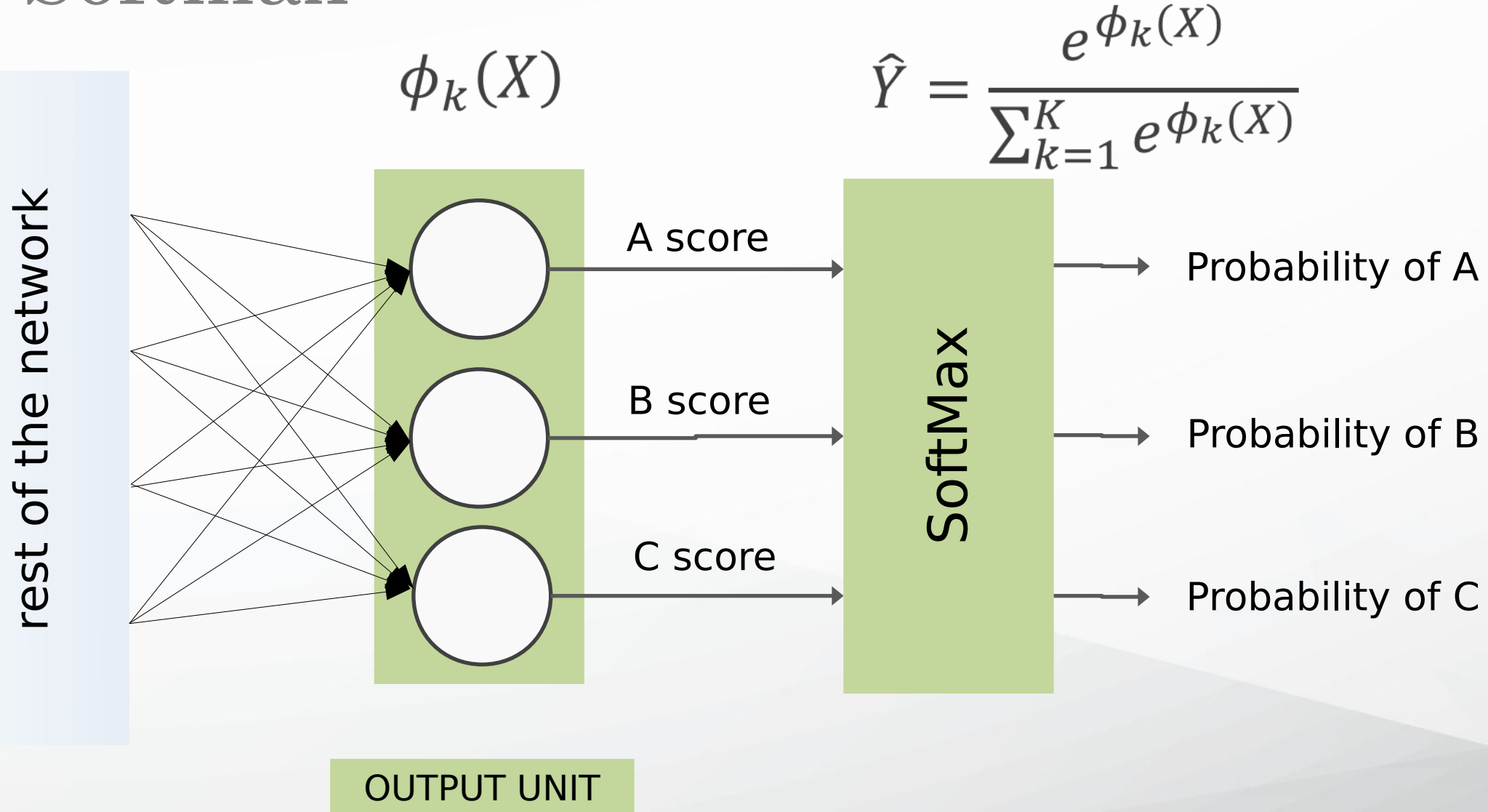


$$X \Rightarrow \phi(X) \Rightarrow P(y = 1) = \frac{1}{1 + e^{-\phi(X)}}$$

Output unit for multi class classification



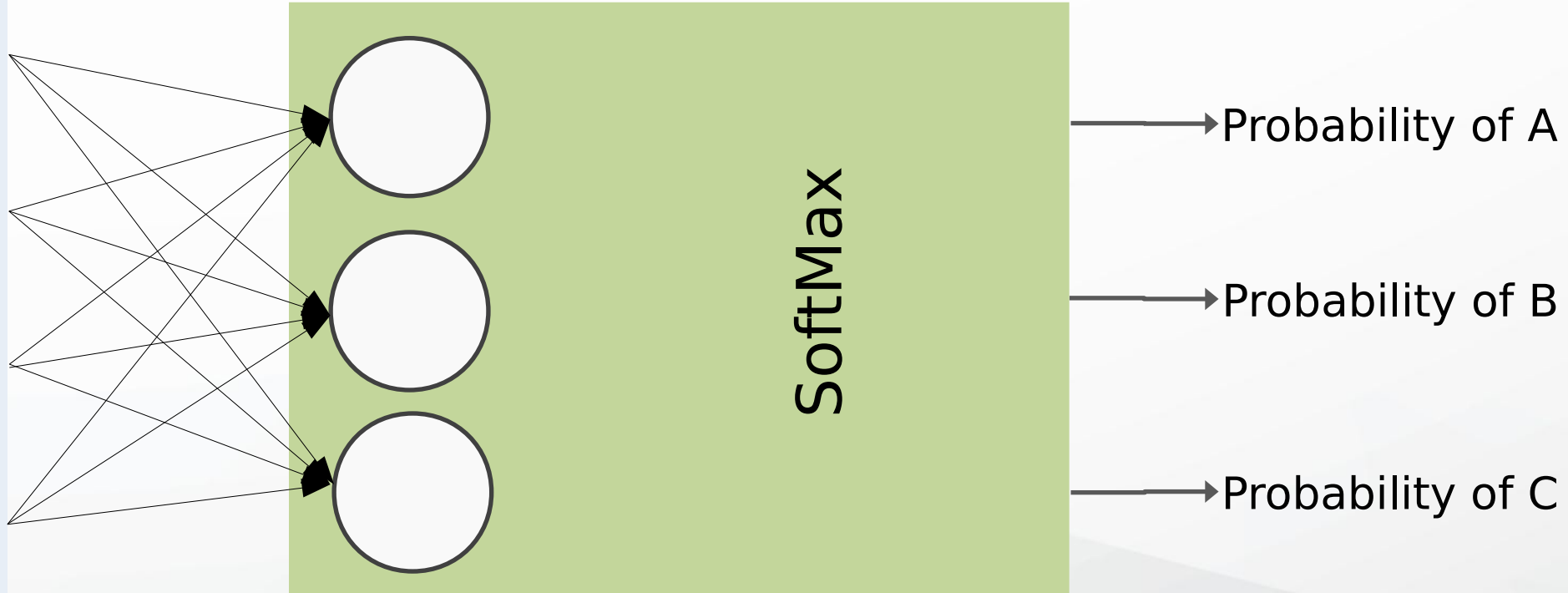
Softmax



Softmax

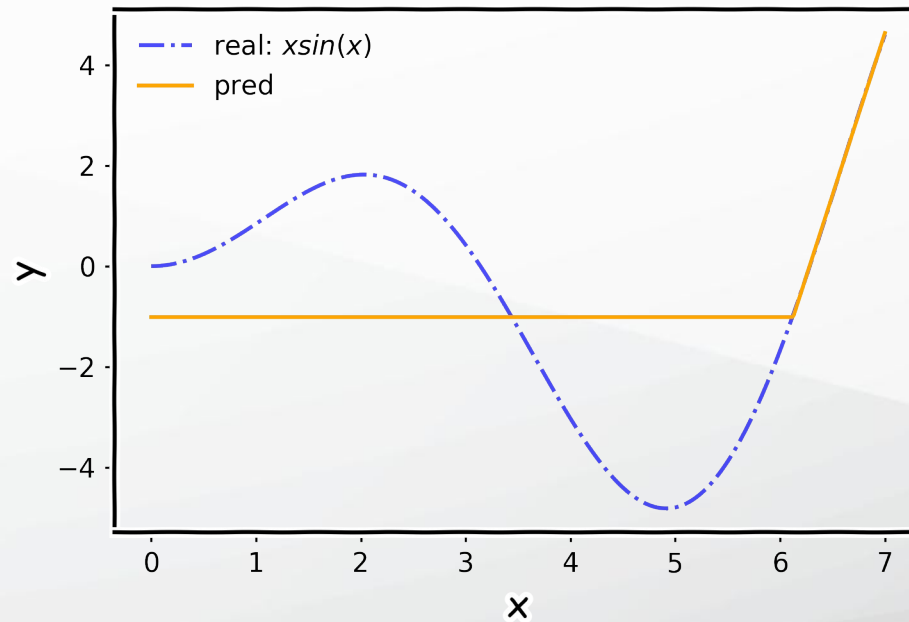
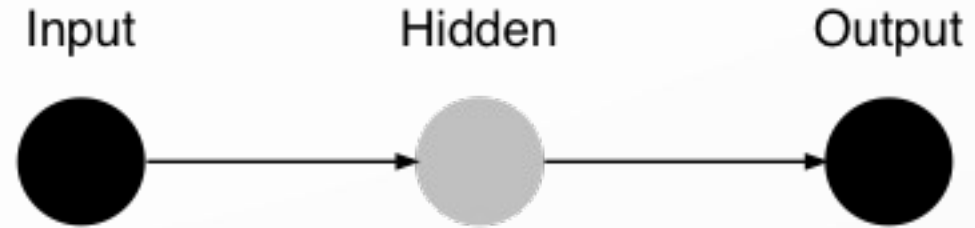
$$\hat{Y} = \frac{e^{\phi_k(X)}}{\sum_{k=1}^K e^{\phi_k(X)}}$$

rest of the network

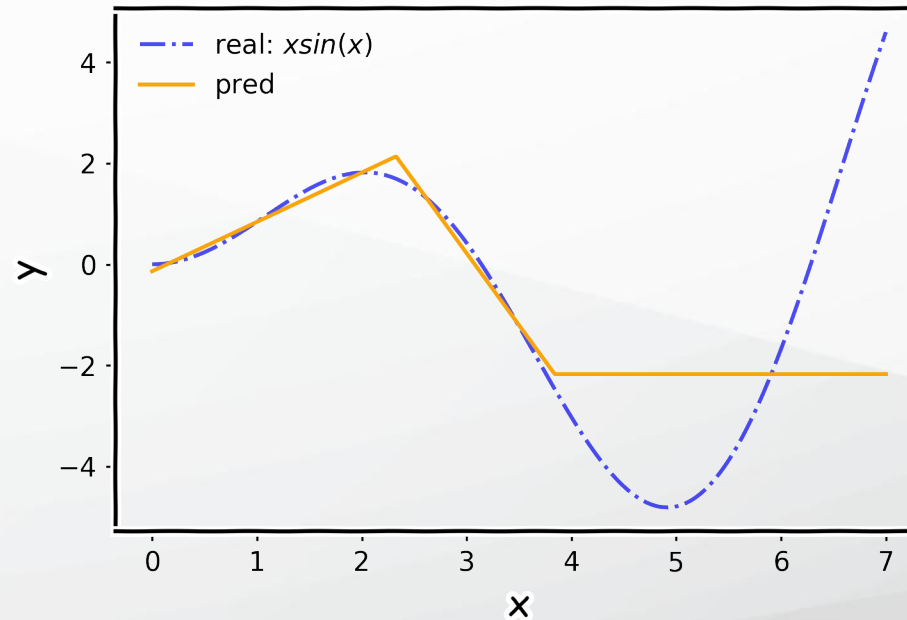
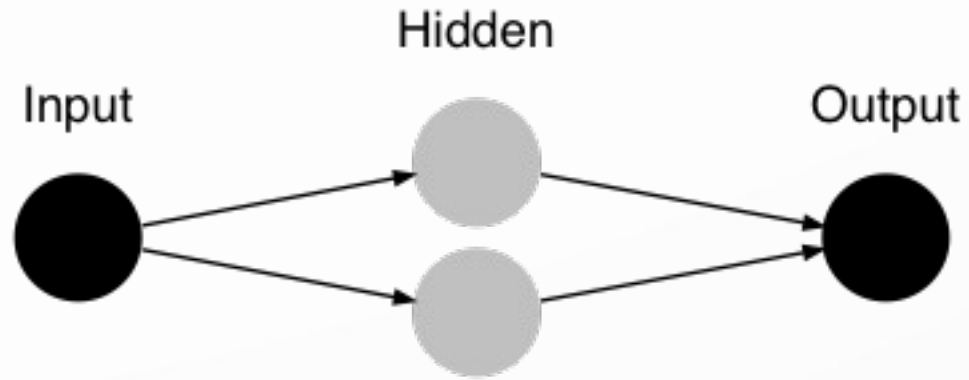


OUTPUT UNIT

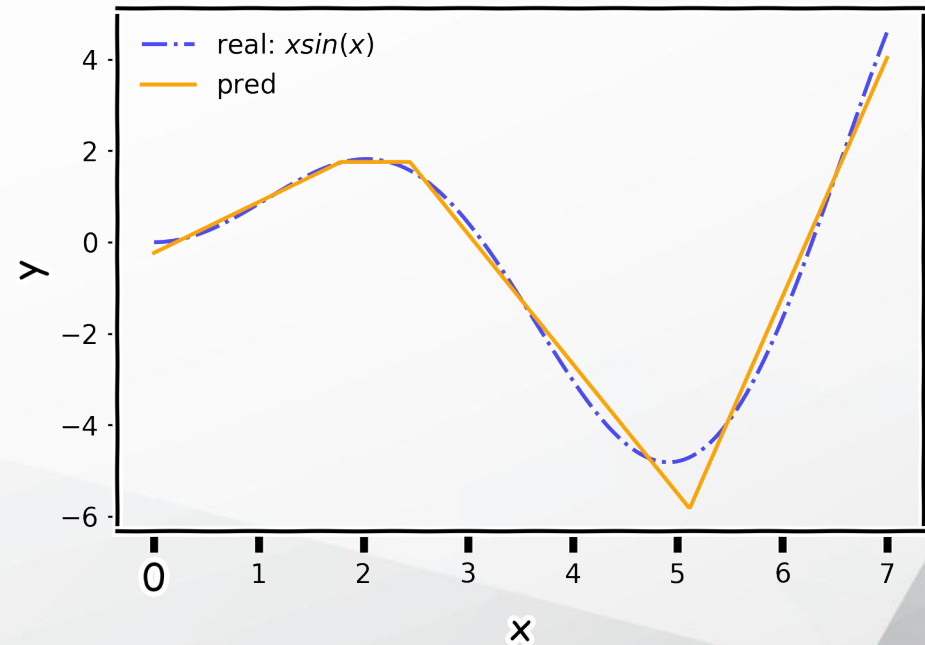
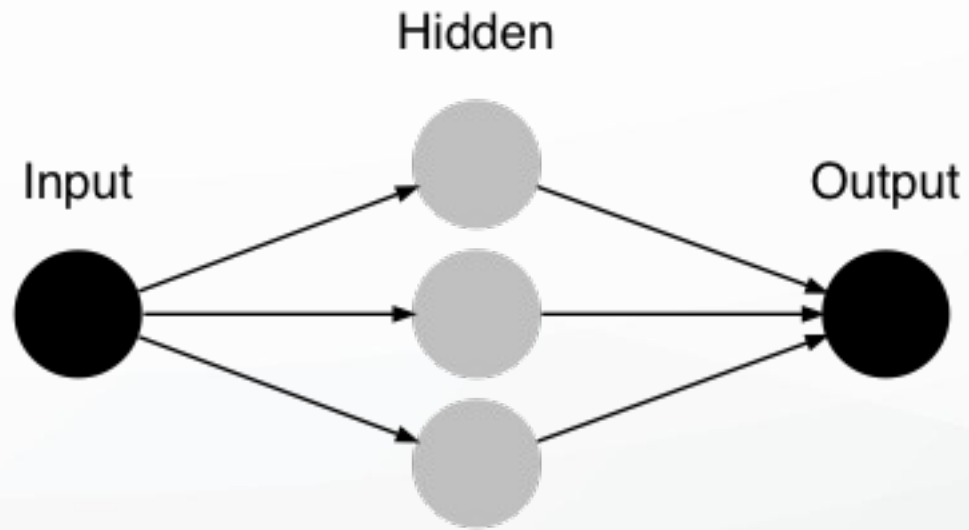
Architecture – performance example



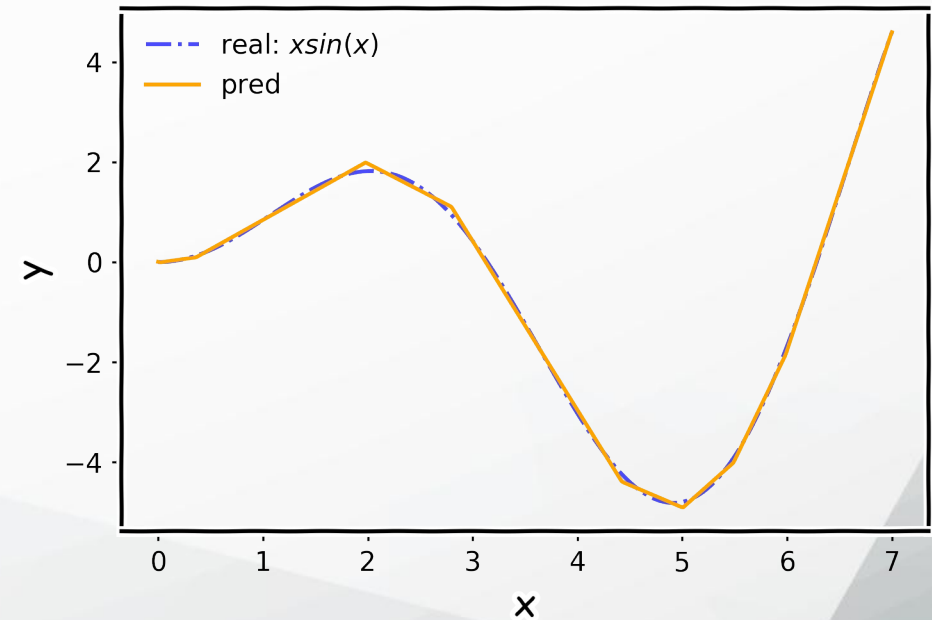
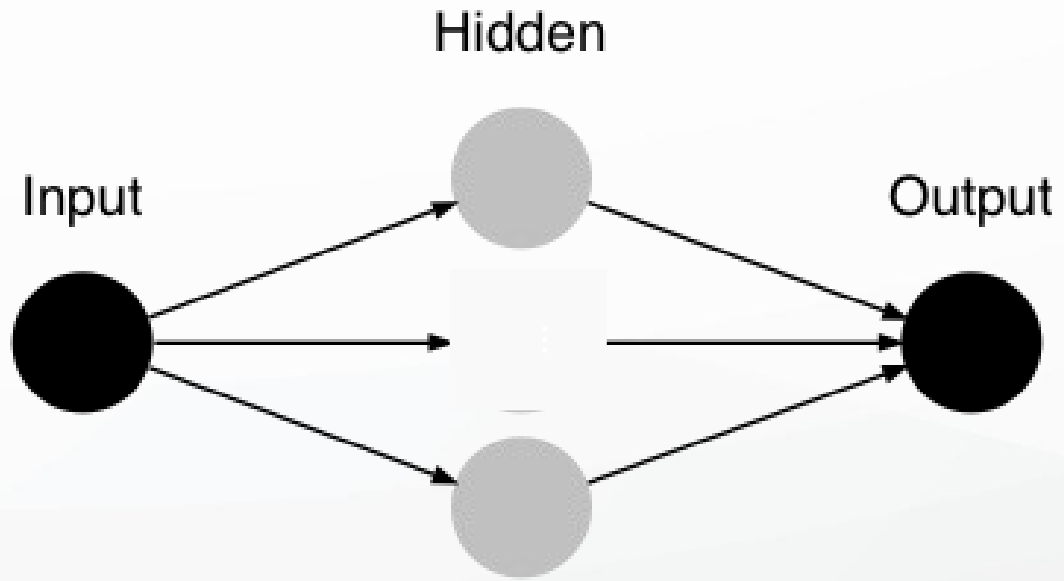
Architecture – performance example



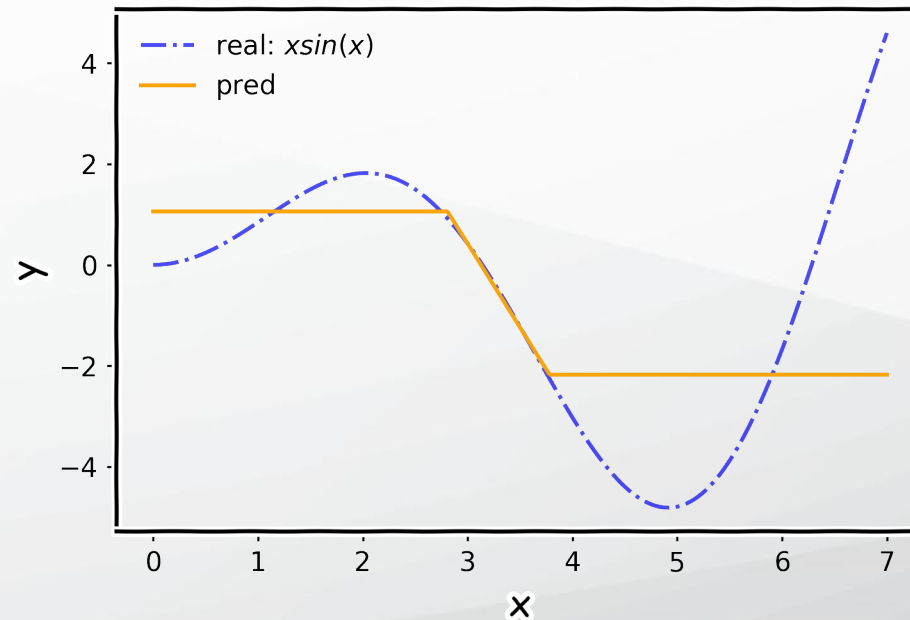
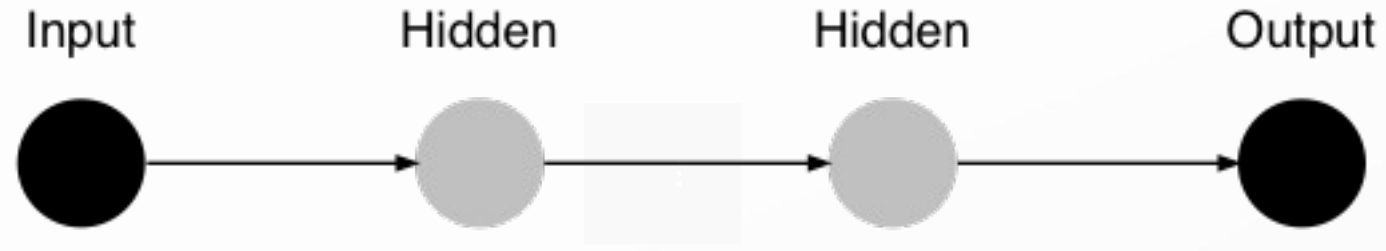
Architecture – performance example



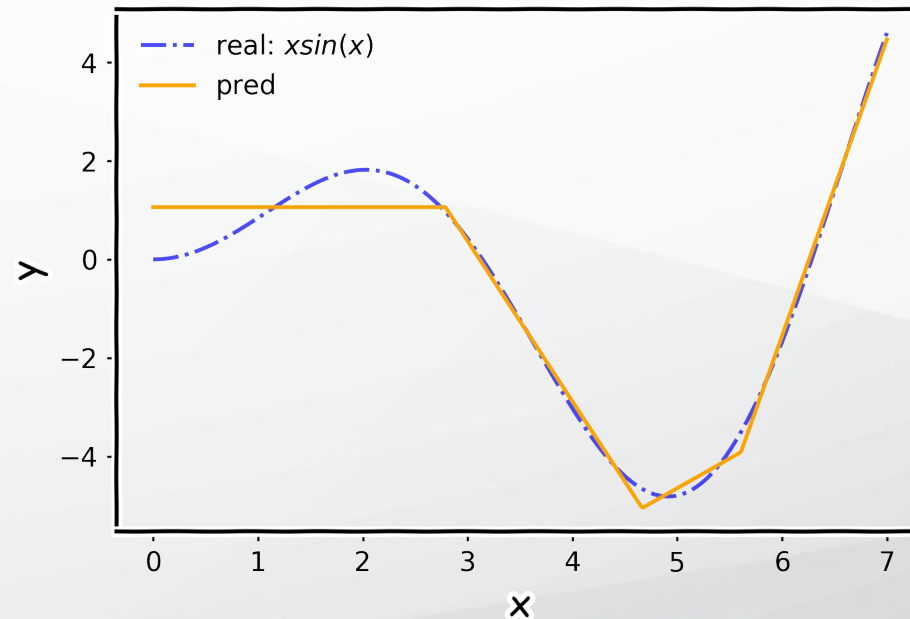
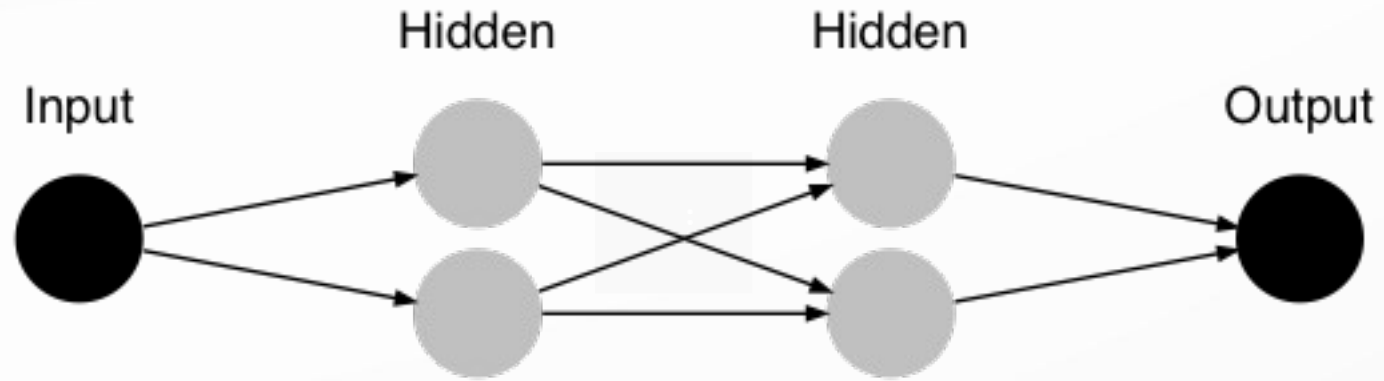
Architecture – performance example



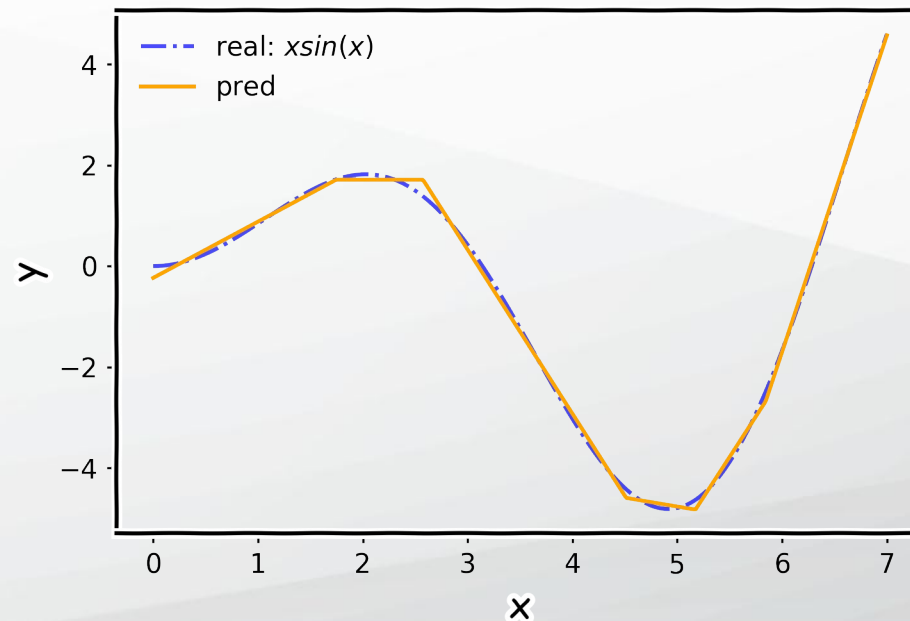
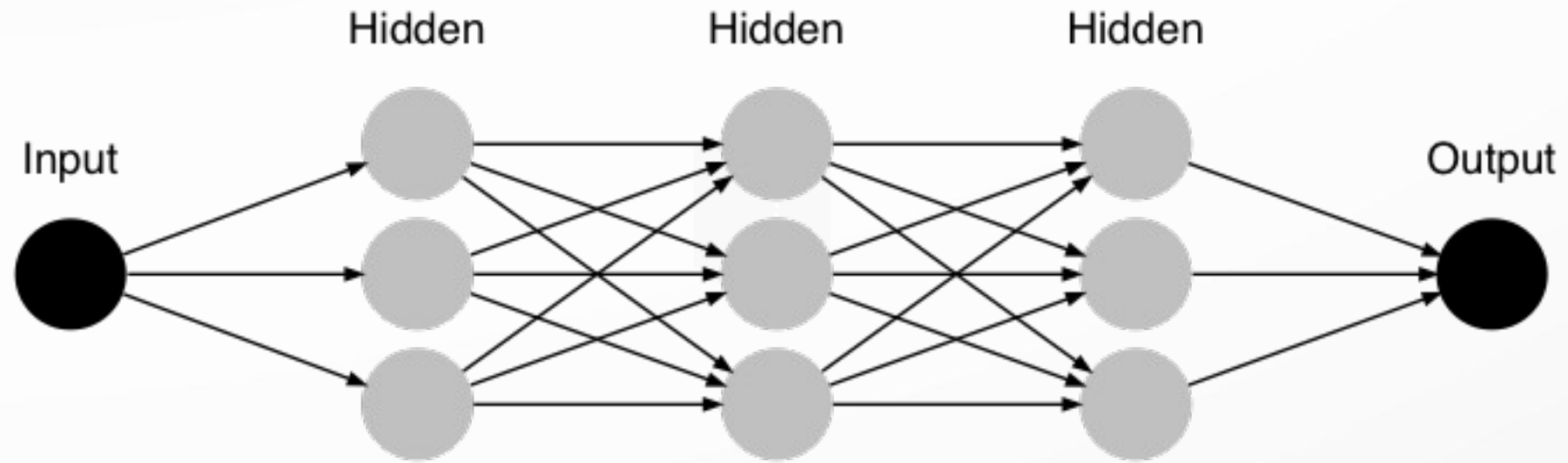
Architecture – performance example



Architecture – performance example



Architecture – performance example



Universal Approximation Theorem

Think of a Neural Network as function approximation.

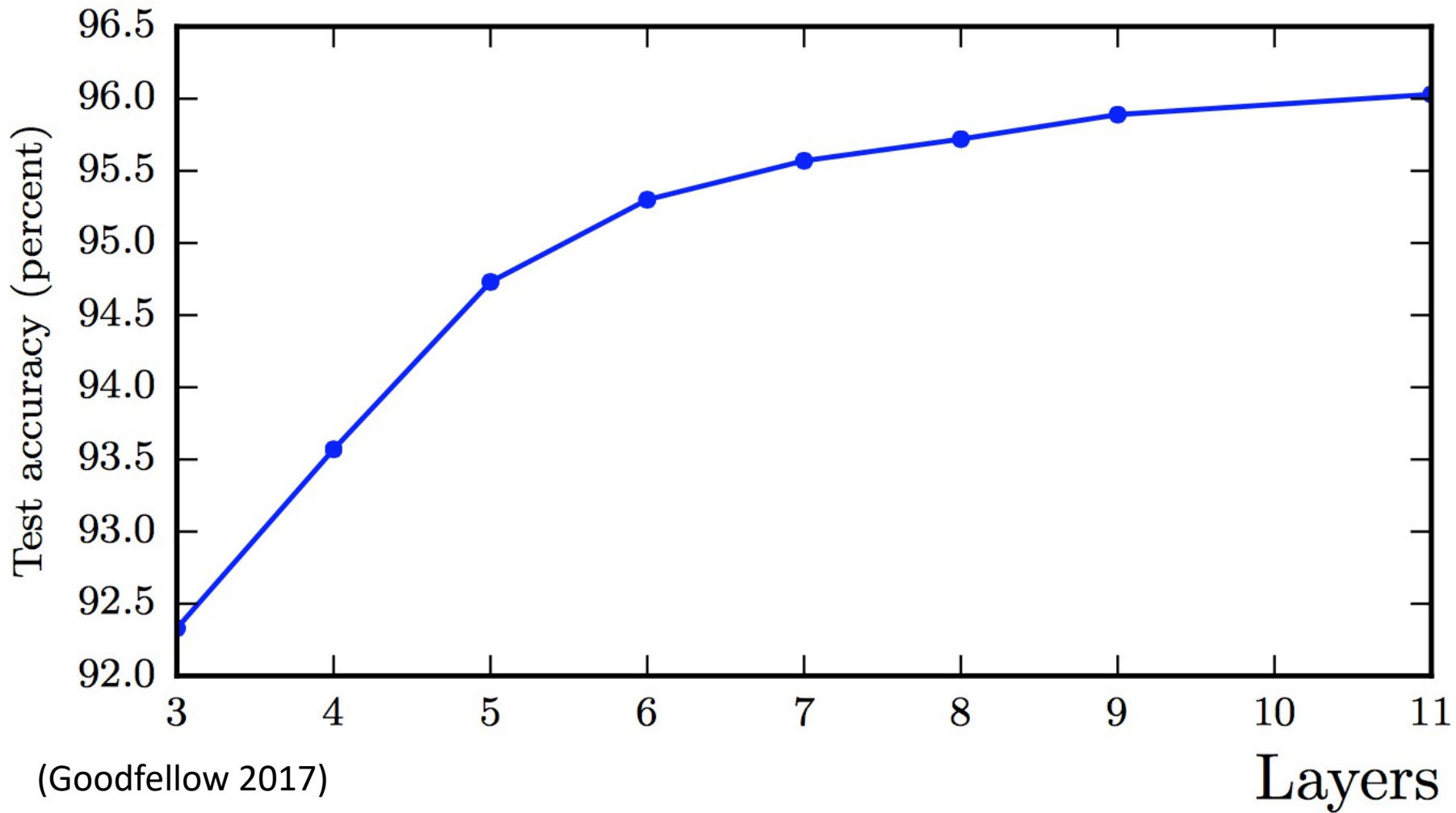
- NN:

One hidden layer is enough to *represent* an approximation of any function to an arbitrary degree of accuracy

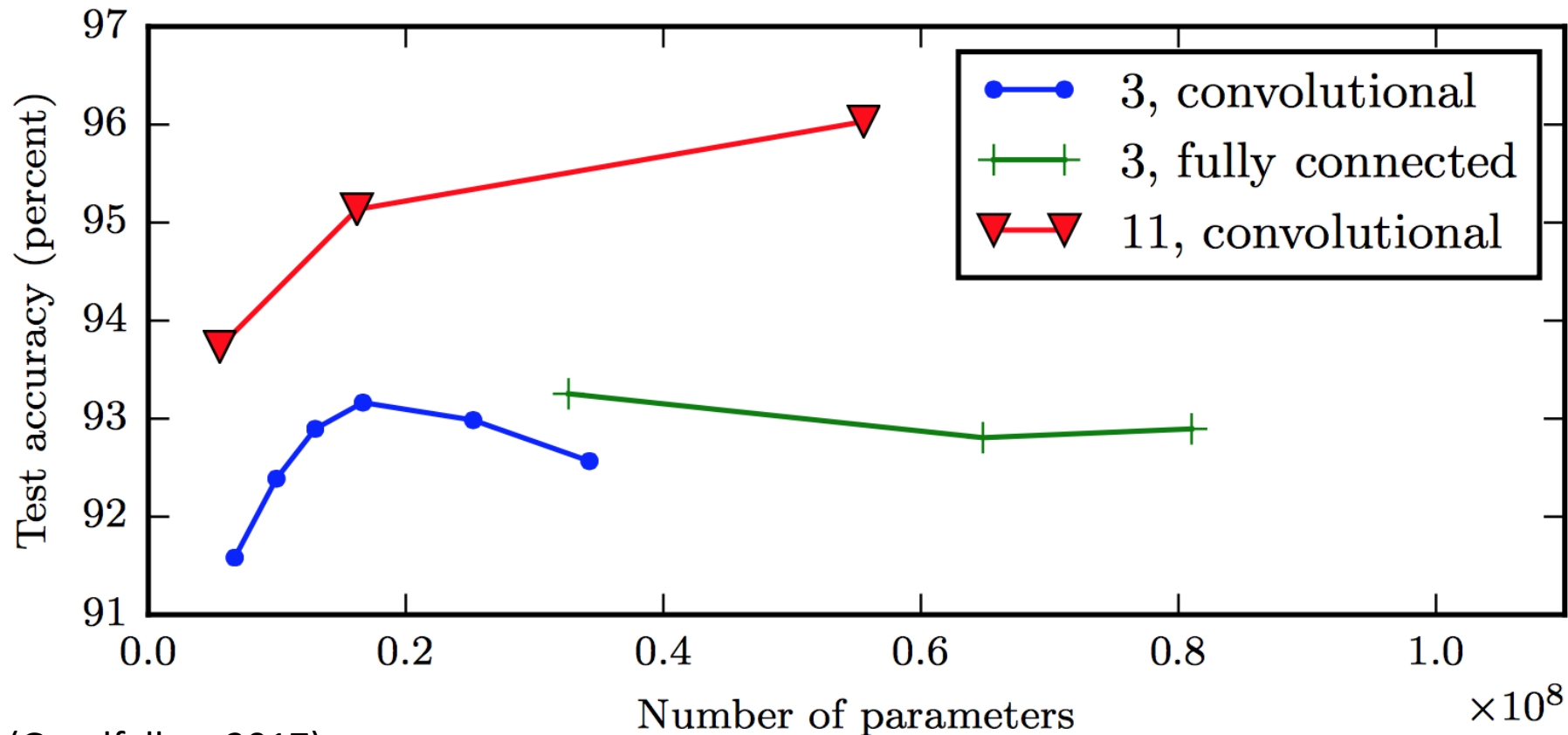
- So why deeper?

- Shallow net may need (exponentially) more width
- Shallow net may overfit more

Better generalization with depth



Overfitting in shallow nets



(Goodfellow 2017)

The **11-layer net** generalizes better on the test set when controlling for number of parameters.

The 3-layer nets perform worse on the test set, even with similar number of total parameters.

Training a feed-forward ANN

Initialisation of network weights $w_{1,1}^1, w_{1,2}^1, w_{1,3}^1, \dots, w_{1,n}^1, \dots, w_{H_{n-1}, H_n}^n, w_{1,1}^O, w_{1,2}^O, \dots, w_{r, H_n}^O$

While not stop condition

For each train example (x^d, t^d) , where $d = 1, 2, \dots, n$

Activate the network and determine the output o^d :

forward propagate the information and determine the output of each neuron

Modify the weights:

establish and backward propagate the error:

- establish the errors of neurons from the output layer $e_r, r=1, \dots, m$
- backward propagate the errors in the entire network $\rightarrow$ distribute the errors on all connections of the network
- modify the weights

EndWhile

Back-propagation of errors in an ANN

- One of the first algorithms for fine tuning of the weights in an ANN
- Very popular

Advantages:

- A gradient descent method
- does not require normalization of input vectors (normalization could improve performance)

Limitations:

- is not guaranteed to find the global minimum of the error function, but only a local minimum
- it has trouble crossing plateaus in the error function landscape.
- requires the derivatives of activation functions to be known at network design time.

Computing the derivatives of error

We consider an error function for the model $E(X, W, t)$

Compute the derivative of this function with respect to every weight $w_{j,k}^l$ (the weight from layer l , between neuron j from layer l and neuron k from layer $l-1$).

In general it is:

$$\frac{\partial E}{\partial w_{j,k}^l} = \frac{\partial E}{\partial o_j^l} \frac{\partial o_j^l}{\partial w_{j,k}^l} = \frac{\partial E}{\partial o_j^l} \frac{\partial o_j^l}{\partial \text{net}_j^l(X^l)} \frac{\partial \text{net}_j^l(X^l)}{\partial w_{j,k}^l}$$

Observe that in the last derivative we have a sum, but only one term depends on the weight so:

$$\frac{\partial \text{net}_j^l(X^l)}{\partial w_{j,k}^l} = \frac{\partial}{\partial w_{j,k}^l} \left(\sum_{q=1}^{n_{l-1}} x_q^l w_{j,q}^l \right) = \frac{\partial (x_k^l w_{j,k}^l)}{\partial w_{j,k}^l} = x_k^l = o_k^{l-1}$$

Here: $X^l = (x_1^l, x_2^l, \dots, x_{n_{l-1}}^l)$ is the input for layer l (aka the output o^{l-1} from the previous layer)

Computing the derivatives of error

For the first hidden layer we have a different situation: the input for this layer is the actual input in the network.

If we evaluate further the partial derivative we get:

$$\frac{\partial o_j^l}{\partial net_j^l(X^l)} = \frac{\partial \phi(net_j^l(X^l))}{\partial net_j^l(X^l)}$$

which is the partial derivative of the activation function ϕ - hence the importance of the derivative

In practice we begin from the last layer because the partial derivatives for this layer are straight forward, and we move backward from layer to layer until we reach the first hidden one.

Thank You!

Convolutional Neural Networks

Lecture 6

April 2, 2026

Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

Convolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Outline

Tensors

Convolution and Cross-Correlation

Convolution Layer Basics

Kernels

Example

Convolutional Network Structure

Convolutional Layer

- Feature learning

- Training the convolutional layer

Pooling Layer

Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

Convolutional
Layer

- Feature learning

- Training the
convolutional layer

Pooling Layer

Deep Convolutional Neural Networks (ConvNets)

ConvNets are deep neural networks widely used in computer vision tasks such as:

- ▶ image classification
- ▶ image retrieval and similarity-based search
- ▶ object detection and recognition in scenes

Examples include systems for face recognition, traffic sign recognition, tumor detection, and optical character recognition (OCR).

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Why ConvNets are useful

- ▶ ConvNets usually require less manual feature engineering than traditional image classification pipelines.
- ▶ They were inspired by the idea of local receptive fields in biological vision.
- ▶ They are designed to capture spatial structure in images.
- ▶ For video or sequence data, related architectures can also model spatiotemporal structure.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Tensors in deep learning

In deep learning, a **tensor** is simply a **multidimensional array**.

Common examples:

- ▶ scalar: a 0D tensor
- ▶ vector: a 1D tensor
- ▶ matrix: a 2D tensor
- ▶ image: often a 3D tensor (height $\times$ width $\times$ channels)
- ▶ mini-batch of images: often a 4D tensor

Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

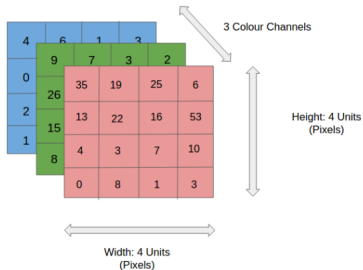
Convolutional
Layer

Feature learning
Training the
convolutional layer

Pooling Layer

Images as tensors

ConvNets ingest and process images as tensors.



Images are represented in a form that is easy to process while preserving important visual features.

Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

Convolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Convolution operation

For two one-dimensional functions $x(t)$ and $w(t)$, the mathematical convolution is defined by

$$s(t) = \int_{-\infty}^{\infty} x(a) w(t - a) da$$

with notation

$$s(t) = (x * w)(t)$$

If t is **discrete**, the integral becomes a sum:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a) w(t - a)$$

[Tensors](#)[Convolution and
Cross-Correlation](#)[Convolution
Layer Basics](#)[Kernels](#)[Example](#)[Convolutional
Network
Structure](#)[Convolutional
Layer](#)[Feature learning](#)[Training the
convolutional layer](#)[Pooling Layer](#)

Convolution Layer Hyperparameters

A convolutional layer is typically defined by:

- ▶ number of filters
- ▶ kernel size
- ▶ stride
- ▶ padding
- ▶ activation function
- ▶ optional bias term

Example:

*Conv2D(in_channels = 32, out_channels = 4,
kernel_size = (3,3), stride = 1, padding = 'same')*

Each filter learns a pattern that is useful for the task.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Convolution vs. cross-correlation in CNNs

In strict mathematical terms, convolution flips the kernel.

In most CNN implementations, the operation commonly called **convolution** is actually **cross-correlation**, which does **not** flip the kernel.

For a 2D input I and kernel K , cross-correlation is often written as

$$S(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

In deep learning practice, this operation is still usually called a *convolution* layer.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Cross-correlation in practice

In practice we work with two tensors:

- ▶ the input: a multidimensional array of data
- ▶ the kernel (or filter): a multidimensional array of learnable parameters

Because real images and kernels are finite, the summation is computed over a finite number of entries.

For a 2D input, the forward operation used in CNNs is commonly written as

$$S(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

This is the standard sliding-filter operation used by modern deep learning libraries.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Using a kernel

We want to apply a filter to an image.

Goal: **extract useful local features** such as edges, corners, or texture patterns.

Example image dimensions: $3 \times 3 \times 1$ (grayscale image)

Kernel / filter:

$$K = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Using a kernel

Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

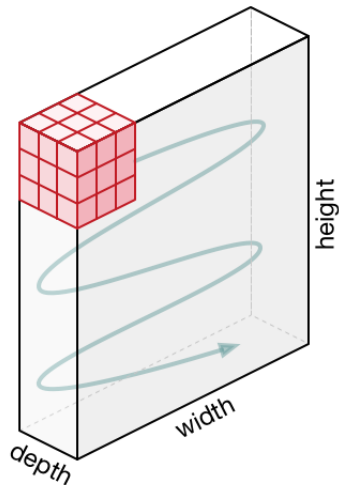
Convolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Using a kernel: movement of the kernel



- ▶ The kernel shifts across the image.
- ▶ At each position, we compute an **element-wise multiplication** between the kernel K and the image patch P , followed by a sum.
- ▶ This produces one value in the output feature map.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Stride

Stride is the number of pixels by which the kernel moves at each step.

- ▶ $\text{stride} = 1$: the kernel moves one pixel at a time
- ▶ $\text{stride} = 2$: the kernel skips one pixel between positions

Larger stride:

- ▶ reduces the spatial size of the output
- ▶ reduces computation
- ▶ may lose fine spatial details

Padding

Padding means adding extra values around the border of the input.

Common choices:

- ▶ **valid padding**: no padding
- ▶ **same padding**: padding chosen so that the spatial size is preserved for stride 1

Why padding is useful:

- ▶ it controls output size
- ▶ it allows border pixels to influence the output more fairly
- ▶ it helps preserve spatial information near image boundaries

[Tensors](#)[Convolution and
Cross-Correlation](#)[Convolution
Layer Basics](#)[Kernels](#)[Example](#)[Convolutional
Network
Structure](#)[Convolutional
Layer](#)[Feature learning](#)[Training the
convolutional layer](#)[Pooling Layer](#)

Output Size of a Convolution Layer

For an input of size $H_{in} \times W_{in}$, kernel size $K_h \times K_w$, padding P_h, P_w , and stride S_h, S_w :

$$H_{out} = \left\lfloor \frac{H_{in} - K_h + 2P_h}{S_h} \right\rfloor + 1 \quad W_{out} = \left\lfloor \frac{W_{in} - K_w + 2P_w}{S_w} \right\rfloor + 1$$

Example:

- ▶ input: 5×5
- ▶ padding: 0
- ▶ kernel: 3×3
- ▶ stride: 1

$$H_{out} = W_{out} = \left\lfloor \frac{5 - 3 + 0}{1} \right\rfloor + 1 = 3$$

So the output is 3×3 .

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Output Size Example with Stride

Let the input be 5×5 , kernel size 3×3 , padding 0, stride 2.

$$H_{out} = W_{out} = \left\lceil \frac{5 - 3 + 0}{2} \right\rceil + 1 = [1] + 1 = 2$$

So the output size is:

$$2 \times 2$$

This shows that increasing the stride reduces the spatial resolution of the output.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Multi-Channel Convolution

Images often have multiple channels.

Example:

- ▶ grayscale image: $H \times W \times 1$
- ▶ RGB image: $H \times W \times 3$

If the input has 3 channels, then each filter must also span all 3 channels.

So, for an RGB image, one filter has shape:

$$K_h \times K_w \times 3$$

One filter produces one output feature map. Using many filters produces many output channels.

[Tensors](#)[Convolution and
Cross-Correlation](#)[Convolution
Layer Basics](#)[Kernels](#)[Example](#)[Convolutional
Network
Structure](#)[Convolutional
Layer](#)[Feature learning](#)[Training the
convolutional layer](#)[Pooling Layer](#)

Multi-Channel Convolution Formula

For an input tensor X and a kernel tensor K , a common CNN operation is:

$$Y(i, j, c_{out}) = \sum_{c_{in}} \sum_m \sum_n X(i + m, j + n, c_{in}) K(m, n, c_{in}, c_{out})$$

Interpretation:

- ▶ we slide the filter spatially over the image
- ▶ at each position, we combine information from all input channels
- ▶ each output channel corresponds to one learned filter

[Tensors](#)[Convolution and Cross-Correlation](#)[Convolution Layer Basics](#)[Kernels](#)[Example](#)[Convolutional Network Structure](#)[Convolutional Layer](#)[Feature learning](#)[Training the convolutional layer](#)[Pooling Layer](#)

Bias Term in a Convolution Layer

In practice, a convolution layer usually includes a bias term.
For each output channel:

$$Y(i, j, c_{out}) = \sum_{c_{in}} \sum_m \sum_n X(i + m, j + n, c_{in}) K(m, n, c_{in}, c_{out}) + b_{c_{out}}$$

The bias:

- ▶ is one scalar per output channel
- ▶ is learned during training
- ▶ shifts the activation values before the nonlinearity

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Why CNNs Work Well for Images

Two important ideas make CNNs efficient:

1. Sparse connectivity

- ▶ each neuron sees only a local region of the input
- ▶ this local region is called the receptive field

2. Parameter sharing

- ▶ the same filter is reused across many spatial locations
- ▶ this greatly reduces the number of trainable parameters

This makes CNNs much more suitable for images than fully connected layers.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Receptive Field

The **receptive field** of a neuron is the region of the input that influences its value.

Examples:

- ▶ one 3×3 convolution layer has a local receptive field of 3×3
- ▶ two stacked 3×3 layers give an effective receptive field of 5×5
- ▶ three stacked 3×3 layers give an effective receptive field of 7×7

Deeper layers therefore capture information from larger portions of the original image.

[Tensors](#)[Convolution and Cross-Correlation](#)[Convolution Layer Basics](#)[Kernels](#)[Example](#)[Convolutional Network Structure](#)[Convolutional Layer](#)[Feature learning](#)[Training the convolutional layer](#)[Pooling Layer](#)

Why an Activation Function is Necessary

A convolution is a linear operation.

If we stack only linear layers, then the whole network is still linear.

Therefore, we add a nonlinear activation function such as ReLU:

$$\text{ReLU}(x) = \max(0, x)$$

This allows the network to learn:

- ▶ complex patterns
- ▶ hierarchical features
- ▶ nonlinear decision boundaries

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Translation Equivariance and Pooling

Convolutional layers are approximately **translation equivariant**:

- ▶ if the input shifts, the feature map also shifts

Pooling can introduce some local **translation invariance**:

- ▶ small shifts in the input may produce similar pooled outputs

This is useful when we care more about whether a feature is present than its exact location.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Using a kernel with 3 channels

Example:

Convolutional
Neural Networks

Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

Convolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Numerical Example: Input and Kernel

We consider the standard CNN forward operation, which in practice is cross-correlation.

Input:

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 2 & 1 & 0 \end{bmatrix}$$

Kernel:

$$K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

We use:

- ▶ stride = 1
- ▶ padding = 0
- ▶ no bias

Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

Convolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Numerical Example: Output Size

The input has size 4×4 and the kernel has size 2×2 .

Using the output size formula:

$$H_{out} = \left\lceil \frac{H_{in} - K_h + 2P_h}{S_h} \right\rceil + 1 \quad W_{out} = \left\lceil \frac{W_{in} - K_w + 2P_w}{S_w} \right\rceil + 1$$

Since $H_{in} = 4$, $W_{in} = 4$, $K_h = K_w = 2$, $P_h = P_w = 0$,
 $S_h = S_w = 1$

we obtain: $H_{out} = W_{out} = \left\lceil \frac{4-2+0}{1} \right\rceil + 1 = 3$

So the output feature map has size: 3×3

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Numerical Example: First Output Value

The first output value is computed from the top-left 2×2 patch:

$$\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} \text{ and the kernel } \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

So:

$$Y_{1,1} = 1 \cdot 1 + 2 \cdot 0 + 3 \cdot (-1) + 1 \cdot 1$$

$$Y_{1,1} = 1 + 0 - 3 + 1 = -1$$

This gives the first value of the output feature map.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Numerical Example: Second and Third Values

For the next position, the kernel moves one step to the right.

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

$$Y_{1,2} = 2 \cdot 1 + 0 \cdot 0 + 1 \cdot (-1) + 2 \cdot 1 = 2 + 0 - 1 + 2 = 3$$

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

ExampleConvolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Numerical Example: Second and Third Values

For the third position in the first row:

$$\begin{bmatrix} 0 & 1 \\ 2 & 2 \end{bmatrix}$$

$$Y_{1,3} = 0 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1) + 2 \cdot 1 = 0 + 0 - 2 + 2 = 0$$

So the first row of the output is:

$$[-1 \quad 3 \quad 0]$$

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

ExampleConvolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Numerical Example: Remaining Output Values

Second row:

$$Y_{2,1} = \begin{bmatrix} 3 & 1 \\ 0 & 1 \end{bmatrix} \star \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = 3 \cdot 1 + 1 \cdot 0 + 0 \cdot (-1) + 1 \cdot 1 = 4$$
$$Y_{2,2} = \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix} \star \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = 1 + 0 - 1 + 3 = 3$$
$$Y_{2,3} = \begin{bmatrix} 2 & 2 \\ 3 & 1 \end{bmatrix} \star \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = 2 + 0 - 3 + 1 = 0$$

So the second row is: $[4 \quad 3 \quad 0]$

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Numerical Example: Final Output

Third row:

$$Y_{3,1} = \begin{bmatrix} 0 & 1 \\ 2 & 2 \end{bmatrix} \star \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = 0 + 0 - 2 + 2 = 0$$

$$Y_{3,2} = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix} \star \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = 1 + 0 - 2 + 1 = 0$$

$$Y_{3,3} = \begin{bmatrix} 3 & 1 \\ 1 & 0 \end{bmatrix} \star \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = 3 + 0 - 1 + 0 = 2$$

Therefore, the final output feature map is: $Y = \begin{bmatrix} -1 & 3 & 0 \\ 4 & 3 & 0 \\ 0 & 0 & 2 \end{bmatrix}$

Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

Convolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Numerical Example with Bias and ReLU

In practice, a convolutional layer often adds a bias and then applies a nonlinear activation.

Suppose the bias is: $b = 1$

Then:

$$Y + b = \begin{bmatrix} 0 & 4 & 1 \\ 5 & 4 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Numerical Example with Bias and ReLU

If we apply ReLU,

$$\text{ReLU}(x) = \max(0, x)$$

the output remains:

$$\text{ReLU}(Y + b) = \begin{bmatrix} 0 & 4 & 1 \\ 5 & 4 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

This illustrates the typical sequence:

cross-correlation $\rightarrow$ bias $\rightarrow$ activation

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Layers of a convolutional network

A common processing pattern is:

- ▶ one or more convolutional (cross-correlation) layers
- ▶ a nonlinear activation function such as ReLU
- ▶ a pooling or subsampling operation

Examples of pooling include max pooling, average pooling, and global average pooling.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

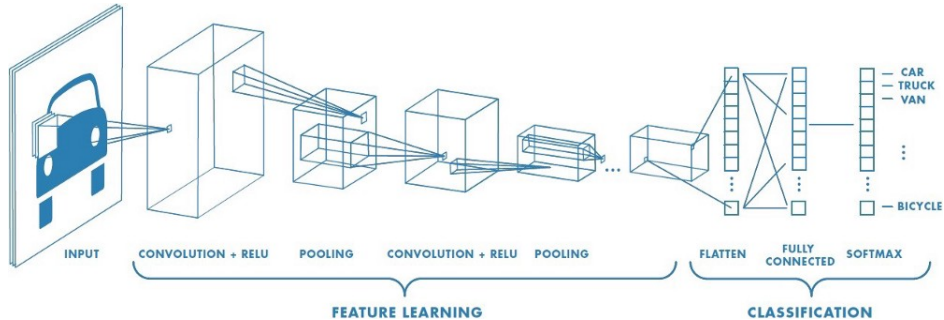
Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Feature learning



Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

Convolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

A Typical CNN Architecture

A common image classification pipeline is:

Input → Conv → ReLU → Conv → ReLU
→ Pooling → Flatten / Global Average Pooling
→ Dense → Softmax

Feature hierarchy:

- ▶ early layers learn edges and textures
- ▶ middle layers learn parts and motifs
- ▶ deeper layers learn object-level patterns

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer**Feature learning**Training the
convolutional layer

Pooling Layer

Training the network

- ▶ Training is similar in spirit to other neural networks.
- ▶ The loss is computed at the output.
- ▶ Gradients are propagated backward through the layers.
- ▶ Parameters are updated using gradient-based optimization methods such as gradient descent or its variants.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer**Feature learning**Training the
convolutional layer

Pooling Layer

Feature learning

Applying the following filter to an image produces strong responses at **vertical edges**. It acts as a vertical edge detector.

1.0	0.0	-1.0
2.0	0.0	-2.0
1.0	0.0	-1.0

Table: A 3×3 Sobel filter for detecting vertical edges

Sliding this filter across the image highlights regions with strong vertical intensity changes.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Feature learning

-2.0	-2.0	-4.0	-2.0	-2.0
-1.0	-1.0	-2.0	-1.0	-1.0
0.0	0.0	0.0	0.0	0.0
1.0	1.0	2.0	1.0	1.0
2.0	2.0	4.0	2.0	2.0

Table: An example of a horizontal line detector

- ▶ Different filters respond to different visual patterns.
- ▶ Combining multiple feature maps allows the network to detect richer image structure.
- ▶ In CNNs, these filters are typically learned automatically from data.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer**Feature learning**Training the
convolutional layer

Pooling Layer

Feature learning

- ▶ The values of a filter are weights learned during training.
- ▶ During gradient-based optimization, the network learns which local patterns are useful for the task.
- ▶ The learned filters are those that help minimize the loss function.

Gradient descent on a convolutional layer

Consider:

- ▶ the input

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{bmatrix}$$

- ▶ the filter

$$F = \begin{bmatrix} F_{1,1} & F_{1,2} \\ F_{2,1} & F_{2,2} \end{bmatrix}$$

- ▶ the output of the forward operation, denoted by O
- ▶ the upstream gradient $\frac{\partial E}{\partial O}$

Tensors

Convolution and
Cross-Correlation

Convolution
Layer Basics

Kernels

Example

Convolutional
Network
Structure

Convolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Forward computation

Using the standard CNN forward operation (cross-correlation), we obtain:

$$O_{1,1} = x_{1,1}F_{1,1} + x_{1,2}F_{1,2} + x_{2,1}F_{2,1} + x_{2,2}F_{2,2}$$

$$O_{1,2} = x_{1,2}F_{1,1} + x_{1,3}F_{1,2} + x_{2,2}F_{2,1} + x_{2,3}F_{2,2}$$

$$O_{2,1} = x_{2,1}F_{1,1} + x_{2,2}F_{1,2} + x_{3,1}F_{2,1} + x_{3,2}F_{2,2}$$

$$O_{2,2} = x_{2,2}F_{1,1} + x_{2,3}F_{1,2} + x_{3,2}F_{2,1} + x_{3,3}F_{2,2}$$

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Partial derivatives with respect to the filter

We compute partial derivatives of the output with respect to filter entries:

$$\frac{\partial O_{1,1}}{\partial F_{1,1}} = x_{1,1}, \quad \frac{\partial O_{1,1}}{\partial F_{1,2}} = x_{1,2}, \quad \frac{\partial O_{1,1}}{\partial F_{2,1}} = x_{2,1}, \quad \frac{\partial O_{1,1}}{\partial F_{2,2}} = x_{2,2}$$

Similarly, we compute the derivatives for $O_{1,2}$, $O_{2,1}$, and $O_{2,2}$.

By the chain rule,

$$\frac{\partial E}{\partial F_{u,v}} = \sum_{i,j} \frac{\partial E}{\partial O_{i,j}} \frac{\partial O_{i,j}}{\partial F_{u,v}}$$

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Example: derivative with respect to one filter entry

For example,

$$\begin{aligned}\frac{\partial E}{\partial F_{1,1}} &= \frac{\partial E}{\partial O_{1,1}} \frac{\partial O_{1,1}}{\partial F_{1,1}} + \frac{\partial E}{\partial O_{1,2}} \frac{\partial O_{1,2}}{\partial F_{1,1}} \\ &\quad + \frac{\partial E}{\partial O_{2,1}} \frac{\partial O_{2,1}}{\partial F_{1,1}} + \frac{\partial E}{\partial O_{2,2}} \frac{\partial O_{2,2}}{\partial F_{1,1}} \\ &= \frac{\partial E}{\partial O_{1,1}} x_{1,1} + \frac{\partial E}{\partial O_{1,2}} x_{1,2} + \frac{\partial E}{\partial O_{2,1}} x_{2,1} + \frac{\partial E}{\partial O_{2,2}} x_{2,2}\end{aligned}$$

This shows how the upstream gradient and the input jointly determine the filter update.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Gradient with respect to the filter

Collecting all partial derivatives, we obtain the gradient with respect to the filter:

$$\begin{bmatrix} \frac{\partial E}{\partial F_{1,1}} & \frac{\partial E}{\partial F_{1,2}} \\ \frac{\partial E}{\partial F_{2,1}} & \frac{\partial E}{\partial F_{2,2}} \end{bmatrix}$$

In implementation terms, this gradient can be expressed as a sliding operation between the input X and the upstream gradient $\frac{\partial E}{\partial O}$.

Depending on the indexing convention, this operation is written as either a correlation or a convolution-like expression.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Gradient with respect to the input

Similarly, the gradient with respect to the input can be written as

$$\begin{bmatrix} \frac{\partial E}{\partial x_{1,1}} & \frac{\partial E}{\partial x_{1,2}} & \frac{\partial E}{\partial x_{1,3}} \\ \frac{\partial E}{\partial x_{2,1}} & \frac{\partial E}{\partial x_{2,2}} & \frac{\partial E}{\partial x_{2,3}} \\ \frac{\partial E}{\partial x_{3,1}} & \frac{\partial E}{\partial x_{3,2}} & \frac{\partial E}{\partial x_{3,3}} \end{bmatrix} = \begin{bmatrix} F_{2,2} & F_{2,1} \\ F_{1,2} & F_{1,1} \end{bmatrix} * \frac{\partial E}{\partial O}$$

where the filter is rotated by 180° .

This flipped filter appears naturally when computing how changes in the output affect the input.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Note: Stride and Pooling

Historically, many CNNs used pooling layers to reduce spatial size.

For example:

- ▶ convolution
- ▶ ReLU
- ▶ max pooling

A strided convolution can both:

- ▶ extract features

in a single operation.

In modern architectures, spatial downsampling is often performed by:

- ▶ pooling layers, or
- ▶ convolutions with stride greater than 1

- ▶ reduce spatial resolution

[Tensors](#)[Convolution and Cross-Correlation](#)[Convolution Layer Basics](#)[Kernels](#)[Example](#)[Convolutional Network Structure](#)[Convolutional Layer](#)[Feature learning](#)[Training the convolutional layer](#)[Pooling Layer](#)

Pooling is Useful, but Not Mandatory

Pooling is still useful because it:

- ▶ reduces feature map size
- ▶ lowers computation
- ▶ can improve robustness to small local shifts

However, pooling is not mandatory in every CNN.

Many modern architectures use:

- ▶ strided convolutions for downsampling
- ▶ global average pooling near the output instead of flattening large tensors

So, in current practice:

- ▶ pooling is a common design choice
- ▶ but it is no longer the only standard way to reduce resolution

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Design Choice: Strided Convolution vs. Pooling

Both approaches reduce spatial resolution, but they play slightly different roles.

Pooling

- ▶ fixed operation
- ▶ no learned weights
- ▶ often used for simple spatial summarization

Strided convolution

- ▶ learned operation
- ▶ can downsample while also learning useful filters
- ▶ widely used in modern CNN architectures

In practice, one chooses depending on the task and model design.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

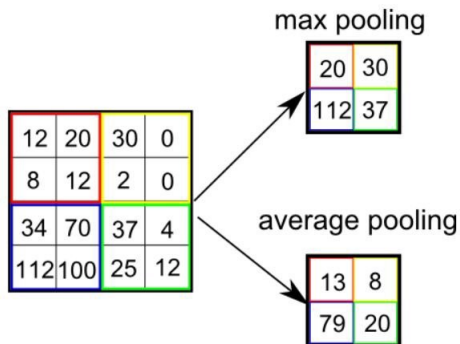
Pooling

Pooling reduces the spatial resolution of feature maps and makes the representation more robust to small translations of the input.

It is useful when we care more about whether a feature is present than about its exact location.

Pooling is also a common way to reduce computation and control feature map size.

Pooling



- ▶ **Max pooling:** returns the maximum value in the pooling window.
- ▶ **Average pooling:** returns the average value in the pooling window.

Training the pooling layer

For max pooling over an $N \times N$ region, during backpropagation the gradient is passed only to the input element that achieved the maximum value in the forward pass.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Practical Notes and Limitations

In practice, CNNs are often combined with:

- ▶ batch normalization
- ▶ dropout
- ▶ data augmentation
- ▶ weight decay (L2 regularization)

Some limitations:

- ▶ they may require large amounts of data
- ▶ training can be computationally expensive
- ▶ they may struggle with very long-range dependencies

Despite this, CNNs remain a core tool in computer vision.

Tensors

Convolution and
Cross-CorrelationConvolution
Layer Basics

Kernels

Example

Convolutional
Network
StructureConvolutional
Layer

Feature learning

Training the
convolutional layer

Pooling Layer

Artificial Intelligence

Lecture slides - 7

Tudor Mihoc
UBB

May 31, 2026

What You Will Learn in This Particular Lecture:

- Batches
- Recurrent neural networks,
- LSTM networks
- Resnet
- Mechanisms of attention
- Autoencoders
- Examples

Batches

What Is a Batch?

- In deep learning, a **batch** is a small group of training examples processed together.
- Instead of updating the model after only one example, we usually process multiple examples at once.
- This makes training more efficient on modern hardware such as GPUs.
- A batch is one subset of the training data used in a single forward and backward pass.

Example:

- Dataset size: 10,000 samples
- Batch size: 32
- The model processes 32 samples at a time

Why use batches?

- faster computation
- more stable gradient estimates
- better memory control than using the whole dataset at once

How Batches Are Used in Sequence Models

- In sequence models, inputs are often stored as 3D tensors.
- A common shape in PyTorch is: (batch size, sequence length, input size)

Meaning:

- **batch size:** number of sequences processed together
- **sequence length:** number of time steps in each sequence
- **input size:** number of features at each time step

Example:

(4, 6, 10)

- 4 sequences in one batch
- each sequence has 6 time steps
- each time step has 10 features

Batches allow the model to learn from many sequences in parallel during training.

How Batches Are Used in CNNs

- CNNs process images in batches instead of one image at a time.
- The input is typically a 4D tensor: (batch size, channels, height, width)

Example:

(32, 3, 64, 64)

- 32 images are processed together
- each image has 3 channels (RGB)
- each image has size 64×64

During training:

- convolutional filters are applied to every image in the batch
- the model computes predictions for all images
- the loss is aggregated across the batch
- parameters are updated using the average gradient

Recurrent Neural Networks

RNNs: Why Sequence Models?

- Many tasks depend on order: text, speech, sensor streams, DNA, logs.
- A feed-forward network sees inputs independently.
- A recurrent neural network (RNN) reuses the same cell across time steps.
- Hidden state acts as a compressed memory of previous inputs.

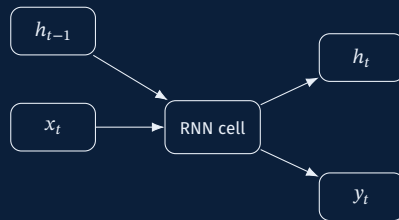
A model processes $x_1, x_2, \dots, x_T$ while carrying context from past to future.

Basic RNN Cell

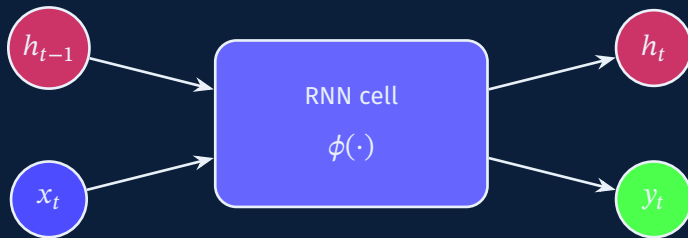
$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

- x_t : input at time t
- h_t : hidden state
- y_t : output
- ϕ : usually tanh or ReLU



Compact View of an RNN Cell



$$h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad y_t = W_{hy}h_t + b_y$$

Simple RNN in PyTorch

```
import torch

# input: (batch_size,
#         sequence_length, input_size)
x = torch.randn(4, 6, 10)

rnn = torch.nn.RNN(
    input_size=10,
    hidden_size=8,
    num_layers=1,
    batch_first=True)

output, h_n = rnn(x)
```

```
print(output.shape)
# (4, 6, 8)
```

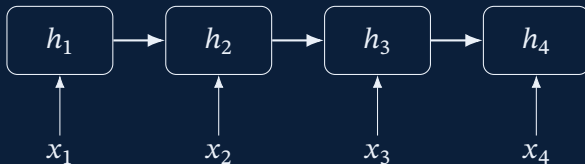
Input shape: (batch, seq_len, input_size)

Output shape: (batch, seq_len, hidden_size)

Final hidden state:
(num_layers, batch, hidden_size)

Unrolling Through Time

same weights reused



- The same parameters are applied at every time step.
- This enables variable-length inputs.
- It also creates long computational chains, which complicate training.

Typical Input-Output Patterns

Pattern	Shape	Example
One-to-many	single input → sequence	image captioning
Many-to-one	sequence → label	sentiment analysis
Many-to-many	aligned sequence output	part-of-speech tagging
Seq2seq	sequence → sequence	translation, summarization

RNNs became popular because one framework can handle several sequence settings. Humans do love turning every problem into arrows in boxes.

Training With Backpropagation Through Time

- Forward pass computes states from $t = 1$ to T .
- Backpropagation Through Time (BPTT) unfolds the network and propagates gradients backward.
- Full BPTT is expensive for long sequences.
- Truncated BPTT updates parameters on shorter windows to reduce memory cost.

Optimization challenge

The gradient is repeatedly multiplied by recurrent weights, which can shrink or explode.

Vanishing and Exploding Gradients

- **Vanishing:** early inputs have almost no influence on the gradient.
- **Exploding:** gradient norm becomes extremely large.
- Long-term dependencies become hard to learn.

$$\frac{\partial L}{\partial h_t} \propto \prod_{k=t+1}^T W_{hh}^T D_k$$

Repeated matrix products are the culprit.

Common remedies

Gradient clipping, careful initialization, gated architectures, layer normalization.

Practical Variants of RNNs

- **Bidirectional RNN:** reads sequence left-to-right and right-to-left.
- **Stacked RNN:** multiple recurrent layers for richer representations.
- **Character-level RNN:** models raw text characters instead of words.
- **Encoder-decoder RNN:** compresses one sequence and generates another.

Trade-off

More context and depth improve expressiveness, but increase training time and instability.

Applications of Classical RNNs

- language modeling
- speech recognition
- handwriting generation
- anomaly detection in time series
- music generation
- log/event forecasting
- tagging and chunking
- machine translation

Main limitation

Plain RNNs struggle when the useful signal is far away in the sequence.

Improving Classical RNN Training

- Gradient clipping limits unstable updates.
- Better initialization keeps activations in a useful range.
- Teacher forcing helps decoder-style RNNs learn faster.
- Sequence padding and masking are essential in minibatch training.

Practical note

Even a simple RNN can work well on short sequences when training is carefully controlled.

Strengths and Weaknesses of RNNs

Strengths

- handles variable-length data
- parameter sharing across time
- natural for temporal reasoning

Weaknesses

- hard to parallelize
- unstable gradients
- weak long-range memory

RNNs were an essential step, but they needed a better memory mechanism. Enter LSTM, because apparently one gate was not enough and humans built four.

LSTM Networks

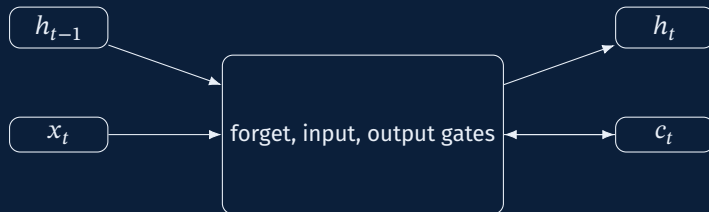
Why LSTM?

- Long Short-Term Memory (LSTM) was designed to preserve information for longer periods.
- It introduces a dedicated cell state c_t and gating mechanisms.
- Gates decide what to forget, what to store, and what to expose.

Goal

Make gradient flow easier across many time steps while keeping sequence modeling power.

LSTM Architecture at a Glance



- Hidden state h_t is the exposed output.
- Cell state c_t is the internal memory highway. c_t is passed to the next time step as internal memory.

The Forget Gate

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

- f_t contains values in $[0, 1]$.
- It controls how much of the old cell state c_{t-1} should be kept.
- Values near 1 preserve memory; values near 0 erase it.

Intuition

The network learns when old context is still useful and when it should be discarded.

The Input Gate and Candidate Memory

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

- i_t decides what new content should enter memory.
- $\tilde{c}_t$ proposes candidate information.

Update rule contribution

Only selected candidate values are written into the cell state.

Updating the Cell State

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

- The first term preserves previous memory.
- The second term writes new information.
- Additive updates help gradients travel more reliably through time.

The cell state behaves like a controlled memory conveyor belt instead of a fragile echo.

$\odot$ denotes element-wise multiplication

$$[a_1, a_2, a_3] \odot [b_1, b_2, b_3] = [a_1 b_1, a_2 b_2, a_3 b_3]$$

The Output Gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t)$$

- The output gate decides how much internal memory becomes visible.
- The hidden state is a filtered view of the cell state.

Effect

LSTMs can keep information internally without exposing all of it at every step.

Complete LSTM Equation Set

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

Why LSTMs Learn Long-Term Dependencies Better

- Cell-state updates are partly additive rather than fully multiplicative.
- Gates provide data-dependent control over memory retention.
- Important signals can persist across dozens or hundreds of steps.

Typical gains over plain RNNs

Better language modeling, stronger sequence classification, and more stable training on long sequences.

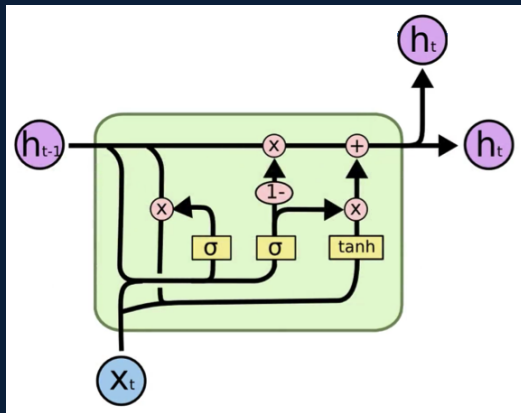
Useful LSTM Variants

- **Stacked LSTM:** multiple LSTM layers.
- **Bidirectional LSTM:** accesses past and future context.
- **Peephole LSTM:** gates can inspect the cell state directly.
- **Seq2seq LSTM:** encoder-decoder setup for translation and generation.

Design principle

Choose variants based on context needs, latency constraints, and sequence length.

Example of a LSTM Variant Node



Typical Applications of LSTMs

- sentiment analysis
- named entity recognition
- speech and phoneme modeling
- machine translation
- caption generation
- demand forecasting
- medical signal analysis
- trajectory prediction

LSTMs were the default sequence workhorse before attention-based models took the spotlight.

Residual Networks (ResNet)

Why Deep CNNs Need Residual Learning

- Deeper networks should represent more complex functions.
- In practice, very deep plain CNNs become hard to optimize.
- Accuracy can saturate or degrade as depth increases.

Residual idea

Learn a residual mapping $F(x)$ and output $y = F(x) + x$ instead of learning y directly.

A Residual Block



- Skip connection creates a short path for information and gradients.
- Identity mapping is easy to preserve if deeper transformations are not needed.

Why Skip Connections Help

- Gradients can pass through the identity path more directly.
- Optimization becomes easier in very deep models.
- The network learns refinements relative to the input representation.

Interpretation

Residual blocks act like iterative feature refinement rather than complete re-computation.

ResNet Families

Model	Typical depth	Notes
ResNet-18 / 34	moderate	basic blocks
ResNet-50 / 101 / 152	deep	bottleneck blocks
Wide ResNet	moderate	wider channels
ResNeXt	deep	grouped convolutions

Residual design influenced much of modern computer vision.

Bottleneck Blocks and Scaling

- Bottleneck block uses 1×1 , 3×3 , 1×1 convolutions.
- First layer reduces channels, middle layer processes, last restores dimension.
- This enables very deep networks with manageable compute.

Engineering lesson

Architectural structure matters as much as raw parameter count.

Attention Mechanisms

Why Attention?

- Fixed-size hidden states can bottleneck sequence models.
- Attention lets the model focus on the most relevant parts of the input.
- Instead of compressing everything into one vector, it computes context on demand.

Benefit:

Dynamic information retrieval improves long-range reasoning and interpretability.

Query, Key, Value Intuition

- **Query:** what the current position is looking for.
- **Keys:** descriptors of available information.
- **Values:** the actual information to combine.

$$\text{Attention}(Q, K, V) = \text{weights}(Q, K) \cdot V$$

Analogy

Query asks a question, keys tell where answers might be, values carry the content.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Dot products measure relevance between queries and keys.
- Softmax turns scores into weights.
- Scaling by $\sqrt{d_k}$ stabilizes large-dimensional dot products.

Self-Attention and Multi-Head Attention

- In self-attention, queries, keys, and values come from the same sequence.
- Each token can directly interact with every other token.
- Multi-head attention learns several relation patterns in parallel.

Result

Models can capture syntax, semantic similarity, and positional dependencies simultaneously.

Cross-Attention and Practical Limits

- Cross-attention links one sequence to another, e.g. decoder to encoder states.
- It is central in translation, captioning, and multimodal models.
- Main limitation: quadratic cost with sequence length in vanilla attention.

Takeaway

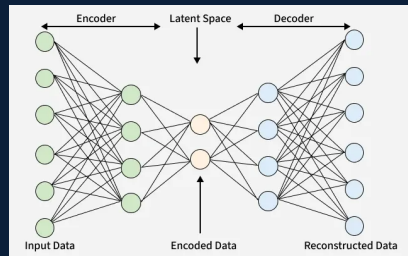
Attention solved many memory problems of RNNs.

Autoencoders

What Is an Autoencoder?

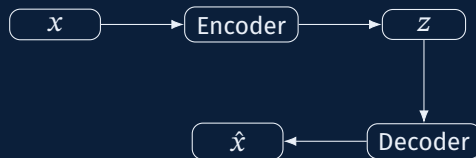
- An autoencoder learns to reconstruct its input.
- It has two parts: an **encoder** and a **decoder**.
- Training forces the model to learn compact structure in the data.

$$x \rightarrow z \rightarrow \hat{x}$$



Encoder, Latent Code, Decoder

- Encoder maps input x to latent vector z .
- Decoder maps z back to reconstruction $\hat{x}$.
- The latent space can capture useful low-dimensional features.



Training Objective

- Minimize reconstruction error between input and output.
- Common losses:
 - Mean squared error for continuous data
 - Binary cross-entropy for normalized binary-like inputs
- The choice of bottleneck and regularization determines what the model learns.

$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2 \quad \text{or related reconstruction losses}$$

Undercomplete vs Overcomplete Autoencoders

Type	Latent dimension	Risk
Undercomplete	smaller than input	may lose detail
Overcomplete	larger than input	may copy input too easily

Key point

Without constraints, an autoencoder can learn a trivial.

Regularized Autoencoders

- **Sparse autoencoder:** encourages most hidden activations to stay near zero.
- **Contractive autoencoder:** penalizes sensitivity of hidden features to small input changes.
- **Denoising autoencoder:** reconstructs clean input from corrupted input.

Purpose

Regularization forces the latent space to represent meaningful structure instead of simple copying.

Denoising Autoencoders

$\tilde{x} = x + \text{noise}$, train decoder to recover x

- Inputs are intentionally corrupted.
- The model learns robust features useful for recovery.
- This often improves generalization and feature quality.

Use case

Image denoising, missing-value repair, robust representation learning.

Variational Autoencoders (VAEs)

- VAE is a probabilistic autoencoder.
- Encoder outputs a distribution, often Gaussian parameters μ and σ .
- Sampling is made differentiable using the reparameterization trick.

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Benefit

VAEs support smooth latent spaces and generative sampling.

Latent Space and Representation Learning

- Nearby latent vectors often represent similar inputs.
- Interpolation in latent space can reveal learned structure.
- Autoencoders are useful for dimensionality reduction beyond linear PCA.

What makes a good latent space?

Compact, smooth, informative, and useful for downstream tasks.

Applications of Autoencoders

- compression
- anomaly detection
- denoising
- recommendation features
- pretraining
- medical imaging support
- generative modeling
- representation learning

Autoencoders are especially useful when labels are scarce but data is abundant.

Limitations of Autoencoders

- Good reconstruction does not guarantee task-relevant features.
- Powerful decoders can ignore weak latent codes.
- VAEs may produce blurry outputs.
- Modern self-supervised methods often outperform plain autoencoders for representation learning.

Lesson

Autoencoders remain valuable, but they are not a universal shortcut to useful embeddings.

Examples

Example 1: Sentiment Analysis With BiLSTM

- Input: tokenized movie review.
- Embedding layer converts words to vectors.
- Bidirectional LSTM captures left and right context.
- Final classifier predicts positive or negative sentiment.

Why this model?

Sentence meaning often depends on earlier and later words, including negation and contrast.

Example 2: Machine Translation With Attention

- Encoder reads source sentence.
- Decoder generates target sentence one token at a time.
- Attention aligns each generated word with relevant source positions.

Benefit

The decoder no longer depends on a single fixed-size summary vector.

Example 3: Image Classification With ResNet

- Input image passes through convolutional and residual blocks.
- Deeper layers learn increasingly abstract visual features.
- Global pooling and a classifier produce class probabilities.

Why ResNet?

Residual connections make deep vision models trainable and reusable for transfer learning.

Example 4: Autoencoder for Anomaly Detection

- Train autoencoder on normal system behavior.
- At test time, unusual patterns reconstruct poorly.
- Large reconstruction error can flag anomalies.

Typical domains

Industrial sensors, fraud signals, network traffic, predictive maintenance.

Example 5: Choosing the Right Model

Task	Suitable model family
short temporal patterns	RNN / GRU
long sequential dependencies	LSTM or attention-based model
very deep image recognition	ResNet
feature compression / anomaly detection	autoencoder
mapping between sequence elements	attention mechanisms

There is no universally best model.

ARTIFICIAL INTELLIGENCE



Solving search problems

Evolutionary Algorithms

Nature-inspired search

Best method for solving a problem

- Human brain
Has created the wheel, car, town, etc.
- Mechanism of evolution
Has created the human brain

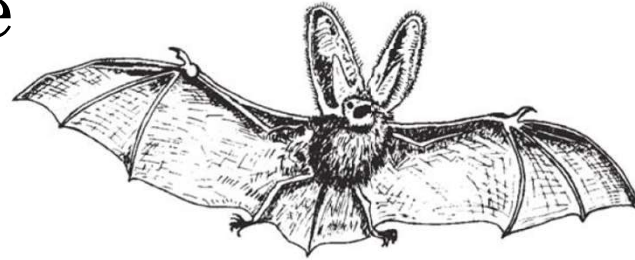
Simulation of nature

- By machines' help → the artificial neural networks simulate the brain
Flying vehicles, DNA computers, membrane-based computers
- By algorithms' help
Evolutionary algorithms simulate the evolution of nature
Particle Swarm Optimisation simulates the collective and social behaviour
Ant Colony Optimisation

Basic elements

■ Simulation of nature

■ Fly of bats



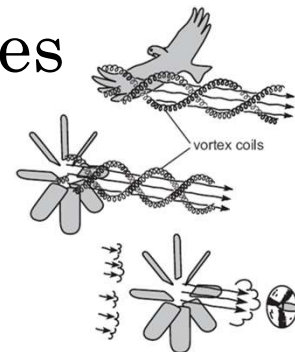
■ Leonardo da Vinci – sketch of a flying machine



■ Flies of birds and planes



■ Flies of birds and wind-turbines



Basic elements

Main characteristics of EAs

- Iterative and parallel processes
- Based on random search
- Bio-inspired – involve mechanisms as:

Natural selection

Reproduction

Recombination

Mutation

Basic elements

Historical points

■ Jean Baptise de Lamarck (1744-1829)

■ Has proposed in 1809 an explanation

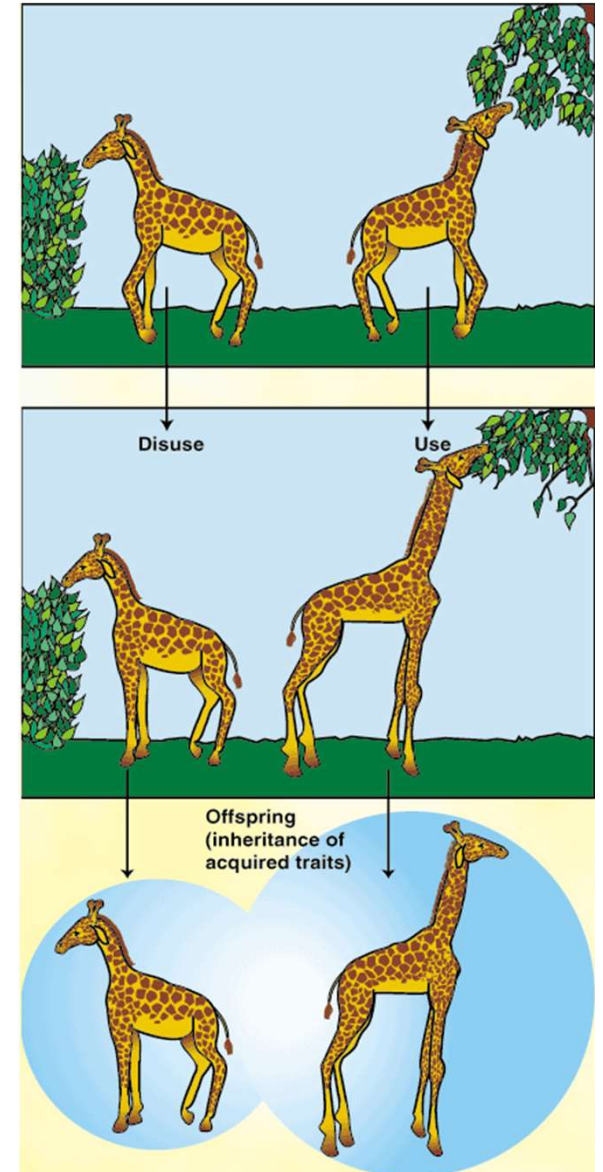
For origin of species in the book

Zoological Philosophy:

■ Needs of an organism determine the evolving characteristics

■ Useful characteristics could be transferred to offspring

■ *use and disuse law*



Basic elements

Historical points

? Charles Darwin (1807-1882)

- In the book *Origin of Species* he proved that all the organisms have evolved based on:

? Variation

- Overproduction of offspring

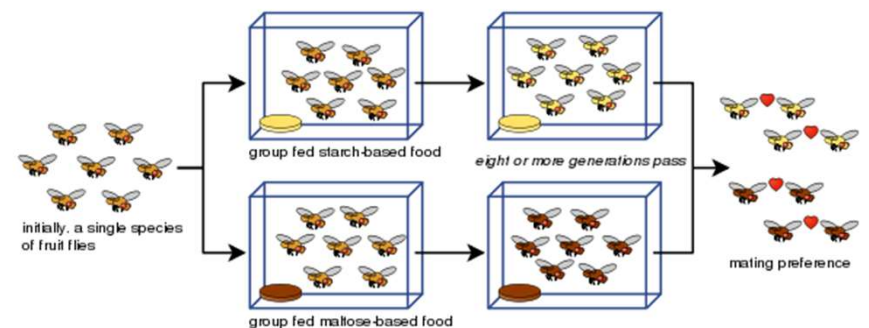
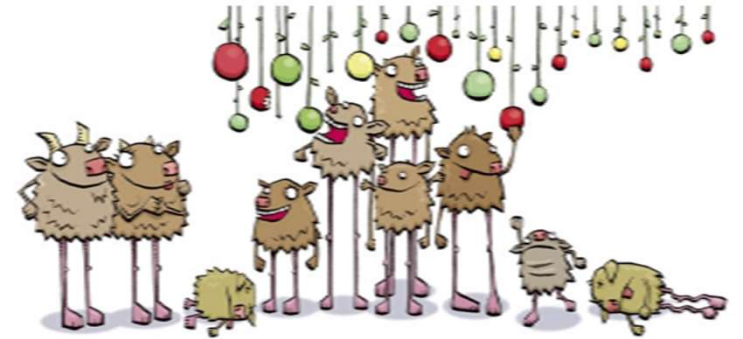
? Natural selection

- Competition (generation of constant size)

- Fitness survival

- Reproduction

- Occurrence of new species



Basic elements

Historical points

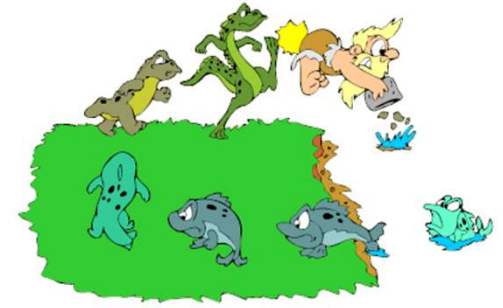
Modern theory of evolution

- Darwin's theory is improved by mechanism of genetic inheritance
- Genetic variance is produced by
 - Mutation and
 - Sexual recombination
- L. Fogel 1962 (San Diego, CA) → *Evolutionary Programming (EP)*
- J. Holland 1962 (Ann Arbor, MI) → *Genetic Algorithms (GAs)*
- I. Rechenberg & H.-P. Schwefel 1965 (Berlin, Germany) → *Evolution Strategies (ESs)*
- J. Koza 1989 (Palo Alto, CA) → *Genetic Programming (GP)*

Basic elements

Evolutionary metaphor

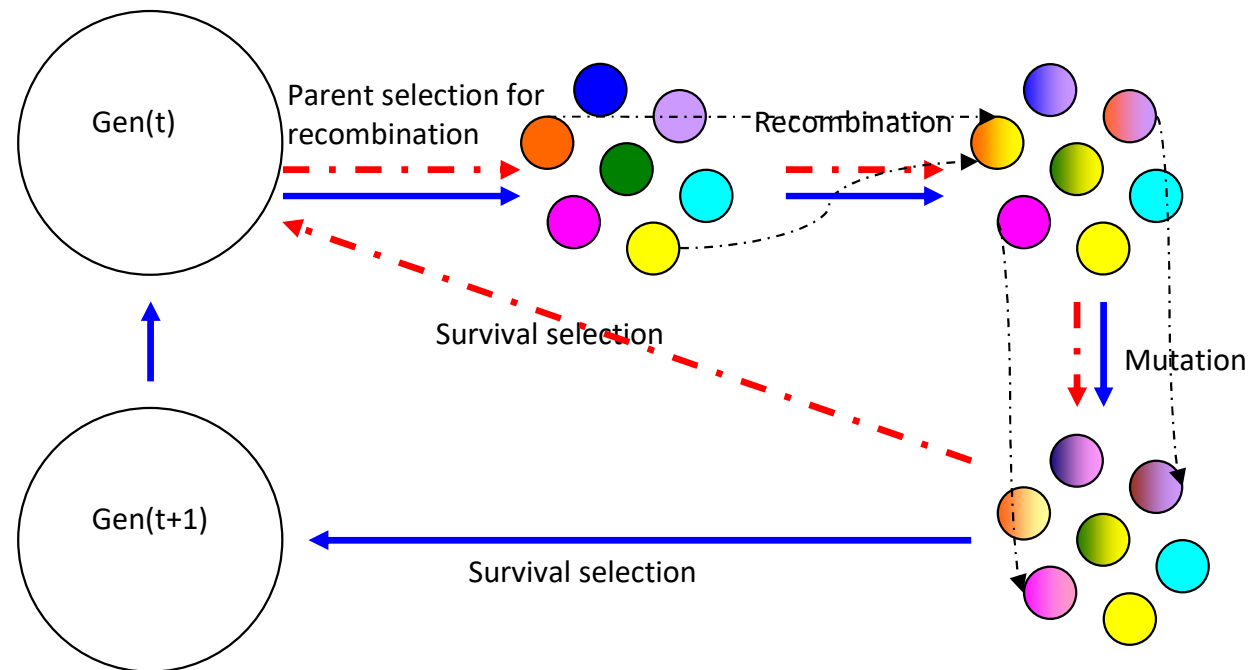
Natural evolution		Problem solving
Individual	↔	Possible solution
Population	↔	Set of possible solutions
Chromosome	↔	Coding of a possible solution
Gene	↔	Part of coding
Fitness	↔	Quality
Crossover and Mutation	↔	Search operators
Environment	↔	Problem

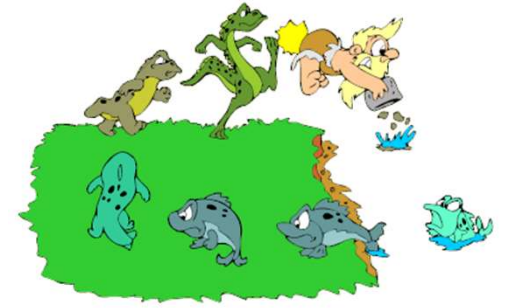


Algorithm

■ General sketch of an EA

- Generational 
- Steady-state 





Algorithm

- Design
 - Chromosome representation
 - Population model
 - Fitness function
 - Genetic operators
 - Selection
 - Mutation
 - Crossover
 - Stop condition

Representation

■ 2 levels of each possible solution

■ External level → phenotype

- Individual – original object in the context of the problem
- The possible solutions are evaluated here
- Ant, knapsack, elephant, towns, ...



■ Internal level → genotype

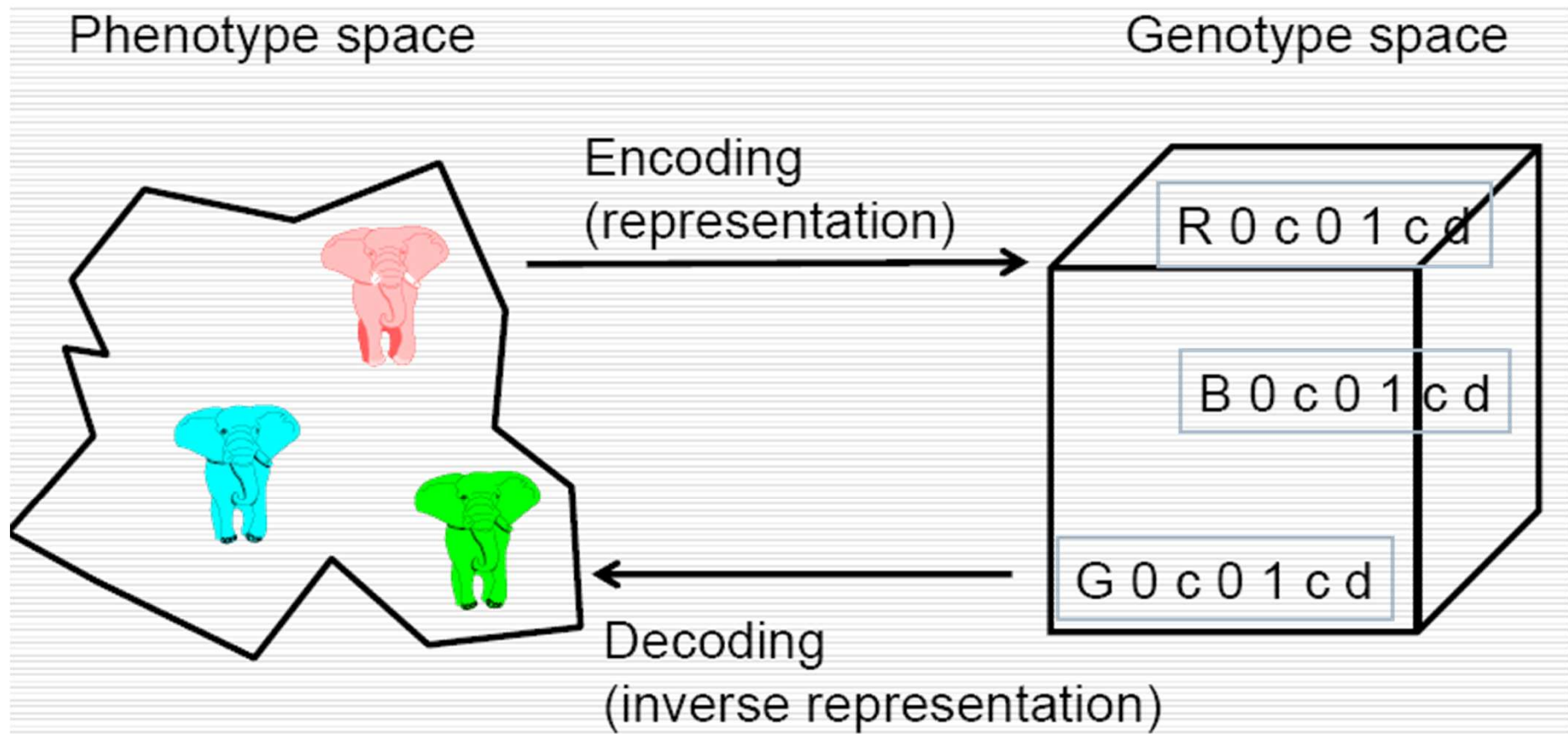
- Chromosome – code associated to an object
 - Composed by genes, located in loci (fix positions) and having some values (alleles)
- The possible solutions are searched here
- One-dimensional vector (with numbers, bits, characters), matrix, ...



Representation

❓ Representation must be representative for:

- Problem
- Fitness function and
- Genetic operators



Representation

Linear

- Discrete

 - Binary → knapsack problem

 - Not-binary

 - Integers

 - Random → image processing

 - Permutation → travelling salesman problem (TSP)

 - Class-based → map colouring problem

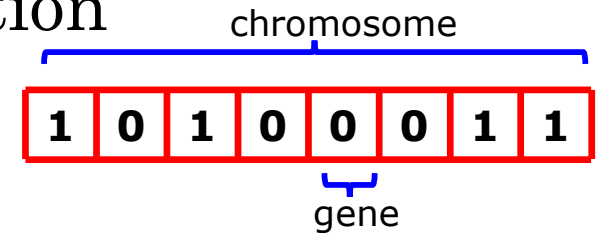
- Continuous (real) → function optimization

Tree-based → regression problems

Representation

Linear discrete and binary representation

- Genotype
Bit-strings



Representation

■ Linear discrete and binary representation

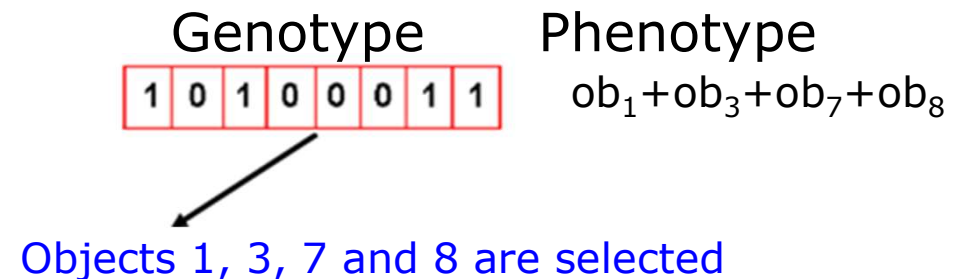
■ Genotype

■ Bit-strings

■ Phenotype

■ Boolean elements

- Ex. Knapsack problem – selected objects for the bag



Representation

■ Linear discrete and binary representation

■ Genotype

■ Bit-strings

■ Phenotype

■ Boolean elements

- Ex. Knapsack problem – selected objects for the bag

■ Integers

Genotype Phenotype

1	0	1	0	0	0	1	1
---	---	---	---	---	---	---	---

= 163

↓

$$1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 =$$
$$128 + 32 + 2 + 1 = 163$$

Representation

? Linear discrete and binary representation

■ Genotype

? Bit-strings

■ Phenotype

? Boolean elements

- Ex. Knapsack problem – selected objects for the bag

? Integers

? Real numbers from a range
(ex. [2.5, 20.5])

Genotype Phenotype

1	0	1	0	0	0	1	1
---	---	---	---	---	---	---	---

 = 13.9609



$$x = 2.5 + \frac{163}{256} (20.5 - 2.5) = 13.9609$$

Representation

Transformation of real values from binary representation

- Let be $z \in [x,y] \subseteq \mathbb{R}$ represented as $\{a_1, \dots, a_L\} \in \{0, 1\}^L$
- Function $[x,y] \rightarrow \{0,1\}^L$ must be inversely (a phenotype corresponds to a genotype)
- Function $\Gamma: \{0,1\}^L \rightarrow [x,y]$ defines the representation

$$\Gamma(a_1, \dots, a_L) = x + \frac{y-x}{2^L - 1} \cdot \left(\sum_{j=0}^{L-1} a_{L-j} \cdot 2^j \right) \in [x, y]$$

Remarks

- 2^L values can be represented
- L – maximum precision of solution
- For a better precision $\rightarrow$ long chromosomes $\rightarrow$ slowly evolution

Representation

Linear discrete non-binary integer random representation

- Genotype

Vector of integers from a given range

- Phenotype

Utility of numbers in the problem

- Ex. Pay a sum S by using different n coins

Genotype $\rightarrow$ vector of n integers from range $[0, S/\text{value of current coin}]$

Phenotype $\rightarrow$ how many coins of each type must be considered

Representation

Linear discrete non-binary integer permutation representation

- Genotype

Permutation of n elements (n – number of genes)

- Phenotype

Utility of permutation in problem

- Ex. Traveling Salesman Problem

Genotype $\rightarrow$ permutation of n elements

Phenotype $\rightarrow$ visiting order of towns (each town has associated a number from $\{1, 2, \dots, n\}$)

Representation

Linear discrete non-binary integer class-based representation

- Similarly to integer one, but labels are used instead numbers
- Genotype
Vector of labels from a given set
- Phenotype
Labels' meaning
- Ex. Map colouring problem
Genotype $\rightarrow$ vector of n colours (n – number of countries)
Phenotype $\rightarrow$ what colour has to be used for each country

Representation

Linear continuous (real) representation

- Genotype

Vector of real numbers

- Phenotype

Number meaning

- Ex. Function optimisation $f: R^n \rightarrow R$

Genotype $\rightarrow$ more real numbers $X=[x_1, x_2, \dots, x_n]$, $x_i \in R$

Phenotype $\rightarrow$ values of function f arguments

Representation

? Tree-based representation

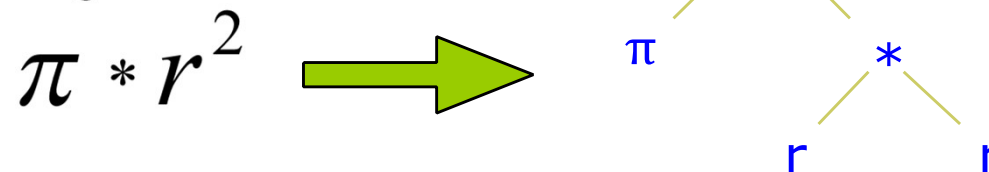
■ Genotype

- ? Trees that encode S-expressions
- ? Internal nodes □ functions (F)
 - Mathematical
 - Arithmetic operators
 - Boolean operators
 - Statements
 - Of a given programming language
 - Of other language type
- ? Leaf → terminals (T)
 - Real or Boolean values, constants or variables
 - Sub-programs

■ Phenotype

- ? Meaning of S-expressions

■ Ex. Computing the circle area



Population

Population – concept

- Aim

- Keeps a collection of possible solutions (candidate solutions)

- Repetitions are allowed

- Is entirely utilised during selection for recombination

- Properties

- (usually) fixed dimension μ

- Diversity

- Number of different fitnesses/phenotypes/genotypes

- Remarks

- Represents the basic unit that evolves

- The entire population evolves, not only the individuals!

Population

Population - initialisation

- Uniformly distributed in the search space (if it is possible)

Binary strings

- Randomly generation of 0 and 1 with a 0.5 probability (fifty-fifth)

Arrays of real numbers uniformly generated (in a given range)

Permutations

- Generation of identical permutation and making some changes

Population

Population - initialisation

- Uniformly distributed in the search space (if it is possible)

Trees

- *Full* method – complete trees
 - Nodes of depth $d < D_{\max}$ are randomly initialised by a function from function set F
 - nodes of depth $d = D_{\max}$ are randomly initialised by a terminal from the terminal set T
- *Grow* method – incomplete trees
 - Nodes of depth $d < D_{\max}$ are randomly initialised by an element from $F \cup T$
 - nodes of depth $d = D_{\max}$ are randomly initialised by a terminal from the terminal set T
- *Ramped half and half* method
 - $\frac{1}{2}$ of population is initialised by *Full* methods
 - $\frac{1}{2}$ of population is initialised by *Grow* methods
 - By using different depths

Population

Population model:

- Generational EA

 - Each generation creates μ offspring

 - Each individual survives a generation only

 - Set of parents is totally replaced by set of offspring

- Steady-state EA

 - Each generation creates a single offspring

 - A single parent (the worst one) is replaced by the offspring

Generation Gap

- Proportion of replaced population

- $1 = \mu / \mu$, for generational model

- $1 / \mu$, for steady-state model

Fitness function

Aim

- Reflects the adaptation to environment
- Quality function or objective function
- Associates a value to each candidate solution
 - Consequences over selection → the more different values, the better

Properties

- Costly stage
 - Unchanged individuals could not be re-evaluated

Typology:

- Number of objectives
 - One-objective
 - Multi-objective → Pareto fronts
- Optimisation direction
 - Maximisation
 - Minimisation
- Degree of precision
 - Deterministic
 - Heuristic

Fitness function

Examples

- Knapsack problem

Representation $\rightarrow$ linear, discrete and binary

Fitness $\rightarrow \text{abs}(\text{knapsack's capacity} - \text{weight of selected objects}) \rightarrow \min$

- Problem of paying sum s by using different coins

Representation $\rightarrow$ linear, discrete and integer

Fitness $\rightarrow \text{abs}(\text{sum to be paid} - \text{sum of selected coins}) \rightarrow \min$

- TSP

Representation $\rightarrow$ linear, discrete, integer, permutation

Fitness $\rightarrow$ cost of path $\rightarrow \min$

- Numerical function optimization

Representation $\rightarrow$ linear, continuous, real

Fitness $\rightarrow$ value of function $\rightarrow \min/\max$

- Computing the circle's area

Representation $\rightarrow$ tree-based

Fitness $\rightarrow$ sum of square errors (difference between the real value and the computed value for a given set of examples) $\rightarrow \min$

Selection



? Aim:

- Gives more reproduction/survival chances to better individuals
 - ? Weaker individuals have chances also because they could contain useful genetic material
- Orients the population to improve its quality

? Properties

- Works at population-level
- Is based on fitness only (is independent to representation)
- Helps to escape from local optima (because its stochastic nature)

Selection



■ Aim

- Parent selection (from current generation) for reproduction
- Survival selection (from parents and offspring) for next generation

■ Winner strategy

- Deterministic – the best wins
- Stochastic – the best has more chances to win

■ Mechanism

- Selection for recombination
 - Proportional selection (based on fitness)
 - Rank-based selection
 - Tournament selection
 - Survival selection
 - Age-based selection
 - Fitness-based selection
- Based on a part of population
- } Based on entire population

Recombination selection



? Proportional selection (fitness-based selection) - PS

? Main idea

- Roulette algorithm for entire population
- Estimation of the copies # of an individual (selection pressure)

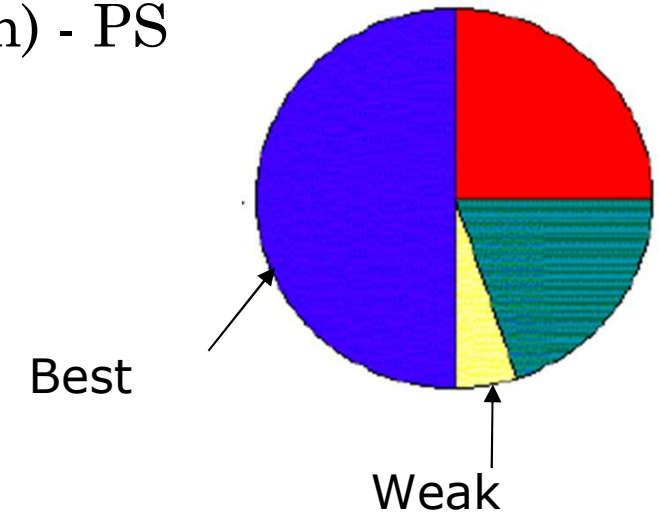
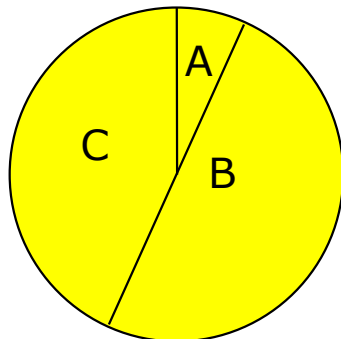
$$E(n_i) = \mu \frac{f(i)}{\langle f \rangle}, \text{ where:}$$

- μ = size of population,
- $f(i)$ = fitness of individual i ,
- $\langle f \rangle$ = mean fitness of population

? Better individuals

- Have more space on roulette
- Have more chances to be selected

? Ex. A population of $\mu = 3$ individuals



	f(i)	P_{selPS}(i)
A	1	1/10=0.1
B	5	5/10=0.5
C	4	4/10=0.4
Sum	10	1

Recombination selection



Proportional selection (fitness-based selection) – PS

Advantages

- Simple algorithm

Disadvantages

- Premature convergence
 - Best chromosomes predispose to dominate the population
- Low selection pressure when fitness functions are very similar (at the end of a run)
- Real results are different to theoretical probabilistic distribution
- Works at the entire population level

Solutions

- Fitness scaling

Windowing

- $f'(i) = f(i) - \beta^t$, where β is a parameter that depends on evolution history
 - eg. β is the fitness of the weakest individual of current population (the t^{th} generation)

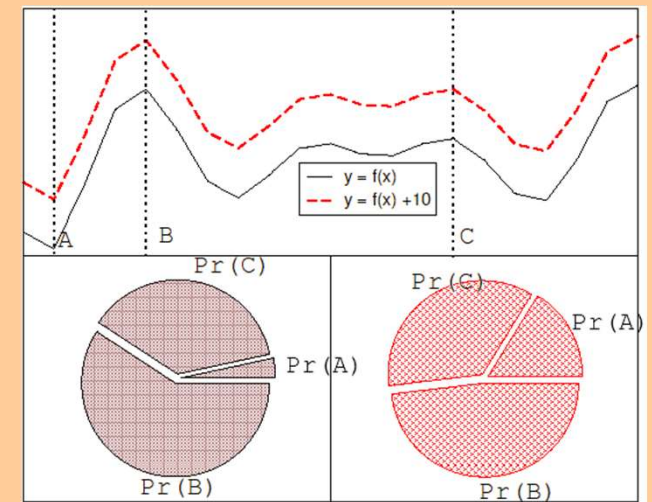
Sigma scaling (Goldberg type)

- $f'(i) = \max\{f(i) - (\langle f \rangle - c * \sigma_f), 0.0\}$, where:
 - C – a constant (usually, 2)
 - $\langle f \rangle$ – average fitness of population
 - σ_f – standard deviation of population fitness

Normalisation

- Starts by absolute (initial) fitnesses
- Standardize these fitnesses such as the fitnesses:
 - Belong to $[0,1]$
 - Best fitness is the smallest one (equal to 0)
 - Sum of them is 1

- Another selection mechanism





Selection for recombination

Ranking selection – RS

■ Main idea

- Sort the entire population based on fitness
 - Increases the algorithm complexity, but it is negligible related to the fitness evaluation
- Each individual receives a rank
- Computes the selection probabilities based on these ranks
 - Best individual has rank μ
 - Worst individual has rank 1
- Tries to solve the problems of proportional selection by using relative fitness (instead of absolute fitness)



Selection for recombination

? Ranking selection - RS

■ Ranking procedures

? Linear (LR)
$$P_{lin_rank}(i) = \frac{2-s}{\mu} + \frac{2i(s-1)}{\mu(\mu-1)}$$

- s – selection pressure
 - Measures the advantages of the best individual
 - $1.0 < s \leq 2.0$
 - In the generational algorithm s represents the copies number of an individual
- Ex. For a population of $\mu = 3$ individuals

	f(i)	P_{selPS}(i)	Rank	P_{selLR}(i) for s=2	P_{selRL}(i) for s=1.5
A	1	1/10=0.1	1	0.33	0.33
B	5	5/10=0.5	3	1.00	0.33
C	4	4/10=0.4	2	0.67	0.33
Sum	10	1			

? Exponential (ER)
$$P_{exp_rank}(i) = \frac{1-e^{-i}}{c}$$

- Best individual can have more than 2 copies
- C – normalisation factor
 - Depends on the population size (μ)
 - Must be choose such as the sum of selection probabilities to be 1



Selection for recombination

□ Ranking selection - RS

■ Advantages

- Keep the selection pressure constant

■ Disadvantages

- Works with the entire population

■ Solutions

- Another selection procedure

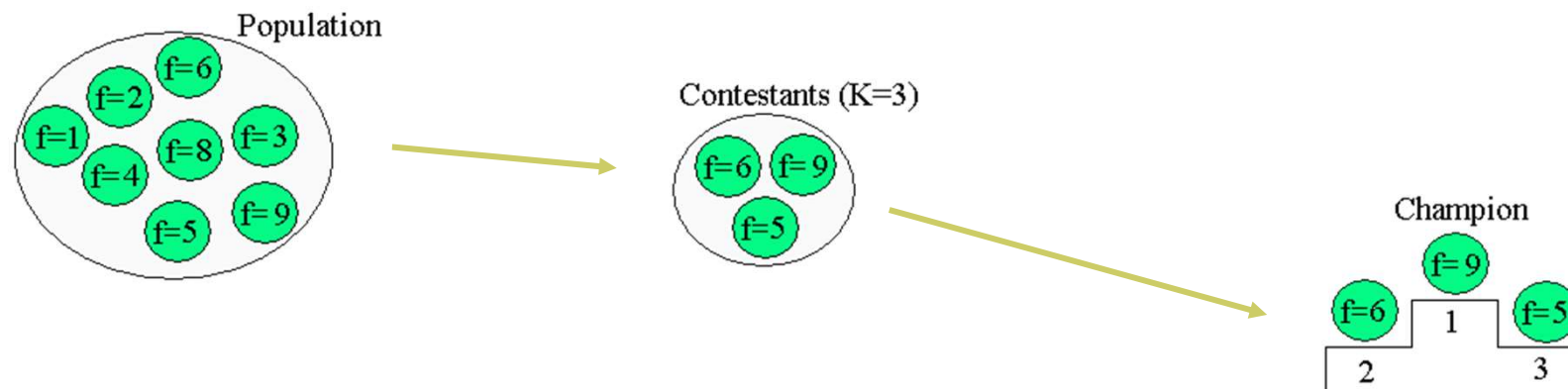
Selection for recombination



? Tournament selection

■ Main idea

- ? Chooses k individuals $\rightarrow$ sample of k individuals (k – tournament size)
- ? Selects the best individual of the sample
- ? Probability of sample selection depends on
 - Rank of individual
 - Sample size (k)
 - The larger k is, the greater selection pressure is
 - Choosing manner – with replacement (steady-state model) or without replacement
 - Selection without replacement increases the selection pressure
 - For $k=2$ the time required by the best individual to dominate the population is the same to that from linear ranking selection with $s = 2 * p$, p – selection probability of the best individual from population



Selection for recombination



? Tournament selection

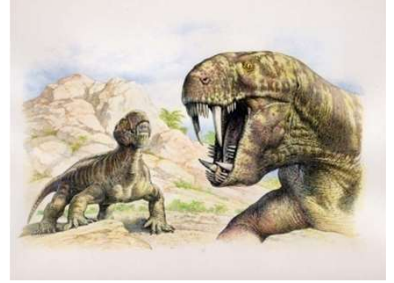
■ Advantages

- ? Does not work with the entire population
- ? Easy to implement
- ? Easy to control the selection pressure by using parameter k

■ Disadvantages

- ? The real results of this selection are different to theoretical distribution (similarly to roulette selection)

Survival selection



☐ Survival selection (selection for replacement)

■ Based on age

- ☐ Eliminates the oldest individuals

■ Based on fitness

- ☐ Proportional selection

- ☐ Ranking selection

- ☐ Tournament selection

- ☐ Elitism

- Keep the best individuals from a generation to the next one (if the offspring are weaker than parents, then keep the parents)

- ☐ GENITOR (replaces the worst individual)

- Elimination of the worst λ individuals

Variation operators



- ❑ Aim :
 - Generation of new possible solutions

- ❑ Properties
 - Works at individual level
 - Is based on individual representation (fitness independent)
 - Helps the exploration and exploitation of the search space
 - Must produce valid individuals

- ❑ Typology
 - Arity criterion
 - ❑ Arity 1 → mutation operators
 - ❑ Arity > 1 → recombination/crossover operators

Mutation

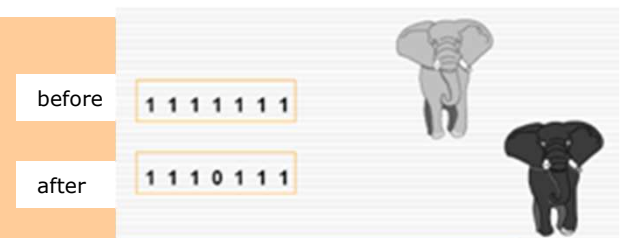


? Aim

- Reintroduces in population the lost genetic material
- Unary search operator (continuous space)
- Introduces the diversity in population (discrete space)

? Properties

- ? Works at genotype level
- ? Based on random elements



- ? Responsible to the exploration of promising regions of the search space
- ? Responsible to escape from local optima
- ? Must introduce small and stochastic changes for an individual
- ? Size of mutation must be controllable
- ? Can probabilistic take place (by a given probability p_m) at the gene level

Mutation



- Binary representation
 - ▢ Strong mutation – bit-flipping
 - ▢ Weak mutation
- Integer representation
 - ▢ Random resetting
 - ▢ Creep mutation
- Permutation representation
 - ▢ Insertion mutation
 - ▢ Swap mutation
 - ▢ Inverse mutation
 - ▢ scramble mutation
 - ▢ K-opt mutation
- Real representation
 - ▢ Uniform mutation
 - ▢ Non-uniform mutation
 - Gaussian mutation
 - Cauchy mutation
 - Laplace mutation
- Tree-based representation → future lecture
 - ▢ Grow mutation
 - ▢ Shrink mutation
 - ▢ Switch mutation
 - ▢ Cycle mutation
 - ▢ Koza mutation
 - ▢ Mutation for numerical terminals

Mutation (binary representation)



■ A chromosome $c = (g_1, g_2, \dots, g_L)$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in \{0, 1\}$, for $i = 1, 2, \dots, L$.

■ Strong mutation – *bit flipping*

■ Main idea

■ Changes by probability p_m (mutation rate) all the genes in their complement

■ $1 \rightarrow 0$

■ $0 \rightarrow 1$

■ Ex. A chromosome of $L = 8$ genes, $p_m = 0.1$



Mutation (binary representation)



- A chromosome $c = (g_1, g_2, \dots, g_L)$ becomes
 $c' = (g_1', g_2', \dots, g_L')$,
where $g_i, g_i' \in \{0, 1\}$, for $i = 1, 2, \dots, L$

□ Weak mutation

■ Main idea

- Changes by probability p_m (mutation rate) some of the genes in 0 or 1
 - $1 \rightarrow 0/1$
 - $0 \rightarrow 1/0$
- Eg. A chromosome of $L = 8$ genes, $p_m = 0.1$

1 0 1 0 0 0 1 1



1 1 1 0 0 0 0 1

Mutation (integer representation)



- A chromosome $c = (g_1, g_2, \dots, g_L)$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in \{val_1, val_2, \dots, val_k\}$ for $i = 1, 2, \dots, L$.

□ Random resetting mutation

■ Main idea

- The value of a gene is changed (by probability p_m) into another value (from the definition domain)

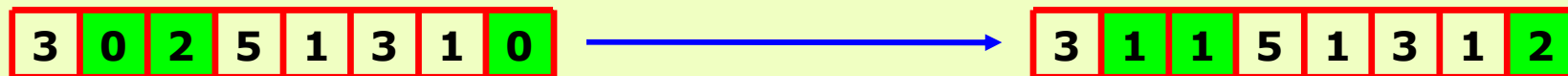


Mutation (integer representation)



- A chromosome $c = (g_1, g_2, \dots, g_L)$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in \{val_1, val_2, \dots, val_k\}$, for $i = 1, 2, \dots, L$.

- *Creep* mutation
 - Main idea
 - The value of a gene is changed (by probability p_m) by adding a positive/negative value
 - New value follows a 0 symmetric distribution
 - The performed change is very small



Mutation (permutation representation)



- A chromosome $c = (g_1, g_2, \dots, g_L)$ with $g_i \neq g_j$ for all $i \neq j$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in \{val_1, val_2, \dots, val_L\}$, for $i = 1, 2, \dots, L$ s. a. $g_i' \neq g_j'$ for all $i \neq j$.

□ *Swap mutation*

■ Main idea

- Randomly choose 2 genes and swap their values



Mutation (permutation representation)

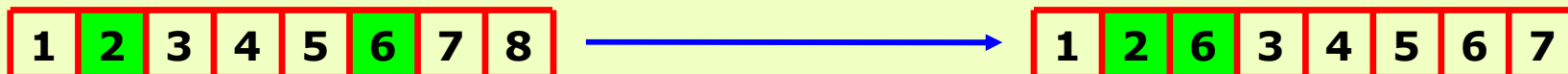


- A chromosome $c = (g_1, g_2, \dots, g_L)$ with $g_i \neq g_j$ for all $i \neq j$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in \{val_1, val_2, \dots, val_L\}$, for $i = 1, 2, \dots, L$ s. a. $g_i' \neq g_j'$ for all $i \neq j$.

□ Insertion mutation

■ Main idea

- Randomly choose 2 genes g_i and g_j with $j > i$
- Insert gene g_j after gene g_i s.a. $g_i' = g_i, g_{i+1}' = g_j, g_{k+2}' = g_{k+1}$, for $k = i, i+1, i+2, \dots$



Mutation (permutation representation)

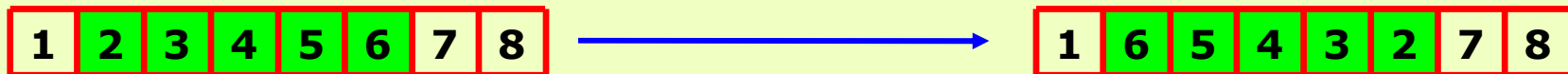


- A chromosome $c = (g_1, g_2, \dots, g_L)$ with $g_i \neq g_j$ for all $i \neq j$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in \{val_1, val_2, \dots, val_L\}$, for $i = 1, 2, \dots, L$ s.a. $g_i' \neq g_j'$ for all $i \neq j$.

□ Inversion mutation

■ Main idea

- Randomly choose 2 genes and inverse the order of genes between them (sub-string of genes)



Mutation (permutation representation)

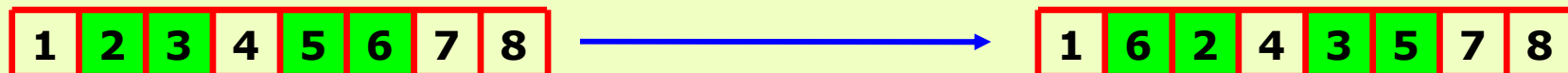


- A chromosome $c = (g_1, g_2, \dots, g_L)$ with $g_i \neq g_j$ for all $i \neq j$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in \{val_1, val_2, \dots, val_L\}$, for $i = 1, 2, \dots, L$ s.a. $g_i' \neq g_j'$ for all $i \neq j$.

□ *scramble mutation*

■ Main idea

- Randomly choose a (continuous or discontinuous) sub-array of genes and re-organise that genes





Mutation (permutation representation)

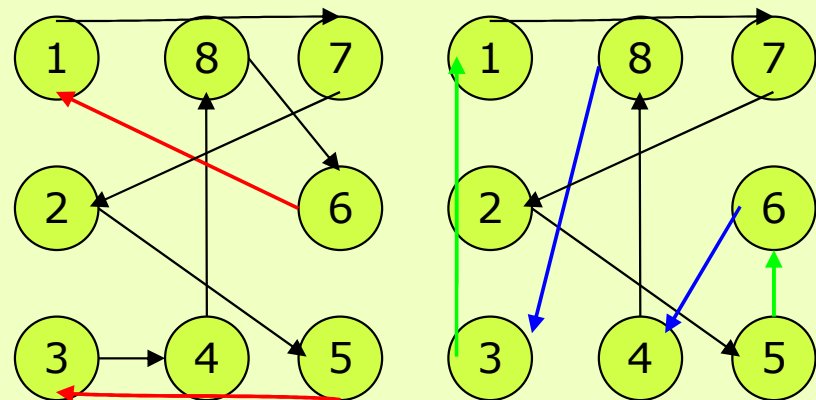
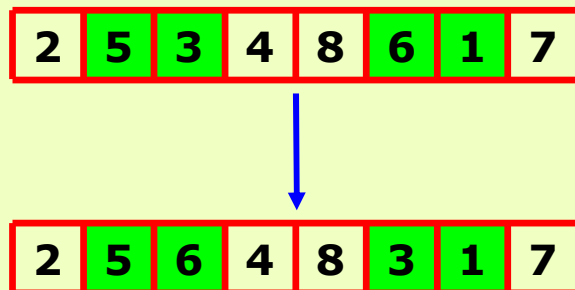
- A chromosome $c = (g_1, g_2, \dots, g_L)$ with $g_i \neq g_j$ for all $i \neq j$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in \{val_1, val_2, \dots, val_L\}$, for $i = 1, 2, \dots, L$ s.a. $g_i' \neq g_j'$ for all $i \neq j$.

□ K-opt mutation

■ Main idea

- Choose 2 disjoint sub-strings of length k
- Interchange 2 elements of these sub-strings

$k=2$



Mutation (real representation)



- A chromosome $c = (g_1, g_2, \dots, g_L)$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in [a_i, b_i]$, for $i=1, 2, \dots, L$.

□ Uniform mutation

■ Main idea

- g_i' is changed by probability p_m into a new value that is randomly uniform generated in $[a_i, b_i]$ range

Mutation (real representation)



- A chromosome $c = (g_1, g_2, \dots, g_L)$ becomes $c' = (g_1', g_2', \dots, g_L')$, where $g_i, g_i' \in [a_i, b_i]$, for $i = 1, 2, \dots, L$.
- Non-uniform mutation
 - Main idea
 - The value of a gene is probabilistically (p_m) changed by adding a positive/negative value
 - The added value belongs to a distribution of type
 - $N(\mu, \sigma)$ (Gaussian) with $\mu = 0$
 - Cauchy (x_0, γ)
 - Laplace (μ, b)
 - And it is re-introduced in $[a_i, b_i]$ range (if it is necessary) – *clamping*

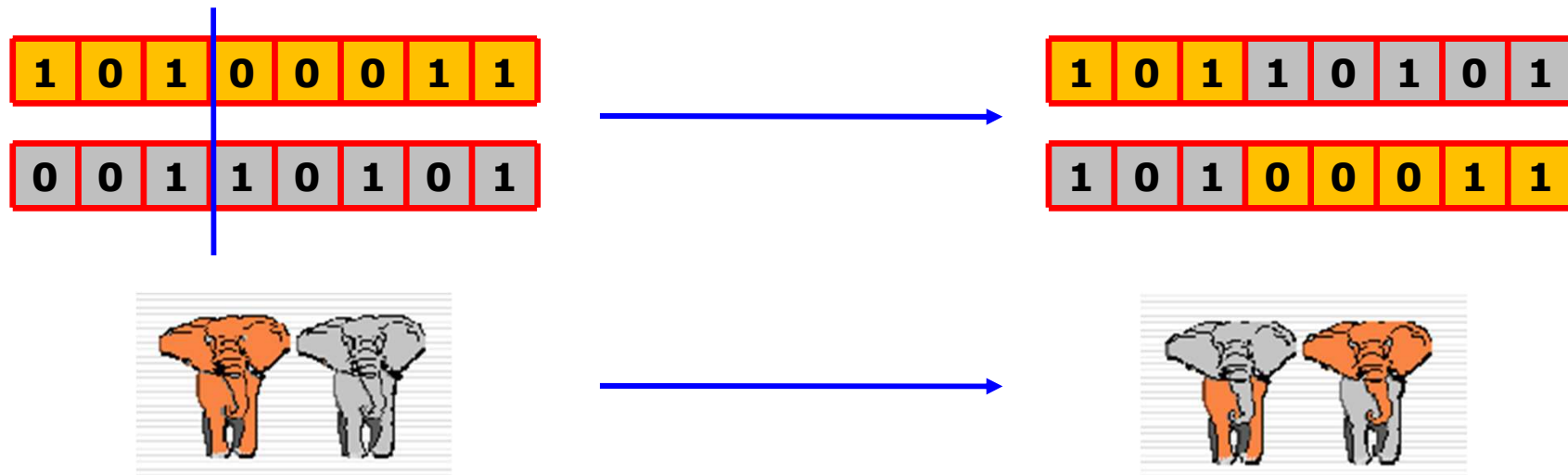


Recombination

- Aim
 - Mix the parents' information

□ Properties

- The offspring has to inherit something from both parents
 - Selection of mixed information is randomly performed
- Operator for exploitation of already discover possible solutions
- The offspring can be better, the same or weaker than their parents
- Its effects are reducing while the search converges





Recombination

□ Types

□ Binary and integer representation

- With cutting points
- Uniforme

□ Permutation representation

- Order crossover (version 1 and version 2)
- Partially Mapped Crossover
- Cycle crossover
- Edge-based crossover

□ Real representation

- Discrete
- Arithmetic
 - Singular
 - Simple
 - Complete
- Geometric
- Shuffle crossover
- Simulated binary crossover

□ Tree-based representation

- Sub-tree based crossover → future lecture



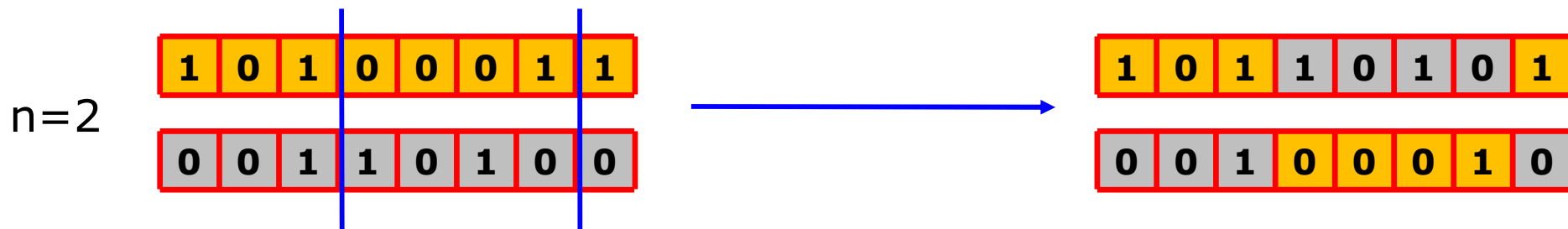
Recombination (binary and integer representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - where $g_i^1, g_i^2, g_i', g_i'' \in \{0, 1\} / \{val_1, val_2, \dots, val_k\}$, for $i = 1, 2, \dots, L$

□ N-cutting point crossover

■ Main idea

- Choose n cutting-points ($n < L$)
- Cut the parents through these points
- Put together the resulted parts, by alternating the parents





Recombination (binary and integer representation)

□ N cutting point crossover

■ Properties

- Average of values encoded by parents = average of values encoded by offspring
 - Eg binary representation on 4 bits of integer numbers – XO with $n = 1$ after second bit
 - $p_1 = (1, 0, 1, 0), p_2 = (1, 1, 0, 1)$
 - $c_1 = (1, 0, 0, 1), c_2 = (1, 1, 1, 0)$
 - $val(p_1) = 10, val(p_2) = (13) \rightarrow (val(p_1) + val(p_2)) / 2 = 23 / 2 = 11.5$
 - $val(c_1) = 9, val(c_2) = (14) \rightarrow (val(c_1) + val(c_2)) / 2 = 23 / 2 = 11.5$
 - Eg. Binary representation on 4 bits for knapsack problem ($K=10$, 4 items of weight and value: $(2,7), (1,8), (3,1), (2,3)$)
 - $p_1 = (1, 0, 1, 0), p_2 = (1, 1, 0, 1)$
 - $c_1 = (1, 0, 0, 1), c_2 = (1, 1, 1, 0)$
 - $val(p_1) = 8, val(p_2) = 18 \rightarrow (val(p_1) + val(p_2)) / 2 = 26 / 2 = 13$
 - $val(c_1) = 10, val(c_2) = 16 \rightarrow (val(c_1) + val(c_2)) / 2 = 26 / 2 = 13$
- Probability of $\beta \approx 1$ is the largest one

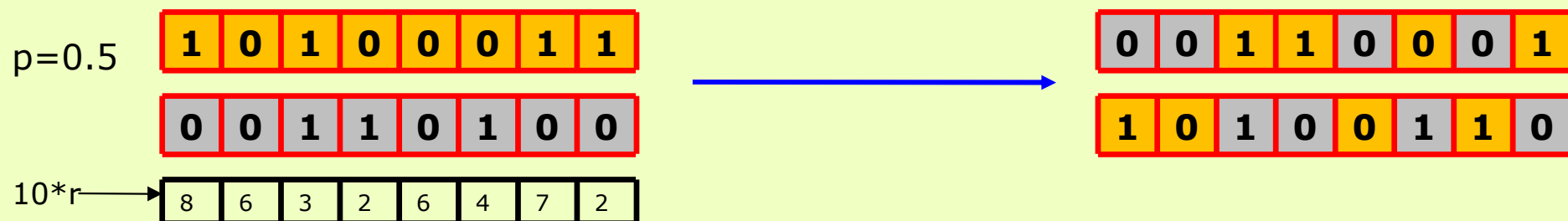
$$\beta = \left| \frac{val(d_1) - val(d_2)}{val(p_1) - val(p_2)} \right|$$

- Contracting crossover $\beta < 1$
 - Offspring values are between parent values
- Expanding crossover $\beta > 1$
 - Parent values are between offspring values
- Stationary crossover $\beta = 1$
 - Offspring values are equal to parent values



Recombination (binary and integer representation)

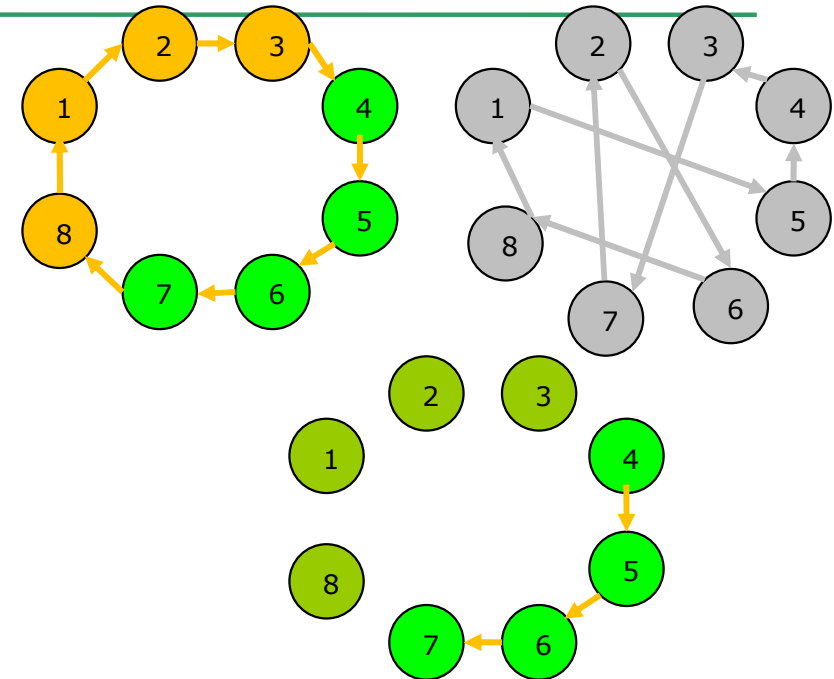
- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - where $g_i^1, g_i^2, g_i', g_i'' \in \{0, 1\} / \{val_1, val_2, \dots, val_k\}$, for $i = 1, 2, \dots, L$
- Uniform crossover
 - Main idea
 - Each gene of an offspring comes from a randomly and uniform selected parent:
 - For each gene a uniform random number r is generated
 - If $r < \text{probability } p$ (usually, $p=0.5$), c_1 will inherit that gene from p_1 and c_2 from p_2 ,
 - otherwise, c_1 will inherit p_2 and c_2 will inherit p_1



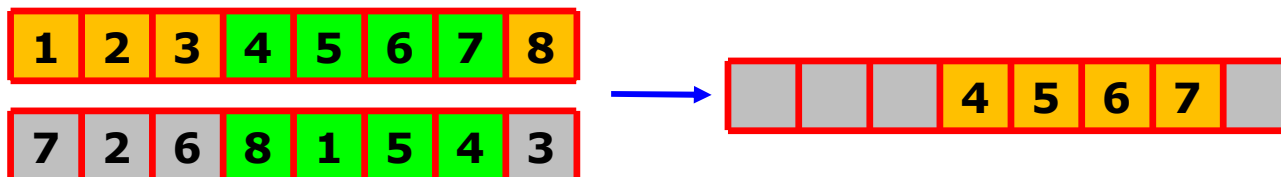


Recombination (permutation representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [1, L] \cap \mathbb{Z}$, for $i = 1, 2, \dots, L$.



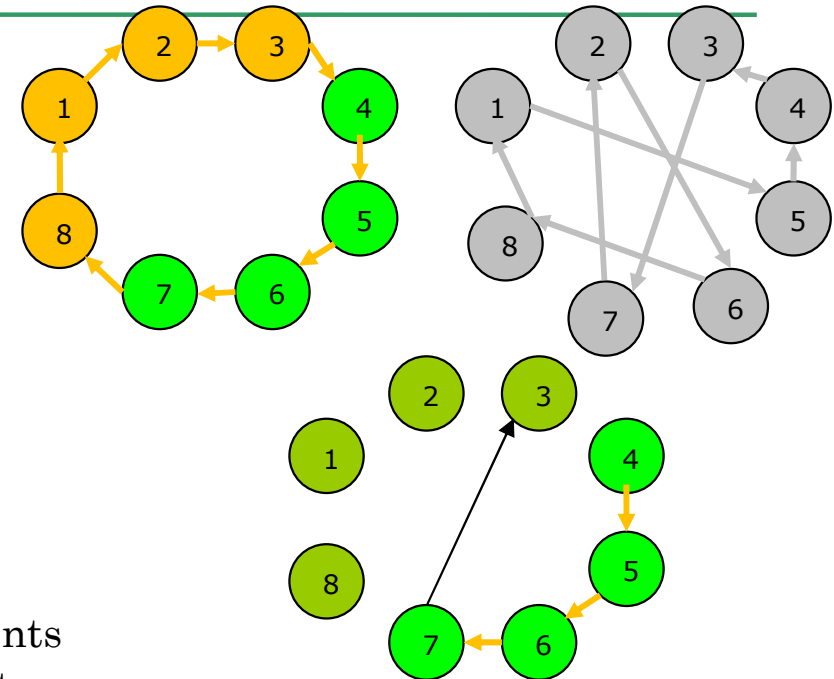
- Order crossover
 - Main idea
 - Offspring keep the order of genes from parents
 - Choose a substring of genes from the parent p_1
 - Copy the substring from p_1 into offspring d_1 (on corresponding positions)



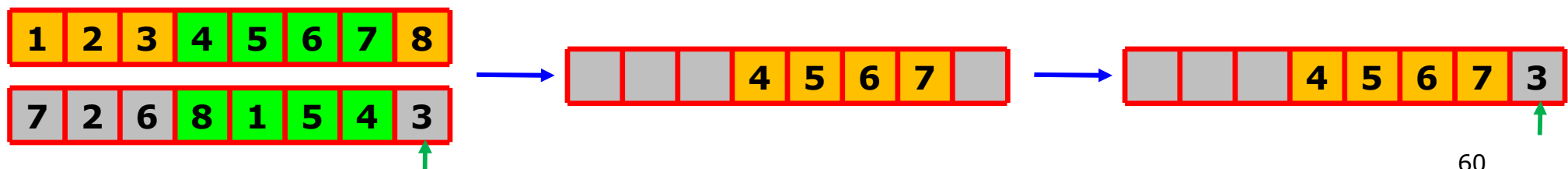


Recombination (permutation representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [1, L] \cap \mathbb{Z}$, for $i = 1, 2, \dots, L$.



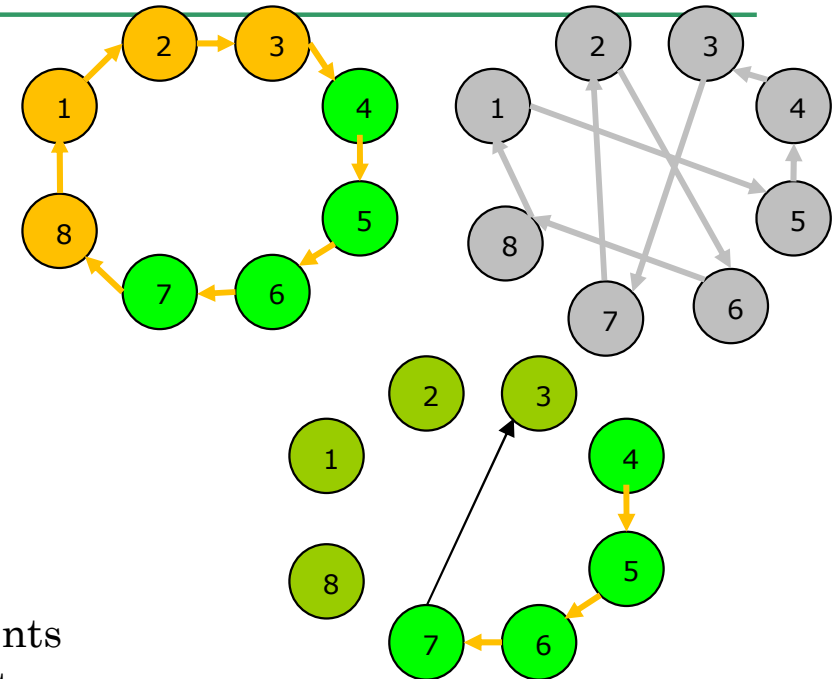
- Order crossover
 - Main idea
 - Offspring keep the order of genes from parents
 - Choose a substring of genes from the parent p_1
 - Copy the substring from p_1 into offspring d_1 (on corresponding positions)
 - Copy the genes of p_2 in offspring d_1 :
 - Starting with the first position after sub-string
 - Respecting gene's order from p_2 and
 - Re-loading the genes from start (if the end of chromosome is reached)



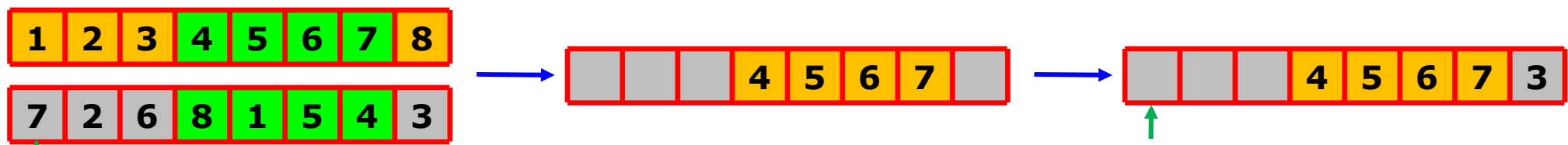


Recombination (permutation representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [1, L] \cap \mathbb{Z}$, for $i = 1, 2, \dots, L$.



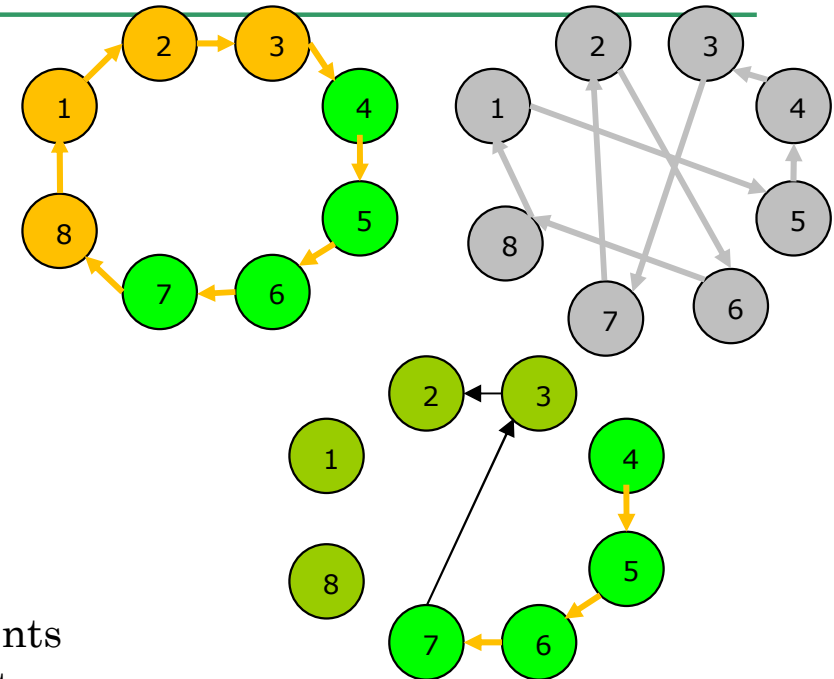
- Order crossover
 - Main idea
 - Offspring keep the order of genes from parents
 - Choose a substring of genes from the parent p_1
 - Copy the substring from p_1 into offspring d_1 (on corresponding positions)
 - Copy the genes of p_2 in offspring d_1 :
 - Starting with the first position after sub-string
 - Respecting gene's order from p_2 and
 - Re-loading the genes from start (if the end of chromosome is reached)





Recombination (permutation representation)

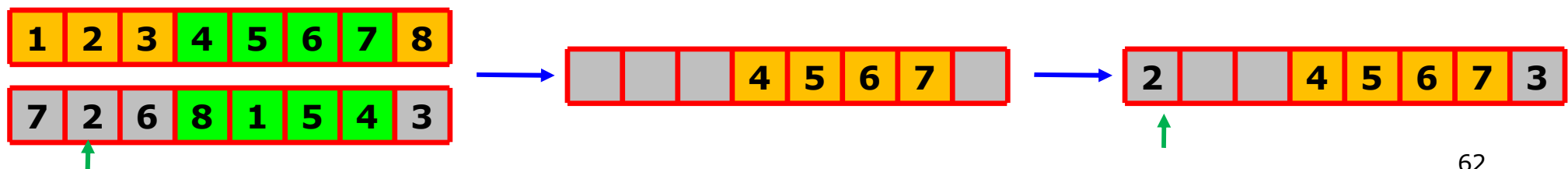
- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [1, L] \cap \mathbb{Z}$, for $i = 1, 2, \dots, L$.



□ Order crossover

■ Main idea

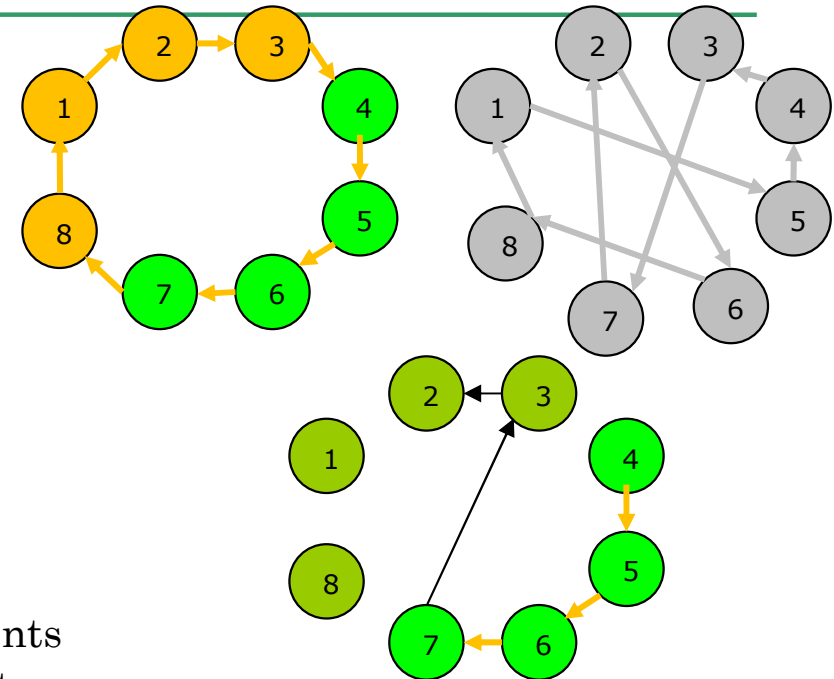
- Offspring keep the order of genes from parents
- Choose a substring of genes from the parent p_1
- Copy the substring from p_1 into offspring d_1 (on corresponding positions)
- Copy the genes of p_2 in offspring d_1 :
 - Starting with the first position after sub-string
 - Respecting gene's order from p_2 and
 - Re-loading the genes from start (if the end of chromosome is reached)





Recombination (permutation representation)

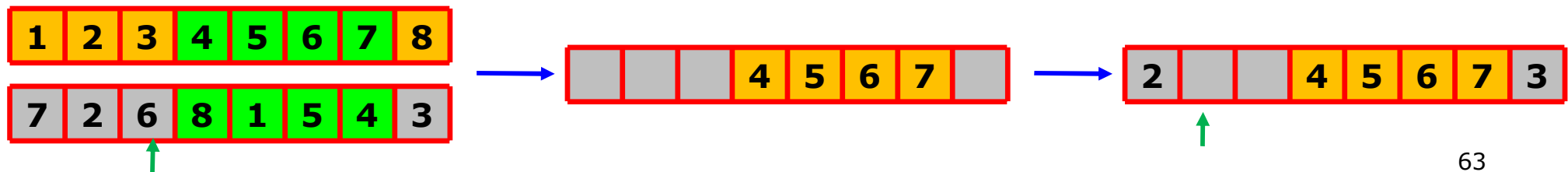
- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [1, L] \cap \mathbb{Z}$, for $i = 1, 2, \dots, L$.



□ Order crossover

■ Main idea

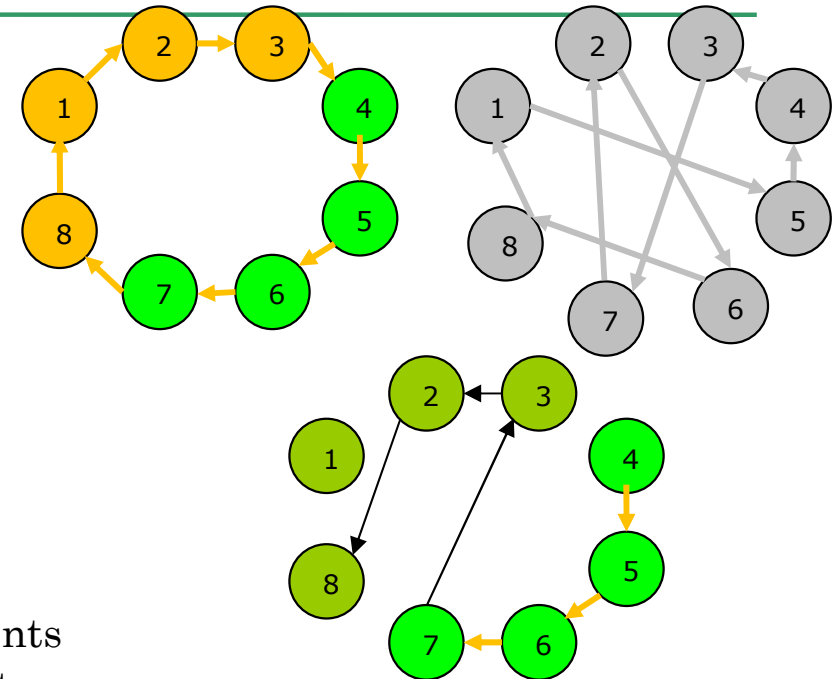
- Offspring keep the order of genes from parents
- Choose a substring of genes from the parent p_1
- Copy the substring from p_1 into offspring d_1 (on corresponding positions)
- Copy the genes of p_2 in offspring d_1 :
 - Starting with the first position after sub-string
 - Respecting gene's order from p_2 and
 - Re-loading the genes from start (if the end of chromosome is reached)





Recombination (permutation representation)

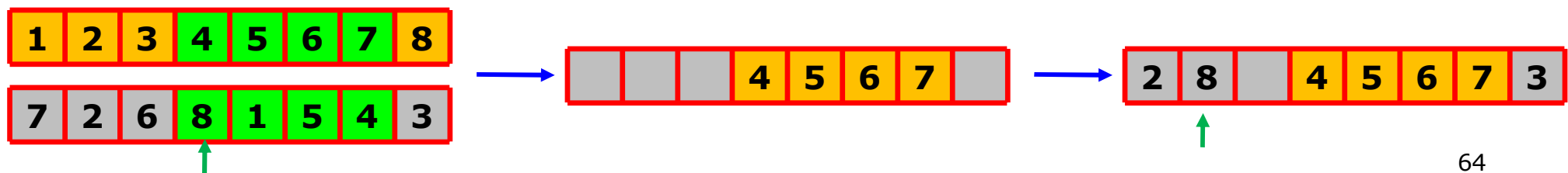
- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [1, L] \cap \mathbb{Z}$, for $i = 1, 2, \dots, L$.



□ Order crossover

■ Main idea

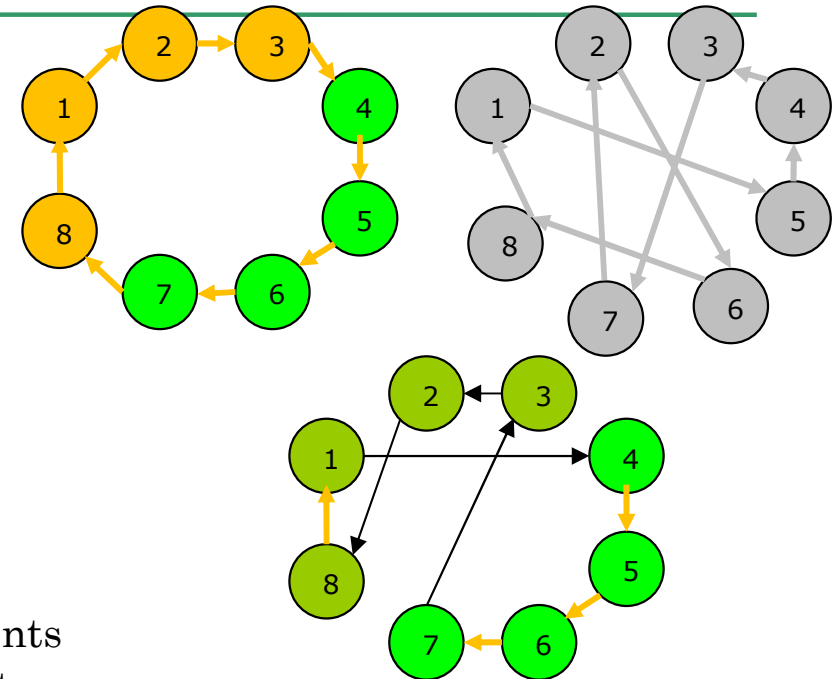
- Offspring keep the order of genes from parents
- Choose a substring of genes from the parent p_1
- Copy the substring from p_1 into offspring d_1 (on corresponding positions)
- Copy the genes of p_2 in offspring d_1 :
 - Starting with the first position after sub-string
 - Respecting gene's order from p_2 and
 - Re-loading the genes from start (if the end of chromosome is reached)





Recombination (permutation representation)

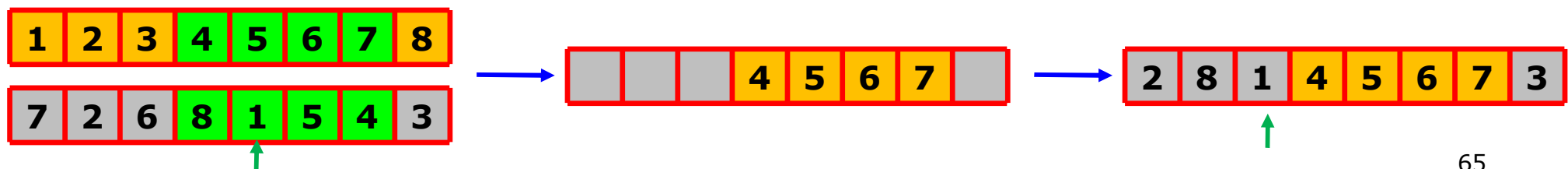
- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [1, L] \cap \mathbb{Z}$, for $i = 1, 2, \dots, L$.



□ Order crossover

■ Main idea

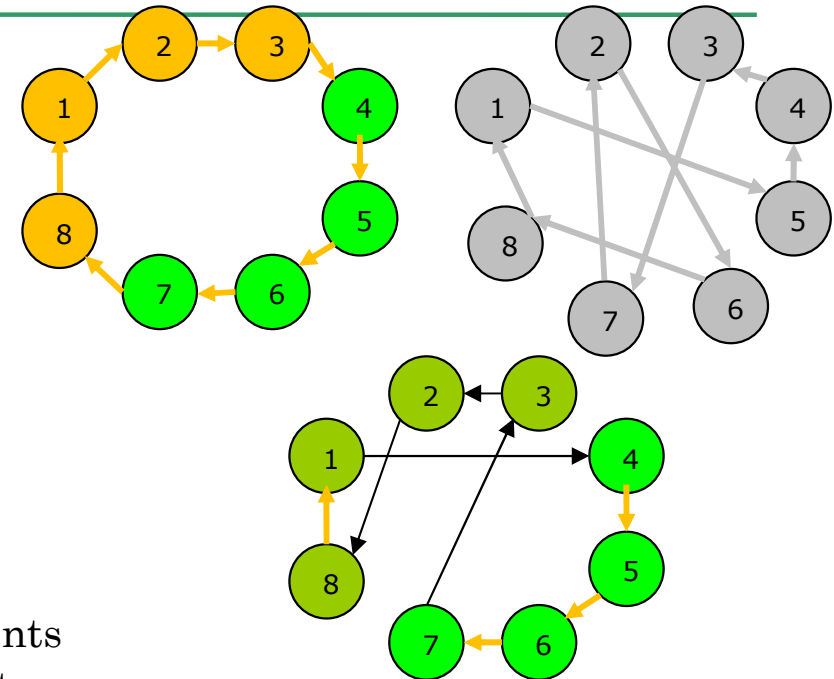
- Offspring keep the order of genes from parents
- Choose a substring of genes from the parent p_1
- Copy the substring from p_1 into offspring d_1 (on corresponding positions)
- Copy the genes of p_2 in offspring d_1 :
 - Starting with the first position after sub-string
 - Respecting gene's order from p_2 and
 - Re-loading the genes from start (if the end of chromosome is reached)





Recombination (permutation representation)

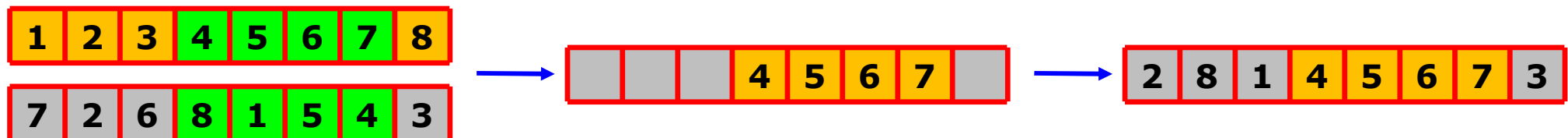
- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [1, L] \cap \mathbb{Z}$, for $i = 1, 2, \dots, L$.



□ Order crossover

■ Main idea

- Offspring keep the order of genes from parents
- Choose a substring of genes from the parent p_1
- Copy the substring from p_1 into offspring d_1 (on corresponding positions)
- Copy the genes of p_2 in offspring d_1 :
 - Starting with the first position after sub-string
 - Respecting gene's order from p_2 and
 - Re-loading the genes from start (if the end of chromosome is reached)





Recombination (real representation)

□ From 2 parent chromosomes

- $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$

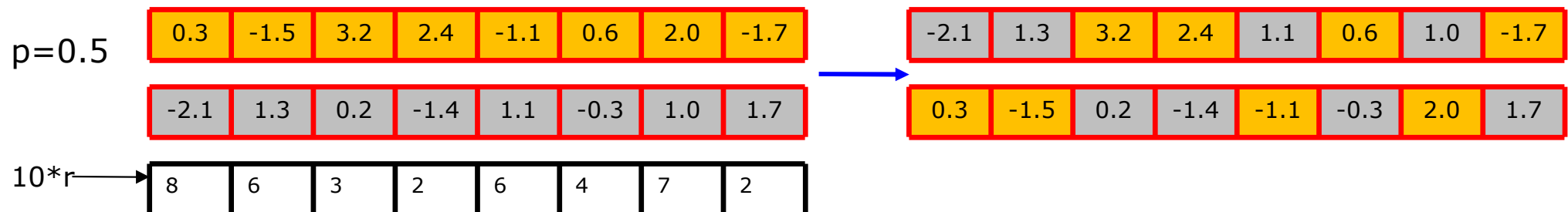
□ 2 offspring are obtained

- $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
- Where $g_i^1, g_i^2, g_i', g_i'' \in [a_i, b_i]$, for $i = 1, 2, \dots, L$

□ Discrete crossover

■ Main idea

- Each gene offspring is taken (by the same probability, $p = 0.5$) from one of the parents
- Similarly to uniform crossover for binary/integer representation
- The absolute values of genes are not changed (no new information is created)





Recombination (real representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [a_i, b_i]$, for $i = 1, 2, \dots, L$

□ Arithmetic crossover

■ Main idea

- Create offspring between parents → arithmetic crossover
 - $z_i = \alpha x_i + (1 - \alpha) y_i$ where $0 \leq \alpha \leq 1$.
- Parameter α can be:
 - Constant → uniform arithmetic crossover
 - Variable → eg. Depends on the age of population
 - Random → generated for each new XO that is performed
- New values of a gene can appear

■ Types:

- Singular arithmetic crossover
- Simple arithmetic crossover
- Complete arithmetic crossover



Recombination (real representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [a_i, b_i]$, for $i = 1, 2, \dots, L$

□ Singular arithmetic crossover

- Choose one gene from two parents (of the same position k) and combine them

$$\square g_k' = \alpha g_k^1 + (1 - \alpha) g_k^2$$

$$\square g_k'' = (1 - \alpha) g_k^1 + \alpha g_k^2$$

- The rest of genes are unchanged

$$\square g_i' = g_i^1$$

$$\square g_i'' = g_i^2, \text{ for } i = 1, 2, \dots, L \text{ and } i \neq k$$

$$[a, b] = [-2.5, +3]$$

$$k = 3$$

$$\alpha = 0.6$$

0.3	-1.5	3.2	2.4	-1.1	0.6	2.0	-1.7
-----	------	-----	-----	------	-----	-----	------

-2.1	1.3	0.2	-1.4	1.1	-0.3	1.0	1.7
------	-----	-----	------	-----	------	-----	-----



0.3	-1.5	2.0	2.4	-1.1	0.6	2.0	-1.7
-----	------	-----	-----	------	-----	-----	------

-2.1	1.3	1.4	-1.4	1.1	-0.3	1.0	1.7
------	-----	-----	------	-----	------	-----	-----

$$0.6 * 3.2 + (1 - 0.6) * 0.2 = 2.0$$

$$(1 - 0.6) * 3.2 + 0.6 * 0.2 = 1.4$$



Recombination (real representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [a_i, b_i]$, for $i = 1, 2, \dots, L$

□ Simple arithmetic crossover

- Select a position k and combine all the genes after that position

$$\text{□ } g_i' = \alpha g_i^1 + (1 - \alpha) g_i^2$$

$$\text{□ } g_i'' = (1 - \alpha) g_i^1 + \alpha g_i^2, \text{ for } i = k, k + 1, \dots, L$$

- Genes from positions $< k$ rest unchanged

$$\text{□ } g_i' = g_i^1$$

$$\text{□ } g_i'' = g_i^2, \text{ for } i = 1, 2, \dots, k - 1$$

$$[a, b] = [-2.5, +3]$$

$$k = 6$$

$$\alpha = 0.6$$

0.3	-1.5	3.2	2.4	-1.1	0.6	2.0	-1.7
-2.1	1.3	0.2	-1.4	1.1	-0.3	1.0	1.7



0.3	-1.5	3.2	2.4	-1.1	0.24	1.6	-0.34
-2.1	1.3	0.2	-1.4	1.1	0.06	1.4	0.34

$$0.6 * 0.6 + (1 - 0.6) * (-0.3) = 0.24$$

$$(1 - 0.6) * 0.6 + 0.6 * (-0.3) = 0.06$$



Recombination (real representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [a_i, b_i]$, for $i = 1, 2, \dots, L$

□ Complete arithmetic crossover

- All of the genes are combined

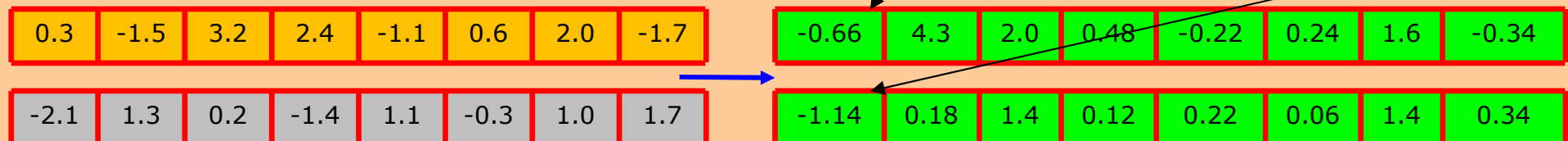
□ $g_i' = \alpha g_i^1 + (1 - \alpha) g_i^2$

□ $g_i'' = (1 - \alpha) g_i^1 + \alpha g_i^2$, for $i = 1, 2, \dots, L$

$[a, b] = [-2.5, +3]$
 $\alpha = 0.6$

$0.6 * 0.3 + (1 - 0.6) * (-2.1) = -0.66$

$(1 - 0.6) * 0.3 + 0.6 * (-2.1) = -1.14$





Recombination (real representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [a_i, b_i]$, for $i = 1, 2, \dots, L$

□ Geometric crossover

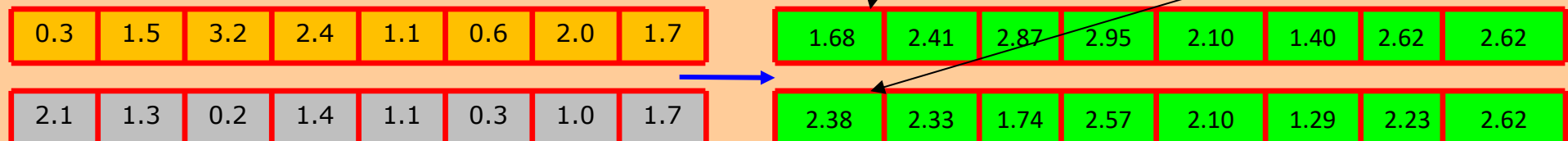
■ Main idea

- Each gene of an offspring represents the product between parent's genes, each of them by a given exponent ω and $1-\omega$, respectively (where ω is a real positive number ≤ 1)

□ $g_i' = (g_i^1)^\omega (g_i^2)^{1-\omega}$

□ $g_i'' = (g_i^1)^{1-\omega} (g_i^2)^\omega$

$[a, b] = [-2.5, +3]$
 $\omega = 0.7$





Recombination (real representation)

- From 2 parent chromosomes
 - $p_1 = (g_1^1, g_2^1, \dots, g_L^1)$ and $p_2 = (g_1^2, g_2^2, \dots, g_L^2)$
- 2 offspring are obtained
 - $c_1 = (g_1', g_2', \dots, g_L')$ and $c_2 = (g_1'', g_2'', \dots, g_L'')$,
 - Where $g_i^1, g_i^2, g_i', g_i'' \in [a_i, b_i]$, for $i = 1, 2, \dots, L$

□ Blend crossover – BLX

■ Main idea

- A single offspring is created
- Offspring's genes are randomly generated from $[Min_i - I * a, Max_i + I * a]$ range, where:
 - $Min_i = \min\{g_i^1, g_i^2\}$, $Max_i = \max\{g_i^1, g_i^2\}$
 - $I = Max - Min$, a – parameter from $[0, 1]$

$$[a, b] = [-2.5, +3]$$

$$a = 0.7$$

0.3	1.5	3.2	2.4	1.1	0.6	2.0	1.7
2.1	1.3	0.2	1.4	1.1	0.3	1.0	1.7



1.25	1.45	-1.11	2.37	1.10	0.11	0.70	1.70
------	------	-------	------	------	------	------	------

Min	0.3	1.3	0.2	1.4	1.1	0.3	1.0	1.7
Max	2.1	1.5	3.2	2.4	1.1	0.6	2.0	1.7
I	0.8	0.2	3.0	1.0	0	0.3	1.0	0.0

Min-Ia	-0.26	1.16	-1.90	0.70	1.10	0.09	0.30	1.70
Max+Ia	2.66	1.50	3.20	2.40	1.10	0.60	2.00	1.70

Recombination or mutation?

□ Intense debates

■ Questions:

- Which is the best operator?
- Which is the most necessary operator?
- Which is the most important operator?

■ Answers:

- Depend on problem, but,
- In general, is better to use both operators
- Each of them having another role (purpose).
- EAs with mutation only are possible, but EAs with crossover only are not possible

□ Search aspects:

- Exploration → discovering promising regions in the search space (accumulating useful information about the problem)
- Exploitation → optimising in a promising region of the search space (by using the existent information)
- Cooperation and competition must exist between these 2 aspects

□ Recombination

- Exploitation operator → performs a large jump into a region somewhere between the regions associated to parents
 - Effects of exploitation decrease while AE is converging
- Binary/n-ary operator that can combine information from 2/more parents
- Operator that does not change the frequency of values from chromosome at the population level

□ Mutation

- Exploration operator → performs small random diversions, remaining in a neighbourhood of parent
 - Local optima escape
- Operator that can introduce new genetic information
- Operator that change the frequency of values from chromosome at the population level

Stop condition



- Choosing a stop condition
 - An optimal solution was found
 - The physical resources were ended
 - A given number of fitness evaluation has been performed
 - The user resources (time, patience) were ended
 - Several generation without improvements have been born

Evaluation



- Performance evaluation of an EA
 - After more runs
 - Statistical measures are computed
 - Average of solutions
 - Median of solutions
 - Best solution
 - Worst solution
 - Standard deviation of solutions – for comparisons
 - The number of independent runs must be large enough

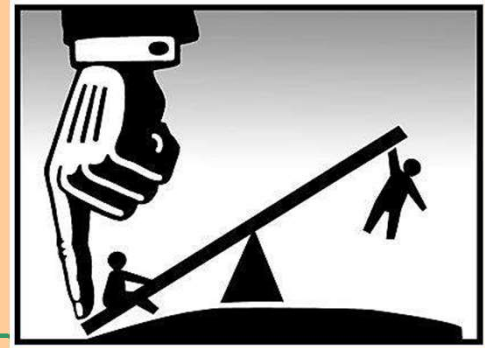
EAs



Analyse of complexity

- The most costly part → fitness evaluation

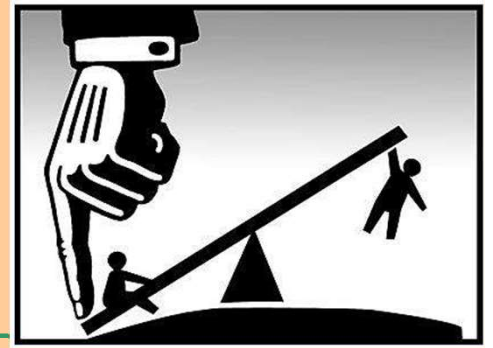
EAs



Advantages

- AEs have a general sketch for all the problems
 - Only
 - representation
 - fitness function
 - are changed
- AEs are able to give better results than classical optimisation methods because
 - They do not require linearization
 - They are not based on some presumptions
 - They do not ignore some possible solutions
- AEs are able to explore more possible solutions than human can

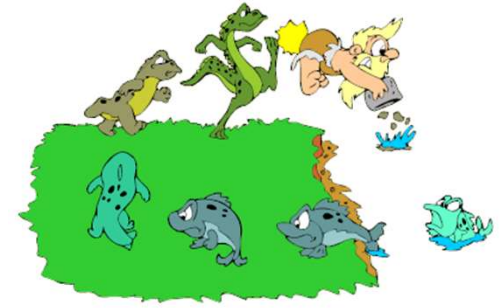
AEs



Disadvantages

- Large running time

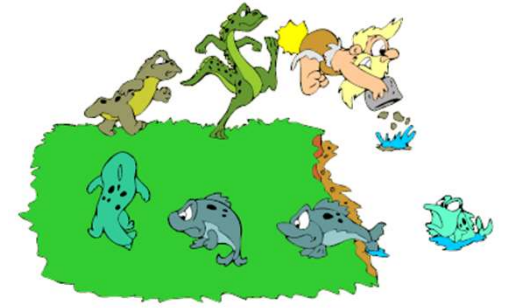
AEs



Applications

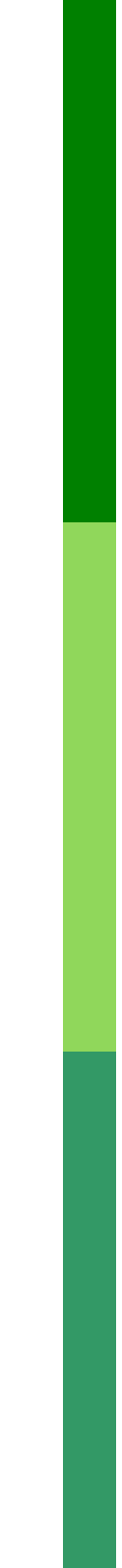
- Vehicle design
 - Material composition
 - Vehicle shape
- Engineering design
 - Structural and organisational optimisation of constructions (buildings, robots, satellites, turbines)
- Robotics
 - Design and components optimisation
- Hardware evolution
 - Digital circuits optimisation
- Telecommunication optimisation
- Cross-word game generation
- Biometric inventions (inspired by natural architectures)
- Traffic and transportation routing
- PC games
- Cryptography
- Genetics
- Chemical analysis of kinematics
- Financial and marketing strategies

AEs



Types

- Evolutionary strategies
- Evolutionary programming
- Genetic algorithms
- Genetic programming



Thank you for your attention!

Modelling and Optimization from the Perspective of Evolutionary Computation

Lecture - 10

Tudor Mihoc

UBB

May 7, 2026

Course content

- 1 Modelling optimization problems.
- 2 Search spaces, decision variables, constraints, and representations.
- 3 Heuristics and evolutionary computation.
- 4 Single-objective, multiobjective, and multimodal optimization.
- 5 Pareto dominance and Pareto fronts.
- 6 Population spreading, diversity, niching, and archiving.

Learning Outcomes

By the end of the course, students should be able to:

- formulate an optimization problem as a mathematical model;
- identify the search space induced by a representation;
- explain why evolutionary algorithms use populations, variation, and selection;
- distinguish single-objective, multiobjective, and multimodal optimization;
- use dominance and Pareto-front concepts correctly;
- select diversity-preserving mechanisms for evolutionary search.

Modelling Optimization Problems

What Is an Optimization Problem?

An optimization problem asks for the best feasible solution according to one or more criteria.

$$\begin{aligned} & \text{minimize} && f(\mathbf{x}) \\ & \text{subject to} && \mathbf{x} \in \Omega, \\ & && g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \\ & && h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p. \end{aligned}$$

From an evolutionary perspective, this model defines the environment in which candidate solutions compete.

Core Modelling Components

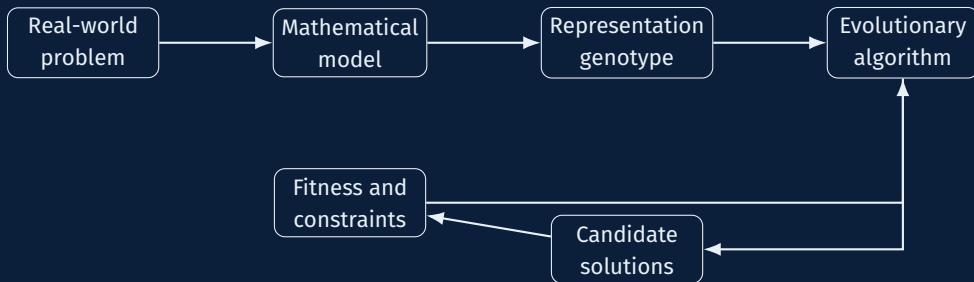
Problem side

- decision variables;
- objective function(s);
- constraints;
- feasible region;
- data, parameters, assumptions.

Evolutionary side

- chromosome or genotype;
- phenotype or decoded solution;
- fitness evaluation;
- variation operators;
- selection pressure.

From Real Problem to Evolutionary Model



Decision Variables and Representations

Problem type	Typical variable	Evolutionary representation
Continuous design	$x_i \in \mathbb{R}$	real-valued vector
Binary selection	$x_i \in \{0, 1\}$	bit string
Scheduling	job order	permutation
Routing	path or tour	permutation / graph encoding
Rule discovery	IF-THEN rules	tree, list, grammar
Neural design	architecture	graph / variable-length encoding

A poor representation can make a simple problem difficult by destroying useful neighbourhood structure.

Objective Function vs Fitness Function

- The **objective function** expresses what the model wants to optimize.
- The **fitness function** is what the evolutionary algorithm uses for selection.
- They are often the same, but not always.

Examples where they differ:

- constrained problems with penalty terms;
- multiobjective problems with ranking and diversity measures;
- noisy problems where fitness is an estimate;
- surrogate-assisted optimization where fitness is predicted.

Constraint Handling in Evolutionary Computation

Common strategies:

- **Penalties:** reduce fitness for constraint violations.
- **Repair:** transform infeasible candidates into feasible ones.
- **Decoders:** map every genotype to a feasible phenotype.
- **Feasibility rules:** feasible candidates dominate infeasible ones.
- **Specialized operators:** preserve feasibility during mutation and crossover.

Bad constraint handling wastes the population on candidates that cannot solve the actual problem.

Search Space

The Search Space

The search space is the set of all candidate solutions that the algorithm can generate.

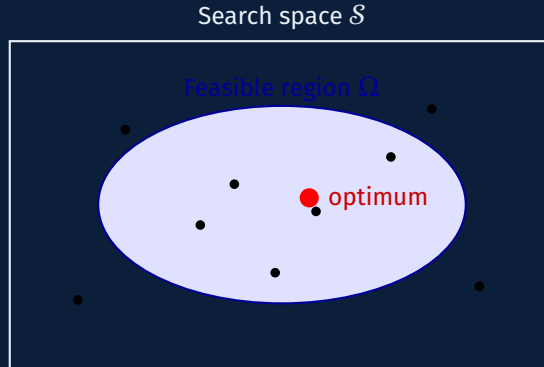
$$\mathcal{S} = \{\mathbf{x} : \mathbf{x} \text{ is representable by the chosen encoding}\}.$$

The feasible space is usually smaller:

$$\Omega \subseteq \mathcal{S}.$$

Evolutionary computation explores $\mathcal{S}$ through populations, variation operators, and selection.

Search Space, Feasible Region, and Optimum



Landscape Thinking

Evolutionary algorithms do not only see a formula; they experience a landscape.

- **Peaks or valleys:** high-quality regions.
- **Ruggedness:** many local optima.
- **Neutrality:** many candidates with similar fitness.
- **Epistasis:** variable interactions.
- **Deception:** local improvements lead away from the global optimum.

The same mathematical objective may become easy or hard depending on the representation and operators.

Exploration and Exploitation

Exploration

- sample new regions;
- avoid premature convergence;
- maintain diversity;
- use larger mutations or recombination.

Exploitation

- refine good regions;
- increase selection pressure;
- use local search or small mutations;
- preserve elite solutions.

Optimization quality depends on balancing both, not blindly maximizing either.

Heuristics and Evolutionary Computation

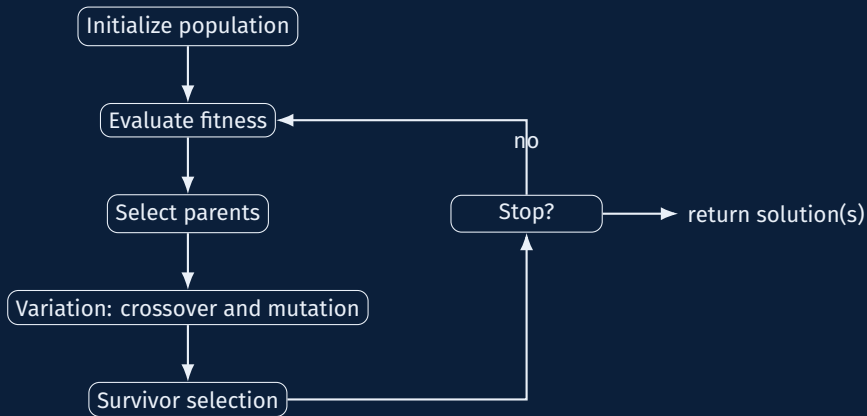
Heuristics

A heuristic is a practical rule for finding good solutions without guaranteeing optimality.

- Useful when exact optimization is too expensive.
- Often problem-specific.
- May provide fast but approximate answers.

A metaheuristic is a higher-level search framework that can be adapted to many problems. Evolutionary algorithms are population-based metaheuristics.

Evolutionary Algorithm: Generic Loop



Main Evolutionary Design Choices

- Population size.
- Encoding and decoding method.
- Initialization strategy.
- Parent selection method.
- Mutation and crossover operators.
- Replacement and elitism policy.
- Fitness assignment.
- Diversity preservation method.
- Termination criterion.

Each choice changes the effective search process.

Single-Objective Optimization

Single-Objective Optimization

A single-objective problem has one criterion:

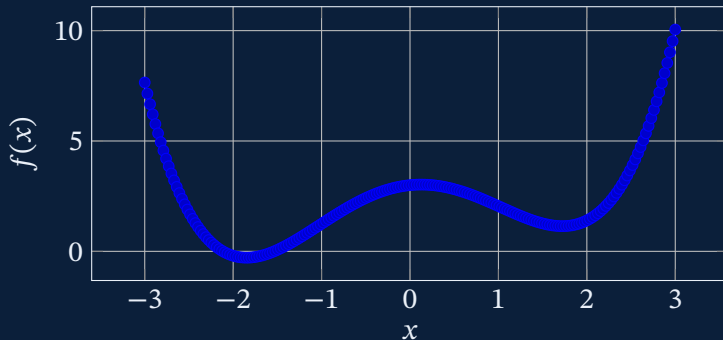
$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x} \in \Omega} f(\mathbf{x}).$$

Evolutionary algorithms approximate $\mathbf{x}^*$ by repeatedly improving a population.

Typical performance indicators:

- best fitness found;
- convergence speed;
- robustness across runs;
- number of function evaluations.

Fitness Landscape Example



A population can sample several regions before selection concentrates search around promising basins.

Selection Pressure

Selection pressure determines how strongly good individuals influence the next generation.

- Low pressure: slow progress but higher diversity.
- High pressure: fast progress but higher risk of premature convergence.

Common methods:

- fitness-proportionate selection;
- rank selection;
- tournament selection;
- elitist replacement.

Multiobjective Optimization

Multiobjective Optimization

A multiobjective problem optimizes several criteria at once:

$$\text{minimize } \mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_k(\mathbf{x})), \quad \mathbf{x} \in \Omega.$$

Usually, objectives conflict.

The goal is no longer one best solution. The goal is a set of trade-off solutions.

Why Evolutionary Algorithms Fit Multiobjective Problems

Evolutionary algorithms naturally maintain a population of alternatives.

- A population can approximate many trade-offs in one run.
- Selection can use dominance instead of scalar fitness.
- Diversity mechanisms can distribute solutions along the front.
- Archiving can preserve non-dominated solutions discovered over time.

This is why algorithms such as NSGA-II, SPEA2, and MOEA/D became standard references in evolutionary multiobjective optimization.

Standard Evolutionary Multiobjective Algorithms

Three Standard Evolutionary Multiobjective Algorithms

Common goal

- approximate the Pareto front;
- keep a diverse set of trade-off solutions;
- balance convergence and spread.

Shared evolutionary structure

- population of candidate solutions;
- variation through crossover and mutation;
- selection based on dominance or decomposition;
- elitism to preserve high-quality solutions.

Algorithm

Main idea

NSGA-II

Non-dominated sorting + crowding

SPEA2

External archive + strength fitness

MOEA/D

Decompose MOP into subproblems

Perspective

All three are evolutionary algorithms, but they define selection pressure differently.

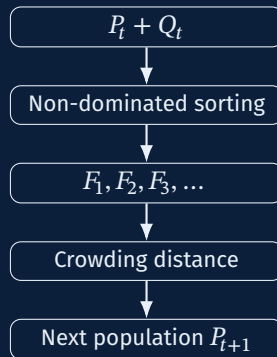
NSGA-II: Non-Dominated Sorting Genetic Algorithm II

Mechanism: rank the population into Pareto fronts, then preserve diversity using crowding distance.

- 1 Combine parents and offspring: $R_t = P_t \cup Q_t$.
- 2 Sort R_t into fronts $F_1, F_2, \dots$.
- 3 Fill the next population with the best fronts first.
- 4 If the last accepted front does not fit, select the least crowded individuals.

Selection rule:

- lower Pareto rank is better;
- for equal rank, larger crowding distance is better.

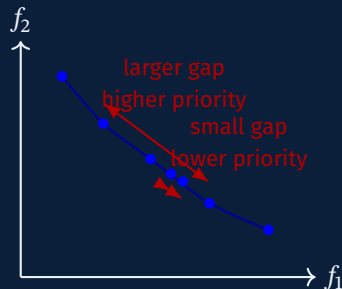


NSGA-II: Crowding Distance for Population Spread

Crowding distance estimates how isolated a solution is inside the same Pareto front.

For each objective, solutions are sorted. Boundary solutions receive very large distance. Interior solutions receive distance based on neighbouring objective values:

$$CD_i = \sum_{k=1}^M \frac{f_k(i+1) - f_k(i-1)}{f_k^{\max} - f_k^{\min}}.$$



Effect: sparse regions survive more often; clusters are reduced; extreme trade-offs are preserved.

SPEA2: Strength Pareto Evolutionary Algorithm 2

Mechanism: use an external archive and assign fitness using dominance strength plus density.

Strength value

$$S(i) = |\{j \mid i \text{ dominates } j\}|.$$

Raw fitness

$$R(i) = \sum_{j \text{ dominates } i} S(j).$$

Final fitness

$$F(i) = R(i) + D(i).$$

Here $D(i)$ is a density estimate, usually based on the distance to the k -th nearest neighbour.

Algorithmic steps

- 1 Combine population and archive.
- 2 Compute strength, raw fitness, and density.
- 3 Copy non-dominated solutions into the archive.
- 4 If archive is too large, truncate crowded regions.
- 5 Generate offspring from archive members.

SPEA2: Archive and Truncation

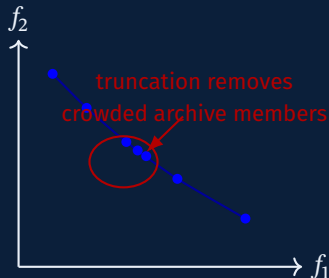
The archive is an elitist memory of high-quality solutions.

If archive is too small:

- add dominated individuals with best fitness.

If archive is too large:

- remove individuals from the most crowded regions;
- preserve boundary and well-spaced solutions;
- maintain a compact approximation of the front.



Interpretation

SPEA2 separates evolutionary search from elite preservation more explicitly than NSGA-II.

MOEA/D: Multiobjective Evolutionary Algorithm Based on Decomposition

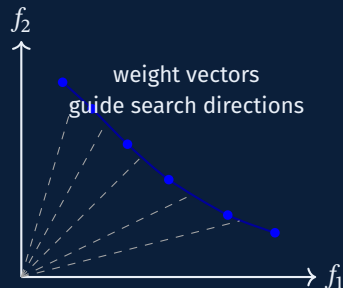
Mechanism: transform one multiobjective problem into many scalar subproblems.

Each subproblem has a weight vector λ^i and optimizes one scalar aggregation, for example the weighted sum:

$$g(x \mid \lambda^i) = \sum_{m=1}^M \lambda_m^i f_m(x).$$

A common alternative is the Chebyshev function:

$$g(x \mid \lambda^i, z^*) = \max_m \lambda_m^i |f_m(x) - z_m^*|.$$



Neighbourhood: similar weight vectors define neighbouring subproblems; mating and replacement are often local.

NSGA-II vs SPEA2 vs MOEA/D

Aspect	NSGA-II	SPEA2	MOEA/D
Main selection idea	Pareto rank + crowding distance	Strength fitness + archive density	Scalar subproblems with weight vectors
Elitism	Parent-offspring union	Explicit external archive	Best solutions for sub-problems
Diversity control	Crowding distance	Density and archive truncation	Weight-vector distribution
Best suited for	General multiobjective GA baseline	Strong archive-based Pareto search	Structured fronts and decomposition-friendly problems
Main weakness	Crowding can degrade in many objectives	Archive management can be costly	Performance depends on decomposition and weight vectors

Pareto Dominance

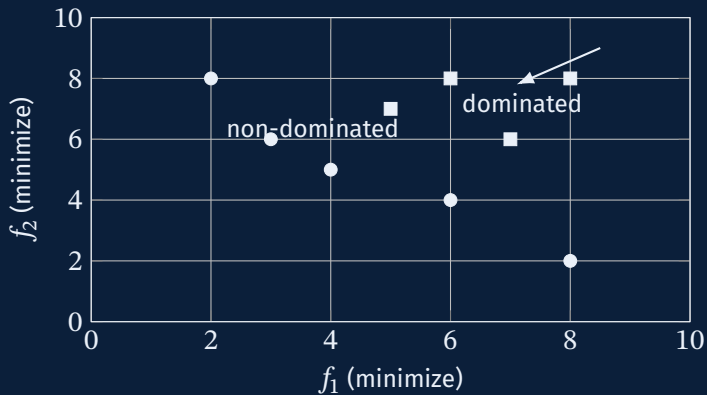
For minimization, solution **a** dominates solution **b** if:

$$\forall i : f_i(\mathbf{a}) \leq f_i(\mathbf{b}) \quad \text{and} \quad \exists j : f_j(\mathbf{a}) < f_j(\mathbf{b}).$$

Interpretation: **a** is no worse in every objective and strictly better in at least one objective. A

solution is **non-dominated** if no other known solution dominates it.

Dominance Example



Pareto Set and Pareto Front

- The **Pareto set** is the set of non-dominated solutions in decision space.
- The **Pareto front** is their image in objective space.

$$PF = \{f(\mathbf{x}) : \mathbf{x} \in \Omega, \nexists \mathbf{y} \in \Omega \text{ such that } \mathbf{y} \text{ dom } \mathbf{x}\}.$$

Evolutionary multiobjective optimization tries to approximate both convergence to the true front and diversity along the front.

Convergence vs Diversity in Multiobjective Search

Convergence

- move toward the true Pareto front;
- prefer non-dominated or less dominated solutions;
- improve objective values.

Diversity

- cover the front broadly;
- avoid clustering in one region;
- preserve extreme and intermediate trade-offs.

A front with excellent convergence but poor spread is still a weak approximation.

Multimodal Optimization

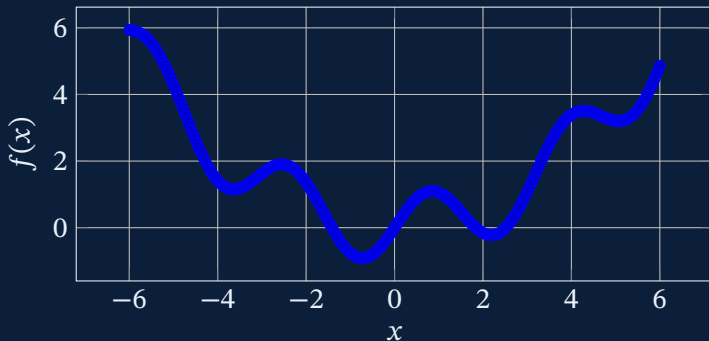
Multimodal Optimization

A multimodal problem has multiple local or global optima.

- In single-objective optimization, the aim may be to find the global optimum.
- In multimodal optimization, the aim may be to identify several good optima.
- In real applications, alternative optima may represent useful design choices.

Evolutionary populations are suitable because they can maintain multiple subpopulations around different basins.

Multimodal Landscape



Without diversity preservation, selection may collapse the whole population into one basin.

Niching in Evolutionary Computation

Niching methods encourage the population to occupy several promising regions.

- **Fitness sharing:** reduce fitness in crowded niches.
- **Crowding:** offspring compete with similar individuals.
- **Speciation:** divide the population into groups.
- **Clearing:** keep a few winners per niche and suppress nearby competitors.
- **Island models:** evolve subpopulations with limited migration.

Spreading the Population

Why Spread the Population?

Population spreading is used to prevent genetic collapse and improve coverage.

- In single-objective search: avoid premature convergence.
- In multiobjective search: approximate the whole Pareto front.
- In multimodal search: preserve multiple optima.
- In dynamic problems: retain adaptability when the landscape changes.

Mechanisms for Spreading a Population

Mechanism	Main effect
Random or stratified initialization	broad initial coverage
Mutation adaptation	controlled exploration
Crowding distance	spread along Pareto fronts
Fitness sharing	penalize dense regions
Niching / speciation	preserve distinct basins
Archive management	retain diverse elite solutions
Island model migration	balance isolation and information flow
Restart strategies	recover from stagnation

Crowding Distance

Crowding distance estimates how isolated a solution is in objective space.

- Boundary solutions usually receive high priority.
- Interior solutions receive larger distance if neighbours are far away.
- In NSGA-II, non-dominated sorting handles convergence, while crowding distance handles spread.

Crowding distance does not require a user-defined niche radius, but it can struggle in many-objective settings.

Fitness Sharing

Fitness sharing reduces the reward of individuals in crowded areas.

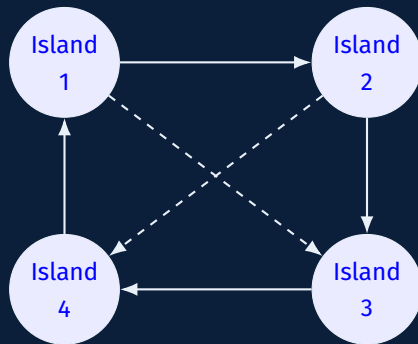
$$F'_i = \frac{F_i}{\sum_{j=1}^N sh(d(i, j))}.$$

A common sharing function is:

$$sh(d) = \begin{cases} 1 - \left(\frac{d}{\sigma_{share}}\right)^\alpha, & d < \sigma_{share}, \\ 0, & d \geq \sigma_{share}. \end{cases}$$

It is powerful, but sensitive to the distance metric and sharing radius.

Island Models



- Each island evolves mostly independently.
- Migration shares useful genetic material.
- Topology, frequency, and migrant selection control the exploration-exploitation balance.

Diversity Measures

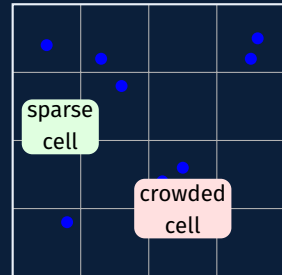
Useful diversity indicators include:

- average pairwise distance in decision space;
- average pairwise distance in objective space;
- entropy of genes or alleles;
- number of occupied niches;
- spread or spacing along a Pareto front;
- hypervolume contribution distribution.

There is no universal diversity measure. The correct measure depends on the modelling goal.

Grid-Based Dispersion: Main Idea

- The search space (or objective space) is divided into a **grid of cells**.
- Each individual is assigned to one cell according to its position.
- The number of individuals per cell estimates the **local density**.
- **Crowded cells** are penalized, while **sparse occupied cells** are encouraged.
- The goal is to maintain **population diversity** and avoid premature convergence.



Evolutionary meaning: better coverage of the search space; support for multimodal and multiobjective search; a simple alternative to fitness sharing or crowding.

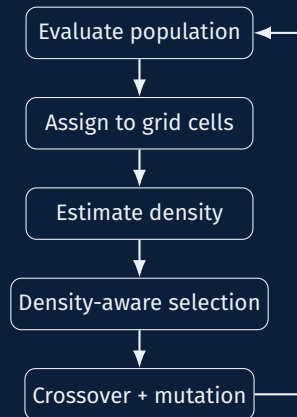
Mechanism in a Genetic Algorithm

- 1 Initialize and evaluate the population.
- 2 Map each individual to a grid cell.
- 3 Compute the cell occupancy n_c .
- 4 Modify selection or survival using density information.

Example:

$$F'_i = \frac{F_i}{1 + \alpha n_c}$$

- F_i = original fitness of individual i ;
- n_c = number of individuals in its cell;
- α = strength of the dispersion pressure.



Alternative uses of the grid

- select parents from less crowded cells;
- delete offspring from overcrowded cells;
- increase mutation for dense regions.

Benefits, Design Choices, and Limitations

Benefits

- preserves diversity;
- improves exploration;
- helps discover multiple peaks or trade-offs;
- easy to combine with standard GA operators.

Limitations

- too coarse a grid hides structure;
- too fine a grid becomes unstable or sparse;
- effectiveness depends on the distance notion induced by the grid;
- may require tuning for high-dimensional problems.

Design choices: grid built in **decision space** or **objective space**; number of cells / resolution; penalty strength α ; when to apply the density correction.

Conclusions

Choosing the Right Evolutionary Approach

Problem structure	Main goal	Suitable EC idea
Single objective	best solution	GA, ES, DE, local hybrid
Constrained	feasible high-quality solution	repair, penalties, decoders
Multiobjective	trade-off set	NSGA-II, SPEA2, MOEA/D
Multimodal	several optima	niching, sharing, clearing
Dynamic	adapt over time	memory, diversity, immigrants
Expensive evaluation	reduce evaluations	surrogate-assisted EA

Example: Modelling a Scheduling Problem

Problem: assign jobs to machines and minimize completion time.

- Decision variables: job order and machine assignment.
- Objective: minimize makespan C_{max} .
- Constraints: machine capacity, precedence, release times.
- Representation: permutation plus assignment vector.
- Variation: order crossover, swap mutation, reassignment mutation.
- Constraint handling: decoder that builds feasible schedules.

A multiobjective version may also minimize energy use, lateness, or cost.

Common Modelling Errors

- Optimizing the wrong objective because the real goal was not translated correctly.
- Using an encoding that generates mostly infeasible candidates.
- Ignoring scale differences between objectives or constraints.
- Treating a multiobjective problem as single-objective too early.
- Confusing population diversity with solution quality.
- Reporting one run without measuring stochastic variability.

Practical Checklist

Before running an evolutionary algorithm, ask:

- 1 What exactly is being optimized?
- 2 What is a valid solution?
- 3 What does the representation allow the algorithm to generate?
- 4 How are constraints handled?
- 5 Is the problem single-objective, multiobjective, multimodal, or several of these?
- 6 How will convergence and diversity be measured?
- 7 What baseline methods will be used for comparison?

Suggested Exercises

- 1 Model a knapsack problem as a binary evolutionary optimization problem.
- 2 Convert a single-objective routing problem into a two-objective problem.
- 3 Given ten objective vectors, identify the non-dominated set.
- 4 Design a mutation operator for a permutation representation.
- 5 Compare crowding distance and fitness sharing for population spreading.

Main Points

- Optimization starts with modelling, not with algorithms.
- The representation defines the search space seen by evolution.
- Evolutionary computation is useful because it searches with populations.
- Single-objective, multiobjective, and multimodal problems require different success criteria.
- Pareto dominance replaces a single fitness ranking in multiobjective search.
- Population spreading is not decorative; it is central to robust evolutionary optimization.

Artificial Intelligence
Intelligent Systems and Rule-Based Systems under Uncertainty
Fuzzy Logic and Fuzzy Inference Systems

Babeş-Bolyai University
Faculty of Mathematics and Computer Science

- Intelligent systems and knowledge-based systems
- Sources and representations of uncertainty
- Fuzzy sets, fuzzy variables, and membership functions
- Fuzzification, fuzzy rule bases, and inference
- Mamdani, Sugeno, and Tsukamoto inference models
- Aggregation, defuzzification, advantages, limitations, and applications

Knowledge-based intelligent systems

General structure

A knowledge-based system is a computational system whose behaviour is driven by explicit domain knowledge and an inference mechanism.

Knowledge base

- domain-specific facts;
- concepts and relationships;
- rules supplied by experts or extracted from data.

Inference engine

- derives new information;
- applies rules to known facts;
- uses domain-independent reasoning procedures.

Knowledge base: information and classification

The knowledge base contains information about the environment and the problem to be solved.

Perfect information

- classical logic;
- propositions are either true or false;
- if A is true, then $\neg A$ is false;
- if B is false, then $\neg B$ is true.

Imperfect information

- non-exact information;
- incomplete information;
- vague or incommensurable observations;
- partial confidence in facts or rules.

Uncertainty in knowledge-based systems

Sources of uncertainty

- imperfect rules;
- doubtful or weak rules;
- vague natural language;
- noisy or incomplete observations.

Common representations

- probabilities;
- fuzzy logic;
- Bayes' theorem;
- Dempster–Shafer theory;
- certainty factors, confidence values, truth values;
- intervals: lower and upper uncertainty bounds.

Fuzzy systems in knowledge-based AI

- Fuzzy systems model imprecise concepts such as *tall*, *cold*, *young*, *high risk*, or *moderate speed*.
- They use degrees of membership rather than strict yes/no membership.
- They are suitable for rule-based control and decision systems where expert knowledge is linguistic.

Central idea

Instead of asking whether an observation belongs to a class, fuzzy logic asks *to what degree* it belongs to that class.

Why fuzzy logic?

Problem

How should a program represent statements such as:

- “George is tall.”
- “It is cold outside.”
- “The temperature is almost normal.”

Fuzzy logic is useful for:

- natural-language queries;
- knowledge representation in knowledge-based systems;
- control systems affected by noisy, gradual, or imprecise phenomena.

A short historical note

- **Parmenides and Aristotle**: classical bivalent reasoning; propositions are true or false.
- **Aristotle's law of the excluded middle**: every proposition is either true or false.
- **Plato**: philosophical discussion of a third region between strict truth and strict falsity.
- **Łukasiewicz**: many-valued logic, including truth, falsehood, and possibility.
- **Lotfi A. Zadeh, 1965**: formal theory of fuzzy sets and fuzzy logic, where truth values may belong to $[0, 1]$.

Fuzzy logic versus Boolean logic

Classical logic

- binary truth values;
- true or false;
- yes or no;
- black or white.

Fuzzy logic

- gradual truth values;
- degrees between 0 and 1;
- partial membership;
- intermediate states.

$$a \wedge b = \min(a, b), \quad a \vee b = \max(a, b), \quad \neg a = 1 - a.$$

Certainty theory

- “Popescu is young.”
- The statement is treated as true or false.

Probability theory

- “There is an 80% chance that Popescu is young.”
- The uncertainty concerns the occurrence of the event.

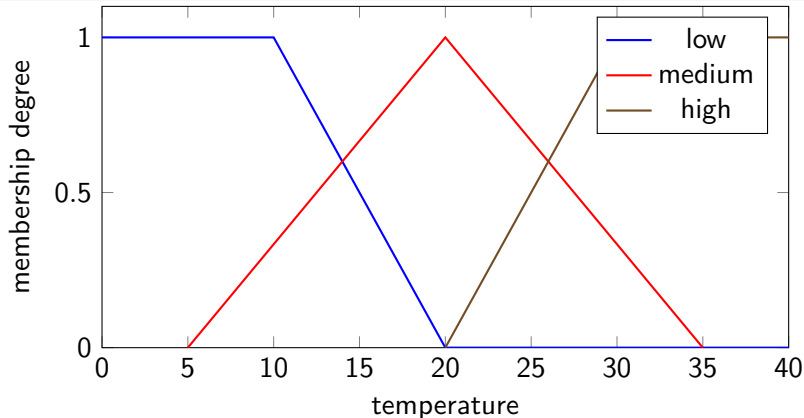
Fuzzy logic

- “Popescu belongs to the class of young people with degree 0.8.”
- The uncertainty concerns the gradual meaning of the concept *young*.

Real phenomena involve fuzzy sets

Example: room temperature

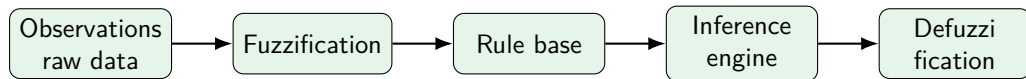
The temperature of a room may be described as *low*, *medium*, or *high*. These categories may overlap.



Steps for constructing a fuzzy system

1. Define the raw inputs and outputs.
2. Define fuzzy variables and fuzzy sets using membership functions.
3. Construct the rule base, usually with expert knowledge.
4. Evaluate the rules with a fuzzy inference mechanism.
5. Aggregate the rule outputs into a single fuzzy output set.
6. Defuzzify the aggregated fuzzy set into a crisp value.
7. Interpret the result in the application context.

Fuzzy system pipeline



Interpretation

The pipeline translates numerical observations into linguistic degrees, reasons with rules, and returns a crisp action or decision.

Core elements of fuzzy logic

- **Fuzzy facts / fuzzy sets:** definition, representation, and operations.
- **Set operations:** complement, containment, intersection, union, equality, algebraic product, and algebraic sum.
- **Properties:** associativity, commutativity, distributivity, transitivity, idempotence, identity, and involution.
- **Hedges:** linguistic modifiers such as *very*, *more or less*, or *approximately*.
- **Fuzzy variables:** symbolic variables defined by labels and membership functions.

Classical sets and fuzzy sets

Classical set

Let X be a universe and $R \subseteq X$ a regular set. Its characteristic function is

$$\mu_R : X \rightarrow \{0, 1\}.$$

An element either belongs to R or it does not.

Fuzzy set

A fuzzy set $F \subseteq X$ is described by a membership function

$$\mu_F : X \rightarrow [0, 1], \quad \mu_F(x) = g.$$

Here g is the degree to which x belongs to F .

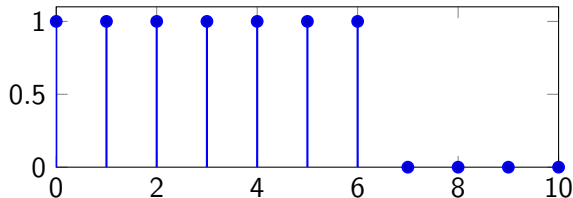
Example: regular set versus fuzzy set

Let $X = \{0, 1, \dots, 10\}$.

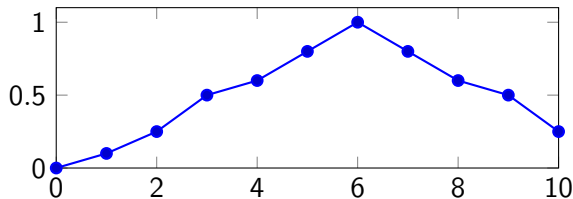
- R : natural numbers smaller than 7.
- F : natural numbers close to 6.

x	$\mu_R(x)$	$\mu_F(x)$
0	1	0
1	1	0.1
2	1	0.25
3	1	0.5
4	1	0.6
5	1	0.8
6	1	1
7	0	0.8
8	0	0.6
9	0	0.5
10	0	0.25

Regular set R

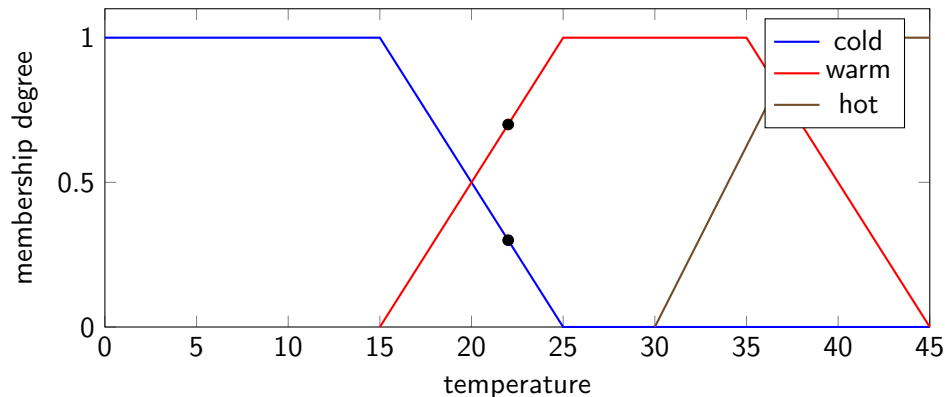


Fuzzy set F



Example: fuzzy temperature labels

A temperature t can belong to several linguistic labels at the same time.



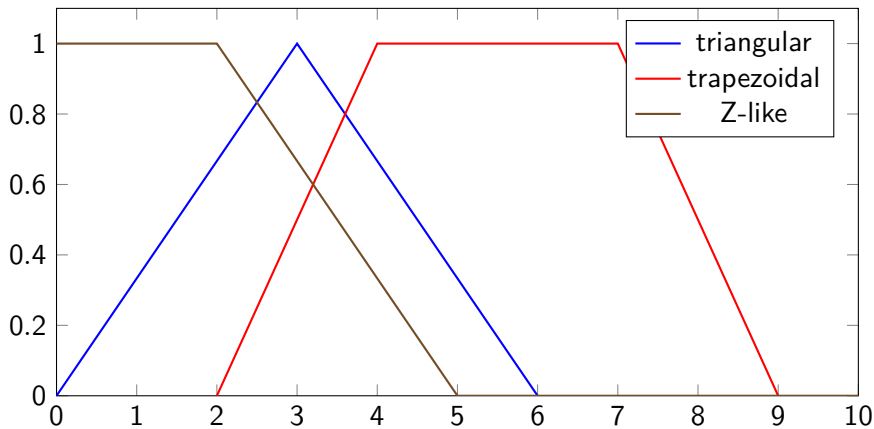
At one temperature, the membership degree may be 0.7 for *cold* and 0.3 for *warm*, depending on the membership functions.

Membership function shapes

- **Singleton:** $\mu(x) = s$ at one point, with zero elsewhere.
- **Triangular:** gradual increase up to a peak, followed by gradual decrease.
- **Trapezoidal:** gradual increase, plateau, and gradual decrease.
- **Z function:** decreasing S-shaped function, often written as $Z(x) = 1 - S(x)$.
- **Π function:** plateau-like function obtained by combining increasing and decreasing shapes.

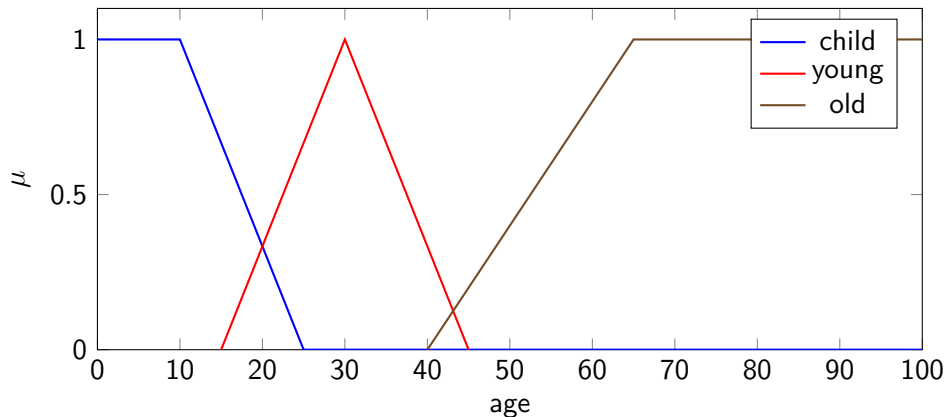
$$\mu_{\Delta}(x) = \max \left(0, \min \left\{ \frac{x - a}{b - a}, \frac{c - x}{c - b} \right\} \right).$$

Common membership functions



Membership functions are design choices. Their shape may be expert-defined, data-driven, or adjusted empirically.

Example: age as a fuzzy variable



A person aged 42 may still partly belong to *young* and may begin to belong to *old*, depending on the modelling convention.

Fuzzy variable

A fuzzy variable is usually defined as

$$V = (x, L, U, M),$$

where:

- x is the name of the symbolic variable;
- L is the set of linguistic labels;
- U is the universe of discourse;
- M is the set of semantic regions, represented by membership functions.

Example

For temperature: $L = \{\text{low, medium, high}\}$ and $U = [-70, 70]$ degrees.

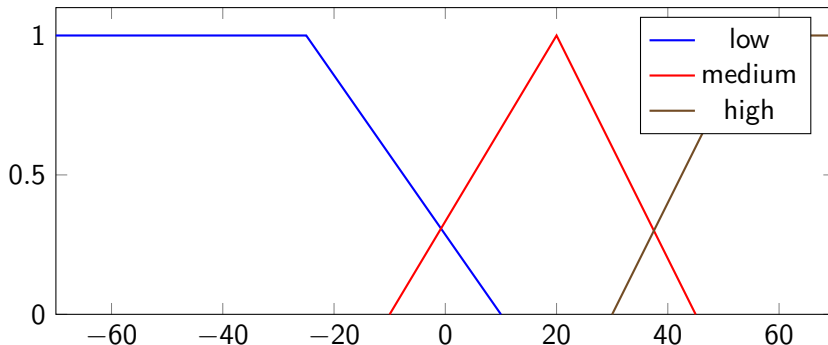
How membership functions are chosen

- **Subjective assessment:** experts define the shapes.
- **Ad-hoc assessment:** simple functions are selected because they solve the practical problem.
- **Data-based assessment:** distributions and measurements are used to shape the functions.
- **Adaptive assessment:** functions are adjusted through testing.
- **Automated assessment:** learning algorithms infer suitable membership functions from training data.

Property: completeness

A fuzzy variable is complete if every value in the universe belongs to at least one fuzzy set with positive degree:

$$\forall x \in X, \quad \exists A \text{ such that } \mu_A(x) > 0.$$



Without completeness, some input values cannot be interpreted by the rule base.

Property: unit partition

A fuzzy variable forms a unit partition if, for every input value x , the sum of the memberships is one:

$$\sum_{i=1}^p \mu_{A_i}(x) = 1,$$

where p is the number of fuzzy sets that cover x .

- There is no universal rule for defining neighbouring fuzzy sets.
- In practice, neighbouring sets often overlap by roughly 25–50%.

Normalising a complete fuzzy variable

A complete fuzzy variable can be transformed into a unit partition by normalising its membership degrees:

$$\mu'_{A_i}(x) = \frac{\mu_{A_i}(x)}{\sum_{j=1}^p \mu_{A_j}(x)}, \quad i = 1, \dots, p.$$

Purpose

Normalisation ensures that all membership degrees associated with an input value form a coherent distribution over the fuzzy labels.

Fuzzification of input data

Fuzzification transforms crisp observations into degrees of membership.

1. Identify raw inputs and outputs.
2. Define a set of membership functions for each variable.
3. Associate each membership function with a linguistic label.
4. For a given raw input, compute the membership degree for each fuzzy set.

Example

For temperature $T = 5^\circ$, one may obtain

$$\mu_{A_1}(T) = 0.6, \quad \mu_{A_2}(T) = 0.4.$$

Example: air-conditioner inputs and output

Inputs

- x : temperature with labels *cold*, *normal*, *hot*;
- y : humidity with labels *low*, *high*.

Output

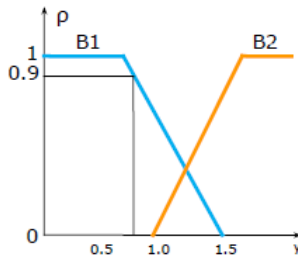
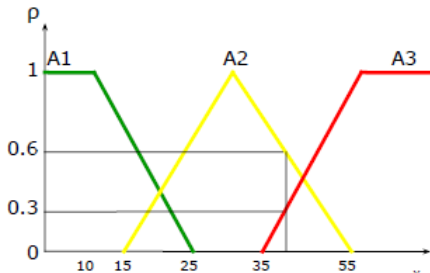
- z : machine power with labels *low*, *medium*, *high*.

Input data

$$x = 37, \quad y = 0.8.$$

$$\mu_{A_1}(x) = 0, \quad \mu_{A_2}(x) = 0.6, \quad \mu_{A_3}(x) = 0.3.$$

$$\mu_{B_1}(y) = 0.9, \quad \mu_{B_2}(y) = 0.$$



A rule is a linguistic construction that links conditions to consequences.

- Non-conditional rule: *Save energy*.
- Simple conditional rule: if x is A_i , then z is C_k .
- Conjunctive rule: if x is A_i and y is B_j , then z is C_k .
- Disjunctive rule: if x is A_i or y is B_j , then z is C_k .

Terminology

The condition part is the *premise*; the conclusion part is the *consequence*.

Classical rules versus fuzzy rules

	Classical logic	Fuzzy logic
R_1	If temperature is -5° , then it is cold.	If temperature is low, then it is cold.
R_2	If temperature is 15° , then it is warm.	If temperature is medium, then it is warm.
R_3	If temperature is 35° , then it is hot.	If temperature is high, then it is hot.

Key distinction

Fuzzy rules operate on linguistic regions rather than on isolated exact values.

For two inputs x and y , a rule base can be organised as a grid of rules:

$$R_{ij} : \quad \text{if } x \text{ is } A_i \text{ and } y \text{ is } B_j, \text{ then } z \text{ is } C_{ij}.$$

- $A_1, \dots, A_m$ are fuzzy sets for input x .
- $B_1, \dots, B_n$ are fuzzy sets for input y .
- C_{ij} is the output fuzzy set assigned by the expert.

Decision matrix for the air conditioner

Temperature / humidity	low humidity	high humidity
cold	low power	medium power
normal	medium power	high power
hot	high power	high power

Example rule

If the temperature is normal and the humidity is low, then the machine power is medium.

Fuzzy inference

- Fuzzy rules are evaluated in parallel.
- Each activated rule contributes to the shape of the final output.
- The output fuzzy sets are aggregated after all rules have been evaluated.
- The aggregated result is then defuzzified into a crisp value.

Practical consequence

A fuzzy system usually does not select one single winning rule. It combines the effects of several partially satisfied rules.

Evaluating rule premises

For each rule premise, compute the membership degree of the input in the relevant fuzzy sets.

$$\mu_{A \cap B}(x) = \min\{\mu_A(x), \mu_B(x)\},$$

$$\mu_{A \cup B}(x) = \max\{\mu_A(x), \mu_B(x)\},$$

$$\mu_{\neg A}(x) = 1 - \mu_A(x).$$

The result of premise evaluation is also called the *rule firing strength* or *degree of fulfilment*.

The consequence part determines how a rule contributes to the output.

- **Mamdani model:** the output variable belongs to a fuzzy set.
- **Sugeno model:** the output variable is a crisp function of the inputs.
- **Tsukamoto model:** the output variable belongs to a fuzzy set with a monotone membership function, producing a crisp value per rule.

Consequents in fuzzy inference models

Distinction

The main difference between Mamdani, Sugeno, and Tsukamoto inference is the representation of the **rule consequent**.

IF x is A AND y is B THEN consequent

Model	Consequent representation
Mamdani	Fuzzy set: z is C
Sugeno	Crisp function: $z = f(x, y)$
Tsukamoto	Monotone fuzzy set: z is C_{monotone}

- Mamdani produces fuzzy output regions that require aggregation and defuzzification.
- Sugeno and Tsukamoto produce crisp rule outputs that are usually combined by weighted averaging.

Main idea

In a Mamdani rule, the consequence is a fuzzy set.

if x is A and y is B , then z is C .

The firing strength of the premise is applied to the membership function of the consequence.

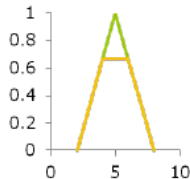
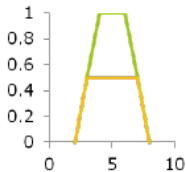
Two common variants:

- clipped fuzzy sets;
- scaled fuzzy sets.

Mamdani consequence: clipping versus scaling

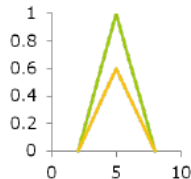
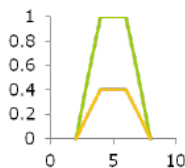
Clipped fuzzy set

- cuts the output membership function at the firing-strength level;
- easy to compute;
- may lose shape information.



Scaled fuzzy set

- multiplies the output membership function by the firing strength;
- preserves more shape information;
- computationally more demanding.



Mamdani example: rule firing strengths

Given:

$$\begin{aligned}\mu_{A_1}(x) &= 0, & \mu_{A_2}(x) &= 0.6, & \mu_{A_3}(x) &= 0.3, \\ \mu_{B_1}(y) &= 0.9, & \mu_{B_2}(y) &= 0.\end{aligned}$$

R_1 : if $x \in A_1$ and $y \in B_1$ then $z \in C_1$,

$$\alpha_1 = \min(0, 0.9) = 0,$$

R_2 : if $x \in A_2$ or $y \in B_1$ then $z \in C_2$,

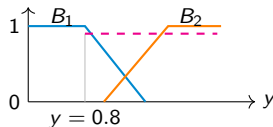
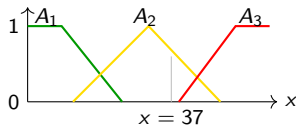
$$\alpha_2 = \max(0.6, 0.9) = 0.9,$$

R_3 : if $x \in A_3$ or $y \in B_2$ then $z \in C_3$,

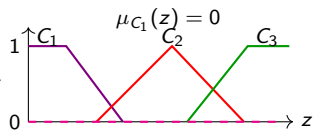
$$\alpha_3 = \max(0.3, 0) = 0.3.$$

Mamdani model: rule evaluation

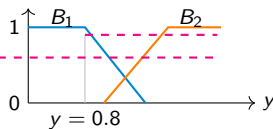
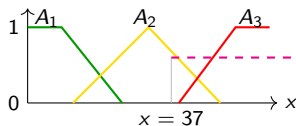
R1: if x is in A_1 and y is in B_1 then z is in C_1



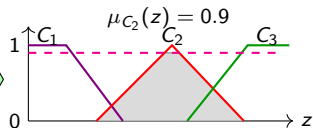
AND
(min)



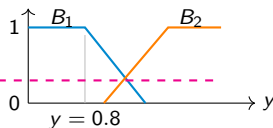
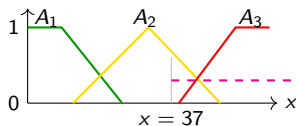
R2: if x is in A_2 or y is in B_1 then z is in C_2



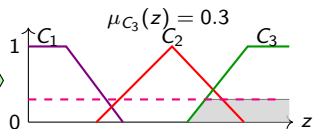
OR
(max)



R3: if x is in A_3 or y is in B_2 then z is in C_3



OR
(max)



Main idea

In a Sugeno rule, the consequence is a crisp function of the inputs.

if x is A and y is B , then $z = f(x, y)$.

- **Zero-order Sugeno:** $f(x, y) = k$, a constant.
- **First-order Sugeno:** $f(x, y) = ax + by + c$.

The final output is commonly computed as a weighted average of rule outputs.

Sugeno model: weighted output

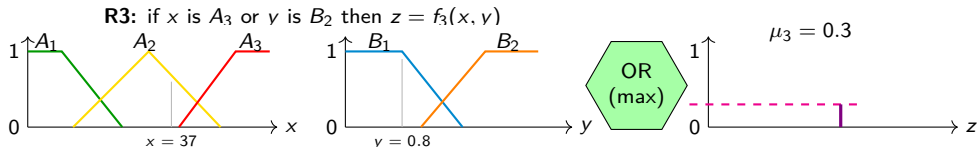
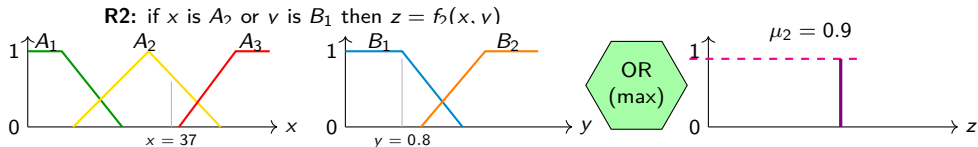
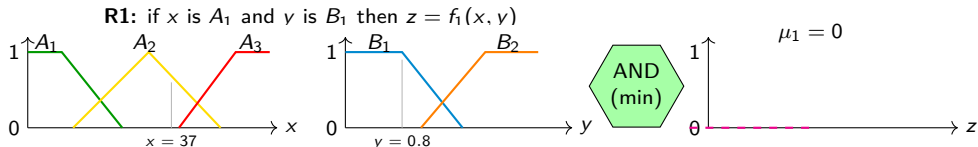
If α_i is the firing strength of rule i and $z_i = f_i(x, y)$ is its crisp consequence, the output is often

$$z^* = \frac{\sum_{i=1}^m \alpha_i z_i}{\sum_{i=1}^m \alpha_i}.$$

Interpretation

Rules with higher firing strength have stronger influence on the final crisp output.

Sugeno model of order zero: rule evaluation

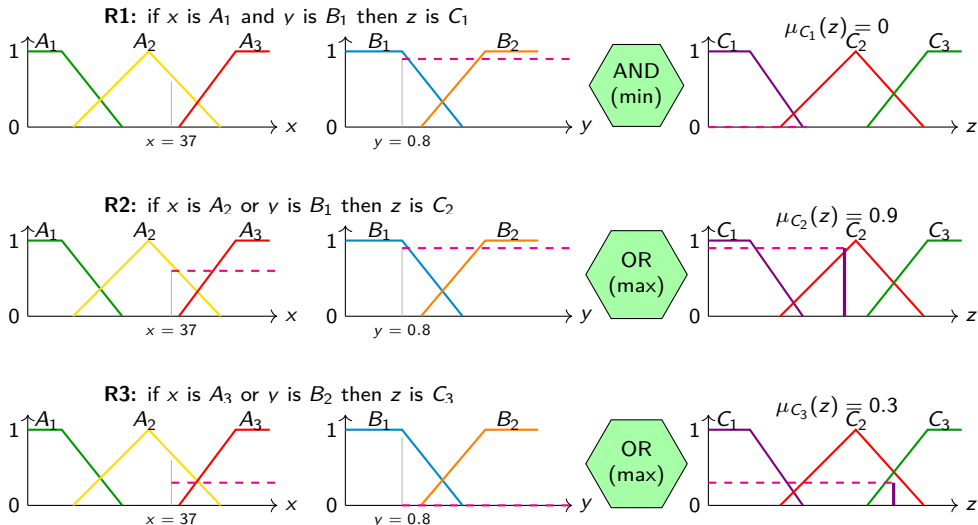


Main idea

In the Tsukamoto model, each rule consequence is represented by a fuzzy set with a monotone membership function.

- The firing strength is matched to the monotone consequent function.
- Each rule produces a crisp value.
- The final result is obtained as a weighted average of these crisp values.

Tsukamoto model: rule evaluation



Comparing Mamdani, Sugeno, and Tsukamoto

	Mamdani	Sugeno	Tsukamoto
Consequence	fuzzy set	crisp function	monotone fuzzy set
Rule output	fuzzy region	crisp value	crisp value
Aggregation	union of fuzzy regions	weighted average	weighted average
Interpretability	high	medium	medium
Common use	expert systems, control	adaptive control, optimisation	monotone-output systems

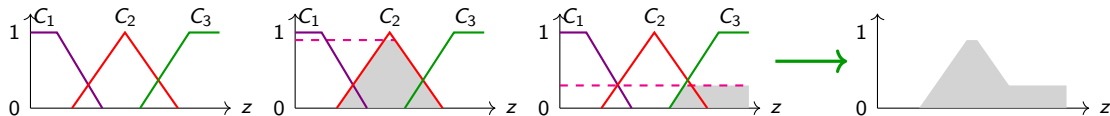
Aggregation of rule outputs

- Aggregation combines the outputs of all activated rules.
- For Mamdani systems, it combines clipped or scaled consequent fuzzy sets into one output fuzzy set.
- For Sugeno and Tsukamoto systems, aggregation is typically performed through a weighted average of crisp outputs.

Output of aggregation

A single fuzzy set for Mamdani systems, or a single crisp weighted expression for Sugeno/Tsukamoto systems.

Aggregation in Mamdani model



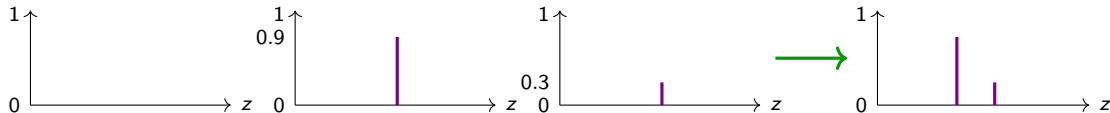
Mamdani aggregation

Each rule produces a **clipped fuzzy set**. The aggregated output is obtained by the union (usually the **max** operator) of all rule consequents:

$$\mu_{\text{agg}}(z) = \max_i \mu_i(z).$$

- **Mamdani:** aggregate fuzzy sets first, then defuzzify.

Aggregation in Sugeno model



Sugeno aggregation

Each rule produces a **crisp singleton output** z_i together with a firing strength w_i . The final output is computed by a weighted average:

$$z^* = \frac{\sum_{i=1}^m w_i z_i}{\sum_{i=1}^m w_i}.$$

- **Sugeno:** aggregate crisp rule outputs directly.

Definition

Defuzzification transforms an aggregated fuzzy result into a crisp value.

Main methods

- centre of area / centroid of area (COA);
- bisector of area (BOA);
- mean of maximum (MOM);
- smallest of maximum (SOM);
- largest of maximum (LOM).

COA: centroid of area

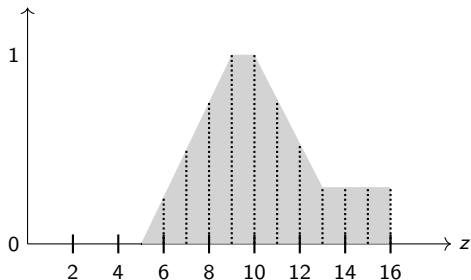
For a sampled Mamdani output set, the centroid can be approximated by

$$\text{COA} = \frac{\sum_{i=1}^n x_i \mu_A(x_i)}{\sum_{i=1}^n \mu_A(x_i)}.$$

For Sugeno and Tsukamoto models, the centroid-like computation becomes a weighted average:

$$z^* = \frac{\sum_{i=1}^m \alpha_i z_i}{\sum_{i=1}^m \alpha_i}.$$

Defuzzification by centroid: Mamdani model



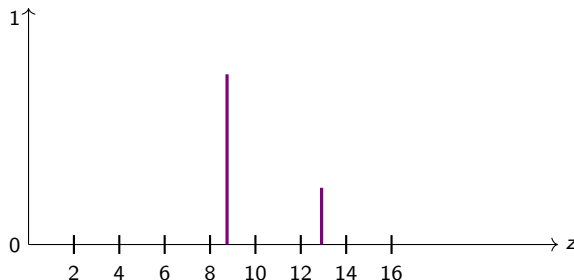
Mamdani centroid: It returns the **center of gravity** of the aggregated fuzzy region.

$$COA = \frac{5 \cdot 0 + 6 \cdot 0.3 + 7 \cdot 0.5 + 8 \cdot 0.7 + 9 \cdot 0.9 + 10 \cdot 0.9 + 11 \cdot 0.7 + 12 \cdot 0.5 + 13 \cdot 0.3 + 14 \cdot 0.3 + 15 \cdot 0.3 + 16 \cdot 0.3}{0 + 0.3 + 0.5 + 0.7 + 0.9 + 0.9 + 0.7 + 0.5 + 0.3 + 0.3 + 0.3 + 0.3}$$

$$COA \simeq 13.7$$

- **Mamdani:** centroid of an aggregated fuzzy area.

Defuzzification by centroid: Sugeno model



Sugeno output: **weighted average** of singleton consequents

In a zero-order Sugeno model, each rule produces a singleton output z_i with firing strength w_i . The crisp output is:

$$z^* = \frac{\sum_{i=1}^m w_i z_i}{\sum_{i=1}^m w_i}.$$

BOA: bisector of area

The bisector of area identifies the point z that divides the aggregated fuzzy set into two regions with equal area:

$$\int_{\alpha}^z \mu_A(x) dx = \int_z^{\beta} \mu_A(x) dx,$$

where α and β are the lower and upper bounds of the support of the aggregated fuzzy set.

Interpretation

BOA returns the point that splits the fuzzy mass into two equal parts.

Maximum-based defuzzification methods

- **MOM – Mean of maximum:** returns the mean of the points where the membership function reaches its maximum.
- **SOM – Smallest of maximum:** returns the smallest point where the maximum is reached.
- **LOM – Largest of maximum:** returns the largest point where the maximum is reached.

When they are useful

Maximum-based methods are simple and fast, but they ignore much of the shape of the aggregated fuzzy set.

Advantages of fuzzy systems

- Imprecise real-world concepts can be expressed through rules.
- Rules are usually easy to understand, test, and maintain.
- Fuzzy systems can be robust when rules or measurements are not perfectly precise.
- They may require fewer rules than some other knowledge-based systems.
- Rules can be evaluated in parallel.

Limitations of fuzzy systems

- They often require many simulations and empirical tests.
- Classical fuzzy systems do not automatically learn unless combined with learning methods.
- It can be difficult to identify the most appropriate rules and membership functions.
- The quality of the model depends strongly on expert knowledge and design choices.
- Fuzzy systems are not a universal substitute for probabilistic or statistical modelling.

Applications of fuzzy systems

- **Control:** satellite altitude, aircraft control, automatic transmission, traffic control, braking systems.
- **Business:** decision support, personnel evaluation, fund management, market prediction.
- **Industry:** energy exchange, water purification, pH control, chemical distillation, polymer production.
- **Consumer electronics:** camera exposure, humidity control, air conditioners, freezers, washing machines.

More application areas

- food production, for example cheese production;
- military and naval decision support;
- underwater and infrared image recognition;
- automatic navigation and route selection;
- medical diagnosis and anaesthesia pressure control;
- modelling neuropathological results in Alzheimer's disease;
- robotic kinematics, especially robotic arms.

- Fuzzy logic extends classical logic by allowing gradual truth values in $[0, 1]$.
- Fuzzy systems combine membership functions, linguistic variables, and rule bases.
- Fuzzy inference evaluates multiple partially satisfied rules in parallel.
- Mamdani, Sugeno, and Tsukamoto models differ mainly in their rule consequences.
- Defuzzification converts the fuzzy conclusion into a crisp action or decision.



BABEȘ-BOLYAI UNIVERSITY
Faculty of Computer Science and Mathematics



ARTIFICIAL INTELLIGENCE

Intelligent systems

Machine learning

Support Vector Machines

K-means

Intelligent Systems – Support Vector Machines

□ Support Vector Machines (SVMs)

- Definition
- Solved problems
- Advantages
- Difficulties
- Tools

Intelligent Systems – Support Vector Machines

□ Definition

- Developed by Vapnik in 1970
- Popularised after 1992
- Linear classifiers that identify the hyperplane that separates the positive and negative classes
- Have a theoretical foundation
- Work very well for large data (text mining, image analysis)

■ Remember

- Supervised learning problem – a data set:
 - (x^d, t^d) , with:
 - $x^d \in \mathbf{R}^m \square x^d = (x^d_1, x^d_2, \dots, x^d_m)$
 - $t^d \in \mathbf{R} \square t^d \in \{1, -1\}$, $1 \square$ positive class, $-1 \square$ negative class
 - where $d = 1, 2, \dots, n, n+1, n+2, \dots, N$
- First n data (x^d and t^d are known) are used as training data
- Last $N-n$ data (x^d is known, t^d is unknown) are used as testing data

Intelligent Systems – Support Vector Machines

□ Definition

- SVM finds a linear function $f(\mathbf{x}) = \langle \mathbf{w} \cdot \mathbf{x} \rangle + b$, ($\mathbf{w}$ –weight vector) such as

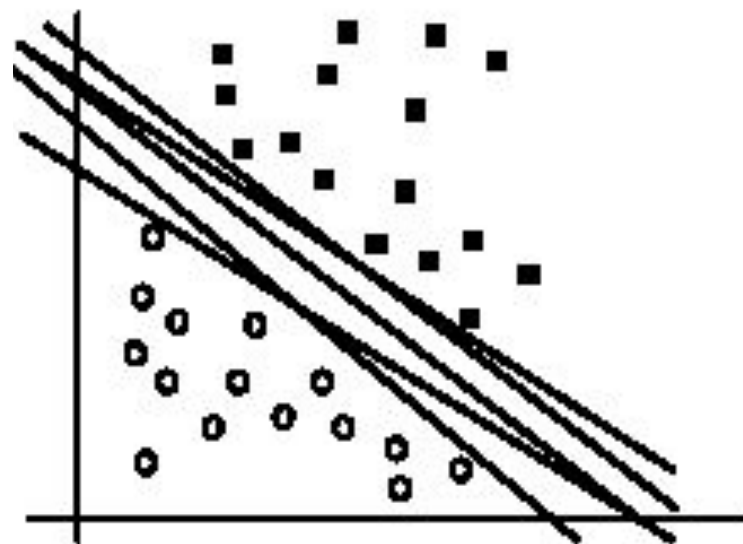
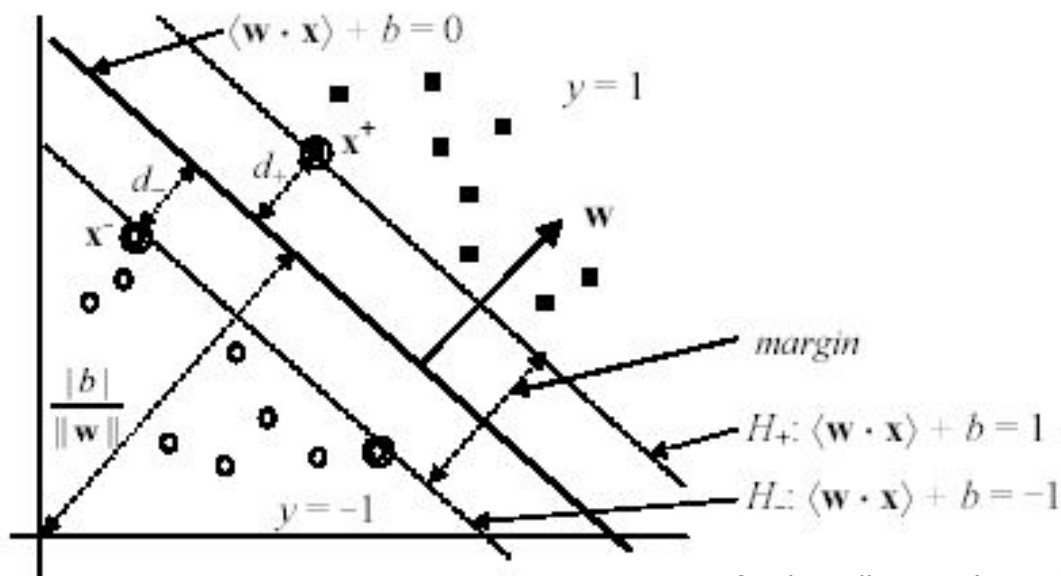
$$y_i = \begin{cases} 1 & \text{if } \langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b \geq 0 \\ -1 & \text{if } \langle \mathbf{w} \cdot \mathbf{x}_i \rangle + b < 0 \end{cases}$$

- $\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = 0$ □ decision hyperplane that separates the two classes

Intelligent Systems – Support Vector Machines

□ Definition

- There are more hyper-planes
 - Which is the best hyper-plane?
- SVM searches the hyper-plane with the largest margin (that minimises the generalisation error)
 - SMO (*Sequential minimal optimization*) algorithm



Intelligent Systems – Support Vector Machines

□ Solved problems

- Classification problems □ more cases (based on the data type):

- Linear separable

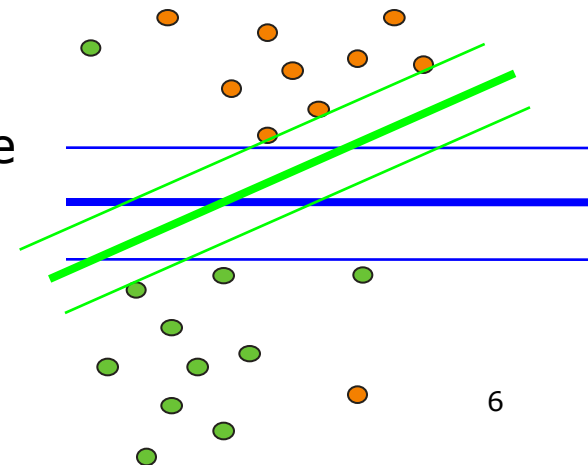
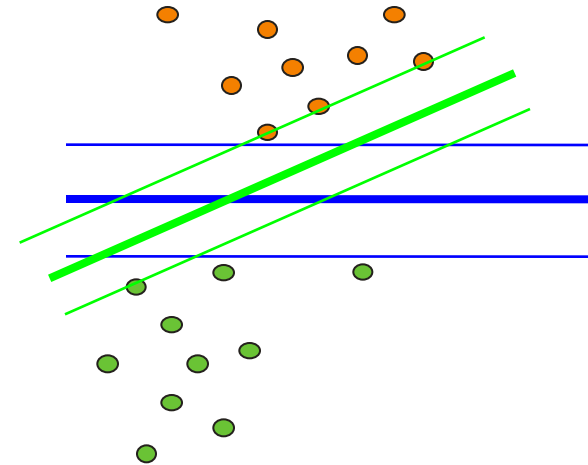
- Separable

- Error = 0

- Non-separable

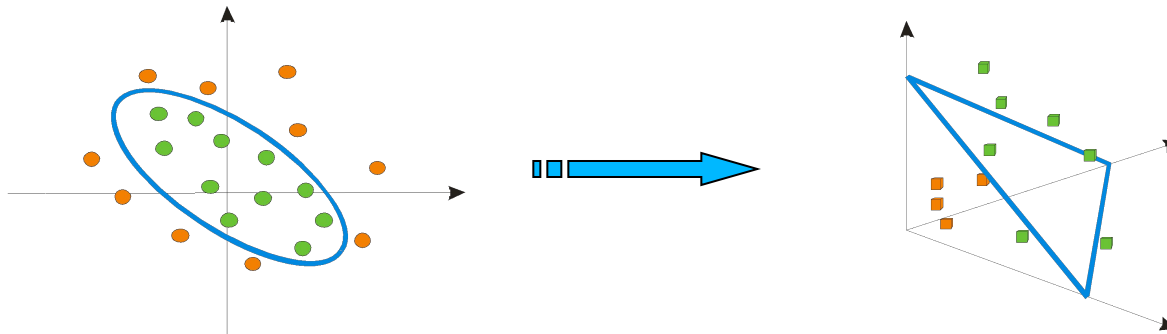
- Constraints are relaxed □ some error are allowed

- C – penalisation coefficient



Intelligent Systems – Support Vector Machines

- Solved problems □ classification problems □ data cases:
 - Non-linear separable
 - Input space is transformed (mapped) into a space of more dimensions (feature space) by using kernel function – in this new space the data becomes linear separable
 - In SVMs the kernel function computes the distance among 2 points
 - □ kernel $\sim$ similarity function



Intelligent Systems – Support Vector Machines

□ Solved problems □ classification problems □ data cases:

■ Non-linear separable □ possible kernels

□ Classic kernels

- Polynomial kernel: $K(\mathbf{x}^{d1}, \mathbf{x}^{d2}) = (\mathbf{x}^{d1}, \mathbf{x}^{d2} + 1)^p$
- RBF kernel: $K(\mathbf{x}^{d1}, \mathbf{x}^{d2}) = \exp(- ||\mathbf{x}^{d1} - \mathbf{x}^{d2}||^2 / 2\sigma^2)$

□ Multiple Kernels

- Linear : $K(\mathbf{x}^{d1}, \mathbf{x}^{d2}) = \sum w_i K_i(\mathbf{x}^{d1}, \mathbf{x}^{d2})$
- Non-linear
 - Without coefficients: $K(\mathbf{x}^{d1}, \mathbf{x}^{d2}) = K_1(\mathbf{x}^{d1}, \mathbf{x}^{d2}) + K_2(\mathbf{x}^{d1}, \mathbf{x}^{d2}) * \exp(K_3(\mathbf{x}^{d1}, \mathbf{x}^{d2}))$
 - With coefficients: $K(\mathbf{x}^{d1}, \mathbf{x}^{d2}) = K_1(\mathbf{x}^{d1}, \mathbf{x}^{d2}) + c_1 * K_2(\mathbf{x}^{d1}, \mathbf{x}^{d2}) \exp(c_2 + K_3(\mathbf{x}^{d1}, \mathbf{x}^{d2}))$

□ Kernels for strings

□ Kernels for images

□ Kernels for graphs

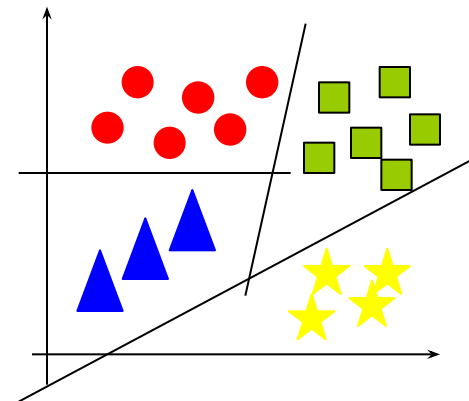
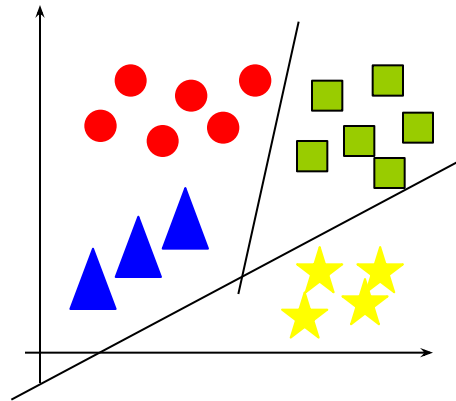
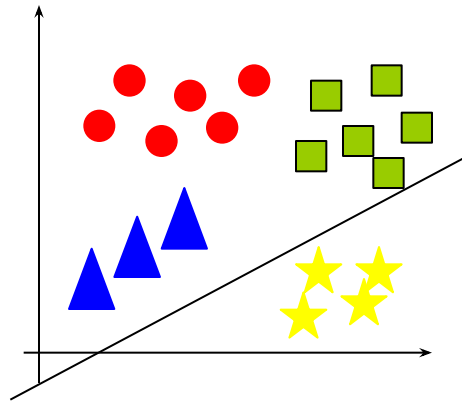
Intelligent Systems – Support Vector Machines

SVM parameters setting:

- Penalisation coefficient C
 - C – small $\square$ slowly convergence
 - C – large $\square$ fast convergence
- Kernel parameters
 - If m (# of attributes) is larger than n (# of data)
 - SVM with a linear kernel (SVM without kernel) $\square$
$$K(\mathbf{x}^{d1}, \mathbf{x}^{d2}) = \mathbf{x}^{d1} \cdot \mathbf{x}^{d2}$$
 - If m (# of attributes) is large and n (# of data) is medium
 - SVM with Gaussian kernel
$$K(\mathbf{x}^{d1}, \mathbf{x}^{d2}) = \exp(- ||\mathbf{x}^{d1} - \mathbf{x}^{d2} ||^2 / 2\sigma^2)$$
 - σ – dispersion of training data
 - Attributes must be normalised (scaled to (0,1))
 - If m (# of attributes) is small and n (# of data) is large
 - Add new attributes and use SVM with linear kernel

Intelligent Systems – Support Vector Machines

- SVM for multi-class classification problems (more than 2 classes)
 - one vs. all



Intelligent Systems – Support Vector Machines

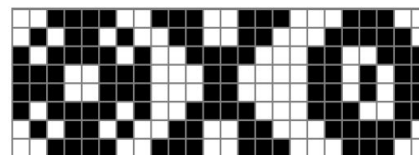
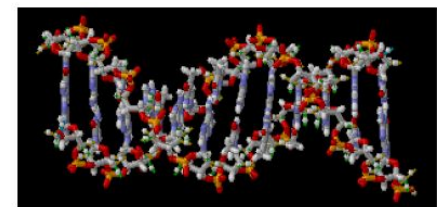
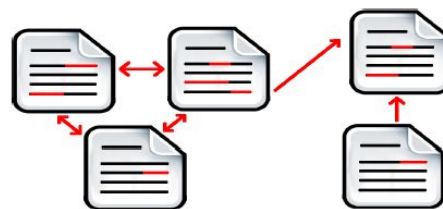
□ Structured SVMs

■ Machine Learning

- Simple SVM $f: \mathcal{X} \rightarrow \mathbb{R}$
 - Any type of inputs
 - Numerical outputs (natural numbers, integers, real numbers)
- Structured SVM: $\mathcal{X} \rightarrow \mathcal{Y}$
 - Any type of inputs
 - Any type of outputs (numerical or structured outputs)

■ Structured information

- Texts and hyper-texts
- Molecules and molecular structures
- Images



Intelligent Systems – Support Vector Machines

□ Structured SVMs

■ Applications

□ Natural Language Processing

- Automatic translation (outputs □ sentences)
- Syntactic and/or morphologic analysis of sentences (outputs □ syntactic and/or morphologic tree)

□ Bioinformatic

- Prediction of secondary structures (outputs □ bi-partite graphs)
- Prediction of enzyme function (outputs □ paths in trees)

□ Speech processing

- Automatic transcriptions (outputs □ sentences)
- Transformation of texts in voice (outputs □ audio signal)

□ Robotics

- Planning (outputs □ sequences of actions)

Intelligent Systems – Support Vector Machines

□ Advantages

- Can work with any type of data (linear or non-linear separable, uniform distributed or not, with known or unknown distribution)
 - Kernel function that creates new attributes (features) □ hidden layers of an ANN
- If the problem is convex SVM finds a unique solution □ global optima
 - ANNs can associate more solutions □ local optima
- Automatic selection of the learnt model (by support vectors)
 - In ANNs hidden layers have to be configured apriori
- Avoid overfitting
 - ANNs have overfitting problems even the cross-validation is involved

□ Difficulties

- Real attributes only
- Binary classification problems only
- Difficult mathematical background

□ Tools

- LibSVM □ <http://www.csie.ntu.edu.tw/~cjlin/libsvm/>
- Weka □ SMO
- SVMLight □ <http://svmlight.joachims.org/>
- SVMtorch □ <http://www.torch.ch/>
- <http://www.support-vector-machines.org/>

Intelligent systems – Machine Learning

□ Typology

■ Experience criteria:

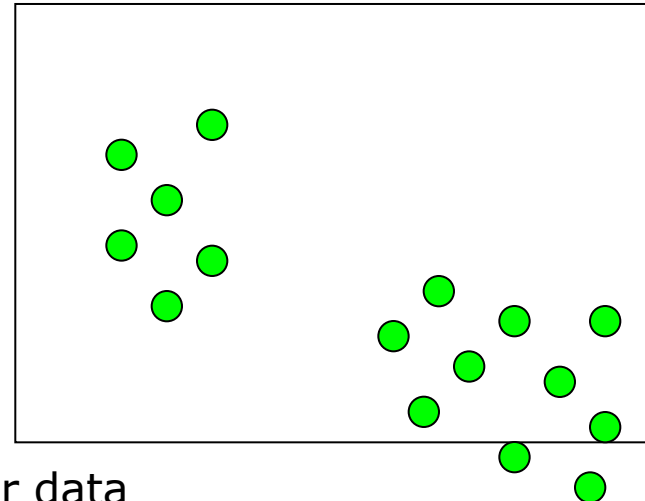
- Supervised learning
- Unsupervised learning
- Active learning
- Reinforcement learning

■ Algorithm criteria

- Decision trees
- Artificial Neural Networks
- Evolutionary Algorithms
- Support Vector Machines
- Hidden Markov Models
- **K-means**

Unsupervised learning

- Aim
 - to find a model or a structure of data
- Solved problems
 - Identification of groups (clusters)
 - Analysis of genes
 - Image processing
 - Analysis of social networks
 - Market segmentation
 - Analysis of astronomical data
 - Clusters of computers
 - Dimension reduction
 - Identification of causes (explanations) for data
 - Modelling the data densities
- Specific
 - **Data are not annotated** (labelled)



Unsupervised learning – definition

Separates the un-labelled examples in disjoint subsets (clusters) such as:

- Examples of the same cluster are similar
- Examples of different clusters are different

Definition

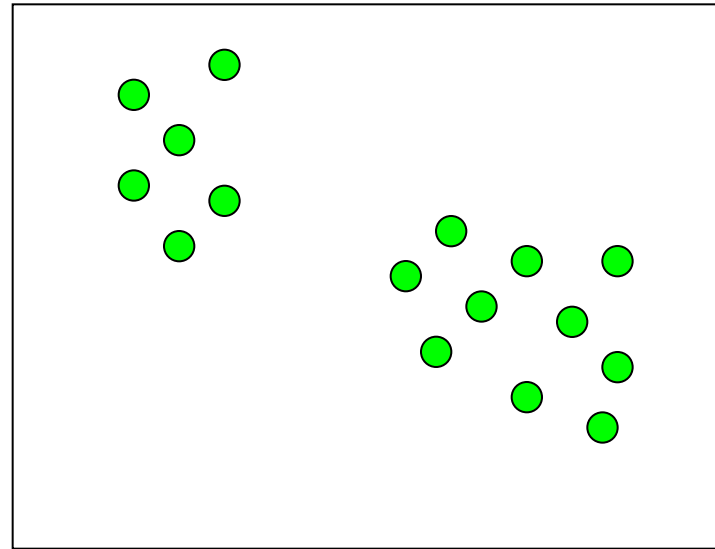
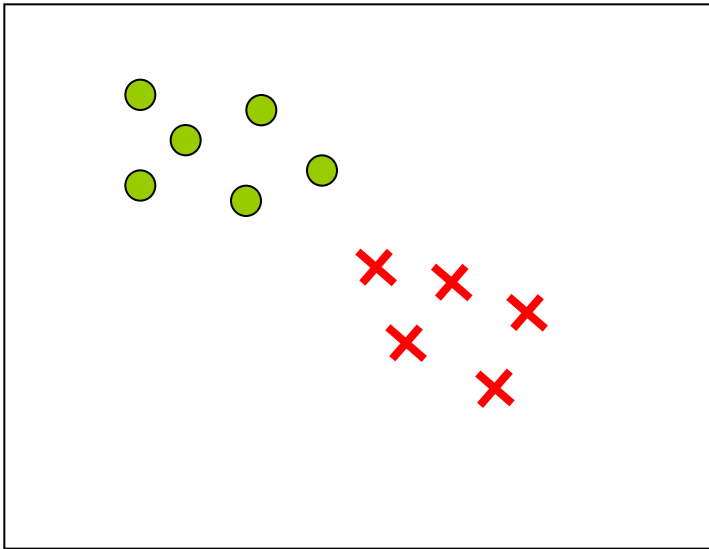
- Given
 - A set of data (examples, instances, cases)
 - Training data
 - As **attribute_data_i**, where
 - $i = 1, N$ ($N = \#$ of training data)
 - **attribute_data_i** = ($atr_{i1}, atr_{i2}, \dots, atr_{im}$), $m = \#$ of attributes (characteristics, properties) of data
 - Testing data
 - As (**attribute_data_i**), $i = 1, n$ ($n = \#$ of testing data)
- Determine
 - An unknown function that groups the training data in more classes
 - # of classes can be pre-defined (k) or unknown
 - Data of the same class are similar
 - The class of a new testing data by using the learnt grouping (on training data)

Other names

- Clustering

Unsupervised learning – definition

■ Supervised vs. unsupervised



Unsupervised learning – definition

□ Distance between 2 elements $\mathbf{p}$ and $\mathbf{q} \in R^m$

- Euclid distance

- $d(\mathbf{p}, \mathbf{q}) = \text{sqrt}(\sum_{j=1,2,\dots,m} (p_j - q_j)^2)$

- Manhattan distance

- $d(\mathbf{p}, \mathbf{q}) = \sum_{j=1,2,\dots,m} |p_j - q_j|$

- Mahalanobis distance

- $d(\mathbf{p}, \mathbf{q}) = \text{sqrt}(\mathbf{p} - \mathbf{q})^T \mathbf{S}^{-1} (\mathbf{p} - \mathbf{q})$,

- Where $\mathbf{S}$ is the covariance matrix ($\mathbf{S} = E[(\mathbf{p} - E[\mathbf{p}])(\mathbf{q} - E[\mathbf{q}])^T]$)

- Internal product

- $d(\mathbf{p}, \mathbf{q}) = \sum_{j=1,2,\dots,m} p_j q_j$

- Cosine distance

- $d(\mathbf{p}, \mathbf{q}) = \sum_{j=1,2,\dots,m} p_j q_j / (\text{sqrt}(\sum_{j=1,2,\dots,m} p_j^2) * \text{sqrt}(\sum_{j=1,2,\dots,m} q_j^2))$

- Hamming distance

- # of differences between $\mathbf{p}$ and $\mathbf{q}$

- Levenshtein distance

- Minimal # of operations required for transforming $\mathbf{p}$ in $\mathbf{q}$

□ Distance vs. similarity

- Distance □ minimisation

- Similarity □ maximisation

Unsupervised learning – definition

Application

- Gene clustering
- Market segmentation (for client clustering)
- news.google.com

Unsupervised learning – process

Process

- 2 steps:
 - Learning
 - Determine (learn), by using an algorithm, the existing clusters
 - Testing
 - Include a new data in one of the identified (during training) clusters

Learning quality (clustering validation)

- Internal criteria
 - Large similarity inside the cluster and reduce similarity between clusters
- External criteria
 - Using benchmarks composed of *apriori* grouped data

Unsupervised learning – evaluation

Performance measures:

□ Internal criteria

- Distance inside the cluster
- Distance between clusters
- Davies-Bouldin index
- Dunn index

□ External criteria

- Comparison with known data – impossible in real-world applications
- Precision
- Recall
- F-measure

Unsupervised learning – evaluation

Internal criteria

- Distance inside cluster c_j that contains n_j instances
 - Average distance (among instances)
 - $D_a(c_j) = \sum_{x_{i1}, x_{i2} \in c_j} ||x_{i1} - x_{i2}|| / (n_j(n_j-1))$
 - Nearest neighbour distance
 - $D_{nn}(c_j) = \sum_{x_{i1} \in c_j} \min_{x_{i2} \in c_j} ||x_{i1} - x_{i2}|| / n_j$
 - Distance to centroids
 - $D_c(c_j) = \sum_{x_i \in c_j} ||x_i - \mu_j|| / n_j$, where $\mu_j = 1/n_j \sum_{x_i \in c_j} x_i$

Unsupervised learning – evaluation

Internal criteria

- Distance between two clusters c_{j1} and c_{j2}
 - Simple link
 - $d_s(c_{j1}, c_{j2}) = \min_{x_{i1} \in c_{j1}, x_{i2} \in c_{j2}} \{ ||x_{i1} - x_{i2} || \}$
 - Complete link
 - $d_{co}(c_{j1}, c_{j2}) = \max_{x_{i1} \in c_{j1}, x_{i2} \in c_{j2}} \{ ||x_{i1} - x_{i2} || \}$
 - Average link
 - $d_a(c_{j1}, c_{j2}) = \sum_{x_{i1} \in c_{j1}, x_{i2} \in c_{j2}} \{ ||x_{i1} - x_{i2} || \} / (n_{j1} * n_{j2})$
 - Link between centroids
 - $d_{ce}(c_{j1}, c_{j2}) = ||\mu_{j1} - \mu_{j2} ||$

Unsupervised learning – evaluation

Internal criteria

- Davies-Bouldin index □ min □ compact clusters

- $DB = 1/nc * \sum_{i=1,2,\dots,nc} \max_{j=1, 2, \dots, nc, j \neq i} ((\sigma_i + \sigma_j)/d(\mu_i, \mu_j))$

- where:

- nc – # of clusters
 - μ_i – centroid of cluster i
 - σ_i – average of distances between elements from cluster i and the centroid μ_i
 - $d(\mu_i, \mu_j)$ – distance between centroid μ_i and centroid μ_j

- Dunn index

- Identifies the dense clusters and well separated

- $D = d_{min} / d_{max}$

- where:

- d_{min} – minimal distance between 2 elements from different clusters - intra-cluster distance
 - d_{max} – maximal distance between 2 elements from the same cluster - inter-cluster distance

Unsupervised learning – typology

- How the clusters are forming
 - Hierarchic clustering
 - Non-hierarchical (partitioned) clustering
 - Clustering based on data density
 - Clustering based on a grid

Unsupervised learning – typology

□ How the clusters are forming

■ Hierarchic clustering

- Creates a dendrogram (taxonomic tree)
 - Creates the clusters (recursively)
 - k (# of clusters) is unknown
- Agglomerative clustering (bottom-up) □ small clusters to large clusters
- Divisive clustering (top-down) □ large clusters to small clusters
- Eg.
 - Clustering ierarhic agglomerative

Unsupervised learning – typology

How the clusters are formed

■ Non-hierarchical

- Partitional □ determine a data separation □ all the clusters in the same time
- Optimises an objective function defined
 - Locally – by using some features only
 - Globally – by using all attributesthat can be:
 - squared error – sum of squared distances between data and the cluster's centroid □ min
 - Ex. *K-means*
 - Graph-based
 - Ex. Clustering based in minimum spanning tree
 - Based on probabilistic models
 - Ex. Identify the data distribution □ expectation maximisation
 - Based on the nearest neighbour
- Required to fix k apriori □ fix the initial clusters
 - Algorithm is run more times with different parameters and the most efficient version is selected
- Ex. *K-means*, *ACO*

Unsupervised learning – typology

How the clusters are forming

- Based on data densities
 - Data density and data connectivity
 - Cluster formation is based on data density from a given region
 - Cluster formation is based on data connectivity from a given region
 - Function of data density
 - Tries to model the data distribution
 - Advantage:
 - Modeling of clusters of any shape

Unsupervised learning – typology

□ How the cluster are forming

■ Based on a grid

- Is not a distinct approach
 - Can be hierarchic, partitional or density-based
- Involves data space segmentation in regular areas
- Objects are placed on a multi-dimensional grid
- Eg. ACO

Unsupervised learning – typology

□ How the algorithms work

■ Agglomerative clustering

1. Initially, each instance form a cluster
2. Compute the distance between any 2 clusters
3. Reunion the closest 2 clusters
4. Repeat steps 3 and 4 until a single cluster is obtained or other stop criterion is satisfied

■ Divisive clustering

1. Establish the number of clusters (k)
2. Initialise the centre of each cluster
3. Determine a data separation
4. Re-compute the centre of each cluster
5. Repeat steps 3 and 4 until the partition is unchanged (algorithm converges)

□ How the algorithm takes into account the attributes (features)

- Monotonic – attributes are taken into account one-by-one
- Polytonic – attributes are simultaneous taken into

Unsupervised learning – typology

□ How the data belong to clusters

■ Exact clustering (hard clustering)

- Each instance x_i has associated a label (class) c_j

■ Fuzzy clustering

- Associates to each input $\mathbf{x}_i$ a degree (probability) of membership f_{ij} to a certain class c_j □ an instance $\mathbf{x}_i$ that can belong to several clusters

Unsupervised learning – algorithms

- Agglomerative hierarchical clustering
- K-means
- AMA
- Probabilistic models
- Nearest neighbour
- Fuzzy
- Artificial Neural Network
- Evolutionary algorithms
- ACO

Unsupervised learning – algorithms

Agglomerative hierarchical clustering

- a. Consider a distance between 2 instances $d(x_{i1}, x_{i2})$
- b. Form N clusters, each of them containing an instance
- c. Repeat
 - Determine the closest 2 clusters
 - Reunion the 2 clusters $\square$ a cluster

Until *a single cluster is obtained*

Unsupervised learning – algorithms

Agglomerative hierarchical clustering

Distance between 2 clusters c_i and c_j :

- Simple link $\square$ minimal distance between the objects of 2 clusters
 - $d(c_i, c_j) = \min_{x_{i1} \in c_i, x_{j2} \in c_j} \text{sim}(\mathbf{x}_{i1}, \mathbf{x}_{j2})$
- Complete link $\square$ maximal distance between the objects of 2 clusters
 - $d(c_i, c_j) = \max_{x_{i1} \in c_i, x_{j2} \in c_j} \text{sim}(\mathbf{x}_{i1}, \mathbf{x}_{j2})$
- Average link $\square$ mean of distances between the objects of 2 clusters
 - $d(c_i, c_j) = 1 / (n_i * n_j) \sum_{x_{i1} \in c_i} \sum_{x_{j2} \in c_j} d(\mathbf{x}_{i1}, \mathbf{x}_{j2})$
- Average link over group $\square$ distance between the means (centroids) of 2 clusters
 - $d(c_i, c_j) = \rho(\boldsymbol{\mu}_i, \boldsymbol{\mu}_j)$, ρ – distance, $\boldsymbol{\mu}_j = 1/n_j \sum_{x_{j2} \in c_j} \mathbf{x}_{j2}$

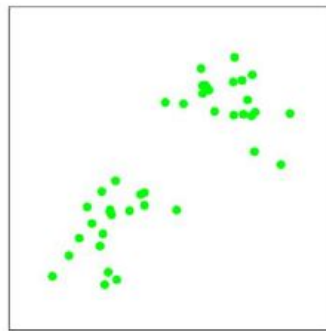
Unsupervised learning – algorithms

K-means (Lloyd algorithm / Voronoi iteration)

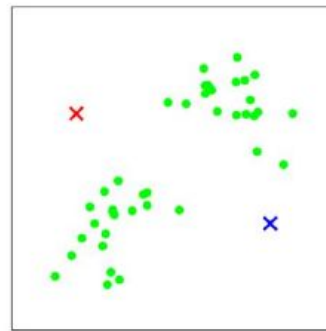
- Suppose that k clusters will form
- Initialise k centroids $\mu_1, \mu_2, \dots, \mu_k$
 - A centroid μ_j ($j=1, 2, \dots, k$) is a vector of m values (m - # of features)
- Repeat until convergence
 - Associated to each instance the nearest centroid □ for each instance $\mathbf{x}_i$, $i = 1, 2, \dots, N$
 - $c_i = \arg \min_{j=1, 2, \dots, k} \|\mathbf{x}_i - \mu_j\|^2$
 - Re-compute the centroids by moving them in the mean of instances associated to it □ for each cluster c_j , $j = 1, 2, \dots, k$
 - $\mu_j = \sum_{i=1, 2, \dots, N} 1_{c_i=j} \mathbf{x}_i / \sum_{i=1, 2, \dots, N} 1_{c_i=j}$

Unsupervised learning – algorithms

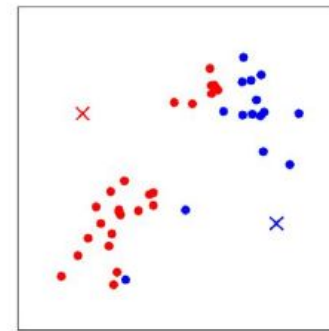
□ K-means



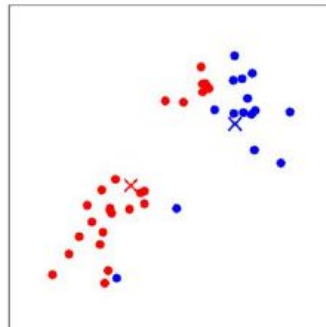
(a)



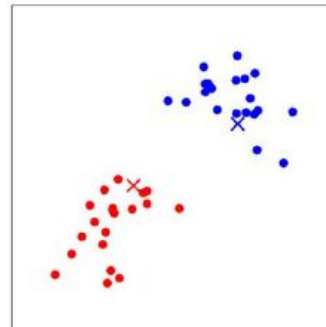
(b)



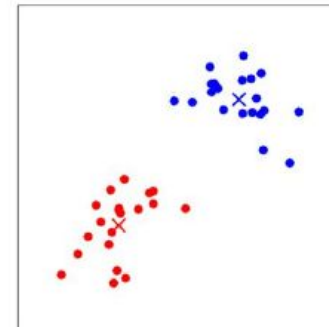
(c)



(d)



(e)



(f)

Unsupervised learning – algorithms

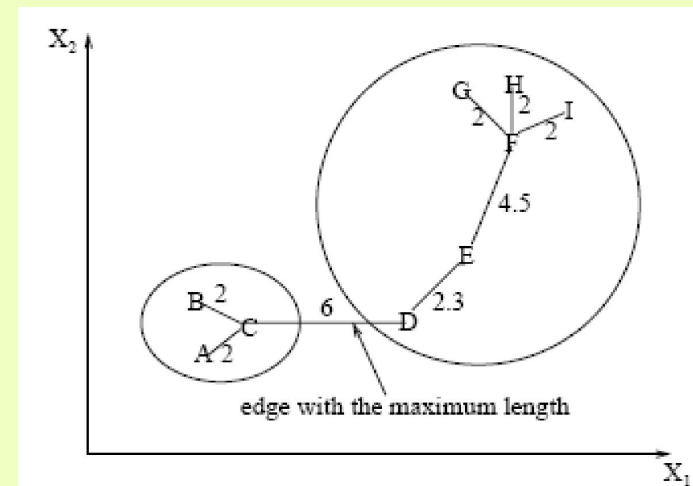
K-means

- Initialisation of k centroids $\mu_1, \mu_2, \dots, \mu_k$
 - With random values (in the definition domain of the problem)
 - With k instances of N (randomly selected)
- Does the algorithm always converge?
 - Yes, because of distortion function J
 - $J(c, \mu) = \sum_{i=1,2, \dots, N} ||\mathbf{x}_i - \mu_{cj}||^2$
which is decreasing
 - Converges in a local optima
 - Finding the global optima $\square$ NP-difficult problem

Unsupervised learning – algorithms

Clustering based on minimum spanning tree (AMA)

- Construct the minimum spanning tree of data
- Eliminate from the tree the longest edges and form clusters



Unsupervised learning - algorithms

Probabilistic models

- <http://www.gatsby.ucl.ac.uk/~zoubin/course04/ul.pdf>
- <http://learning.eng.cam.ac.uk/zoubin/nipstut.pdf>

Unsupervised learning - algorithms

Nearest neighbor

- Some of the instances are labeled
- It is repeated until all instances are labeled
 - An unlabeled instance will be included in the closest instance cluster
 - if the distance between the unlabeled instance and the labeled one is less than a threshold

Unsupervised learning - algorithms

Fuzzy clustering

- An initial fuzzy partitioning is established
 - The membership degrees matrix U , is constructed, where u_{ij} – the degree of membership of instance $\mathbf{x}_i$ ($i=1,2, \dots, N$) to the cluster c_j ($j = 1, 2, \dots, k$) ($u_{ij} \in [0,1]$)
 - The higher u_{ij} is, the higher the confidence that instance $\mathbf{x}_i$ is part of cluster c_j
- An objective function is established
 - $E^2(U) = \sum_{i=1,2, \dots, N} \sum_{j=1,2, \dots, k} u_{ij} ||\mathbf{x}_i - \boldsymbol{\mu}_j||^2$,
 - where $\boldsymbol{\mu}_j = \sum_{i=1,2, \dots, N} u_{ij} \mathbf{x}_i$ – the center of the j^{th} fuzzy cluster
 - which is optimized (min) by re-assigning instances (in new clusters)
- Fuzzy Clustering $\square$ Hard Clustering
 - imposing a threshold on the membership function u_{ij}

Thank you for your attention!